Knowledge-Based Editor

Volume 3

Programmer's Apprentice Group

MIT AI Lab

January, 1985

Table of Contents: Knowledge Based Editor

Volume 1

OZ:<KBE.DOC>358.TEXT
OZ:<KBE.TEST>358.LISP

OZ:<KBE>KBE.INDEX

OZ:<KBE>LISPM.INIT
OZ:<KBE>SYSDEF.LISP

OZ:<KBE.SOURCE>BASIC.LISP
OZ:<KBE.SOURCE>NOISE.LISP
OZ:<KBE.SOURCE>SETS.LISP
OZ:<KBE.SOURCE>CONDITIONS.LISP
OZ:<KBE.SOURCE>ENGLISH.LISP
OZ:<KBE.SOURCE>SYSTEM-MAGIC.LISP
OZ:<KBE.SOURCE>FLAVOR-MAGIC.LISP
OZ:<KBE.SOURCE>MACRO.LISP
OZ:<KBE.SOURCE>KVARS.LISP
OZ:<KBE.SOURCE>KMAC.LISP
OZ:<KBE.SOURCE>INTERFACE-MACROS.LISP
OZ:<KBE.SOURCE>MBOX.LISP
OZ:<KBE.SOURCE>POST-OFFICE.LISP
OZ:<KBE.SOURCE>BEHAVIOR.LISP
OZ:<KBE.SOURCE>EVENTS.LISP
OZ:<KBE.SOURCE>ACTOR.LISP
OZ:<KBE.SOURCE>COMPLETE.LISP
OZ:<KBE.SOURCE>COMMENT-MACRO.LISP
OZ:<KBE.SOURCE>READMACRO.LISP
OZ:<KBE.SOURCE>OUTIL.LISP
OZ:<KBE.SOURCE>DTRACE.LISP
OZ:<KBE.SOURCE>DUTIL.LISP
OZ:<KBE.SOURCE>DMAC.LISP
OZ:<KBE.SOURCE>DVARS.LISP
OZ:<KBE.SOURCE>DB.LISP
OZ:<KBE.SOURCE>GC.LISP
OZ:<KBE.SOURCE>DDEBUG.LISP

Volume 2

OZ:<KBE.SOURCE>SURFACE.LISP
OZ:<KBE.SOURCE>EXPAND.LISP
OZ:<KBE.SOURCE>PRIMOP.LISP
OZ:<KBE.SOURCE>DFLOW.LISP
OZ:<KBE.SOURCE>UTIL.LISP
OZ:<KBE.SOURCE>SEGTRN.LISP
OZ:<KBE.SOURCE>ANALYZE.LISP
OZ:<KBE.SOURCE>CODER.LISP

Volume 3

OZ:<KBE.SOURCE>PSEUDO.LISP
OZ:<KBE.SOURCE>DICT.LISP
OZ:<KBE.SOURCE>GRAM.LISP
OZ:<KBE.SOURCE>REFER.LISP
OZ:<KBE.SOURCE>VERBS.LISP
OZ:<KBE.SOURCE>IMPLEMENT.LISP
OZ:<KBE.SOURCE>ADA-EVAL.LISP
OZ:<KBE.SOURCE>ADA-ANAL.LISP
OZ:<KBE.SOURCE>ADA-PARSE.LISP
OZ:<KBE.SOURCE>ADA-UNPARSE.LISP
OZ:<KBE.SOURCE>VIEW.LISP
OZ:<KBE.SOURCE>PLI.LISP
OZ:<KBE.SOURCE>DEFLIB.LISP
OZ:<KBE.SOURCE>SYSLIB.LISP
OZ:<KBE.SOURCE>KBELIB.LISP
OZ:<KBE.SOURCE>DEMO-LIB.LISP
OZ:<KBE.SOURCE>DEMO-ADA-LIB.LISP

Volume 4

OZ:<KBE.SOURCE>BRIDGE.LISP
OZ:<KBE.SOURCE>KAOS.LISP
OZ:<KBE.SOURCE>SOCIETY.LISP
OZ:<KBE.SOURCE>STANDARD-ACTOR.LISP
OZ:<KBE.SOURCE>REMOTE.LISP
OZ:<KBE.SOURCE>OVERLAPPING-REGIONS.LISP
OZ:<KBE.SOURCE>MOUSE-SENSITIVE-REGIONS.LISP
OZ:<KBE.SOURCE>OPTIONS.LISP
OZ:<KBE.SOURCE>SIMPLE.LISP
OZ:<KBE.SOURCE>SIMPLE-ACTOR.LISP
OZ:<KBE.SOURCE>MAGIC-ACTOR.LISP
OZ:<KBE.SOURCE>TV-MAGIC.LISP
OZ:<KBE.SOURCE>ZWEI-MAGIC.LISP
OZ:<KBE.SOURCE>POSSIBILITIES.LISP
OZ:<KBE.SOURCE>BELIEF.LISP
OZ:<KBE.SOURCE>EDIT.LISP
OZ:<KBE.SOURCE>EDIT-ACTOR.LISP
OZ:<KBE.SOURCE>CONSISTENCY.LISP
OZ:<KBE.SOURCE>EDIT-COMMANDS.LISP
OZ:<KBE.SOURCE>EDIT-MISC.LISP
OZ:<KBE.SOURCE>EDIT-SIMPLE.LISP
OZ:<KBE.SOURCE>EDIT-DEBUG.LISP
OZ:<KBE.SOURCE>KBE-MODE.LISP
OZ:<KBE.SOURCE>FOCUS.LISP
OZ:<KBE.SOURCE>TEST.LISP
OZ:<KBE.SOURCE>DOC.LISP
OZ:<KBE.SOURCE>PARSE.LISP
OZ:<KBE.SOURCE>EXPLAIN.LISP
OZ:<KBE.SOURCE>DOCUMENT.LISP
OZ:<KBE.SOURCE>PBM-HANDLERS.LISP
OZ:<KBE.SOURCE>INIT.LISP
OZ:<KBE.SOURCE>DEBUG.LISP

```
;;; -*- Mode: LISP; Package: KBE; Base: 10; Fonts: MEDFNB -*-

;;; (c) Copyright 1983, Massachusetts Institute of Technology
;;;     All Rights Reserved.
;;;     Permission to copy all or part of this material is granted,
;;;     provided that the copies are not made or distributed for resale,
;;;     the MIT copyright notice and reference to the source file and
;;;     the software distribution version appear, and that notice is
;;;     given that copying is by permission of Massachusetts Institute
;;;     of Technology.

;;; PSEUDO: Support for the Pseudo-English grammar.

;;;; Declarations, etc.

(defvar *cliches*  nil "A list of all active cliches.")
(defvar *roles*    nil "A list of all active roles.")
```

```
;;;; Dictionary Support

;;; The dictionary, which is held by the variable *DICTIONARY-LIST*
;;; has the following format, which is relied upon only by these
;;; dictionary utilities.
;;;
;;;   <dictionary-list> ::= (<entry> . <dictionary-list>)
;;;   <entry> ::= (<word> <part-of-speech> <syntax> <description>)

(defvar *dictionary-list* '() "A list of dictionary entries")

;;;  Externally, the functions ADD-DICTIONARY-ENTRY and PRINT-DICTIONARY
;;;  should be the only things that get used.

;;;  (ADD-DICTIONARY-ENTRY word part-of-speech syntax description)
;;;    Adds a word to the dictionary. Warns if a definition already
;;;    exists which conflicts with the definition being added.

(defun add-dictionary-entry (word part-of-speech syntax description)
  (let ((new-entry (list word part-of-speech syntax description)))
    (let ((probe (mem #'(lambda (x y)
                          (and (eq (car  x) (car  y))
                               (eq (cadr x) (cadr y))))
                      new-entry
                      *dictionary-list*)))
      (cond (probe
             (let ((entry (if probe (car probe))))
               (if (not (equal (cddr entry) (cddr new-entry)))
                   (format error-output
                           "~&Dictionary entry for ~A ~A being redefined."
                           (string-downcase part-of-speech) word))
               (setf (cddr entry) (cddr new-entry))))
            (t
             (push new-entry *dictionary-list*))))))
```

```
;;;  (PRINT-DICTIONARY [[style] stream])
;;;    STYLE may be one of:
;;;      :RAW           -- Output the dictionary with no sorting
;;;                        or grouping.
;;;      :SORT          -- Output the dictionary sorted alphabetically.
;;;      :GROUP         -- Output the dictionary grouped by part of speech
;;;                        but with no sorting done within groups.
;;;      :GRUP-AND-SORT -- Output the dictionary grouped by part of speech
;;;                        and sorted alphabetically within each group.
;;;    Probably a STYLE of :RAW is fastest and :GROUP looks nicest.
;;;    STREAM controls where the output goes.

(defun print-dictionary (&optional
                         (style ':raw)
                         (stream standard-output))
  (cond ((or (not style) (eq style ':raw))
         (format stream "~&KBE Dictionary")
         (print-dictionary-internal *dictionary-list* stream))
        ((eq style ':sort)
         (format stream "~&KBE Dictionary (Sorted)")
         (print-dictionary-internal
           (sortcar (copylist *dictionary-list*) #'string-lessp)
           stream))
        ((eq style ':group)
         (let ((parts-of-speech '()) (pl nil))
           (dolist (entry *dictionary-list*)
             (if (not (memq (cadr entry) parts-of-speech))
                 (push (cadr entry) parts-of-speech)))
           (dolist (entry *dictionary-list*)
             (push entry (get (locf pl) (cadr entry))))
           (format stream "~&KBE Dictionary (Grouped)")
           (do ((l pl (cddr l))
                (result nil (cons (cons (car l) (cadr l)) result)))
               ((null l)
                (dolist (l (sortcar result #'string-lessp))
                  (format stream "~&~/|~2%~A~2%"
                          (string-capitalize-words (car l)))
                  (print-dictionary-internal (cdr l) stream))))))
        ((eq style ':group-and-sort)
         (let ((parts-of-speech '()) (pl nil))
           (dolist (entry *dictionary-list*)
             (if (not (memq (cadr entry) parts-of-speech))
                 (push (cadr entry) parts-of-speech)))
           (dolist (entry *dictionary-list*)
             (push entry (get (locf pl) (cadr entry))))
           (format stream "~&KBE Dictionary (Grouped and Sorted)")
           (do ((l pl (cddr l))
                (result nil (cons (cons (car l) (cadr l)) result)))
               ((null l)
                (dolist (l (sortcar result #'string-lessp))
                  (format stream "~&~/|~2%~A~2%"
                          (string-capitalize-words (car l)))
                  (print-dictionary-internal
                    (sortcar (cdr l) #'string-lessp)
                    stream))))))
        (t (ferror "~S is not a valid KBE dictionary style." style)))
  (format stream "~%~/|~2%Cliche~2%")
  (dolist (cliche *cliches*)
    (let ((comments (get cliche 'cliche-comments)))
      (let ((parts  (get comments'roles)))
        (format stream "~2%~A, cliche" cliche)
        (when parts
          (format stream "~% Role~P: ~A"
                  (length parts)
                  (phrase-append "and" (mapcar #'string-downcase parts))))
        (format stream ".~% ~A~&"
                (let ((data (or (get comments 'detail)
```

```
                        (get comments 'main))))
                (cond ((null data) "No available documentation.")
                      ((symbolp data)
                       (string-capitalize-words data stream))
                      ((stringp data) data)
                      (t
                       (auto-fill-string
                         (remove-misplaced-whitespace
                           (with-output-to-string (str)
                             (dolist (text data)
                               (format str "~A "
                                       (cond ((stringp text) text)
                                             ((get comments text))
                                             (t (string-downcase
                                                  text)))))))
                         nil " "))))))))
  (values))

(defun print-dictionary-internal (entry-list stream)
  (dolist (entry entry-list)
    (format stream "~&~A, ~A~@[~% Syntax: /"~A/"~]~% ~A"
            (car entry)
            (string-downcase (cadr entry))
            (caddr entry)
            (auto-fill-string (cadddr entry) nil " "))))
```

```
;;;; Utilities

;;; Type predicates

(defun setup-plural (name)
  (putprop (intern (string-pluralize name) *kbe-package*)
           name
           'singular))

;for the convenience of other places in the KBE that call this function,
;it works harder than just doing a get.  It may still easily fail
;however.

(defun singular-form (name)
  (cond ((get name 'singular))
        ((< (string-length name) 2) nil)
        (t (let (singular-string)
             (prog ()
               (when (> (string-length name) 3)
                 (setq singular-string
                       (string-append
                         (substring name 0 (- (string-length name) 3))
                         "Y"))
                 (if (string-equal (string name)
                                   (string-pluralize singular-string))
                     (return nil)))
               (setq singular-string
                     (substring name 0 (1- (string-length name))))
               (if (string-equal (string name)
                                 (string-pluralize singular-string))
                   (return nil))
               (setq singular-string nil))
             (if singular-string
                 (intern singular-string (symbol-package name)))))))

(defvar *completion-table* nil "List of completions")

(defun *define-completion (name)
  (if (symbolp name) (putprop name t 'in-dictionary))
  (setq *completion-table*
        (add-to-completion-table *completion-table*
                                 (cond ((stringp name) name)
                                       (t (string-downcase name))))))

(defun in-dictionary? (name) (get name 'in-dictionary))

(defun *define-cliche  (name)
  (when (not (memq name *cliches*))
    (push name *cliches*)
    (*define-completion name)
    (setup-plural name)))

(defun *define-role    (name)
  (when (not (memq name *roles*))
    (push name *roles*)
    (*define-completion name)
    (setup-plural name)))

(defun *undefine-cliche  (name) (setq *cliches*  (delq name *cliches*)))
(defun *undefine-role    (name) (setq *roles*    (delq name *roles*)))

(defun cliche-name?  (x) (mem #'string-equal x *cliches*))
(defun role-name?    (x) (or (mem #'string-equal x *roles*)
                             (and (symbolp x) (not (in-dictionary? x))
                                  (not (memq x '(nil s))))))
(defun program-name? (x) (fboundp x))

(defun in-chars? (thing chars)
```

```
  (cond ((fixp thing) (memq thing chars))
        ((not (or (symbolp thing) (stringp thing))) nil)
        ((> (string-length thing) 1) nil)
        (t (in-chars? (character thing) chars))))

(defvar comment-chars        '(#/;)         "list of comment chars")
(defvar sentence-terminators '(#/. #/? #/!) "list of sentence terminators")
(defvar word-separators      '(#/,)         "list of word separators")
(defvar whitespace-chars     '(#\space
                               #\tab
                               #\return)    "list of whitespace chars")
(defvar code-char-matches    '((#/[ #/])
                               (#/{ #/})
                               (#/( #/))
                               (#/" #/"))   "code-bracket pairs")
(defvar code-chars (append '(#/' #/0 #/1 #/2 #/3 #/4 #/5 #/6 #/7 #/8 #/9)
                           (mapcar #'car code-char-matches))
  "list of code-bracket chars")

;This is prepared to be called with either something in balanced
;() {} or "" which is read in literally as a string;
;or something in balanced [] in which case the stuff between them
;is read in as a string; or any other thing (eg a number or symbol)
;in which case the the lisp reader is used to read the thing.

;note that [] is now the way to indicate an arbitrary code string
;instead of "".  The unparser has been changed to reflect that.

;also, $ is now used to quote an arbitrary lisp thing rather than
;ordinary quote.  The unparser has been changed to reflect this too.

(defun read-code (c)
  (cond ((assq (character c) code-char-matches)
         (let ((string (read-matched-code (tyi))))
           (if (= c #/[)
               (setq string
                     (substring string 1 (1- (string-length string)))))
           '',string))
        (T (let ((thing (read-hacking-dots)))
             '',(let ((base 10.) (*nopoint nil))
                  (format nil "~S" thing))))))

(defun read-matched-code (c)
  (let* ((c1 (character c))
         (c2 (cadr (assq c1 code-char-matches))))
    (do ((chars (ncons c1) (cons c chars))
         (c (tyi) (tyi))
         (level 0))
        ((and (= level 0) (= c c2))
         (apply #'string-append (nreverse (cons c chars))))
      (cond ((= c c1) (incf level))
            ((= c c2) (decf level))))))

(defun comment-char?        (thing) (in-chars? thing comment-chars))
(defun sentence-terminator? (thing)
  (or (typep thing 'sys:error)
      (in-chars? thing sentence-terminators)))
(defun word-separator?      (thing) (in-chars? thing word-separators))
(defun whitespace-char?     (thing) (in-chars? thing whitespace-chars))
(defun code-char?           (thing) (in-chars? thing code-chars))

(defun punctuation? (thing)
  (or (word-separator? thing)
      (sentence-terminator? thing)))

(defun word-break? (thing)
  (or (comment-char? thing)
      (punctuation? thing)
      (whitespace-char? thing)))
```

```
;;;; Tokenization

;;; To get a fresh tokenization environment locally, use the
;;; special form:
;;;  (WITH-FRESH-TOKEN-BUFFER ...forms...)
;;;
;;; To peek at a token, use PEEK-TOKEN. To read a token, use
;;; READ-TOKEN. These each take an optional extra argument saying
;;; whether to force the word to be a dictionary token in spite of
;;; its case.

(defvar *token-buffer* nil "Buffer for token peeking.")

(defmacro with-fresh-token-buffer (&rest body)
  '(let ((*token-buffer* nil)) ,@body))

(defun peek-token (&optional (force-keyword? nil))
  (cond (*token-buffer*
         (car *token-buffer*))
        (t
         (push (read-token-internal force-keyword?) *token-buffer*)
         (car *token-buffer*))))

(defun unread-token (token)
  (push token *token-buffer*))

(defun really-read-token (force-keyword?)
  (cond (*token-buffer* (pop *token-buffer*))
        (t (read-token-internal force-keyword?))))

(defun read-token (&optional (force-keyword? nil))
  (let ((result (really-read-token force-keyword?)))
    (cond ((sentence-terminator? result)
           (parse-error "Premature termination of input."))
          (t result))))

(defvar code-predicted nil
  "T if the next thing to parse is predicted to be code")

(defvar entered-code-environment nil
  "T if we have gotten to place in the sentence where we had code.")

(defun read-token-internal (&optional (force-keyword nil))
  (condition-case (condition)
      (let ((c (tyipeek t)))
        (cond ((comment-char? c)
               (readline) ;for effect, discard rest of line
               (read-token-internal force-keyword))
              ((punctuation? c)
               (let ((package *kbe-package*))
                 (intern (tyi))))
              ;; here is where we interface new $ to old ' read in style
              ((= c #/$) (tyi)
               (list 'quote (list 'quote (read-hacking-dots))))
              ((code-char? c)
               (read-code c))
              ((or force-keyword (and (>= c #/a) (<= c #/z)))
               (let* ((package *kbe-package*)
                      (word (scan-word)))
                 (setq code-predicted
                       (or (memq word '(as of compute))
                           ;;triggering of and like this isn't quite right.
                           (and entered-code-environment (eq word 'and))))
                 (setq entered-code-environment
                       (or entered-code-environment code-predicted))
                 word))
              ((and (>= c #/A) (<= c #/Z))
               (if code-predicted (read-code c) (list 'quote (scan-word))))
```

```
            (t
             (list 'quote (read-hacking-dots)))))
    (sys:read-end-of-file condition)))

(defun read-hacking-dots ()
  (let ((c (tyipeek)))
    (cond ((= c #/')
           (tyi) (tyipeek t) ;eat char and whitespace
           (list 'quote (read-hacking-dots)))
          ((= c #/;)
           (readline)
           (read-hacking-dots))
          ((or (and (>= c #/A) (<= c #/Z))
               (and (>= c #/a) (<= c #/z)))
           (scan-word))
          ((or (and (>= c #/0) (<= c #/9)) (= c #/.) (= c #/-))
           (read-from-string
             (with-output-to-string (str)
               (do ((dot nil)) (nil)
                 (let ((c (tyi standard-input -1)))
                   (cond ((= c #/.)
                          (when dot
                            (send standard-input ':untyi c)
                            (return))
                          (setq dot t))
                         ((minusp c) (return))
                         ((word-break? c)
                          (send standard-input ':untyi c)
                          (return)))
                   (tyo c str))))))
          (t (read-hacking-/{)))))

(defun scan-word ()
  (symbolize-scanned-word (really-scan-word)))

(defun symbolize-scanned-word (token)
  (let ((the-symbol (read-from-string token)))
    (cond ((symbolp the-symbol) the-symbol)
          (t (parse-error "Invalid word syntax: ... ~A ..." the-symbol)))))

(defun really-scan-word ()
  (with-output-to-string (str)
    (do ((c (tyi standard-input -1) (tyi standard-input -1)))
        ((cond ((minusp c) t)
               ((word-break? c) (send standard-input ':untyi c) t)))
      (tyo c str)
      (if (= c #//) (tyo (tyi standard-input) str)))))

;;; (TOKEN-STRING token)
;;;
;;;  This function returns a string which represents a rough approximation
;;;  to the input format probably used for TOKEN. It is intended only for
;;;  use with error diagnostics, so is allowed to be heuristic.

(eval-when (eval compile load) ;needed, for example, in define-preposition

(defun token-string (token)
  (cond ((symbolp token)
         (format nil "/"~A/"" (string-downcase token)))
        (t
         (format nil "~S" (cadr token)))))

) ;nehw-lave
```

```
;;;; Driver Functions

(defun parse-command (string &rest parse-options)
  (with-input-from-string (str string)
    (lexpr-funcall #'read-command str parse-options)))

(defun read-command (&optional
                     (standard-input standard-input)
                     (parse-handler #'parse-sentence)
                     (parse-type "sentence"))
  (with-user-package
    (with-fresh-token-buffer
      (prog1 (funcall parse-handler)
             (when (not (sentence-terminator? (peek-token)))
               (parse-error
                 "Text follows logical end of ~A." parse-type))))))

;;; (PROMPT-AND-READ-COMMAND [stream])
;;;
;;;  This command is an entry point to the text reader which reads
;;;  whole sentences, interfacing appropriately to a rubout-handler
;;;  facility if one is available.

(defun prompt-and-read-command (&optional (stream standard-input)
                                &rest read-command-args)
  (cond ((send stream ':operation-handled-p ':rubout-handler)
         (read-command-prompt standard-input nil)
         (do () (nil) ;forever
           (when (eq (tyipeek) #\c-Z)
             (tyi)
             (null-sentence)      ;Allow graceful handling if possible
             (signal 'sys:abort)) ;Else simulate an Abort
           (condition-restart-case
             (return-if
               (lexpr-send-if-handles stream
                        ':rubout-handler '((:reprompt read-command-prompt)
                                           (:full-rubout t))
                        #'read-command stream read-command-args))
             ((sys:abort "Start over with typing this command.")
              (read-command-prompt standard-input nil)))))
        (t
         (lexpr-funcall #'read-command stream read-command-args))))

;;; Prompting function for the sentence reader.

(define-internal-option *command-prompt*
                        "Prompt for text input to KBE"
  ">" :sexp)

(defun read-command-prompt (stream char)
  char ;ignored
  (format stream "~&~A" *command-prompt*))
```

```
;;;; Sentence Parser

(define-condition null-sentence ())

(defun parse-sentence ()
  (peek-token t) ; Capitalization DWIM for first word
  (let ((token (peek-token)) code-predicted entered-code-environment)
    (cond ((typep token 'sys:error) nil)
          ((sentence-terminator? token)
           (null-sentence)
           (parse-error "Empty sentence parsed; ~
                         the delimiter ~A was encountered prematurely."
                        (token-string token)))
          (t
           (let ((fn (get token 'verb-semantics)))
             (cond ((not fn)
                    ;; Only if these names are not defined as verbs,
                    ;; we allow some interfaces to use these as a
                    ;; simulated end of input.
                    (if (memq token '(abort exit quit done terminate))
                        (null-sentence))
                    (parse-error "Not a verb: ~A"
                                 (token-string token)))
                   (t
                    (read-token)
                    (funcall fn))))))))
```

```
;;;; Name Parser

(defvar *token-type* nil "Keyword saying what token type to peek for.")

(defun parse-name (type)
  (declare (values token type))
  (let ((*token-type* type))      ;for the sake of an interactive reader
    (let ((token (peek-token)))
      (when (punctuation? token)
        (parse-error "Found punctuation out of place: ~A"
                     (token-string token)))
      (selectq type
        ((:STRING)
         (cond ((and (not (atom token))
                     (stringp (cadr token)))
                (values (cadr (read-token)) ':expression))
               (t (parse-error "Found ~A where a string was expected."
                               (token-string token)))))
        ((:KEYWORD)
         (cond ((symbolp token)
                (values (intern (string (read-token)) "") ':keyword))
               ((symbolp (cadr token))
                (values (intern (string (cadr (read-token))) "")
                        ':keyword))
               (t
                (parse-error "Found ~A where a keyword was expected."
                             (token-string token)))))
        ((:TOKEN)
         (cond ((symbolp token)
                (values (read-token) ':token))
               (t
                (parse-error
                  "Found ~A where a pseudo-English word was expected."
                  (token-string token)))))
        ((:TOKEN-NAME)
         (cond ((or (symbolp token)
                    (not (symbolp (cadr token))))
                (parse-error
                  "Found ~A where a name was expected."
                  (token-string token)))
               (t
                (values (intern-KBE (string (cadr (read-token))))
                        ':token))))
        ((:IDENTIFIER)
         (cond ((symbolp token)
                ;; Use default package, not KBE package!
                (values (intern (string (read-token))) ':identifier))
               ((symbolp (cadr token))
                (values (cadr (read-token)) ':identifier))
               (t
                (parse-error "Found ~A where an identifier was expected."
                             (token-string token)))))
        ((:NAME)
         (cond ((symbolp token)
                (values (read-token) ':token))
               ((symbolp (cadr token))
                (values (cadr (read-token)) ':identifier))
               (t
                (parse-error "Found ~A where a name was expected."
                             (token-string token)))))
        ((:EXPRESSION)
         (cond ((symbolp token)
                (values (intern (string (read-token))) ':expression))
               (t
                (values (cadr (read-token)) ':expression))))
        ((:CODE)
         (cond ((or (atom token)
                    (and (not (stringp (cadr token)))
```

```
                    (or (atom (cadr token))
                        (neq (car (cadr token)) 'quote))))
             (parse-error "Found ~A where a quoted expression expected."
                          (token-string token)))
            (t
             (values (cadr (read-token)) ':expression))))
    ((:ANY)
     (cond ((symbolp token)
            (values (read-token) ':token))
           ((symbolp (cadr token))
            (values (cadr (read-token)) ':identifier))
           (t
            (values (cadr (read-token)) ':expression))))
    (otherwise
     (ferror "Bad argument to PARSE-NAME: ~S" type))))))
```

```
;;;; Miscellaneous Utilities

(defun require-one-of (words &optional context)
  (cond ((let ((w (peek-token)))
           (dolist (word words)
             (if (or (eq w word)
                     (memq w (get word 'synonyms)))
                 (return t))))
         (read-token))
        (t (parse-error "Missing ~A~@[ ~A~]."
                        (phrase-append "or" (mapcar #'token-string words))
                        context))))

(defmacro define-synonyms (&rest words)
  '(progn 'compile
          ,@(mapcan
              #'(lambda (a)
                  (mapcan #'(lambda (b)
                              (if (neq a b)
                                  (ncons '(push ',a
                                                (get ',b 'synonyms)))))
                          words))
              words)))

(defun parse-definition-spec (&optional (error? nil))
  (caseq (peek-token)
    ((program function package procedure)
     (let ((first-token (read-token)))
       '(name program-definition
              ,(if (not (atom (peek-token)))
                   (parse-name ':identifier))
              ,(string-downcase first-token))))
    ((cliche)
     (read-token)
     '(name cliche-definition
            ,(if (not (atom (peek-token)))
                 (parse-name ':token-name))))
    (otherwise
      (if error? (parse-error "Missing name of definition.")))))

(defun parse-variable-name ()
  '(name variable ,(parse-name ':identifier)))
```

```
;;;; Generalized References

(defun parse-reference ()
  (if (or (pronoun? (peek-token)) (eq (peek-token) 'the))
      (parse-definite-reference)
      (parse-indefinite-reference)))
```

```
;;;; Definite References

(defun parse-definite-reference ()
  (or (parse-seg-reference)
      (parse-port-reference-minus-the)
      (parse-error "Missing definite reference.")))

(defun parse-seg-reference ()
  (cond ((parse-pronoun-reference))
        (t
         (when (neq (peek-token) 'the)
           (parse-error "Missing /"the/" in definite reference."))
         (read-token)
         (parse-the-reference))))

(defun parse-toplevel-reference ()
  (let ((ref (parse-seg-reference)))
    (when (neq (car ref) 'root-reference)
      (parse-error "Not a toplevel reference: ~A" (unparse-command ref)))
    ref))

(defun parse-pronoun-reference ()
  (if (pronoun? (peek-token))
      (list (read-token))))

(defun parse-the-reference ()
  (or (parse-top-reference)
      (parse-root-or-relative-reference (parse-ordinal))))

(defun parse-top-reference ()
  (if (memq (peek-token) '(top))
      (list (read-token))))

(defun parse-root-or-relative-reference (ordinal)
  (let ((descriptors (do ((descriptor (parse-descriptor)
                                      (parse-descriptor))
                          (l '() (cons descriptor l)))
                         ((not descriptor) (reverse l)))))
    (cond ((not (memq (peek-token) '(program cliche segment
                                     function procedure package)))
           (parse-relative-reference ordinal descriptors))
          (ordinal
           (parse-error "Invalid syntax in definite reference."))
          (t
           (parse-root-reference descriptors)))))

(*define-completion 'program)
(*define-completion 'cliche)
(*define-completion 'segment)
(*define-completion 'function)
(*define-completion 'package)
(*define-completion 'procedure)

(defun parse-descriptor ()
  (let ((word (peek-token)))
    (cond ((punctuation? word) nil)
          ((not (atom word))
           (if (program-name? (cadr word))
               '(name program ,(cadr (read-token)))))
          ((cliche-name? word)
           '(name cliche ,(read-token)))
          ((role-name? word)
           '(name role ,(read-token)))
          ((descriptor? word)
           (read-token))
          (t
           (let ((singular (singular-form word)))
             (cond ((descriptor? singular)
```

```
                    (read-token)
                    (unread-token 's)
                    singular)
                   ((role-name? singular)
                    (read-token)
                    (unread-token 's)
                    '(name role ,singular))
                   ((cliche-name? singular)
                    (read-token)
                    (unread-token 's)
                    '(name cliche ,singular))))))))

(defun parse-plural ()
  (cond ((eq (peek-token) 's)
         (read-token)
         t)))

(defun add-descriptor (reference descriptor)
  (selectq (car reference)
    (root-reference '(root-reference ',(cons descriptor
                                             (cadr (cadr reference)))
                                     ,@(cddr reference)))
    (relative-reference '(relative-reference
                          ,(cadr reference)
                          ',(cons descriptor (cadr (caddr reference)))
                          ,@(cdddr reference)))
    (otherwise reference))) ;eg, pronouns

(defun parse-relative-reference (ordinal descriptors)
  (cond ((not descriptors)
         (if ordinal (unread-token
                       (intern (string-upcase ;yuck
                                 (ordinal-unparse ordinal))
                               *kbe-package*)))
         nil)
        (t
         (let* ((plural (parse-plural))
                (preposition (if (memq (peek-token) '(in of))
                                 (read-token)))
                (reference (cond (preposition
                                  (or (parse-seg-reference)
                                      (parse-error
                                        "Invalid relative reference.")))
                                 (t '(top)))))
           '(relative-reference ,ordinal
                                ',descriptors
                                ,plural
                                ',(or preposition 'in)
                                ,reference))))))

(defun parse-root-reference (descriptors)
  (let ((name
          (cond ((parse-definition-spec))
                ((eq (peek-token) 'segment)
                 '(name ,(read-token) ,(parse-name ':identifier)))
                (descriptors
                 (parse-error
                   "Root reference must name a definition or segment.")))))
    (if name '(root-reference ',descriptors ,name))))

(defun parse-port-reference-minus-the ()
  (let ((ordinal (parse-ordinal)))
    (let ((port-descriptors
            (do ((p '() (cons d p))
                 (d (parse-port-descriptor)
                    (parse-port-descriptor)))
                ((not d)
                 (if (not p)
                     (parse-error
```

```
                    "Missing descriptor in definite reference."))
              (nreverse p)))))
  (let ((plural (parse-plural)))
    (let ((ref (cond ((eq (peek-token) 'of)
                      (read-token)
                      (parse-seg-reference))
                     (t '(top)))))
      '(port-reference ,ordinal ',port-descriptors ,plural ,ref))))))
```

```
;;;; Indefinite References

(defun parse-indefinite-reference ()
  (let ((expected-type ':any))
    (when (memq (peek-token) '(a an))
      (read-token)
      (setq expected-type ':name))
    (multiple-value-bind (token type)
        (parse-name expected-type)
      (caseq type
        ((:TOKEN :IDENTIFIER)
         '(cliche-instance
            ,(caseq type
               ((:TOKEN) '(name cliche ,token))
               ((:IDENTIFIER) '(name program ,token))
               (T (ferror "PARSE-INDEFINITE-REFERENCE lost.")))
            ,@(when (eq (peek-token) 'of)
                (read-token) ;pop the OF
                (do ((l () (cons (parse-reference) l))
                     (at-least-once? nil t))
                    ((and at-least-once?
                          (not (memq (peek-token) '(/, and))))
                     (nreverse l))
                  (when (eq (peek-token) '/,) ;maybe pop ","
                    (read-token))
                  (when (eq (peek-token) 'and)  ;maybe pop AND
                    (read-token))))))
        ((:EXPRESSION)
         '(code ',token))                ; =>    (code (quote (quote exp)))
                                         ;    or (code (quote "..."))
        (t (ferror "PARSE-INDEFINITE-REFERENCE lost."))))))
```

```
;;;; Modifier phrases

(defun parse-optional-cliche-name ()
  (if (cliche-name? (peek-token))
      '(name cliche ,(read-token))))

(defun parse-position ()
  (cond ((eq (peek-token) 'in)
         (read-token)
         (parse-definite-reference))
        ((eq (peek-token) 'here)
         (read-token)
         '(top))
        (t nil)))

(defun parse-roles-or-parameters (&optional spec)
  (selectq (cadr spec)
    (program-definition
     (parse-roles-or-parameters-internal
       '(parameter) '(parameters)
       "in a program specification"))
    (cliche-definition
     (parse-roles-or-parameters-internal
       '(role) '(roles)
       "in a cliche specification"))
    (otherwise
     (parse-roles-or-parameters-internal
       '(parameter role)
       '(parameters roles)
       "somewhere"))))

(defun parse-roles-or-parameters-internal (singular plural context)
  (cond ((eq (peek-token) 'a)
         (read-token)
         (let ((key (require-one-of singular context)))
           (cons key (list (parse-variable-name)))))
        (t
         (let ((key (require-one-of plural context)))
           (do ((p nil))
               ((let ((next (peek-token)))
                  (and (atom next) (not (memq next '(/, and)))))
                (cons key (nreverse p)))
             (push (parse-variable-name) p)
             (when (eq (peek-token) '/,)
               (read-token))
             (when (eq (peek-token) 'and)
               (read-token)))))))
```

```
;;; Ordinal Adjectives

(defvar *ordinals* () "data about ordinal adjectives")

(defmacro define-ordinal (name numeric-value)
  (check-arg numeric-value fixnump "a fixnum")
  '(*define-ordinal ',name ,numeric-value))

(defun *define-ordinal (name numeric-value)
  (check-arg numeric-value fixnump "a fixnum")
  (let ((entry (assq name *ordinals*)))
    (cond (entry (setf (cdr entry) numeric-value))
          (t (push (cons name numeric-value) *ordinals*))))
  (*define-completion name)
  (add-dictionary-entry name 'ordinal nil
                        (format nil "Selects the ~A of several of things."
                                (string-downcase name)))
  name)

(defun parse-ordinal ()
  (let ((word (peek-token)))
    (let ((val (cdr (assq word *ordinals*))))
      (if val (read-token))
      val)))
```

```
;;;; Port Descriptors


(defvar *port-descriptors* '() "List of defined port descriptors")

(defun port-descriptor? (word) (memq word *port-descriptors*))

(defmacro define-port-descriptor (name &optional documentation)
  '(progn 'compile
          (if (not (memq ',name *port-descriptors*))
              (push ',name *port-descriptors*))
          (setup-plural ',name)
          (*define-completion ',name)
          (add-dictionary-entry
            ',name 'port-descriptor nil
            ,(or documentation
                 (format nil "Used to designate ~A in a port reference."
                         (a-or-an (string-downcase name)))))
          ',name))

(defun parse-port-descriptor ()
  (let ((word (peek-token)))
    (cond ((punctuation? word) nil)
          ((port-descriptor? word) (read-token))
          ((let ((singular (singular-form word)))
             (cond (singular
                    (read-token)
                    (unread-token 's)
                    singular))))
          ((symbolp word) nil)
          (t (parse-variable-name)))))
```

```
;;;; Verbs

(eval-when (eval compile load)

(defun construct-verb-bindings (name bvl form)
  (cond ((null bvl) form)
        (t
         (construct-verb-bindings
           name
           (cdr bvl)
           (cond ((eq (car bvl) '...) form)
                 ((symbolp (car bvl))
                  `(progn (require-one-of
                            '(,(car bvl))
                            ,(format nil "in a usage of the verb /"~A/""
                                     (string-downcase name)))
                          ,form))
                 (t
                  `(let ((,(caar bvl)
                          ,(if (cadddr (car bvl))
                               `(or ,(cadr   (car bvl))
                                    ',(cadddr (car bvl)))
                               (cadar bvl))))
                     ,form)))))))

) ;NEHW-LAVE

(defmacro define-verb-parse (name bvl documentation &body body)
  (check-arg documentation stringp
             "a documentation string for the KBE dictionary")
  (let* ((capitalized-name (string-capitalize (string-downcase name))))
    `(progn 'compile
            (defun (,name verb-semantics) ()
              ,(construct-verb-bindings name
                                        (reverse bvl)
                                        (cond ((cdr body) `(progn ,@body))
                                              (t (car body)))))
            ,@(mapcan #'(lambda (x)
                          (if (and (atom x) (neq x '...))
                              (ncons `(*define-completion ',x))))
                      bvl)
            (*define-completion ',name)
            (*define-completion ,capitalized-name) ;Allow stylized casing
            (add-dictionary-entry
              ',name 'verb
              ,(format nil "~A~{ ~A~}."
                       capitalized-name
                       (mapcar #'(lambda (x)
                                   (cond ((symbolp x) (string-downcase x))
                                         (t (format nil "<~A>"
                                                    (string-downcase
                                                      (car x))))))
                               bvl))
              ,documentation))))
```

```
(defdefmacro define-verb-unparse "KBE verb unparser" (name bvl . body)
  (let ((var (gensym)))
    '(local-declare ()
       (defun (,name unparse) (,(if bvl var 'ignore))
         (let ,(if bvl '(((nil ,@bvl) ,var)) '())
           ,@body))
       ',name)))

(defdefmacro define-verb-action "KBE verb action" (name bvl . body)
  (let ((var (gensym)))
    '(local-declare ()
       (defun (,name verb-action) (&rest ,(if bvl var 'ignore))
         (let ,(if bvl '((,bvl ,var)) '())
           ,@body))
       ',name)))

(defflavor vocabulary () ()
  (:method-combination (:case :base-flavor-last :interpret)))

(defmethod (vocabulary :otherwise :interpret) (verb &rest verb-args)
  (ferror "~S is not a verb known to ~A.~% Context: ~A"
          verb self (unparse-command (cons verb verb-args))))

(defmacro define-vocabulary (name &optional bvl &body forms)
  '(local-declare ()
     (defflavor ,name () (vocabulary))
     ,(if forms
          '(defmethod (,name :otherwise :interpret) ,bvl
             ,@forms))
     ',name))

(comment ;too bad this doesn't work.

(defdefmacro define-additional-interpretation "KBE verb interpretation"
             (name bvl . forms)
  (let ((flavor (if (atom name) 'default-vocabulary (car name)))
        (method (if (atom name) name (cadr name)))
        (var (gensym)))
    (multiple-value-bind (arglist spread-p)
        (parse-argument-list bvl)
      '(defwhopper (,flavor :interpret) (,var ,@bvl)
         (cond ((eq ,var ',method)
                (progn ,@forms)))
         (,(if spread-p 'lexpr-continue-whopper 'continue-whopper)
          ,var ,@arglist)))))

; something like this would almost work but needs more support

(defmacro define-additional-interpretation (name bvl &body forms)
  (let ((flavor (if (atom name) 'default-vocabulary (car name)))
        (method (if (atom name) name (cadr name))))
    (let ((proxy (symbolconc flavor '-interpretation-proxy)))
      (multiple-value-bind (arglist spread-p)
          (parse-argument-list bvl)
        '(defwhopper (,proxy ,method) (,var ,@bvl)
           (progn ,@forms)
           (,(if spread-p 'lexpr-continue-whopper 'continue-whopper)
            ,@arglist))))))

);tnemmoc

(defdefmacro define-interpretation "KBE verb interpretation"
             (name bvl . forms)
  (let ((flavor (if (atom name) 'default-vocabulary (car name)))
        (method (if (atom name) name (cadr name))))
    '(defmethod (,flavor :case :interpret ,method) ,bvl ,@forms)))
```

```
(define-vocabulary default-vocabulary)

(defprop define-verb "KBE verb" si:definition-type-name)

(defmacro define-verb (name bvl doc)
  (let* ((question? (let ((qmark (mem #'(lambda (x y)
                                          (and (atom y)
                                               (string-equal x y)))
                                      "?" bvl)))
                      (cond ((not qmark) nil)
                            (t
                             (delq (car qmark) bvl)
                             t))))
         (bvl-vars (mapcan #'(lambda (x)
                               (if (not (atom x)) (ncons (car x))))
                           bvl)))
    (make-defdefmacro-source-file-wrapper
      'define-verb
      name
      '(local-declare ()
         (define-verb-parse ,name ,bvl ,doc
           (list ',name ,@bvl-vars))
         (define-verb-unparse ,name ,bvl-vars
           (append-delimiter
             (text ,(text "~A~{~A~}"
                          (string-capitalize (string-downcase name))
                          (mapcar #'(lambda (x)
                                      (if (atom x)
                                          (text " ~A" (string-downcase x))
                                          (text "~~@[~@[ ~A~] ~~A~~]"
                                                (caddr x))))
                                  bvl))
                   ,@(mapcar #'(lambda (x data)
                                 (cond ((cadddr data)
                                        '(if (not (equal ,x ',(cadddr data)))
                                             (unparse-command ,x)))
                                       (t '(unparse-command ,x))))
                             bvl-vars
                             (mapcan #'(lambda (x) (if (not (atom x))
                                                       (ncons x)))
                                     bvl))))
           ,(if question? '#/? '#/.)))
         (define-verb-action ,name ,bvl-vars
           (send-delegate ':interpret ',name ,@bvl-vars))
         ',name))))

(defprop define-action "KBE action" si:definition-type-name)

(defmacro define-action (name bvl doc &body forms)
  (make-defdefmacro-source-file-wrapper
    'define-action
    name
    '(local-declare ()
       (define-verb ,(cond ((atom name) name)
                           (t (cadr name)))
                    ,bvl ,doc)
       ,(if forms
            '(define-interpretation ,name
                                    ,(mapcan #'(lambda (x)
                                                 (if (not (atom x))
                                                     (ncons (car x))))
                                             bvl)
               ,@forms))
       ',name)))
```

```
;;;; Other Word Types

(defmacro define-part-of-speech (name pos default-doc &rest context-info)
  '(progn 'compile
          (*define-completion ',name)
          (putprop ',name t ',pos)
          (add-dictionary-entry
            ',name ',pos nil
            ,(format nil "~A~%~@[(See also ~A)~]"
               (cond ((stringp (car context-info))
                      (pop context-info))     ;exploits evaluation order!
                     (t
                      default-doc))
               (if context-info
                   (phrase-append "and" (mapcar #'token-string
                                                context-info)))))
          ',name))

(defmacro define-preposition (name &rest context-info)
  '(define-part-of-speech ,name preposition
     "Used as a relational or guide word."
     ,@context-info))

(defmacro define-pronoun (name &rest context-info)
  '(define-part-of-speech ,name pronoun
     "Used as a context-dependent definite reference."
     ,@context-info))

(defun pronoun? (word) (get word 'pronoun))

(defmacro define-article (name doc)
  '(define-part-of-speech ,name article ,doc))

(defmacro define-descriptor (name doc)
  '(progn 'compile
          (define-part-of-speech ,name descriptor ,doc)
          (setup-plural ',name)
          ',name))

(defun descriptor? (word) (get word 'descriptor))

(defmacro define-adverb (name doc)
  '(define-part-of-speech ,name adverb ,doc))
```

```
;;;; Testing Utilities

(defun test-parser ()
  (catch-error-restart ((sys:abort sys:error null-sentence)
                        "Exit parser test routine.")
    (let ((*command-prompt* "^X"))
      (do ((c (prompt-and-read-command)
              (prompt-and-read-command)))
          ((not c))
        (let ((u (ignore-errors (unparse-command c))))
          (let ((package *kbe-package*))
            (let (prinlevel prinlength)
              (format t "~& ^Y ~S~@[~% ^^ ~A~]" c u)))))))
  (format t "~&Leaving the KBE test-parser loop.~%")
  (values))
```

```
;;; -*- Mode: LISP; Package: KBE; Base: 10; Fonts: MEDFNB -*-

;;; (c) Copyright 1983, Massachusetts Institute of Technology
;;;     All Rights Reserved.
;;;     Permission to copy all or part of this material is granted,
;;;     provided that the copies are not made or distributed for resale,
;;;     the MIT copyright notice and reference to the source file and
;;;     the software distribution version appear, and that notice is
;;;     given that copying is by permission of Massachusetts Institute
;;;     of Technology.

;;; DICT: The main core of the dictionary.
;;;
;;;       This is not the complete dictionary. Some additional
;;;       special-purpose terms may be defined in other files.
```

```
;;;; Adjective Definitions

;;; Ordinals

(define-ordinal first    1)
(define-ordinal second   2)
(define-ordinal third    3)
(define-ordinal fourth   4)
(define-ordinal fifth    5)
(define-ordinal sixth    6)
(define-ordinal seventh  7)
(define-ordinal last    -1)
```

```
;;;; Port Descriptors

(define-port-descriptor parameter)
(define-port-descriptor argument)
(define-port-descriptor free-variable)
(define-port-descriptor value)
(define-port-descriptor side-effect)
(define-port-descriptor input)
(define-port-descriptor output)
(define-port-descriptor temporal)
(define-port-descriptor non-temporal)
(define-port-descriptor port)
```

```
;;;; Verb Definitions

(define-action typeout ((text (parse-name ':string)))
               "Requests that the given <text> be typed out."
  (format t "~&~:A~&" text))

(define-action remark ((text (parse-name ':string)))
               "Requests that the given <text> be remarked about."
  (format t "~&* ~A~&" text))

(define-action clear (display)
               "Requests that the display area be cleared."
  (values)) ;Ignored in most contexts

(define-action how (is (cliche-or-program (parse-toplevel-reference))
                       defined ?)
               "Asks what how <cliche-or-program> is defined."
  cliche-or-program ;ignored
  nil) ;Default is to not know.

(define-action Enable ((switch (parse-name ':keyword)) mode)
               "Enables the KBE feature named <switch>."
  (multiple-value-bind (success? comment-string)
      (set-switch switch t)
    (cond ((not success?)
           (complain "~A" comment-string))
          (t '(:text :text ,comment-string)))))

(define-action Disable ((switch (parse-name ':keyword)) mode)
               "Disables the KBE feature named <switch>."
  (multiple-value-bind (success? comment-string)
      (set-switch switch nil)
    (cond ((not success?)
           (complain "~A" comment-string))
          (t '(:text :text ,comment-string)))))

(*define-completion 'patches)

(define-action Load ((thing-to-load (parse-name ':any)))
               "Loads the <thing-to-load>. Normally, this is a filename.
                If it is the word /"patches/" then KBE patches will
                be loaded instead."
  (cond ((stringp thing-to-load)
         (list ':text
               ':text (text "The file ~A was loaded."
                            (load thing-to-load))))
        ((eq thing-to-load 'patches)
         (multiple-value-bind (old-major old-minor)
             (si:get-system-version "KBE")
           old-major ;ignore
           (load-patches 'kbe ':silent)
           (multiple-value-bind (new-major new-minor)
               (si:get-system-version "KBE")
             new-major ;ignore
             (list ':text
                   ':text (cond ((= old-minor new-minor)
                                 "No patches were loaded.")
                                (t
                                 (text "Patches loaded for ~A"
                                       (system-version-string
                                         "KBE")))))))))
        (t
         (complain "I don't know how to load ~A."
                   (string-downcase thing-to-load)))))

(define-action Compile ((thing-to-compile (parse-name ':any)))
               "Compiles the <thing-to-compile>, which should be a
                filename."
```

```
(cond ((stringp thing-to-compile)
       (list ':text
             ':text (text "The file ~A was compiled.~
                                   The file ~A contains the output."
                          thing-to-compile
                          (compiler:compile-file thing-to-compile))))
      (t
       (complain "I don't know how to compile ~A."
                 (string-downcase thing-to-compile)))))
```

```
;;;; Misc other words

;;; Prepositions, guide words

(define-preposition in)
(define-preposition of)


;;; Pronouns

(define-pronoun it)
(define-pronoun this)
(define-pronoun that)

;;; Articles

(define-article a  "Designates an indefinite reference.")
(define-article an "Synonym for /"a/".")
(define-article a<n> "Synonym for /"a/" and /"an/".")

(define-synonyms a an a<n>)

(define-article the "Designates a definite reference.")

;;; Definite reference descriptors

(define-descriptor implemented
                   "Declares a definite reference to already exist.")
(define-descriptor unimplemented
                   "Declares a definite reference to not already exist.")
(define-descriptor role
                   "Denotes the named roles of a program fragment.")

;;; Adverbs

(define-adverb here "In /"What needs to be done here/" means like
                     /"in the top/".")
```

```
;;; -*- Mode:LISP; Package:KBE; Base:10; Fonts: MEDFNB; -*-

;;; (c) Copyright 1983, Massachusetts Institute of Technology
;;;     All Rights Reserved.
;;;     Permission to copy all or part of this material is granted,
;;;     provided that the copies are not made or distributed for resale,
;;;     the MIT copyright notice and reference to the source file and
;;;     the software distribution version appear, and that notice is
;;;     given that copying is by permission of Massachusetts Institute
;;;     of Technology.

;This describes the grammer of the command language used with the KBE.
;It shows both what the surface form is, and what it parses as.
;for example consider the following rule:

;<if-then-else> =: if <exp> then <statement1>; {else <statement2>;}
;               => (cond (<exp> <statement1>) {(T <statement2>)})

;This rule could be used to specify what an if-then-else statement looks
;like and how it is parsed into an internal representation (Lisp).  The
;language used to specify the grammar is BNF like.  It uses the following
;constructs

;=:  specifies what a non-terminal expands into (ie a grammer rule)
;=>  specifies the parse result corresponding to a grammar rule
;<name>  this is a non-terminal grammer symbol. When a non-terminal appears
;        in a parse-result it means that the corresponding parse should be
;        included at this point.  Numerical suffixes can be used on
;        non-terminal names in order to make the correspondance between
;        rule and parse clear.
;foo  anything else corresponds to terminal symbols in the rule or result
;!  OR means that one of two things can occur.
;{}  means that something is optional (it may occur 0 or 1 times)
;{}1  means that this must occur exactly once (used if there is a set of
;     alternatives inside).
;{}*  means that this can occur 0 or more times.
;{}+  means that this must occur 1 or more times.
```

```
;the top level node in the grammer specifies the commands which can be
;used to request actions from the system.  It is intended that this
;list will expand.  Also note that the verbs make heavy use of noise
;words for euphony.

;<command> =: <display> ! <move-to> ! <clear-display> ! <describe>
;             <what-needs-to-be-done> ! <use-language> ! <implement> !
;             <reimplement> ! <make-definition> ! <reinitialize>
;             <add-parameter> ! <add-role> ! <remove> ! <share> !
;             <analyze> ! <eval> ! <compute> ! <code-up> ! <document> !
;             <explain> ! <locate>

;<end> =: "." ! "!" ! "?"
;      => ;ignored just signals end of sentance.

;<display> =: Display <definite-reference> <end>
;          => (DISPLAY <definite-reference>)

;<move-to> =: Move to <definite-reference> <end>
;          => (MOVE-TO <definite-reference>)

;<document> =: Document <definite-reference> <end>
;           => (DOCUMENT <definite-reference>)

;<explain> =: Explain <definite-reference> <end>
;          => (EXPLAIN <definite-reference>)

;<clear-display> =: Clear display <end>
;                => (CLEAR-DISPLAY)
;(This is not meaningful in the ZWEI interface mode.)
;
;<reinitialize> =: Reinitialize <end>
;               => (REINITIALIZE)
;
;<describe> =: Describe <definite-reference> <end>
;           => (describe <definite-reference>)

;<what-needs-to-be-done> =: What needs to be
;                                done {in <seg-reference> ! here ! } <end>
;                        => (WHAT-NEEDS-TO-BE-DONE
;                               {<seg-reference> ! (TOP) ! })
;
; If HERE is used, <seg-reference> becomes (TOP).
; If HERE and IN <seg-reference> are omitted, so is arg to result.

;<use-language> =: Use language <keyword> <end>
;               => (USE-LANGUAGE (QUOTE <keyword>))
;Note that the language name is passed in quoted.

;<keyword> =: <string> ! <STRING>
;          => a symbol on the keyword package with the given print name.

;<implement> =: Implement <seg-reference> as <reference> <end>
;            => (IMPLEMENT <seg-reference> <reference>)
;Demands that <seg-reference> not be implemented yet.  Forces the adjective
;UNIMPLEMENTED to appear in the reference.

;<reimplement> =: Reimplement <seg-reference> as <reference> <end>
;              => (REIMPLEMENT <seg-reference> <reference>)
;Demands that <seg-reference> already be implemented.  Forces the adjective
;IMPLEMENTED to appear in the reference.
```

```
;<make-definition> =: Define a
;                     {<program-definition-name>
;                      {{with a parameter <variable-name>} !
;                       {with parameters
;                             {{and} <variable-name> {,}}+}}} !
;                     {<cliche-definition-name>
;                      {{with a role <variable-name>} !
;                       {with roles
;                             {{and} <variable-name> {,}}+}}}1
;                     <end>
;                  => (MAKE-DEFINITION {<cliche-name> ! NIL}
;                                      {<program-definition-name> !
;                                       <cliche-definition-name>}1
;                                      . {<variable-name>}*)
;Note that here the program/cliche name cannot already be defined.

;<add-parameter> =: Add a parameter <variable-name>
;                   {to <seg-reference>} <end>
;                => (ADD-PARAMETER <variable-name> <seg-reference>)
;Adding a parameter is the only kind of adding which works now.
;Others may follow.  The <seg-reference> must refer to a program
;definition.  If it is omitted it defaults to "to the top".

;<add-role> =: Add a role <variable-name>
;              {to <seg-reference>} <end>
;           => (ADD-ROLE <variable-name> <seg-reference>)
;Adding a role to a define-cliche is the only other kind of adding
;which works now.  The <seg-reference> must refer to a define-cliche.
;If it is omitted it defaults to "to the top".

;<remove> =: Remove <definite-reference> <end>
;         => (REMOVE <definite-reference>)
;this isn't currently used and probably doesn't work well.

;not implemented at the current time.
;<share> =: Share <definite-reference>
;                 {{,} {and} <definite-reference>}+ <end>
;        => (SHARE . {<definite-reference>}+)

;<analyze> =: Analyze <code> <end>
;          => <code>
;analyzes the code supplied.

;<code-up> =: Code <reference> <end>
;          => (CODE-UP <reference>)

;<compute> =: Compute <reference> <end>
;           => (COMPUTE <reference>)
;shows you what a reference command-evals to.

;<locate> =: Locate
;             {<program-definition-name> ! <cliche-definition-name>}1
;             {{,} <path>} <end>
;          => (LOCATE <program-definition-name> {<path>})
;says where the virtual cursor should be.

;<path> =: ( {<number>}* )
;       => ( {<number>}* )
;a path into an expression, 0-indexed.
;eg,  () is toplevel.
;     (1 3 4) is the 4th thing in the 3rd thing in the 1st thing.

;<eval> =: Eval '<lisp-expr> <end>
;       => (EVAL (QUOTE <lisp-expr>))
;LISP EVALS the thing so that you can see the value.
```

```
;The heart of the language is reference expressions.

;<reference> =: <definite-reference> ! <indefinite-reference>
;            => <definite-reference> ! <indefinite-reference>

;First we have indefinite noun phrases.  These create new pieces of
;plan rather than referring to existing pieces of plan.  Note that
;there is a very great reduction in what you can say here compared to
;the past.  The only thing you can create is a segment.

;<indefinite-reference> =: <code> ! <cliche-instance>

;we work very hard to make it easy for the user to type code.
;basically he can just type anything he wants that is code,
;and we try to figure it out.  There are also two escapes.
;first you can type $ (dollar sign) followed by any lisp
;sexpr.  This forces the code to be lisp code no matter what the
;current language is.  Second, you can type [...] in which case everything
;between the [] is interpreted as a string which should be parsed
;in the current language.

;<code> =: $ <lisp-expr> ! [<current-language-expr>] !
;          <naked-current-language-expr>
;       => (CODE (QUOTE {(QUOTE <lisp-expr>) !
;                        "<current-language-expr>" !
;                        "<naked-current-language-expr>"}1))

;note that in the above we make several key assumptions in order to
;be able to handle the <naked-current-language-expr> and to
;differentiate it from other things.
;first it must be a single piece. ie
; a number.
; a string.
; a symbol.
; a sequence of characters surrounded by matching () or {}.
; a lisp expression that can be read with a single call on read.
;note that if it is a symbol there could be ambiguities as to whether
;it was a keyword or a program name or something.  To prevent this,
;it must be in upper case, and the preceding keyword must be AS, OF,
;or COMPUTE.

;<cliche-instance> =: {a ! an} {<cliche-name> ! <program-name>}1
;                              {of {{and} <reference> {,}}+}
;                  => (CLICHE-INSTANCE {<cliche-name> ! <program-name>}1
;                                      . {<reference>+)
;Note that the A/AN is optional.  Also, you can use commas and "and" in
;the list of role fillers if you want.
```

```
;------

;The most complex form in the command language is the definite
;reference expression.  They make it possible to identify any segment
;or port in a plan.  Note however, that the segments are the key
;thing.  Port references are only used as output by commands that
;describe things.  Also good reference expressions are only available
;for segments that play some role in a program.

;<definite-reference> =: <seg-reference> ! <port-reference>
;                     => <seg-reference> ! <port-reference>

;<seg-reference> =: <pronoun-reference> ! <the-reference>
;                => <pronoun-reference> ! <the-reference>

;<pronoun-reference> =: <it> ! <this> ! <that>
;                    => <it> ! <this> ! <that>

;<it> =: it
;     => (it)
;"it" refers to the segment which is both the "conversational"
;and "presentational" reference. When "this" and "that" designate
;the same reference, so does "it". At any other time, "it" is ambiguous.

;<that> =: that
;       => (that)
;"that" refers to the segment most recently referred to in conversation.
;The apprentice notifies the interface when the meaning of this
; reference changes.

;<this> =: this
;       => (this)
;"this" refers to the segment most recently referred to in presentation.
;The interface notifies the apprentice when the meaning of this
; reference changes.
```

```
;<the-reference> =: <top> ! <root-reference> ! <relative-reference>
;                => <top> ! <root-reference> ! <relative-reference>

;<top> =: the top
;      => (TOP)
;This refers to the current program or cliche definition.
;Its value is computed from the active meaning of "this".

;<root-reference> =: the {<descriptor>}* {<segment-name> !
;                                         <program-definition-name> !
;                                         <cliche-definition-name>}1
;
;                => (ROOT-REFERENCE (QUOTE ({<descriptor>}*))
;                                   {<segment-name> !
;                                    <program-definition-name> !
;                                    <cliche-definition-name>}1)

;<relative-reference> =: the {<ordinal>} {<descriptor>}+ {<plural>}
;                           {of <seg-reference> ! in <seg-reference>}
;                    => (RELATIVE-REFERENCE
;                         <ordinal> (QUOTE ({<descriptor>}+)) <plural>
;                         (QUOTE {OF ! IN}1) <seg-reference>)
;plurality is usually specified by making the descriptor plural.  If
;either <ordinal> or <plural> is left out in the expression, then it
;is specified as NIL in the parse.  If neither an "of" or "in" phrase
;is specified, it defaults to "in the top".
;Note that the verbs <implement> and
;<reimplement> force the adjectives unimplemented and implemented
;respectively to appear in their reference pharses.

;<port-reference> =: the {<ordinal>} {<port-descriptor>}+ {<plural>}
;                    {of <seg-reference>}
;                 => (PORT-REFERENCE
;                      <ordinal> (QUOTE ({<port-descriptor>}+)) <plural>
;                      <seg-reference>)
;plurality is usually specified by making the last port-descriptor
;plural.  If either <ordinal> or <plural> is left out in the
;expression, then it is specified as NIL in the parse.  If an "of"
;phrase is not specified, it defaults to "of the top".  Note that you
;cannot tell whether you have a relative-ref or a rel-port-ref until
;you see whether you get a descriptor or a port-descriptor.
```

```
;In the above,
;<descriptor> =: <program-name> ! <cliche-name> ! <role-name> !
;               implemented ! unimplemented ! role ! mark
;            => <program-name> ! <cliche-name> ! <role-name> !
;               IMPLEMENTED ! UNIMPLEMENTED
;A segment can be described either in terms of what program it calls,
;what cliche it is an instance of, what role it plays in a plan,
;or whether it has been implemented yet.  Note
;that you cannot have a program, cliche, or role called implemented,
;or unimplemented.  Note that MARK is implemented as a special role
;name, and is treated as such by the parser and command evaluater.

;<port-descriptor> =: <variable-name> ! parameter ! argument !
;                     free-variable ! value ! side-effect ! input !
;                     output ! temporal ! non-temporal ! port
;                  => <variable-name> !  PARAMETER ! ARGUMENT !
;                     FREE-VARIABLE ! VALUE ! SIDE-EFFECT ! INPUT !
;                     OUTPUT ! TEMPORAL ! NON-TEMPORAL ! PORT
;Note that a variable name can be used as an adjective describing a
;free-variable port.  This is the only place a variable-name can appear.

;<plural> =: s
;         => T
;This can be expressed as a separate adjective (an isolated "s") to
;indicate that the prior word is plural.  However, it is usually
;specified by making that word itself be plural.  Note however, that
;this can only be done if that word is lower case.  This is handled
;special in the parsing of <definite-ref> as will be discussed below.

;<ordinal> =: first ! second ! third ! fourth ! fifth ! sixth ! last
;          => 1     ! 2      ! 3     ! 4      ! 5     ! 6     ! -1
;This is used to identify segments based on their position in the
;code, and to identify the arguments to a funcall by their order.
```

```
;One basic underlying concept is that of a name.  There are many
;different kinds of names that the user can type.

;<number> =: an integer in base 10 (with optional decimal point)
;         => a lisp integer.

;<STRING> =: an uppercase string.  (No completion applied.)
;         => a symbol (corresponding to the string) interned in the package
;            in effect when the user was doing his type in.  You can
;            just read it.
;This refers to an identifier: either a program name or a variable name.

;<string> =: a lowercase string of characters.  (Note that in general lower
;            case strings are subject to completion by the input reader.)
;         => the corresponding symbol for a cliche, role name, or segment
;            name.  Note that these are all symbols with upper case
;            pnames in the KBE package

;---

;<cliche-name> =: <string>
;              => (NAME CLICHE <string>)
;The name of a cliche.  Completion is done based on the list *CLICHES*.

;<role-name> =: <string>
;            => (NAME ROLE <string>)
;The name of a role in a cliche.  Completion is done based on the list
;*roles*.  In order to avoid confusion, it is required that a role
;never have the same name as a cliche.

;<program-name> =: <STRING>
;               => (NAME PROGRAM <STRING>)
;The name of a program.

;<variable-name> =: <STRING>
;                => (NAME VARIABLE <STRING>)
;Any name.  You have to disambiguate between program names and var
;names from the gramatical context.

;<program-definition-name> =: program {<STRING>}
;                          => (NAME PROGRAM-DEFINITION {<STRING> ! NIL})
;This is used to specify a program definition being edited.
;The string must correspond to some program in a buffer that the KBE
;knows about.  A missing <STRING> (ie NIL) means the current top program

;<cliche-definition-name> =: cliche {<STRING>}
;                         => (NAME CLICHE-DEFINITION {<STRING> ! NIL})
;This is used to specify a cliche definition being edited.  The string must
;correspond to some cliche in a buffer that the KBE knows about.
;A missing <string> (ie NIL) means the current top cliche.

;<segment-name> =: segment <string>
;               => (NAME SEGMENT <string>)
;This kind of name only exists for robustness so that it is possible to
;create a definite reference to any arbitrary segment.  The string must
;be one of the segment names on the list *DB-SEGS*.

;Note that NAME is a fexpr.  All of the other functions in the
;parse results are ordinary exprs.
```

```
; This takes a command and converts it into a string which would
;have parsed into it.  Several of the commands result in the printing of
;such strings.

(defun sexp->string (sexp)
  (let ((base 10.)
        (*nopoint nil))
    (with-user-package (let (prinlevel prinlength) (text "~S" sexp)))))

(defun unparse-command (command &aux fn)
  (cond ((null command) nil)
        ((symbolp command) (string-downcase command))
        ((stringp command) (text "[~A]" command))
        ((atom command)
         (barf "UNPARSE-COMMAND: Invalid KBE command?~% Data: ~S"
               command))
        ((setq fn (get (car command) 'unparse))
         ;; Try to standardize the numeric format a bit.
         ;; Too bad packages are not so easy... -kmp
         (let ((base 10.) (*nopoint nil))
           (funcall fn command)))
        ;; Put these after the previous clause to make sure they can
        ;; be overridden.
        ((eq (car command) 'quote)
         (string-append #/$ (sexp->string (cadr command))))
        (t
         (barf "UNPARSE-COMMAND: Invalid KBE command?~% Data: ~S"
               command))))

(defvar *alphanumeric* (string-append *alphabet* "0123456789"))
(defvar *quote-chars* "'")
(defvar *toplevel-chars* (string-append *alphanumeric*
                                        "_-."
                                        *quote-chars*))

(defun append-delimiter-if (text) (if text (append-delimiter text)))

(defun append-delimiter (string &optional (char #/.))
  (text "~A~:[ ~;~]~C"
        string
        (OK-to-append-delimiter-directly? string)
        char))

(defun OK-to-append-delimiter-directly? (string)
  (or (zerop (string-length string)) ;pathological
      (let ((last-char (aref string (1- (string-length string)))))
        (or (= last-char #/)
            ; This next is heuristic, but loses only on symbols
            ; containing only / and | and ending in / such as ///|
            (= last-char #/|)
            (= last-char #/")
            (let* ((p1 (or (string-reverse-search-not-set
                             *toplevel-chars* string)
                           -1))
                   (p2 (string-search-not-set *quote-chars* string
                                              (1+ p1))))
              (and (or (minusp p1)
                       (whitespace-char? (aref string p1)))
                   p2
                   (string-search-char (aref string p2)
                                       *alphabet*)))))))

;Each little unparse fn returns its result without blanks around it.
```

```
;;;; Non-Verbs

(defun (it unparse) (ignore)
  "it")

(defun (this unparse) (ignore)
  "this")

(defun (that unparse) (ignore)
  "that")

(defun (here unparse) (ignore)
  "here")

(defun (top unparse) (ignore)
  "the top")
```

```
(defun (root-reference unparse) (command)
  (let* (((nil (nil dscs) name) command)
         (dscs-string (dsc-unparse dscs nil)))
    (text "the~@[ ~A~] ~A"
          dscs-string
          (unparse-command name))))

(defun (relative-reference unparse) (command)
  (reference-unparse command))

(defun (port-reference unparse) (command)
  (let* (((a b c d e) command))
    (reference-unparse (list a b c d ''of e))))

(defun reference-unparse (command)
  (let* (((nil ordinal (nil dscs) plural (nil of-in) reference) command)
         (ordinal-string (ordinal-unparse ordinal))
         (dscs-string (dsc-unparse dscs plural)))
    (let ((relation (text "~A ~A"
                          (string-downcase of-in)
                          (unparse-command reference))))
      (if (equal relation "in the top") (setq relation nil))
      (text "the~@[ ~A~] ~A~@[ ~A~]"
            ordinal-string dscs-string
            relation))))


(defun ordinal-unparse (ordinal)
  (if ordinal
      (caseq ordinal
        (-1 "last") (1 "first") (2 "second") (3 "third")
        (4 "fourth") (5 "fifth") (6 "sixth") (7 "seventh"))))

(defun dsc-unparse (dscs plural)
  (if dscs
      (let* ((strings (mapcar #'dsc-unparse1 dscs)))
        (if plural
            (let ((last (last strings)))
              (if (not (< #.(1- #/a) (aref (car last) 0) #.(1+ #/z)))
                  (rplacd last (ncons "s"))
                  (rplaca last (string-pluralize (car last))))))
        (text "~{~A~^ ~}" strings))))

(defun dsc-unparse1 (dsc)
  (if (memq dsc '(parameter argument free-variable value side-effect input
                  output temporal non-temporal port
                  implemented unimplemented role))
      (string-downcase dsc)
      (unparse-command dsc)))
```

```
(defun (code unparse) (command)
  (let* (((nil (nil code)) command))
    (if (stringp code)
        (text "[~A]" code)
        (let (prinlevel prinlength) (text "$~S" (cadr code))))))

(defun (cliche-instance unparse) (command)
  (let* (((nil name . refs) command)
         (name-string (unparse-command name))
         (ref-strings (mapcar #'unparse-command refs))
         (refs-string (caseq (length ref-strings)
                        (0 nil)
                        (1 (car ref-strings))
                        (2 (text "~{~A and ~A~}" ref-strings))
                        (T (text "~{~#[~1;and ~]~A~^, ~}" ref-strings)))))
    (text "~A ~A~@[ of ~A~]"
          (select-a-or-an name-string) name-string refs-string)))

(defun (name unparse) (command)
  (let* (((nil type name type-name) command))
    (caseq type
      ((cliche role) (string-downcase name))
      ((program variable) (with-user-package (text "~S" name)))
      (program-definition
        (let ((descriptor (or type-name "program")))
          (cond (name (with-user-package (text "~A ~S"
                                               descriptor name)))
                (t descriptor))))
      (cliche-definition
        (cond (name (with-kbe-package (text "cliche ~S" name)))
              (t "cliche")))
      (segment (string-append "segment " (string-downcase name)))
      (T (barf "bad args to NAME ~S" command)))))
```

```
;;; -*- Mode:LISP; Package:KBE; Base:10; Fonts: MEDFNB; -*-

;;; (c) Copyright 1983, Massachusetts Institute of Technology
;;;     All Rights Reserved.
;;;     Permission to copy all or part of this material is granted,
;;;     provided that the copies are not made or distributed for resale,
;;;     the MIT copyright notice and reference to the source file and
;;;     the software distribution version appear, and that notice is
;;;     given that copying is by permission of Massachusetts Institute
;;;     of Technology.
;;;
;;; REFER: Support for definite noun-phrase hacking in the KBE.

;;; (WITH-CONSISTENT-DB forms)
;;;
;;;  If forms is exited abnormally,

(defmacro with-consistent-db (&body forms)
  (let ((startcontext  (gensym))
        (old-*this*    (gensym))
        (old-*that*    (gensym)))
    '(let ((,startcontext (cdr *db-context*))
           (,old-*that* *that*)
           (,old-*this* *this*))
       (when-abnormal-exit (progn ,@forms)
         (cond (,startcontext (rdb ,startcontext))
               (t (db-init)))
         (setq *that* ,old-*that*)
         (setq *this* ,old-*this*)))))

;;; Actions that access the database need to explicit take a lock
;;; on the database.

(define-vocabulary programmer-vocabulary)

(defmacro define-db-action (name bvl doc &body forms)
  '(define-action (programmer-vocabulary ,name) ,bvl ,doc
     (with-db-locked
       (error-restart (sys:error ',(text "Retry /"~A.../" command." name))
         (with-consistent-db
           ,@forms)))))

(defmacro define-db-interpretation (name bvl &body forms)
  '(define-interpretation (programmer-vocabulary ,name) ,bvl
     (with-db-locked
       (error-restart (sys:error ',(text "Retry /"~A.../" command." name))
         (with-consistent-db
           ,@forms)))))
```

```
;;;; Focus info

(defvar *that*        nil "Keeps track of the referent for 'that'.")
(defvar *this*        nil "Keeps track of the referent for 'this'.")
(defvar *here*        nil "A plist descriptor of here.")
(defvar *almost-this* nil "Like *this*, but not as exact.")

(defmacro bind-focus (loc &body forms)
  (let ((var (gensym)))
    '(let ((,var ,loc))
       (let ((*this*        (car    ,var))
             (*that*        (cadr   ,var))
             (*almost-this* (caddr  ,var))
             (*here*        (cadddr ,var)))
         (unwind-protect (progn ,@forms)
           (setf ,loc
                 (list *this* *that* *almost-this* *here*)))))))

(defun it ()
  (let ((that-ref (eval-command *that*))
        (this-ref (eval-command (or *this* *almost-this*))))
    (unless (equal this-ref that-ref)
      (complain "Ambiguous reference for 'it'. Use 'this' or 'that'."))
    this-ref))

(defun this () (eval-command *this*))
(defun that () (eval-command *that*))

(defun (it action) () (it))

(defun (this action) () (this))
(defun (that action) () (that))

(defun make-be-it (things)
  (setq *that* (make-ref (get-segcase (car things))))
  (setq *this* *that*)
  (setq *almost-this* *that*)
  (setq *here*
        (ignore-errors
          (let ((fn-name (~01 ~name (??& (top-segcase
                                           (car (eval-command *that*)))
                                         @SEG 'lambda-exp))))
            (when fn-name
              '(:name ,fn-name
                :path ,(code->path fn-name (parsed->code *that*)))))))))

(defun make-be-this (things &optional path position)
  (setq *this* nil *almost-this* nil)
  (when things
    (let ((fn (get-toplevel-name (car things))))
      ;; Probably PATH->CODE and CODE->PARSED should not err here, but
      ;; if they do, we want it to be a soft error because this
      ;; focus-setting is generally not important since we are relatively
      ;; error-tolerant about absence of focus anyway. -kmp 5/12/84
      (setq *this* (ignore-errors (code->parsed (path->code fn path))))
      (setq *almost-this* *this*)
      (setq *here* position)
      (when (not *almost-this*)
        (do ((l (reverse path) (cdr l))) (nil)
          (let ((parsed (ignore-errors
                          (code->parsed (path->code fn (reverse l))))))
            (when parsed
              (setq *almost-this* parsed)
              (return)))
          (if (null l) (return)))))))

;;;; Default references
```

```
(defun default-segcase ()
  (let ((result (eval-command (or *this* *almost-this*))))
    (cond (result (get-segcase (car result)))
          (t      nil))))

;; Should this really be able to err or are all the functions that
;; call it content with getting NIL (or something else) back if it
;; fails. It was getting an ill-formed fact error in FIND-FACTS when
;; doing (TOP-SEGCASE NIL), so I changed it to say something epsilon
;; more intelligent. We should decide on a general strategy...

(defun (top action) ()
  (let ((sc (default-segcase)))
    (cond (sc (list (top-segcase sc)))
          (t (complain "No default context has been established.")))))
```

```
(define-db-action locate ((position (parse-name ':expression)))
                  "Verbally marks a shift of presentational focus to
                   <position>. Does not shift the presentational focus.
                   The <position> is an arbitrary plist containing data
                   about the position."
  (let ((data (eval-command position)))
    (let ((type (get (locf data) ':type))
          (name (get (locf data) ':name))
          (path (get (locf data) ':path)))
      (let ((ref (or (get (locf data) ':reference)
                     (selectq type
                       ((define-cliche) (text "the cliche ~S" name))
                       ((defun)         (text "the program ~S" name))
                       (otherwise nil)))))
        (when ref
          (let ((things
                  (ignore-errors
                    (eval-command
                      (parse-command ref #'parse-definite-reference)))))
            (when things (make-be-this things path data))))))
    (values)))
```

```
;;;; ...

;this is here so that the parser will always be able to parse
;this even though it isn't actually in any stored cliche.

(*define-role 'mark)

;this takes in a segcase and makes it be a marked part of its
;top level plan segment.

(defun make-be-marked (segcase)
  (>& (top-segcase segcase) slot segcase 'mark 'segment))

;this gets rid of all of the marks in a program.

(defun clear-marks (segcase)
  (--& (top-segcase segcase) slot * 'mark))
```

```
;;; (EVAL-COMMAND command)
;;;
;;;  Special-purpose evaluator for parsed commands or command phrases.

(defun eval-command (command &aux fn)
  (cond ((or (memq command '(t nil)) (numberp command)) command)
        ((not (listp command))
         (barf "A KBE command may not be ~A:~% ~S"
               (a-or-an (string-downcase (typep command)))
               command))
        ((eq (car command) 'quote) (cadr command))
        ((eq (car command) 'name) command)
        ((setq fn (get (car command) 'verb-action))
         ;; Actions are special forms now.
         ;; They must eval their own args. --kmp 2/23/84
         (apply fn (cdr command)))
        ((setq fn (get (car command) 'action))
         (apply fn (mapcar #'eval-command (cdr command))))
        (t
         (barf "The identifier ~S was unrecognized in a KBE command:~% ~S"
               (car command) command))))
```

```
;;; The first part of this file deals with REFERENCE.  It provides a
;;; number of mechanisms (roughly equivalent to noun phrases in English)
;;; the user can use to refer to parts of a plan.  The intent is that the
;;; lisp expressions will read something like English if you don't
;;; scrutinize them too closely.
;;;
;;; Each of these returns a THING which is one of:
;;;  a seg       -- a segcase with the case NIL
;;;  a port      -- input, output, tinput, toutput
;;;
;;; There is a problem with representing the fact that sometimes you
;;; have a list of things, and sometimes you have just one.  In order
;;; to keep this straight, almost every function takes and returns lists
;;; of things even if it demands or produces just a single item.  This
;;; makes the representation uniform.
;;;
;;; A real problem here is that a THING is the actual thing from
;;; the data base. As a result it only stays accurate as long as the
;;; db doesn't change too much.
;;;
;;; In order to deal with this, we do the following: Whenever something
;;; is stored, instead of storing it, we store a path name, which is
;;; designed to be able to still refer to it even if every atom is
;;; renamed, and even if a fair amount of modification in the form
;;; of new segments is added.

;;; (pl? thing)
;;;  Converts NIL or thing to NIL or (thing). It is used to make a
;;;  list of things when you may or may not have a thing.

(defmacro pl? (thing)
  (once-only (thing)
    '(if ,thing (list ,thing))))

;utilities for working with names

(defmacro name-type (name)
  '(cadr ,name))

(defmacro name-symbol (name)
  '(caddr ,name))

(defun check-name (name)
  (if (not (and (listp name)
                (eq (car name) 'name)
                (listp (cdr name))
                (memq (name-type name)
                      '(cliche cliche-definition program
                        program-definition role variable segment))
                (listp (cddr name))
                (symbolp (name-symbol name))))
      (barf "~A bad name" name)))
```

```
;THE is used to refer to item(s).

;Note that it is important that THE never blows up because it is sometimes
;called on an expression that no longer refers to anything in order to
;find out if this is the case.  Root-reference is used to specify
;root segments.  Note tha the list of adjectives is only for euphony and
;is ignored here.

(defun (root-reference action) (ignore name)
  (check-name name)
  (caseq (name-type name)
    (segment
     (if (memq (name-symbol name) *db-segs*)
         (ncons (& (name-symbol name)))))
    (program-definition
      (if (name-symbol name)
          (pl? (~01 ~segcase (??& * @seg 'lambda-exp (name-symbol name))))
          (let ((sc (default-segcase)))
            (if sc (list (top-segcase sc))))))
    (cliche-definition
      (if (name-symbol name)
          (pl? (~01 ~segcase (??& * cliche-def (name-symbol name))))
          (let ((sc (default-segcase)))
            (if sc (list (top-segcase sc))))))
    (T (complain "'The ~A' -- not a valid reference"
                 (unparse-command name)))))

(defun (relative-reference action)
       (ordinal adjectives plural? of-in superiors)
  (let* ((things (Rappend (relative-reference1 adjectives of-in
                                               (Elist superiors)))))
    (cond (plural? things)
          (ordinal (pl? (ordinal-select ordinal things)))
          ((null (cdr things)) things))))

(defun relative-reference1 (adjectives of-in superior)
  (let* (((implemented? pgm cliche role) (the-adj-parse adjectives))
         (candidates
           (caseq of-in
             (of (if role
                     (~set ~slotseg (??& superior slot))
                     (~set ~subseg (??& superior subseg))))
             (in (Rlist (& (Elist (all-subsegs (~seg superior)))))))))
    (letS ((thing (Elist candidates)))
      (Rlist (Fselect thing
               (and (if (eq implemented? 'implemented)
                        (not (not-implemented? (~seg thing))) T)
                    (if (eq implemented? 'unimplemented)
                        (not-implemented? (~seg thing)) T)
                    (if pgm
                        (and (not (fact? thing))
                             (or (??& thing @seg 'funcall pgm)
                                 (??& thing @seg 'funcall
                                      (symbolconc "ADA-FN~" pgm))))
                        T)
                    (if cliche
                        (and (not (fact? thing))
                             (??& thing cliche cliche))
                        T)
                    (if role
                        (if (eq role '*)
                            (??& * slot thing *)
                            (??& * slot thing role))
                        T)))))))

(defun the-adj-parse (adjectives)
  (let (((implemented? pgm cliche role) nil))
    (letS ((name (Elist adjectives)))
```

```
    (cond ((memq name '(implemented unimplemented))
           (setq implemented? name))
          ((eq name 'role) (setq role '*))
          (T (check-name name)
             (caseq (name-type name)
               (program (setq pgm (name-symbol name)))
               (cliche (setq cliche (name-symbol name)))
               (role (setq role (name-symbol name)))
               (T (complain "Bad adjective ~A in RELATIVE-REFERENCE"
                            (unparse-command name)))))))
    (list implemented? pgm cliche role)))
```

```
(defun all-subroles (segcase)
  (declare (values things))
  (let ((*result* '()))
    (declare (special *result*))
    (all-subroles-aux segcase)
    *result*))

(defun all-subroles-aux (thing)
  (declare (special *result*))
  (push thing *result*)
  (if (not (fact? thing))
      (Rignore (all-subroles-aux (~slotseg (Elist (??& thing slot)))))))

(defun (port-reference action) (ordinal adjectives plural? superiors)
  (let* ((things (Rappend (port-reference1 ordinal adjectives
                                           (Elist superiors)))))
    (cond (plural? things)
          ((null (cdr things)) things))))

(defun port-reference1 (argno adjectives superior)
  (let* (((name key kind) (port-reference-adj-parse adjectives))
         (candidates
           (caseq key
             (input (??& superior input))
             (output (??& superior output))
             (tinput (??& superior tinput))
             (toutput (??& superior toutput))
             (temporal (append (??& superior tinput)
                               (??& superior toutput)))
             (non-temporal (append (??& superior input)
                                   (??& superior output)))
             (nil (append (??& superior input)
                          (??& superior output)
                          (??& superior tinput)
                          (??& superior toutput))))))
    (letS ((thing (Elist candidates))
           (@ioF (?& thing @io)))
      (Rlist (Fselect thing
               (and (if argno (eq argno (~argno @ioF)) T)
                    (if name (eq name (~name @ioF)) T)
                    (if kind (eq kind (~kind @ioF)) T)))))))

(defun port-reference-adj-parse (adjectives)
  (let* (name key kind temporal)
    (letS ((adj (Elist adjectives)))
      (caseq adj
        ((input output) (setq key adj))
        ((temporal non-temporal) (setq temporal adj))
        (port )
        (parameter (setq key 'input) (setq kind 'lambda-arg))
        (argument (setq key 'input) (setq kind 'actual-arg))
        (free-variable (setq kind 'free-var))
        (value (setq key 'output) (setq kind 'returnval))
        (side-effect (setq kind 'side-effect))
        (side-effected (setq kind 'side-effected))
        (T (check-name adj)
           (if (not (eq (name-type adj) 'var))
               (complain "Bad adjective ~A in PORT-REFERENCE"
                         (unparse-command adj)))
           (setq name (name-symbol adj)))))
    (if (eq temporal 'temporal)
        (caseq key
          (input (setq key 'tinput))
          (output (setq key 'toutput))))
    (setq key (or key temporal))
    (list name key kind)))
```

```
;this implements reference based on ordinal numbers (first, second,
;last, ...).  This operates in a somewhat ad hoc way for each kind of
;thing that may have been found.

;there are only two basic situations where this works.
;1. if you have a set of subsegs, then this assumes that they are all
;   subsegs in the same program, and uses dflow and cflow in that program
;   to order them, before selecting one.
;2. if they are inputs with ARGNOs then these are used to order them.

;there is a hack in that what is actually used to order the things is
;the order of appearance in the code (taking account of the fact that
;arguments are evaluated before fns).  (Note that if two things have
;the same code, or are atoms then things won't work too well.)  This a
;reasonable hack since it looks reasonable given what the user sees, and it
;disambiguates things a lot.  (It assumes that all of the things are in the
;same piece of code.)

;this is the key fn.  It takes a list of things.  It assumes that all of
;the things are in the same function.
;it returns a cons of two things: the first is a list of those things which
;have code corresponding to them in the function, sorted in the order they
;appear
; the second is a list of the other things in the order they originally
;were in.

(declare (special 2codes 2result))

(defun order-things (things)
  (cond ((null (cdr things)) [things])
        ((let* ((2codes (over things revlist [(corr-code _) ! _]))
                (start (car (corr-code (top-segcase
                                         (get-segcase (car things))))))
                (2result))
           (order-things1 start nil nil)
           [(nreverse 2result) ! (over 2codes select (not (eq (car _) T))
                                   revlist (cdr _))]))))

;this recursive program actually traverses a code tree looking for things.
;note that the decendants of a cell are ordered before it.

(defun order-things1 (cell car-parent cdr-parent)
  (cond ((not (atom cell))
         (order-things1 (car cell) cell nil)
         (order-things1 (cdr cell) nil cell)))
  (over 2codes select (not (memq (car _) '(T nil)))
     (cond ((cond ((eq (caar _) cell))
                  ((atom (caar _)) nil)
                  ((and (eq (caaar _) ' *code-is-car*)
                        (eq (cdaar _) car-parent)))
                  ((and (eq (caaar _) ' *code-is-cdr*)
                        (eq (cdaar _) cdr-parent))))
            (push (cdr _) 2result)
            (setf (car _) T)))))

;this finds the piece of code corresponding to a thing if there is one.

(defun corr-code (thing)
  (caseq (thing-type thing)
    ((seg segcase) (?@seg-code (~seg thing)))
    ((dflow cflow) (corr-code (~to thing)))
    ((input tinput output toutput)
     (or (?@io-code thing) (corr-code (~segcase thing))))))
```

```
;expects a list of things and gives you the numth one

(defun ordinal-select (num things)
  (cond ((null things) nil)
        ((null (cdr things)) (car things))
        (T (let* (((sort . others) (order-things things)))
             (cond ((> num (length things)) nil)
                   ((minusp num) (setq num (length things))))
             (cond ((not (> num (length sort))) (nth (1- num) sort))
                   ((null (cdr others)) (car others)))))))

;this is the inverse of the above it tells you the numeric position of
;a thing in a list of things

(defun ordinal-pos (thing things)
  (cond ((not (member thing things)) nil)
        ((null (cdr things)) 1)
        (T (let* (((sort . others) (order-things things))
                  (spot (member thing sort)))
             (cond ((and spot
                         (null (cdr spot))
                         (null others)
                         (> (length things) 2))
                    -1)
                   (spot (- (length things)
                            (length (cdr spot))))
                   ((null (cdr others)) -1))))))

;this takes in a list of things and returns it sorted in terms of
;there code order.  (This actually tries to deal with things that
;correspond not at all or ambiguously to the code.)

(defun ordinal-sort (things)
  (let (((sort . others) (order-things things)))
    (nconc sort others)))
```

```
;this is the inverse fn of the reference expression THE.

;the goal here is to get an unambiguous reference expression that will
;point to the thing in question.  there are two difficulties here
;which work against each other.  You want the result to be reasonably
;aesthetic, because the user sees these expressions printed out to him,
;and you want it to be unambigous.  If you specify that you want a
;short result, then it tries to be less verbous.

(defun make-ref (thing &optional (short nil))
  (or (make-ref-slot thing short)
      (caseq (thing-type thing)
        (SEG     (make-ref-seg thing short))
        (INPUT   (make-ref-input thing))
        (OUTPUT  (make-ref-output thing))
        (TINPUT  (make-ref-tinput thing))
        (TOUTPUT (make-ref-toutput thing)))))

;this computes the basic type of a thing (the keyword or SEG)

(defun thing-type (thing)
  (cond ((and (fact? thing)
              (memq (~keyword thing) '(input output tinput toutput)))
         (~keyword thing))
        ((and (listp thing)
              (memq (~seg thing) *db-segs*)
              (null (~case thing)))
         'seg)
        (T (barf "THING-TYPE: not a valid thing.~% Thing: ~:=" thing))))

(defun make-ref-slot (thing &optional (short T))
  (let ((slotF (~01 (??& * slot thing))))
    (if slotF
        (let* ((others (~set ~slotseg (??& (~segcase slotF) slot
                                           * (~slotname slotF))))
               (ordinal (if (cdr others) (ordinal-pos thing others))))
          '(relative-reference ,ordinal
                               '(,@(make-ref-extra
                                    thing (or short ordinal))
                                 (name role ,(~slotname slotF)))
                               nil 'of ,(make-ref (~segcase slotF) T))))))


(defvar *internal-names* '(5variable 5noop zeroary-function unary-function
                           binary-function simple-subseg)
  "List of 'function' and 'cliche' names that we don't want user to see.")

(defun make-ref-extra (thing short)
  (if (and (not short) (not (fact? thing)))
      (let* ((@seg (~01 (??& thing @seg)))
             (fn-name (if (and @seg (eq (~kind @seg) 'funcall))
                          (~name @seg)))
             (cliche-name (~01 ~clichename (??& thing cliche))))
        (cond ((and cliche-name
                    (not (memq cliche-name *internal-names*)))
               '((name cliche ,cliche-name)))
              ((and fn-name
                    (not (memq fn-name *internal-names*)))
               '((name program ,fn-name)))))))
```

```
(defun make-ref-seg (thing &optional (short T))
  (let* ((cliche-def-name (~01 ~cliche-def-name (??& thing cliche-def)))
         (@seg (~01 (??& thing @seg))))
    (cond (cliche-def-name
           '(root-reference ',(make-ref-extra thing short)
                            (name cliche-definition ,cliche-def-name)))
          ((and @seg (eq (~kind @seg) 'lambda-exp))
           '(root-reference ',(make-ref-extra thing short)
                            (name program-definition ,(~name @seg))))
          (T '(root-reference nil (name segment ,(~seg thing)))))))

(defun make-ref-input (thing)
  (let* ((@io (?& thing @io))
         (segcase (~segcase thing))
         (seg-ref (make-ref segcase T)))
    (caseq (~kind @io)
      (free-var '(port-reference nil '((name variable ,(~name @io))
                                       free-variable input)
                                 nil ,seg-ref))
      (lambda-arg '(port-reference nil '((name variable ,(~name @io))
                                         parameter)
                                   nil ,seg-ref))
      (actual-arg '(port-reference ,(if (cdr (??& (&* segcase input) @io
                                                  'actual-arg))
                                        (~argno @io))
                                   '(argument) nil ,seg-ref)))))

(defun make-ref-output (thing)
  (let* ((@io (?& thing @io))
         (seg-ref (make-ref (~segcase thing) T)))
    (caseq (~kind @io)
      (free-var '(port-reference nil '((name variable ,(~name @io))
                                       free-variable output)
                                 nil ,seg-ref))
      (side-effect '(port-reference nil '((name variable ,(~name @io))
                                          side-effect output)
                                 nil ,seg-ref))
      (returnval '(port-reference nil '(value) nil ,seg-ref)))))

(defun make-ref-tinput (thing)
  (let* ((@io (?& thing @io))
         (seg-ref (make-ref (~segcase thing) T)))
    '(port-reference nil '(,@(if (~name @io)
                                 '((name variable ,(~name @io))))
                           temporal input)
                     nil ,seg-ref)))

(defun make-ref-toutput (thing)
  (let* ((@io (?& thing @io))
         (seg-ref (make-ref (~segcase thing) T)))
    '(port-reference nil '(,@(if (~name @io)
                                 '((name variable ,(~name @io))))
                           temporal output)
                     nil ,seg-ref)))
```

```
;;; -*- Mode:LISP; Package:KBE; Base:10; Fonts: MEDFNB; -*-

;;; (c) Copyright 1983, Massachusetts Institute of Technology
;;;     All Rights Reserved.
;;;     Permission to copy all or part of this material is granted,
;;;     provided that the copies are not made or distributed for resale,
;;;     the MIT copyright notice and reference to the source file and
;;;     the software distribution version appear, and that notice is
;;;     given that copying is by permission of Massachusetts Institute
;;;     of Technology.

;;; If you run with *AUTOMATIC-REAP* set non-(), you cannot back up
;;; in any way once a verb runs to completion. The only thing
;;; you can do is back up to the ground state -- ie, >I or
;;; *initial-db-context*

(define-internal-option *automatic-reap*
                        "Do a DB-REAP after every command"
  #+3600 nil ; 3600 is fast enough that this doesn't matter much.
  #-3600 t   ; LM-2 needs all the help it can get, even at the expense
             ;  of having Undo not work.
  :boolean)

(defvar *last-db-context* nil "holds old context point for backup")
(def-db-var *undo-stack* nil "...")

(defun top-segcase (segcase)
  (declare (values segcase))
  (repeat ((ts (~seg segcase)) nts)
    (setq nts (?supseg ts))
    (cond ((null nts) (return (list ts)))
          (t (setq ts nts)))))
```

```
(define-db-interpretation how (toplevel-reference)
  (let ((results (copytree (parsed->code toplevel-reference))))
    (when (and *debug* (cdr results))
      (cbarf "Ignore extra code."
             "More than one piece of code for ~A."
             (unparse-command toplevel-reference)))
    (let ((result (car results)))
      '(:code
        :text ,(cond (result (let (prinlevel prinlength)
                               (text "~S" result)))
                     (t (text "I don't know the definition of ~A"
                              (unparse-command toplevel-reference))))
        :definition ,result))))

;;; This is used to create a literal piece of plan.
;;; eg, the arg to ANALYZE is currently a (CODE ...) form.

(defun (code action) (code)
  (if (stringp code)
      (setq code (string-parser code))
      (setq code (cadr code)))
  (list (create-plan-for-code code)))

;;; This is the only entry into the analyzer,etc. which should be used.
;;; note the way it takes care of the {a ...} stuff.

(defun create-plan-for-code (lisp-code)
  (declare (special *pending-dflows*))
  (if (eq *pending-dflows* '*top-level*)
      (let (*pending-dflows*)
        (prog1 (create-plan-for-code1 lisp-code)
               (if *pending-dflows* (complain "pending dflows remain"))))
      (create-plan-for-code1 lisp-code)))

;note when we are redefining a whole program,
;we must be carefull not to evaluate the fillers untill
;after we do the surface-analyze and make the result be IT!
;otherwise we must evaluate them first in order to
;get hold of the right referents.

(defun create-plan-for-code1 (lisp-code)
  (let* ((pending-implements (check-for-implements lisp-code))
         (segcase (analyze (surface-analyze lisp-code)))
         (total-redefinition
          (and (listp lisp-code)
               (memq (car lisp-code) '(defun define-cliche def-ada-top)))))
    (if total-redefinition (make-be-it (list segcase)))
    (setq pending-implements
          (mapcar #'(lambda (x)
                      (list (car x) (car (eval-command (cadr x)))))
                  pending-implements))
    (make-be-it (list segcase))
    (dolist (imp pending-implements)
      (let* ((slot (car imp))
             (slotsegcase (~slotseg (?& segcase slot * slot)))
             (filler (cadr imp)))
        (implement-internal slotsegcase filler)
        (--& * slot * slot)))
    segcase))

(declare (special check-for-implements-list))

(defun check-for-implements (code)
  (let (check-for-implements-list)
    (check-for-implements1 code)
    check-for-implements-list))

;this finds instances of {a ...} and queues up implements to do for them.
```

```
;note we are completely punting the problem of not being able to
;fix the slot in the right place in general.  It happens not to
;be a problem in the scenario chosen.
;also this doesn't handle fexprs well.

(defun check-for-implements1 (code)
  (cond ((not (listp code)))
        ((and (kbe-comment? code)
              (eq (kbe-comment-type code) ':indefinite))
         (let* ((fake-slot (symbolconc 'slot- (gensym)))
                (filler (parse-command
                          (kbe-comment-name->string
                            (kbe-comment-name code))
                          #'parse-indefinite-reference)))
           (push (list fake-slot filler) check-for-implements-list)
           (rplaca (cddr code) (list 'the fake-slot))))
        (T (do ((list code (cdr list)))
               ((not (listp list)))
             (check-for-implements1 (car list))))))
```

```
;;; >Analyze <code>.
;;;
;;;  Constructs the code for a plan designated by <reference>.

 ; Issues: Focus probably isn't set right after this.
 ;         The multiple calls to EVAL-COMMAND need to have the
 ;         command termination signal get deferred.

(define-db-action Analyze ((reference (parse-reference)))
                  "Makes database consistent with the given <reference>."
  (if (not (eq (car reference) 'code))
      (complain "Non literal reference for Analyze: ~A"
                (unparse-command reference)))
  (let* ((code (cadadr reference))
         (expression (cond ((stringp code) (string-parser code))
                           (t (cadr code)))))
    (when (or (atom expression)
              (not (memq (car expression)
                         '(defun define-cliche def-ada-top))))
      (complain "Only able to analyze definitions."))
    (assure-db-consistency expression)
    (suggest-focus '(:name  ,(cadr expression)))
    (values)))

 ;This definition might should be elsewhere. -kmp
(defmacro without-end-of-command (&body forms)
  '(condition-bind ((end-of-command #'(lambda (cond)
                                        (send cond ':proceed ':handled))))
     ,@forms))

(defun eval-commands (commands
                      &optional (end-mode ':normal) (fn #'eval-command)
                      &rest args)
  ; END-MODE should be one of :IGNORE, :DEFER, or :NORMAL
  (cond ((null commands) nil)
        ((null (cdr commands))
         (if (eq end-mode ':ignore)
             (without-end-of-command
               (lexpr-funcall fn (car commands) args))
             (lexpr-funcall fn (car commands) args)))
        (t
         (if (neq end-mode ':normal)
             (without-end-of-command
               (lexpr-funcall fn (car commands) args))
             (lexpr-funcall fn (car commands) args))
         (lexpr-funcall #'eval-commands (cdr commands) end-mode fn args))))

(define-internal-option *trap-db-inconsistency*
                        "Trap when CODE-EQUAL fails in DB"
  nil :boolean)

(defun assure-db-consistency (test-code)
  (let* ((name (cadr test-code))
         (good-code (path->code name)))
    (cond ((not good-code) (force-db-consistency test-code))
          ((code-equal test-code good-code)) ;nothing to do
          (t
           (when *trap-db-inconsistency*
             (fsignal "CODE-EQUAL failed.~% Test code: ~:=~% Good code: ~:="
                      test-code good-code))
           (let ((result (compare-code name good-code test-code)))
             (cond (result
                    (eval-commands (with-input-from-string (str result)
                                     (read-commands-from-stream-contents
                                       str))
                                   ':ignore))
                   (t
                    (force-db-consistency test-code))))))))
```

```
    (make-be-it (list (name->segcase name)))))

(defun force-db-consistency (expression)
  (code (create-plan-for-code expression)))
```

```
;;; Utilities

(*define-completion 'here)

(define-db-action add ((reference (parse-reference))
                       (where (progn
                                (if (eq (peek-token) 'here) (read-token))
                                '(here))))
                  "Adds <reference> at the cursor position.
                   The word /"here/" may optionally follow for euphony."
  where ;ignored.
  (let ((sexp (make-kbe-comment nil (ncons (unparse-command reference))))
        (name (get (locf *here*) ':name))
        (path (get (locf *here*) ':path)))
    (eval-command '(analyze (code '',(splice-code-at-path
                                       (path->code name nil)
                                       path
                                       sexp))))))

(defun splice-code-at-path (old-code path code-to-add)
  (let ((the-copy (copytree old-code)))
    (splice-code-at-path-1 the-copy
                           (or path (complain "Cannot insert code there."))
                           code-to-add
                           nil 0) ;parent info
    the-copy))

(defun splice-code-at-path-1 (old-code path code-to-add parent n)
  (cond ((not path)
         (cond ((not parent) (barf "Parent node not given?"))
               ((zerop n)
                (let ((a (car parent))
                      (d (cdr parent)))
                  (setf (car parent) code-to-add)
                  (setf (cdr parent) (cons a d))))
               (t
                (push code-to-add (nthcdr n parent)))))
        (t
         (splice-code-at-path-1 (nth (car path) old-code)
                                (cdr path)
                                code-to-add
                                old-code
                                (car path)))))
```

```
;;; >Compute <reference>.
;;;
;;;  Returns the referent of the given <reference>.
;;;  Intended mainly for debugging.

(define-db-action compute ((reference (parse-reference)))
                  "Computes referent of the given <reference>."
  (copytree (eval-command reference)))

;evals an expr and returns the result.
;;; i assume this is mostly for debugging? -kmp

(define-db-action eval ((code (let ((exp (parse-name ':expression)))
                                (when (or (atom exp)
                                          (neq (car exp) 'quote))
                                  (parse-error
                                    "EVAL expects a quoted name."))
                                exp)))
                  "Calls EVAL on <code>."
  (copytree (eval (eval-command code))))
```

```
(define-db-action explain ((reference (parse-definite-reference)))
                  "Explains the segment designated by <reference>."
  (explain-reference reference))

(define-KBE-option *explainer* "Explanation format"
  'document-fn :assoc '(("Paragraph" . explain-fn)
                        ("Outline"   . document-fn)))

(defun explanation (fn) (funcall *explainer* fn))

(defun explain-reference (reference)
  (let ((things (eval-command reference)))
    (if (null things)
        (complain "I don't know anything about ~A."
                  (unparse-command reference)))
    (make-be-it things)
    (with-output-to-string (standard-output)
      (dolist (thing things)
        (let ((fn-name (get-toplevel-name thing)))
          (values (format t "~S ...~2%~A~2&"
                          fn-name
                          (format-explanation
                            (explanation fn-name)))))))))

(define-db-action document ((reference (parse-definite-reference)))
                  "Documents the segment designated by <reference>."
  (document-reference reference))

(define-db-action comment ((reference (parse-definite-reference)))
                  "Comments the program designated by <reference>."
  (document-reference reference)) ;a more modest name for this.

(defun document-reference (reference)
  (let ((things (eval-command reference)))
    (if (null things)
        (complain "I don't know anything about ~A."
                  (unparse-command reference)))
    (request-display reference)
    (let ((fn-name (get-toplevel-name (car things))))
      (document-code fn-name (explanation fn-name)))
    (values)))
```

```
(define-db-action move (to (reference (parse-definite-reference)))
                  "Moves to the segment designated by <reference>."
  (request-display reference)
  (values))

;Highlight the specified thing(s) on the tv display...

(define-db-action display ((reference (parse-definite-reference)))
                  "Displays the segment designated by <reference>."
  (request-display reference)
  (values))

(defun request-display (reference)
  (let ((things (eval-command reference)))
    (if (null things)
        (complain "I don't know anything about ~A."
                  (unparse-command reference)))
    (make-be-it things)
    (let ((parsed-forms (Rlist (make-ref (Elist things)))))
      (if (and *debug* (cdr parsed-forms))
          (fsignal "Multiple focus seen by REQUEST-DISPLAY" parsed-forms))
      (change-focus-to (car parsed-forms) T))))

(defun change-focus-to (parsed-form &optional (force-highlight? nil))
  (let* ((things (eval-command parsed-form))
         (topfn  (if things (get-toplevel-name (car things))))
         (paths  (if parsed-form
                     (let ((code (condition-case (cond)
                                        (parsed->code parsed-form)
                                   (sys:error nil))))
                       (if code
                           (rlist (code->path topfn (elist code))))))))
    (suggest-focus
      '(:reference ,(unparse-command parsed-form)
        :name  ,topfn
        :paths ,paths
        :path  ,(car paths)
        :force-highlight ,force-highlight?))))
```

```
(define-db-action reinitialize (database)
                  "Reinitializes the KBE database (for debugging)."
  (db-init)
  (list ':text ':text "Database has been reinitialized."))

(define-db-action use (language (language (parse-name ':keyword)))
                  "Sets the language of reference to <language>."
  (use-language-internal language)
  (suggest-focus '())
  (values))

(defun resolve-language (name)
  (cond ((or (symbolp name) (stringp name))
         (setq name (intern-keyword name)))
        (t
         (complain "Odd language spec: ~S" name)))
  (cond ((memq name '(:pl1 :pli)) 'PLI)
        ((memq name '(:lisp))     'LISP)
        ((memq name '(:ada))      'ADA)
        (t
         (complain "Sorry, I never heard of the ~A language." name))))

(defvar *display-language* 'lisp "For possible use by VIEW")

(defun use-language-internal (name)
  (declare (special *input-language*            ; From PARSE
                    *display-language*))        ; From VIEW
  (*catch 'no-redisplay-needed
    (let ((language (resolve-language name)))
      (setq *input-language* language)
      (if (eq *display-language* language)
          (*throw 'no-redisplay-needed t))
      (setq *display-language* language))
    (let ((things (that)))
      (when things
        (clear-display)
        (request-display (make-ref (car things))))))   ;ie, *that*
  (values))


;;;only works for lisp, but is obsolete anyway.

(defun grind-code (form) (text "~:=~&" form))

(define-db-action view ((toplevel-reference (parse-toplevel-reference))
                        in (language (parse-name ':keyword)))
                  "Requests the code named by <toplevel-reference> be
                   displayed in the given <language>."
  (let ((*display-language* (resolve-language language)))
    (declare (special *display-language*))
    (let ((code (parsed->code toplevel-reference)))
      (cond ((not code)
             (complain "I don't know anything about ~A."
                       (unparse-command toplevel-reference)))
            (t
             '(:text :text .(grind-code (car code))))))))
```

```
;this takes in a set of things, makes up an english description (based
;on make-ref) and prints it on the tty.


(define-db-action describe ((reference (parse-definite-reference)))
                  "Describes the segment designated by <reference>."
  (describe-reference reference))

(defun describe-reference (reference)
  (let ((things (eval-command reference)))
    (list ':text
          ':text (cond ((null things)
                        (text "I don't know about ~A."
                              (unparse-command reference)))
                       (t
                        (text "~{~A.~^~%~}"
                              (determine-ref-sharing
                                (ordinal-sort things))))))))

;this deletes the common tails from a set of reference strings and
;returns only the unique initial parts.

(defun determine-ref-sharing (things &aux strings)
  (setq strings (Rlist (string-reverse
                         (unparse-command
                           (make-ref
                             (Elist things))))))
  (prog (pos)
      L (setq pos (min (or (string-search " fo " (car strings)) 123456)
                       (or (string-search " ni " (car strings)) 123456)))
        (if (eq pos 123456) (return nil))
        (setq pos (+ pos 4))
        (cond ((Rand-fast (string-equal (car strings)
                                        (Elist (cdr strings))
                                        0 0 pos pos))
               (setq strings (Rlist (substring (Elist strings) pos)))
               (go L))))
  (mapS #'string-nreverse (Elist strings))
  strings)
```

```
;;; This function allows you to ask about what still needs to be done
;;; (as recorded in TO-DO assertions).

(define-db-action what (needs to be done (where (parse-position) "in") ?)
                  "Requests a list of things that need to be done.
                   An optional context may be supplied as in /"What needs
                   to be done in the program FOO./""
  (let ((ref (if where
                 (let ((things (car (eval-command where))))
                   (cond (things (make-ref things))
                         (t
                          (barf "I don't know anything about ~A."
                                (unparse-command where))))))))
    (letS (((fn . strings)
            (Elist (alist-of-strings-to-do-in ref))))
      (list ':possibilities
            ':type    ':unfinished-edits
            ':context (if ref (unparse-command ref))
            ':data (Rlist (cons fn strings))))))

(defun alist-of-strings-to-do-in-function (name)
  (alist-of-strings-to-do-in (function-name->parsed name)))

(defun function-name->parsed (function-name)
  (parse-command (text "the program ~S" function-name)
                 #'parse-definite-reference "definite reference"))

(defun alist-of-strings-to-do-in (ref)
  (let ((things (eval-command ref)))
    (if (or (not ref) things) (alist-of-strings-to-do things))))

;this is the main function which gets the key info.

(defun alist-of-strings-to-do (things &aux)
  (if (null things)
      (setq things (~set ~segcase (??& * @seg 'lambda-exp))))
  (let* ((segs
          (makeqset
            (Rnconc (all-subsegs (~seg (get-segcase (Elist things)))))))
         (stuff-by-fn
          (letS* ((segcase (& (Elist segs)))
                  (to-do (not-implemented? (~seg segcase))))
            (Reqalist (get-toplevel-name segcase)
                      (cons (Fselect to-do to-do) segcase)))))
    (letS (((fn . stuff) (Elist stuff-by-fn))
           (refs (determine-ref-sharing
                   (letS () (Rlist (cdr (Elist stuff))))))
           (strings (letS ((code (cdr (assq (car (Elist stuff))
                                            '((feed   . "Supply data to")
                                              (use    . "Use")
                                              (expand . "Implement")))))
                           (ref (Elist refs)))
                      (Rlist (text "~A ~A." code ref)))))
      (Rbag (cons fn strings)))))


(defun not-implemented? (seg)
  (cond ((?empty-slot seg))
        ((and (or (??& (&* seg) @seg 'lambda-exp)
                  (??& (&* seg) cliche-def))
              (null (??& (& seg) subseg)))
         'expand)
        ((and (??& (&* seg) cliche 'pseudo-subcliche)
              (Rand-fast
                (not-implemented?
                  (Elist (~set ~seg ~slotseg (??& (&* seg) slot))))))
         'expand)))
```

```
;;; (FINISH-EDIT marker)
;;;
;;;  Historically, this function was responsible for getting a lot
;;;  of the busy work of editing done. It would recode the modified
;;;  program and redisplay it with the cursor positioned on the item
;;;  to which the pronoun "this" would refer.  It made the assumption
;;;  that after the edit operation, "this" would designate
;;;  something in the modified program unless the whole program has
;;;  been destroyed.
;;;
;;;  In the current model of the world, FINISH-EDIT sees that
;;;  many of these same things get done, but mostly by hooks rather than
;;;  wired code. For a simple demo, we might still want display to happen
;;;  automatically, but for the projected mode of operation (with the
;;;  apprentice working in background), there is no reason for the coder
;;;  to code something up unless it is going to be viewed. Hence we want
;;;  a modularity which will allow us to be able to say that if there is
;;;  more real work to be done, do it first. Call the coder only if the
;;;  apprentice is not otherwise busy. This saves useless coding.
;;;
;;;  Note also that FINISH-EDIT is now a special form, not
;;;  a function. Its argument should be an unquoted symbol which names
;;;  a variable which will be set to *last-db-context* if any such exists.

(defmacro finish-edit (old-context-name &body list-of-one-it-form)
  '(progn (make-be-it ,@list-of-one-it-form)
          (when (and *last-db-context* (not *automatic-reap*))
            (remember-db-context *last-db-context* ,old-context-name)
            (push '(,*last-db-context* . ,(it))
                  *undo-stack*))
          (code-and-display-it)
          (db-context-was *last-db-context*)
          ;; From an outsider's point of view, we're all done...
          (end-of-command)
          ;; Now just clean up locally...
          (db-reap-and-gc)
          (values)))

;;; >Reap database.

(define-db-action Reap (database)
                  "Causes the database to be reaped and GC'd."
  (db-reap)
  (db-gc)
  (list ':text ':text "Database has been reaped and GC'd."))

;;; (DB-REAP-AND-GC)
;;;
;;;  Currently, this will compact the db if *AUTOMATIC-REAP* is non-(),
;;;  and it will unconditionally call DB-GC.
;;;
;;;  At some point, this may want to change so that it doesn't always
;;;  call DB-GC, which is currently a very expensive operation.
;;;
;;;  An important assumption of the recent DB changes is that the database
;;;  is compacted every time you do something. There are places which
;;;  probably depend on this being true. To find them, look for places
;;;  where conditionals hinge on the value of *AUTOMATIC-REAP*.

(defun db-reap-and-gc ()
  (cond (*automatic-reap* (db-reap)))
  (db-gc))

;;; (CODE-AND-DISPLAY-IT)
;;;
;;; If something is being referred to by "it" (the value of *that*),
;;; codes and displays that thing.
;;;
```

```
;;; If nothing is being referred to (presumably due to the whole
;;; program having been destroyed), a new context is pushed and the
;;; display is cleared in case it might have been invalidated.

(defun code-and-display-it ()
  (cond (*that*
         (code (top-segcase (get-segcase (car (eval-command *that*)))))
         (change-focus-to *that*))
        (T (push-db-context) (clear-display))))
```

```
;;; This only works for segments at the moment.

(define-db-action Remove ((definite-reference (parse-definite-reference)))
                  "Remove the thing referred to by <definite-reference>."
  (remove-reference definite-reference))

(defun remove-reference (reference)
  (let ((things (eval-command reference)))
    (cond ((null things)
           (complain "I don't know anything about ~A."
                     (unparse-command reference)))
          ((or (cdr things)
               (neq (thing-type (car things)) 'seg))
           (complain "I don't know how to kill ~A."
                     (unparse-command reference))))
    (finish-edit <IMP
      (prog1 [(& (plan-top (~seg (car things))))]
             (2remove-seg (car things))))))

;;; (UNDO-LAST-EDIT)
;;;
;;;  Running this program will undo an edit. This only makes sense if
;;;  we are not compacting the DB, which destroys the information about
;;;  other contexts besides the initial context to which we could back up.

(define-db-action undo () "Undoes the last request."
  (undo-last-edit))

(defun undo-last-edit ()
  (let (((context . references) (pop *undo-stack*)))
    (cond ((and (not *automatic-reap*)
                context
                (not (> (car context) (car *db-context*))))
           (restore-db-context context)
           (db-context-was *last-db-context*)
           (setq *this* nil)
           (setq *that* (car references))
           (if (cdr references)
               (cbarf "Ignore extra focus info."
                      "Multiple focus seen in UNDO-LAST-EDIT."
                      references))
           (change-focus-to (car references)))
          (t
           (complain "No saved context to undo to!")))))
```

```
(defun unparse-parameters-or-roles (data)
  (when (cdr data)
    (text "~A ~A"
          (string-downcase
            (cond ((cddr data) (car data))
                  (t (a-or-an (or (singular-form (car data))
                                  (car data))))))
          (phrase-append "and" (mapcar #'unparse-command
                                       (cdr data))))))

(defun (parameter  unparse) (x) (unparse-parameters-or-roles x))
(defun (parameters unparse) (x) (unparse-parameters-or-roles x))
(defun (role       unparse) (x) (unparse-parameters-or-roles x))
(defun (roles      unparse) (x) (unparse-parameters-or-roles x))

(define-db-action define
                  (a<n> (optional-cliche-name (parse-optional-cliche-name))
                     (spec
                       (let ((name (parse-definition-spec)))
                         (if (name-symbol name) name
                             (parse-error "Missing definition name."))))
                     (optional-roles-or-parameters
                       (when (eq (peek-token) 'with)
                         (read-token)
                         (parse-roles-or-parameters spec))
                       "with"))
                  "Defines a given program//cliche. Parameters//roles
                   may optionally also be specified as in /"... with a
                   parameter//role X./" or /"... with parameters//roles
                   X and Y./""
  (make-definition-internal
    optional-cliche-name
    spec
    (cdr optional-roles-or-parameters)))

(defun make-home-suggestion (name)
  (let ((this-thing (car (this)))) ; %%% use *almost-this*? -kmp 5/26/84
    (if this-thing
        (let ((near-name (get-toplevel-name this-thing)))
          (suggest-code-home (name-symbol name)
                             '(:near ,near-name))))))

(defun make-definition-internal (cliche-name name args)
  (if cliche-name (check-name cliche-name))
  (check-name name)
  (if (not (name-symbol name))
      (complain "No name was given for the ~A to be defined."
                (unparse-command name)))
  (make-home-suggestion name)
  (caseq (name-type name)
    (program-definition
      (lexpr-funcall #'eval-commands (list name) ':ignore
                     #'make-program-definition args))
    (cliche-definition
      (lexpr-funcall #'eval-commands (list name) ':ignore
                     #'make-cliche-definition  args))
    (T (complain "Cannot define a ~A" (unparse-command name))))
  (if cliche-name
      (eval-command
        (parse-command (text "Implement the ~A as a ~A"
                             (unparse-command name)
                             (unparse-command cliche-name)))))
  (values))

(defun make-cliche-definition (name &rest roles)
  (if (or (get (name-symbol name) 'cliche-plan)
          (??& * cliche-def (name-symbol name)))
```

```
      (complain "The ~A is already defined." (unparse-command name)))
  (let* ((segcase (& (genseg (name-symbol name)))))
    (+& segcase segtype 'simple)
    (+& segcase plantype 'expr)
    (+& segcase @seg 'opencode nil nil)
    (+& segcase cliche-def (name-symbol name) nil)
    (when roles
      (complain "Can't specify roles in /"Define a cliche.../" any more.")
      (comment (Rignore (add-role (name-symbol (Elist roles)) segcase))))
    (finish-edit <DEFINE
      (list segcase))))

(define-db-action Append (a parameter
                            (var (parse-variable-name))
                            (where (when (eq (peek-token) 'to)
                                     (read-token)
                                     (parse-toplevel-reference))
                                   "to" (top)))
                  "Appends the specified parameter to the parameter
                   list of the current program (or to some other
                   program, whose name may be introduced by /"to/")."
  (let ((things (eval-command where)))
    (check-name var)
    (if (not (and (null (cdr things))
                  things
                  (eq (thing-type (car things)) 'seg)
                  (??& (car things) @seg 'lambda-exp)))
        (complain "Can only add a parameter to a defun."))
    (add-parameter (name-symbol var) (car things))
    (finish-edit <ADD things)))
```

```
;;; Note: Better error checking here would be desirable. Right now we
;;;  don't worry about the case where the function being redefined
;;;  had a different IO-spec from the one replacing it. If that happens,
;;;  and if previously analyzed programs have made assumptions about
;;;  that IO-spec, the world will be confused. --kmp

(defun make-program-definition (name &rest variables)
  (if (or (let ((segcase
                  (~01 ~segcase
                       (??& * @seg 'lambda-exp (name-symbol name)))))
            (cond ((not segcase) nil)
                  (T (kill-seg (~seg segcase)) T)))
          (fboundp (name-symbol name)))
      (grumble "Redefining the ~A" (unparse-command name)))
  (let* ((segcase (& (genseg (name-symbol name)))))
    (+& segcase segtype 'simple)
    (+& segcase plantype 'expr)
    (+& segcase @seg 'lambda-exp (name-symbol name) nil)
    (Rignore (add-parameter (name-symbol (Elist variables)) segcase))
    (finish-edit <DEFINE
      (list segcase))))

(defun add-parameter (symbol segcase)
  (if (??& (&* segcase input) @io 'lambda-arg symbol)
      (barf "the program already has an arg: ~A" symbol))
  (let* ((port (+& segcase input nil (genobj (~seg segcase) symbol) nil))
         (argno (1+ (length (~nset ~argno
                                   (??& (&* segcase input) @io 'lambda-arg))))))
    (+& port @io 'lambda-arg symbol argno)
    port))
```

```
;This focuses on the ability to actually do the sharing.
;ie get the new dflow right, and delete any segments that
;are redundant.  It does this at the cost of only weakly checking
;whether sharing is in fact a correct thing to do.
;Note that this assumes that seg1 is not forced to be after seg2.

(define-db-action share ((definite-reference-1 (parse-definite-reference))
                         and
                         (definite-reference-2 (parse-definite-reference)))
                  "Shares <definite-reference-1>
                    and <definite-reference-2>.
                   eg, /"Share the x and the y./""
  (share-references definite-reference-1 definite-reference-2))

(defun share-references (ref1 ref2)
  (let ((thing1 (eval-command ref1))
        (thing2 (eval-command ref2)))
    (let ((failures '()))
      (cond ((not thing1) (push "~0G~A is unknown to me" failures))
            ((or (cdr thing1)
                 (neq (thing-type (car thing1)) 'seg))
             (push "~0G~A is not a seg" failures)))
      (cond ((not thing2) (push "~1G~A is unknown to me" failures))
            ((or (cdr thing2)
                 (neq (thing-type (car thing2)) 'seg))
             (push "~1G~A is not a seg" failures)))
      (if failures
          (let ((unref1 (unparse-command ref1))
                (unref2 (unparse-command ref2)))
            (complain
              (auto-fill-string
                (text "Unable to share ~A and ~A because ~A."
                      unref1 unref2
                      (phrase-append "and"
                                     (mapcar #'(lambda (x)
                                                 (text x unref1 unref2))
                                             failures))))))))
    (finish-edit <SHARE
      (share0 (~seg (car thing1)) (~seg (car thing2))))))

(defun share0 (seg1 seg2)
  (when (not (eq seg1 seg2))
    (let ((seg1ins (sort (??& (&* (&* seg1) input) @io) #'arg-order))
          (seg2ins (sort (??& (&* (&* seg2) input) @io) #'arg-order)))
      (when (not (and (simple-funcall-or-constant seg1)
                      (simple-funcall-or-constant seg2)
                      (equal (?@seg-name seg1) (?@seg-name seg2))
                      (eq (length seg1ins) (length seg2ins))))
        (complain "Unable to share ~A with ~A."
                  (unparse-command (make-ref (& seg1) t))
                  (unparse-command (make-ref (& seg2) t))))
      (letS ((from1 (dflow-source (~deep (Elist seg1ins))))
             (from2 (dflow-source (~deep (Elist seg2ins)))))
        (when (not (eq from1 from2))
          (when (not (and (??& from1 @io 'returnval)
                          (??& from2 @io 'returnval)))
            (barf "Ports not sharable: ~:= and ~:=" from1 from2))
          (share0 (~seg~segcase from1) (~seg~segcase from2))))
      (let* ((((first-seg) (second-seg))
              (ordinal-sort (list (& seg1) (& seg2)))))
        (Rignore (implement-dflow (?& (&* first-seg) output)
                  (Elist (share-dests (?& (&* second-seg) output)))))
        (kill-shared-seg second-seg)
        (list (& first-seg))))))

;we used to do this but it choked in demo.
;      (let* ((expr-seg (shared-expr seg1 seg2))
;             (shared-seg (~seg (copyseg seg1 nil)))
```

```
;            (sharedins (sort (??& (&* (&* shared-seg) input) @io)
;                             #'arg-order)))
;       (>& (& expr-seg) subseg (& shared-seg))
;       (>& (& expr-seg) role (& shared-seg) 'action)
;       (Rignore (implement-dflow (?& (&* shared-seg) output)
;                  (Elist (append (share-dests (?& (&* seg1) output))
;                                 (share-dests (?& (&* seg2) output))))))
;
;       (Rignore (implement-dflow (dflow-source (~deep (Elist seg1ins)))
;                                 (~deep (Elist sharedins))))
;
;       (kill-shared-seg seg1)
;       (kill-shared-seg seg2)
;       (list (& shared-seg)))
```

```
(defun shared-expr (seg1 seg2)
  (let (((lca) (2intermediate-segs seg1 seg2)))
    (prog ()
      L (if (eq (?plantype lca) 'expr) (return lca))
        (setq lca (?supseg lca))
        (if (null lca)
            (barf "No containing expr for ~A and ~A" seg1 seg2))
        (go L))))

(defun kill-shared-seg (seg)
  (let ((supseg (?supseg seg)))
    (2remove-seg (& seg))
    (prog (supsup)
       L (when (and (setq supsup (?supseg supseg))
                    (null (??subseg supseg)))
           (2remove-seg (& supseg))
           (setq supseg supsup)
           (go L)))))

(defun share-dests (ret)
  (prog (to-do found out dflows)
     (setq to-do (list ret))
   L (if (null to-do) (return found))
     (setq out (pop to-do))
     (cond ((and (output-fact? out) (setq dflows (??& * dflow out)))
            (letS ((df (Elist dflows)))
              (push (~to df) to-do)
              (make-fact-be-out df))
            (if (not (eq out ret)) (2remove-output out)))
           (T (push out found)))
     (go L)))

(defun dflow-source (in)
  (let ((source (or (~01 ~from (??& * dflow * in))
                    (~01 (??& * tinput * * in))
                    (and (eq (~keyword in) 'toutput) (~lower-io in)))))
    (cond (source (dflow-source source))
          ((and (??& (~segcase in) @seg 'funcall '5noop)
                (eq (~keyword in) 'output))
           (dflow-source (?& (~segcase in) input)))
          ((and (??& * role (&* (~seg~segcase in)) 'dfjoin)
                (eq (~keyword in) 'output))
           (let* ((join-in (?& (& (~seg~segcase in) 'body)
                               input 'id * (~obj in)))
                  (before-join-in (~01 ~from (??& * dflow * join-in))))
             (dflow-source
               (if (??& (~segcase before-join-in) plantype 'ssr-body)
                   (let ((rec (~subseg (?& (~segcase before-join-in)
                                           recursive))))
                     (?& rec input * (~obj before-join-in)))
                   before-join-in))))
          (T in))))

(defun arg-order (@io1 @io2)
  (and (~argno @io1) (~argno @io2) (< (~argno @io1) (~argno @io2))))

(defun simple-funcall-or-constant (seg)
  (and (memq (?@seg-kind seg) '(funcall constant))
       (??& (&* (&* seg) output) @io 'returnval)
       (null (cdr (??& (&* seg) output)))
       (null (??& (&* (&* seg) input) @io 'free-var))))
```

```
;;; >Finish {definition-name}.
;;;
;;;  Cleans up after having edited {definition-name}.


(defvar *at-end-noticers* nil "list of fns to call by s-end")

(defun add-at-end-noticer (fn)
  (if (not (memq fn *at-end-noticers*))
      (push fn *at-end-noticers*))
  fn)

(define-db-action Finish (editing
                          (definition-name (parse-toplevel-reference)))
                  "Cleans up after editing <definition-name>."
  (let ((code-list (parsed->code definition-name)) fixed-code)
    (when (not (string-equal (definition-type-name (car code-list))
                             "cliche"))
      (let ((ref (lisp-cursor-location code-list)))
        (if ref (complain "You haven't implemented ~A yet." ref)))
      (setq fixed-code (remove-markers (car code-list)))
      (when (and code-list (not (equal (car code-list) fixed-code)))
        (make-be-it (eval-command definition-name))
        (coder-confirmation (cadr fixed-code) fixed-code)))
    (dolist (fn *at-end-noticers*)
      (funcall fn (or fixed-code (car code-list))))))

(defun remove-markers (code)
  (cond ((atom code) code)
        ((eq (car code) 'kbe-/#-comment-macro)
         (when (not (cadr code))
           (barf "Bad code format: ~S" code))
         (remove-markers (car (cadr code))))
        (t
         (cons (remove-markers (car code))
               (remove-markers (cdr code))))))
```

```
;;; Local Modes:;
;;; Mode:LISP;
;;; Lisp WHEN Indent:1;
;;; Lisp REPEAT Indent:1;
;;; Lisp REPEAT* Indent:1;
;;; Lisp WITHOUT-END-OF-COMMAND Indent:0;
;;; End:;
```

```
;;; -*- Mode:LISP; Package:KBE; Base:10; Fonts: MEDFNB; -*-

;;; (c) Copyright 1983, Massachusetts Institute of Technology
;;;     All Rights Reserved.
;;;     Permission to copy all or part of this material is granted,
;;;     provided that the copies are not made or distributed for resale,
;;;     the MIT copyright notice and reference to the source file and
;;;     the software distribution version appear, and that notice is
;;;     given that copying is by permission of Massachusetts Institute
;;;     of Technology.
(comment
;;; the following stuff just exists as part of the Xplan test.
;;; it is not part of the running kbe.

(defun xplan-test (comment &optional (print-plan nil) (reps 6) &aux pas)
  (with-open-file (file "<pa.xplan>test-results.oplan" 'out)
    (print comment file)
    (print comment)
    (letS (((test-name times average-time seg)
            (xplan-test0 (Elist '(:copy :average-imp :enumerator-imp))
                         print-plan reps))
           (text (format nil
                         "~2%times for ~A in oplan in microseconds:~
                          ~{~%    ~8@{~:12D~}~}~
                          ~%Average of middle 50% of times: ~:12D~
                          ~2%Skeleton of the plan produced:~%"
                         test-name times average-time)))
      (multiple-value (nil pas) (construct-plan-def seg))
      (princ text)
      (pl pas)
      (princ text file)
      (pl pas file)))
  (values))

(defun xplan-test0 (test-name print-plan reps)
  (let* ((a-quarter (// reps 4))
         times seg middle-times average-time)
    (do ((i reps (1- i))) ((zerop i))
      (multiple-value-bind (s time) (xplan-test1 test-name)
        (push time times)
        (setq seg s)))
    (setq middle-times (sort (copylist times) #'lessp))
    (setq middle-times (nthcdr a-quarter middle-times))
    (setq middle-times (nthcdr a-quarter (reverse middle-times)))
    (setq average-time (// (apply #'+ middle-times) (length middle-times)))
    (if print-plan
        (with-open-file
            (file (format nil "<pa.xplan>~A-test.oplan" test-name) 'out)
          (format file
                  ";;; -*- Mode:LISP; Package:KBE; Fonts: MEDFNB; -*-~
                   ~2% ;plan resulting from ~A test~2%"
                  test-name)
          (multiple-value-bind (plan pas) (construct-plan-def seg)
            (pl pas file)
            (pl plan file))))
    (list test-name times average-time seg)))

(defun xplan-test1 (name)
  (xplan-test-setup)
  (let* ((start-time (time:fixnum-microsecond-time))
         (seg (caseq name
                (:copy (xplan-test-copy 'mydelete))
                (:average-imp
                  (xplan-test-implement
                    (~slotseg (?& '(mydelete) slot * 'test))
                    'average)
                  'mydelete)
                (:enumerator-imp
                  (xplan-test-implement
```

```
                    (~slotseg (?& '(mydelete) slot * 'enumerator))
                    'list-enumeration)
                  'mydelete)
                (T (barf "invalid test name"))))
          (end-time (time:fixnum-microsecond-time)))
    (values seg (time-difference end-time start-time))))

(defvar mydelete-plan-file nil "saves reading in the file multiple times")

(defun xplan-test-setup ()
  (when (null mydelete-plan-file)
    (with-open-file (file "<pa.xplan>mydelete.oplan" 'in)
      (setq mydelete-plan-file (read file))))
  (db-init)
  (eval mydelete-plan-file))

(defun xplan-test-copy (seg)
  (~seg (copyseg seg nil)))

(defun xplan-test-implement (seg cliche-name)
  (~seg (implement-internal seg (a*seg '(name cliche ,cliche-name)))))
) ;end of comment
```

```
;This stuff allows you to implement something as a reference to a
;port that you cannot make dflow to at first reference.

;it is absolutely essential that the top level value of
;this variable never be changed!
(defconst *pending-dflows* '*top-level*
  "holds dflows that cannot be handled till later")

(defun add-pending-dflow (entry)
  (if (eq *pending-dflows* '*top-level*)
      (barf "*pending-dflows* is not bound"))
  (if (null (eval entry))
      (push entry *pending-dflows*)))

;take care of the ones you can, and leave the rest.

(defun 2check-pending-dflows ()
  (if (eq *pending-dflows* '*top-level*)
      (barf "*pending-dflows* is not bound"))
  (let ((stuff *pending-dflows*))
    (setq *pending-dflows* nil)
    (mapc #'add-pending-dflow stuff)))
```

```
;;; This file contains the functions that support the
;;; verbs implement and reimplement.  as well as indefinite references
;;; which create an instance of a cliche.

(defvar 2ROLE-FILLERS nil "The things that fill in the roles in a cliche")

(defun (cliche-instance action) (name &rest 2ROLE-FILLERS)
  (if (eq *pending-dflows* '*top-level*)
      (let (*pending-dflows*)
        (prog1 (cliche-instance-action name)
               (if *pending-dflows* (complain "pending dflows remain"))))
      (cliche-instance-action name)))

(defun cliche-instance-action (name)
  (let* ((segcase (a*seg name))
         (parts (get (get (~clichename (?& segcase cliche))
                          'cliche-comments)
                     'roles)))
    (if (> (length 2role-fillers) (length parts))
        (barf "cliche-instance -- too many role-fillers"))
    (letS* ((filler (car (Elist 2ROLE-FILLERS)))
            (role-name (Elist parts)))
      (implement-internal0
        (~slotseg (?& segcase slot * role-name)) filler))
    (2check-constraints (~seg segcase))
    (ncons segcase)))

(defun a*seg (name)
  (check-name name)
  (caseq (name-type name)
    (cliche (let ((seg (2get-cliche (name-symbol name))))
              (if (not seg)
                  (& (create-cliche-for-function (name-symbol name)))
                  (& seg))))
    (program (& (create-cliche-for-function (name-symbol name))))
    (T (complain "~A not cliche or function name"
                 (unparse-command name)))))

;this gets hold of an instance of a cliche.  see defprototype
;to see how they get set up in the first place.
;Note that it does some variable renaming stuff based on suggestions.

;Note each kind of constraint either runs at creation time on the
;plan list (eg rename) or later on the plan db (eg default)

(defvar 2creation-time-plan nil)

(declare (special 2cliche-name))

(defun 2get-cliche (2cliche-name)
  (let ((2creation-time-plan (get 2cliche-name 'cliche-plan)))
    (when 2creation-time-plan
      (mapcar #'2check-constraint
              (get (get 2cliche-name 'cliche-comments)
                   'constraints))
      (~seg (eval 2creation-time-plan)))))
```

```
;note that this is pretty dangerous because it renames so much stuff.
;you must make sure that the thing you are renaming isn't some
;special keyword or anything.  This could be fixed better in principle
;however.  It assmes that the old-morphene is singular as is
;the new-morphene.  It finds every place the old-morphene occurs, and
;changes it to the new-morphene.  The old-morphene is said to occur
;anywhere it (or its plural form) exists in a name preceded by -, _,
;space, or the start of the name, and followed by -, _, space, or
;the end of the name.

(defun rename-all (old-morphene new-morphene)
  (when (and 2creation-time-plan
             (not (member new-morphene '(nil "nil" "nils"))))
    (let ((alist (2get-rename-alist
                  old-morphene new-morphene 2creation-time-plan)))
      (if alist (setq 2creation-time-plan
                      (ada-sublis alist 2creation-time-plan))))))

;this looks for every symbol with the old prefix and sets up
;an appropriate alist for renaming.  it assumes that everything
;is upper case at the moment

(declare (special 2rename-alist 2rename-old-singular 2rename-old-plural
                  2rename-new-singular 2rename-new-plural))

(defun 2get-rename-alist (old new stuff)
  (let* (2rename-alist
         (2rename-old-singular (string-upcase old))
         (2rename-old-plural (pluralize 2rename-old-singular))
         (2rename-new-singular (string-upcase new))
         (2rename-new-plural (pluralize 2rename-new-singular)))
    (2gra stuff)
    2rename-alist))

(defun pluralize (string)
  (or (cdr (assoc string '(("datum" . "data"))))
      (string-pluralize string)))

(defun 2gra (stuff)
  (cond ((and (or (symbolp stuff) (stringp stuff))
              (not (assq stuff 2rename-alist))) ;saves time and duplication
         (2gra0 stuff))
        ((listp stuff) (mapc #'2gra stuff))))

;this assumes that everything is upper case for now.

(defun 2gra0 (item)
  (let* ((string (string item))
         (parts)
         (parts-end 0)
         (start 0)
         end
         (slen (string-length 2rename-old-singular))
         (plen (string-length 2rename-old-plural))
         (len (string-length string)))
    (prog ()
      L (cond ((and (%string-equal
                      2rename-old-singular 0 string start slen)
                    (or (>= (+ start slen) len)
                        (memq (aref string (+ start slen))
                              '(#/- #/_ #\space))))
               (when (not (= start parts-end))
                 (push (substring string parts-end start) parts))
               (push 2rename-new-singular parts)
               (setq start (+ start slen))
               (setq parts-end start))
              ((and (%string-equal
                      2rename-old-plural 0 string start plen)
```

```
            (or (>= (+ start plen) len)
                (memq (aref string (+ start plen))
                      '(#/- #/_ #\space))))
        (when (not (= start parts-end))
          (push (substring string parts-end start) parts))
        (push 2rename-new-plural parts)
        (setq start (+ start plen))
        (setq parts-end start)))
(setq end (string-search-set "-_ " string start))
(when (null end)
  (when parts
    (when (not (= parts-end len))
      (push (substring string parts-end) parts))
    (let ((new-item (apply #'string-append (reverse parts))))
      (if (symbolp item)
          (setq new-item
                (maybe-intern new-item (symbol-package item))))
      (push (cons item new-item) 2rename-alist)))
  (return))
(setq start (1+ end))
(go L))))
```

```
;note this works on LISP or ADA.

(defun file-name-root (thing)
  (or (cond ((null thing) nil)
            ((stringp thing)
             (let* ((spot (string-search-set  "._-" thing))
                    (root
                      (intern (if spot
                                  (string-upcase (substring thing 0 spot))
                                  thing)
                              'kbe)))
               (or (singular-form root) root)))
            ((and (listp thing) (memq (car thing) '(quote str-lit)))
             (file-name-root (cadr thing)))
            ((and (listp thing) (equal (car thing) '(ada-string)))
             (file-name-root (string (cadr thing))))
            ((and (listp thing) (null (cdr thing))) ;maybe a segcase
             (file-name-root (car thing)))
            ;; These next two things are a gross cludge, but
            ;; there are lots of ways you could do it right by
            ;; either storing info with the cliche, or searching
            ;; over the plan.  I am just cheating for now.
            ((and (symbolp thing) (memq thing *db-segs*)
                  (??& (&* thing) cliche 'file_summarization))
             'summary)
            ((and (symbolp thing) (memq thing *db-segs*)
                  (??& (&* thing) cliche 'file_selection))
             'selection)
            ((and (symbolp thing) (memq thing *db-segs*))
             (file-name-root
               (or (~01 ~name (??& (&* thing) @seg 'constant))
                   (and (??& (&* thing) @seg 'funcall '5noop)
                        (~01 string ~name
                             (??& (&* (&* thing) input) @io
                                  'actual-arg * 1))))))
            ((symbolp thing) (file-name-root (string thing))))
      (*throw '2constraint-abort nil)))

;this creates a cliche for a function.

(declare (special 2NAME))

(defun create-cliche-for-function (symbol)
  (let* ((type (cond ((fboundp symbol) 'lisp)
                     ((eq (ada-symbol-type symbol) 'fn) 'ada)
                     (T (barf "~A not a function" symbol))))
         (arglist (caseq type
                    (lisp (arglist symbol))
                    (ada (mapcar #'(lambda (x ignore) x)
                                 '(x y z w)
                                 (do ((info ada-info (cdr info)))
                                     ((null info))
                                   (declare (special ada-info))
                                   (when (eq symbol (cadaar info))
                                     (return (caadar info))))))))
         (length (length arglist)))
    (cond ((or (> length 2)
               (memq '&rest (cdr arglist)) ; kludge -- ok if is first
               (memq '&quote arglist)
               (memq '&optional arglist))
           (barf "Cannot create cliche for arglist ~A" arglist)))
    (let* ((template (get (cond ((= length 0) 'zeroary-function)
                                ((= length 1) 'unary-function)
                                ((= length 2) 'binary-function))
                          'cliche-plan)))
      (~seg (eval (subst (caseq type
                           (lisp symbol)
                           (ada (intern (string-append "ADA-FN~" symbol))))
```

```
                'fn template))))))
```

```
;note that some constraints do not run at creation time.
;We must test to see that they don't

(defvar *2focus-seg* nil)
(defvar *supress-constraints* nil "block application of constraints")

(defun 2check-constraints (*2focus-seg*)
  (when (and (not *supress-constraints*) *2focus-seg*)
    (let ((cliche-name
           (~01 ~clichename (??& (&* *2focus-seg*) cliche))))
      (cond ((null cliche-name))
            ((eq cliche-name 'pseudo-subcliche)
             (2check-constraints
               (~01 ~seg ~segcase (??& * slot (&* *2focus-seg*)))))
            (T (mapcar #'2check-constraint
                       (get (get cliche-name 'cliche-comments)
                            'constraints)))))))

(defun 2check-constraint (constraint)
  (*catch '2constraint-abort
    (if (2check-should-eval constraint) (eval constraint))))

;this tries to see if you shouldn't even attempt the constraint.
;it is important because it can save a great deal of time.

(defun 2check-should-eval (constraint)
  (let* ((fn-name (car constraint))
         (first-arg (cadr constraint)))
    (when (eq fn-name 'ada-fn)
      (setq fn-name (caadr constraint))
      (setq first-arg (caddr constraint)))
    (cond (2creation-time-plan (not (memq fn-name '(default derived))))
          ((memq fn-name '(rename-all rename rename_all)) nil)
          ((eq fn-name 'default)
           (let ((segcase (eval first-arg)))
             (and segcase (not-implemented? (~seg segcase)))))
          (T T))))

(defun default (segcase source)
  (when (not 2creation-time-plan)
    (when (and segcase (not-implemented? (~seg segcase)))
      (2check-constraints
        (~seg (implement-seg (~seg segcase)
                             (~seg (plan-for-source source))))))))

(defun derived (segcase source)
  (when (not 2creation-time-plan)
    (let* ((current-code (car (?@seg-code (~seg segcase)))))
      (if (and (listp current-code) (eq (car current-code) '*code-is-car*))
          (setq current-code (cadr current-code)))
      (when (or (not-implemented? (~seg segcase))
                (not (equal current-code source)))
        (implement-seg (~seg segcase) (~seg (plan-for-source source)))))))

(defun plan-for-source (source)
  (if (and (listp source) (memq (car source) *db-segs*)) source
      (create-plan-for-code source)))
```

```
;note that all the fns you write to work in constraint things basically
;have to take in segments and produce either code or a chunk of plan.

;these are terrible cludges but work in simple cases.
;they just assume that every string with ~s in it in a hunk
;of code is a format string.

(defvar total-size 0)

(defun size-in-lines (segcase)
  (if (not segcase) (*throw '2constraint-abort nil))
  (let ((total-size 0))
    (cond ((not-implemented? (~seg segcase)) 1)
          (T (mapcar #'line-size-0 (all-subsegs (~seg segcase)))
             total-size))))

(defun line-size-0 (seg)
  (when (eq (?@seg-kind seg) 'constant)
    (let ((stuff (cadr (?@seg-name seg))))
      (if (stringp stuff) (line-size-1 stuff)))))

(defun line-size-1 (string)
  (do ((chars (nreverse (listarray string)) (cdr chars)))
      ((null chars))
    (when (member (car chars) '(#/% #/&))
      (setq total-size (+ total-size (ls-numeric-arg chars))))))

;this works pretty well (note digits are reversed already in the list)
(defun ls-numeric-arg (list)
  (let (digits)
    (dolist (char list)
      (if (= char #/~) (return nil))
      (if (member char '#.(listarray "0123456789"))
          (push char digits)))
    (if (null digits) (push #/1 digits))
    (readlist digits)))

(defun size-in-characters (segcase)
  (let ((total-size 0))
    (cond ((not-implemented? (~seg segcase)) 1)
          (T (mapcar #'char-size-0 (all-subsegs (~seg segcase)))
             total-size))))

(defun char-size-0 (seg)
  (when (eq (?@seg-kind seg) 'constant)
    (let ((stuff (cadr (?@seg-name seg))))
      (if (stringp stuff) (char-size-1 stuff)))))

;this is real weak - only works if length args specified in format.
(defun char-size-1 (string)
  (do ((chars (nreverse (listarray string)) (cdr chars)))
      ((null chars))
    (when (member (car chars) '#.(listarray "AaSsDdOoFfEeXx"))
      (setq total-size (+ total-size (ls-numeric-arg chars))))))
```

```
;these are some handy primitives for renaming things.

(defun 2rename-vars-avoiding (seg avoid-list)
  (let* ((names (2var-names-used seg))
         (name-clashes (qset^ names avoid-list)))
    (if name-clashes
        (2rename-var-names seg
           (letS ((clash (Elist name-clashes)))
             (Rlist (cons clash (2find-safe-name clash avoid-list))))))))

(defun 2var-names-used (seg)
  (makeqset (mapcan #'2var-names-used-seg (all-subsegs seg))))

(defun 2var-names-used-seg (seg)
  (let (names)
    (lets ((name (5goodname (~name (Elist (??& (&* (&* seg) port) @io))))))
      (if name (push name names)))
    (lets ((name
             (5goodname
               (~varname (Elist (??& (&* (&* seg) port) @createvar))))))
      (if name (push name names)))
    names))

(defun 2rename-var-names (seg alist)
  (Rignore (2rename-var-names-seg (Elist (all-subsegs seg)) alist)))

(defun 2rename-var-names-seg (seg alist)
  (lets* ((@ioF (Elist (??& (&* (&* seg) port) @io)))
          (name (5goodname (~name @ioF)))
          (new-name (cdr (assq name alist))))
    (when new-name
      (make-fact-be-out @ioF)
      (assert-fact (&~name @ioF new-name))))
  (lets* ((@createvarF (Elist (??& (&* (&* seg) port) @createvar)))
          (name (5goodname (~varname @createvarF)))
          (new-name (cdr (assq name alist))))
    (when new-name
      (make-fact-be-out @createvarF)
      (assert-fact (&~varname @createvarF new-name)))))

(defun 2find-safe-name (name avoid-list)
  (let ((root (substring name
                         0
                         (1+ (string-reverse-search-not-set
                               "~_0123456789" (string name))))))
    (do ((i 1 (1+ i)))
        (nil)
      (let ((new-name (let ((base 10.) (*nopoint t))
                        (maybe-intern (text "~A~A" root i)
                                      (symbol-package name)))))
        (if (not (memq new-name avoid-list))
            (return new-name))))))
```

```
;IMPLEMENT and REIMPLEMENT.

(define-db-action copy (to (seg-reference (parse-seg-reference))
                           (reference (parse-reference)))
                  "Copies the cliche <reference> to
                   the cliche <seg-reference>"
  (setq reference ;kludge converts the phrase to a phrase.
        '(cliche-instance (name cliche ,(caddr (caddr reference)))))
  (if (eq *pending-dflows* '*top-level*)
      (let (*pending-dflows*)
        (prog1
          (*implement (add-descriptor seg-reference 'unimplemented)
                      'like reference)
          (if *pending-dflows* (complain "pending dflows remain"))))
      (*implement (add-descriptor seg-reference 'unimplemented)
                  'like reference)))

(*define-completion 'as)
(*define-completion 'like)

(define-db-action implement ((seg-reference (parse-seg-reference))
                             (as-or-like (require-one-of '(as like)))
                             (reference (parse-reference)))
                  "Implements the segment given by <seg-reference>
                   <as//like> the code fragment given by <reference>."
  (if (eq *pending-dflows* '*top-level*)
      (let (*pending-dflows*)
        (prog1
          (*implement (add-descriptor seg-reference 'unimplemented)
                      as-or-like
                      reference)
          (if *pending-dflows* (complain "pending dflows remain"))))
      (*implement (add-descriptor seg-reference 'unimplemented)
                  as-or-like
                  reference)))

(defun *implement (old-ref relation new-ref)
  (let ((old (eval-command old-ref))
        (new (let ((*supress-constraints* (eq relation 'like)))
               (eval-command new-ref))))
    (let ((failures '()))
      (cond ((not old) (push "~0G~A is unknown to me" failures))
            ((or (cdr old)
                 (neq (thing-type (car old)) 'seg))
             (push "~0G~A is not a seg" failures)))
      (cond ((not new) (push "~1G~A is unknown to me" failures))
            ((cdr new)
             (push "~1G~A is not a single seg or port" failures)))
      (if failures
          (let ((unref1 (unparse-command old-ref))
                (unref2 (unparse-command new-ref)))
            (complain
              (auto-fill-string
                (text "Unable to implement ~A as ~A because ~A."
                      unref1 unref2
                      (phrase-append "and"
                                     (mapcar #'(lambda (x)
                                                 (text x unref1 unref2))
                                             failures))))))))
    (if (not (not-implemented? (~seg (car old))))
        (complain "~A was already implemented."
                  ;; Create a fully-qualified reference to the seg...
                  (string-capitalize
                    (unparse-command (make-ref (car old) t)))))
    (finish-edit <IMP
      (list (caseq relation
              (as (implement-internal (car old) (car new)))
              (like (implement-like (car old) (car new))))))))
```

```
(define-db-action reimplement ((seg-reference (parse-seg-reference))
                               (as-or-like (require-one-of '(as like)))
                               (reference (parse-reference)))
                  "Re-implements the segment given by <seg-reference> as
                   the code fragment given by <reference>."
  (if (eq *pending-dflows* '*top-level*)
      (let (*pending-dflows*)
        (prog1
          (*reimplement (add-descriptor seg-reference 'implemented)
                        as-or-like
                        reference)
          (if *pending-dflows* (complain "pending dflows remain"))))
      (*reimplement (add-descriptor seg-reference 'implemented)
                    as-or-like
                    reference)))

(defun *reimplement (old-ref relation new-ref)
  (let ((old (eval-command old-ref))
        (new (let ((*supress-constraints* (eq relation 'like)))
               (eval-command new-ref))))
    (let ((failures '()))
      (cond ((not old) (push "~0G~A is unknown to me" failures))
            ((or (cdr old)
                 (neq (thing-type (car old)) 'seg))
             (push "~0G~A is not a seg" failures)))
      (cond ((not new) (push "~1G~A is unknown to me" failures))
            ((cdr new)
             (push "~1G~A is not a single seg or port" failures)))
      (if failures
          (let ((unref1 (unparse-command old-ref))
                (unref2 (unparse-command new-ref)))
            (complain
              (auto-fill-string
                (text "Unable to reimplement ~A as ~A because ~A."
                      unref1 unref2
                      (phrase-append "and"
                                     (mapcar #'(lambda (x)
                                                 (text x unref1 unref2))
                                             failures))))))))
    (if (not-implemented? (~seg (car old)))
        (complain "~A is not yet implemented."
                  ;; Create a fully-qualified reference to the seg...
                  (string-capitalize
                    (unparse-command (make-ref (car old) t)))))
    (finish-edit <REIMP
      (list (caseq relation
              (as (implement-internal (car old) (car new)))
              (like (implement-like (car old) (car new))))))))
```

```
;this defines a top level thing by copying all of the
;plan for another thing over to it.  Only implementing
;like of cliches are support at the moment.

;currently must say "implement the cliche FOO like a bar."
;later maybe support "Implement the program FOO like the program BAR"

(defun implement-like (oldthing newthing)
  (let* ((cliche-def (~01 (--& oldthing cliche-def)))
         (new-cliche-name (~01 ~clichename (??& newthing cliche)))
         (new-comments (cdr (get new-cliche-name 'cliche-comments))))
    (if (not (and cliche-def new-cliche-name))
        (barf "can only IMPLEMENT a top level cliche LIKE another cliche"))
    (>& newthing cliche-def (~cliche-def-name cliche-def) new-comments)
    (kill-seg-and-its-subsegs (~seg oldthing))
    newthing))
```

```
;this fn allows you to define what the insides of a preexisting
;segment are.  It takes in two segments. it throws away everything
;inside the old original one, and puts everything from the second into
;it.  (note this destroys both of the inputs, producing the result.)
;The hard part is trying to reconcile the way the old seg is
;interfaced to the world, with the way the new one is.  The primary
;vehical for this is looking at the ports.

(defun implement-internal (oldthing newthing)
  (let* ((slot-superior (~01 ~seg ~segcase (??& * slot oldthing)))
         (result (implement-internal0 oldthing newthing)))
    (2check-constraints slot-superior)
    (2check-pending-dflows)
    result))

(defun implement-internal0 (oldthing newthing)
  (cond ((memq (thing-type newthing) '(input tinput output toutput))
         (implement-seg-as-port (~seg oldthing) newthing))
        ((not (eq (thing-type newthing) 'seg))
         (barf "can only implement something as a seg or port"))
        ((?supseg (~seg newthing))
         ;assume is a preexisting seg in same deffn and use dflow.
         (implement-seg-as-port (~seg oldthing)
                                (~deep (?& (&* newthing output)
                                           @io 'returnval))))
        (T (implement-seg (~seg oldthing) (~seg newthing)))))

(defun implement-seg-as-port (oldseg port)
  (let ((newseg (~seg (eval '(plan-def 5noop nil
                               (DEFFACTS 5NOOP
                                 (SEGTYPE (SIMPLE))
                                 (PLANTYPE (NEW-VIEW))
                                 (INPUT (NIL IN NIL))
                                 (OUTPUT (PART OUT IN))
                                 (@SEG (FUNCALL 5NOOP NIL))
                                 (@IO (IN ACTUAL-ARG NIL 1)
                                      (OUT RETURNVAL NIL NIL))))))))
    ;make sure no dflow hooked up to oldseg input
    (2remove*assertions (??& * dflow * (&* (&* oldseg) input)))
    (--& (&* (&* oldseg) input) @io)
    (--& (&* oldseg) input)
    (2imp oldseg newseg)
    (let* ((in (?& (&* newseg) input)))
      (add-pending-dflow '(implement-seg-as-port-dflow ',in ',port)))
    (& newseg)))

(defun implement-seg-as-port-dflow (in port)
  (when (equal (top-segcase (~segcase in)) (top-segcase (~segcase port)))
    (implement-dflow port in)
    T))
```

```
;This implements one segment as another.  The work done by this
;individual fn is to determine how to make a proper interface between
;the two segments.  Note that compatability of segments is implemented
;by using library cliches.  (something should be done so we don't
;loose the info from the ports on the oldseg when one of the
;intermediate cliches is used.)

;we make the following assumptions
; oldseg must be simple and cannot be a recursion, ssr-body, pred, or join.
; newseg must be simple and cannot be a recursion, ssr-body, pred, or join.
;  In addition it cannot be a lambda-exp, or ssr.

(defun implement-seg (oldseg newseg)
  (let* ((old-type (2imp-type oldseg))
         (new-type (2imp-type newseg))
         (seg))
    (if (or (eq old-type 'cannot-implement)
            (memq new-type '(cannot-implement ssr top)))
        (barf "IMPLEMENT-SEG: cannot use these segments in implement~
               ~% oldseg: ~S~% newseg: ~S"
               oldseg newseg))
    (caseq old-type
      (ssr ;this will only be if the coersion below happend already
        (cond ((eq (?plantype newseg) 'tempcomp)
               (setq seg (2imp oldseg newseg)))
              (t (setq seg (2imp oldseg (2get-cliche 'map)))
                 (2imp (?1role seg 'op) newseg))))
      (simple
        (if (??& (&* newseg) plantype 'tempcomp)
            (cond ((and (??& * role (&* oldseg) 'op))
                   (let ((ssr (~seg~segcase (?& * role (&* oldseg) 'op)))
                         (slotF (?& * slot (&* oldseg))))
                     (assert-fact (&~slotseg slotF (& ssr)))
                     (setq oldseg ssr))
                   (setq seg (2imp oldseg newseg)))
                  ((check-isnt-init oldseg (containing-tc oldseg))
                   (setq seg (implement-in-tc oldseg
                                              (containing-tc oldseg)
                                              newseg)))
                  ((check-isnt-init oldseg (refered-to-tc oldseg newseg))
                   (setq seg (implement-in-tc oldseg
                                              (refered-to-tc oldseg newseg)
                                              newseg)))
                  ;this is the only time a nested loop is formed.
                  (T (setq seg (2imp oldseg newseg))))
            (setq seg (2imp oldseg newseg))))
      (top ;this happens a lot.
        (cond ((and (??& (&* oldseg) cliche-def)
                    (??& (&* newseg) cliche))
               (setq seg (2imp-top oldseg (2get-cliche 'simple-subseg)))
               (--& (&* seg) cliche)
               (--& (&* seg) slot)
               (2imp (car (??subseg seg)) newseg))
              ((null (??subseg newseg))
               (setq seg (2imp-top oldseg (2get-cliche 'simple-subseg)))
               (2imp (car (??subseg seg)) newseg))
              (t (setq seg (2imp-top oldseg newseg))))))
      (& seg)))

;note that you cannot implement a cliche-def directly as a cliche because
;you couldn't tell which slots went with what.
```

```
(defun check-isnt-init (seg tc)
  (when tc
    (let ((supseg (?supseg seg)))
      (cond ((or (null supseg) (eq tc supseg)) T)
            ((and (eq (?plantype supseg) 'ssr)
                  (??& (&* supseg) role (&* seg) 'init)) nil)
            (T (check-isnt-init supseg tc))))))


;this deterimes type information needed by implement-seg

(defun 2imp-type (seg)
  (let* ((stype (?segtype seg))
         (ptype (?plantype seg))
         (@type (?@seg-kind seg))
         (role-name (~01 ~rolename (??& * role (&* seg)))))
    (cond ((or (not (eq stype 'simple))
               (memq ptype '(ssr-body join))
               (memq @type '(join recursion))
               (memq role-name '(join rec body dfjoin)))
           'cannot-implement)
          ((or (eq @type 'lambda-exp)
               (??& (&* seg) cliche-def))
           'top)
          ((or (eq ptype 'ssr)
               (memq role-name '(bssr gen acc trn term filter)))
           'ssr)
          (t 'simple))))
```

```
;this implements a topmost seg with another one.  The old ports are
;preserved in addition to any new ones.  Unless the new seg is also a top
;seg things are logically as if the new seg was made a subseg of the old
;one.  Note it is vital that the top seg itself maintains integrity.
;(this really should be hacked so that if one of the outputs of the newseg
;is a part of one of its inputs this fact carries over.)

(defun 2imp-top (oldseg newseg)
  (dolist (fact (seg-facts oldseg))
    (make-fact-be-out fact)
    (fact-key-dispatch fact
      ((input output tinput toutput @io @seg @createvar cliche-def)
       (make-fact-be-in fact))
      ((subseg (kill-seg-and-its-subsegs (~seg~subseg fact))))))
  (over (??& (&* newseg) input) -> in
        (~01 ~name (??& in @io)) -> name
    (2replace-port in
     (or (and name (~1st ~deep (??& (&* (&* oldseg) input) @io * name)))
         (let ((newport (>& (& oldseg) input nil
                            (genobj oldseg (~obj in)) nil)))
           (over (--& in @io) (assert-fact (&~deep _ newport)))
           newport)))
    (make-fact-be-out in))
  (over (??& (&* newseg) output) -> out
        (~01 ~name (??& out @io)) -> name
    (2replace-port out
     (or (and (??& out @io 'returnval)
              (~1st ~deep (??& (&* (&* oldseg) output) @io 'returnval)))
         (and name (~1st ~deep (??& (&* (&* oldseg) output) @io * name)))
         (let ((newport (>& (& oldseg) output nil
                            (genobj oldseg (~obj out)) nil)))
           (over (--& out @io) (assert-fact (&~deep _ newport)))
           newport)))
    (make-fact-be-out out))
  (dolist (fact (seg-facts newseg))
    (make-fact-be-out fact)
    (fact-key-dispatch fact
      ((segtype subseg dflow plantype role slot cliche)
       (assert-fact (&~segcase fact (& oldseg))))
      (@seg (when (eq (~kind fact) 'lambda-exp)
              (--& (&* oldseg) @seg)
              (assert-fact (&~segcase fact (& oldseg)))))
      (cflow (>& (& oldseg) cflow
                 (cond ((not (eq (~seg (~from fact)) newseg))
                        (~from fact))
                       (t (& oldseg (~case (~from fact)))))
                 (cond ((not (eq (~seg (~to fact)) newseg))
                        (~to fact))
                       (t (& oldseg (~case (~to fact)))))))))
  oldseg)
```

```
(defun containing-tc (seg)
  (let ((supseg (?supseg seg)))
    (cond ((null supseg) nil)
          ((eq (?plantype supseg) 'tempcomp) supseg)
          (T (containing-tc supseg)))))

;this just finds first one which is certainly buggy.

(defun refered-to-tc (oldseg newseg)
  (or (letS ((free-in
               (Elist (??& (&* (&* newseg) input) @io))) ;actual-arg too
             (in (~deep free-in))
             (name (~name free-in))
             (source (2find-var-source name oldseg 'before))
             (tc-seg (if source (containing-tc (~seg~segcase source)))))
        (if tc-seg (done tc-seg)))
      (letS ((pending (Elist *pending-dflows*)))
        (caseq (car pending)
          (implement-seg-as-port-dflow
            (let* ((in (eval (cadr pending)))
                   (port (eval (caddr pending)))
                   (tc (containing-tc (~seg~segcase port))))
              (if (and (2seg-inside-seg (~seg~segcase in) newseg) tc)
                  (done tc))))
          (2imp-input-dflow
            (let* ((in (eval (cadr pending)))
                   (name (eval (caddr pending)))
                   (source (2find-var-source name oldseg 'before))
                   (tc (if source
                           (containing-tc (~seg~segcase source)))))
              (if (and (2seg-inside-seg (~seg~segcase in) newseg) tc)
                  (done tc))))))))

(defun 2seg-inside-seg (low-seg high-seg)
  (and low-seg
       (or (eq low-seg high-seg)
           (2seg-inside-seg (?supseg low-seg) high-seg))))

(defun implement-in-tc (oldseg tc-seg newseg)
  (let* ((ssr-seg (genseg 'ssr-seg))
         (ret (~01 ~deep (??& (&* (&* newseg) output) @io 'returnval)))
         (out-slot (~01 ~slotseg (??& (&* newseg) slot * * 'output)))
         (out-ret
           (if out-slot
               (~01 ~deep (??& (&* out-slot output) @io 'returnval)))))
    (>& (& tc-seg) subseg (& ssr-seg))
    (>& (& tc-seg) role (& ssr-seg) 'acc)
    (2imp ssr-seg newseg)
    (~seg (implement-seg-as-port oldseg (or ret out-ret)))))

;this has a GROSS kludge in it a the moment for the demo, because I
;haven't figured out how to make it work in general.  It is the
;coder that should really be figuring this out.

(defun 2rename-vars-for-imp (newseg topseg)
  (let* ((newseg-names (2var-names-used-seg newseg))
         avoid-names)
    (dolist (pending *pending-dflows*)
      (if (and (eq (car pending) '2imp-input-dflow)
               (2seg-inside-seg
                 (~seg~segcase (eval (cadr pending))) newseg))
          (push (eval (caddr pending)) newseg-names)))
    (setq avoid-names (qset- (2var-names-used topseg) newseg-names))
    ;;here come da kludge.
    (setq avoid-names (qset^ '(sum) avoid-names))
    (if avoid-names
        (2rename-vars-avoiding newseg avoid-names))))
```

```
;this is the heart of the matter it replaces an inner seg with
;another.  the ports on the old one, and dflow touching them serve
;only to specify how to hook the new seg up, the old seg info is then
;just thrown away.

(defun 2imp (oldseg newseg)
 (let ((topseg (~seg (top-segcase (& oldseg))))
       (subsegs (--& * subseg (&* oldseg))))
;change variable names in newseg so that we won't confuse coder.
;note that variable names on the top of newseg are ok if they
;clash. we must do this after oldseg is out but before newseg is in.
  (2rename-vars-for-imp newseg topseg)
;connect newseg into parent.
  (over subsegs (assert-fact (&~subseg _ (& newseg))))
  (over (--& * slot (&* oldseg)) (assert-fact (&~slotseg _ (& newseg))))
  (over (--& * role (&* oldseg)) (assert-fact (&~roleseg _ (& newseg))))
  (over (--& * cflow * (&* oldseg)) (assert-fact (&~to _ (& newseg))))
  (over (--& * cflow (&* oldseg)) ;maybe bug if strange case names!
    (assert-fact (&~from _ (& newseg (~case (~from _))))))
;hook up the ports and slots of new.
  (letS* ((old-dummy (filterS #'dummy-port?
                              (Elist (??& (&* oldseg) toutput))))
          (new-dummy (2make-dummy-toutput newseg)))
    (2replace-port old-dummy new-dummy))
  (letS ((free-out (Elist (??& (&* (&* newseg) output) @io 'free-var)))
         (out (~deep free-out))
         (name (~name free-out))
         (old-out (and name (~1st ~deep (??& (&* (&* oldseg) output)
                                             @io 'free-var name)))))
    (cond (old-out
           (2replace-port old-out out)
           (make-fact-be-out old-out))))
  (letS ((se (Elist (??& (&* (&* newseg) output) @io 'side-effect)))
         (out (~deep se))
         (name (~name se))
         (old-out (2find-side-effect-out oldseg name)))
    (cond (old-out
           (2replace-port old-out out)
           (make-fact-be-out old-out))
          (T (2splice-into-parallel-dflows out name))))
  (let ((ret (~01 ~deep (??& (&* (&* newseg) output) @io 'returnval)))
        (old-ret (~01 ~deep (??& (&* (&* oldseg) output) @io 'returnval))))
    (cond ((and ret old-ret)
           (2replace-port old-ret ret)
           (make-fact-be-out old-ret))))
  ;;note we can hook up an output slot again even if it is used.
  (let* ((output-slotFs (??& (&* newseg) slot * * 'output))
         (out (sole-used-output? (?supseg newseg) oldseg)))
    (cond ((and out (null (??& out @io 'returnval))
                output-slotFs (null (cdr output-slotFs)))
           (letS ((to (Elist (~set ~to (??& * dflow out)))))
             (implement-dflow (?& (~slotseg (car output-slotFs)) output)
                              to))
           (make-fact-be-out out))))
```

```
;the next three things are needed so we can find ports for variables
  (assert-fact (&~code (-& (&* newseg) @seg)
                       (~01 ~code (??& (&* oldseg) @seg))))
  (--& (&* oldseg) cflow)
  ;we had to keep the dflow till found side-effect and free-var outputs.
  (--& (&* oldseg) dflow)
;we did the outputs first to help fix the position of newseg
  (letS* ((old-dummy (filterS #'dummy-port?
                              (Elist (??& (&* oldseg) tinput))))
          (new-dummy (2make-dummy-tinput newseg)))
    (2replace-port old-dummy new-dummy))
  (letS ((free-in (Elist (??& (&* (&* newseg) input) @io))) ;actual-arg too
         (in (~deep free-in))
         (name (~name free-in))
         (old-in (and name (or (~1st ~deep (??& (&* (&* oldseg) input)
                                                @io 'free-var name))
                               (2input-from-var oldseg name)))))
    (cond (old-in
           (2replace-port old-in in)
           (make-fact-be-out old-in))
          (name (add-pending-dflow '(2imp-input-dflow ',in ',name)))))
  (let* ((input-slotFs
          (Rlist (filterS #'(lambda (x)
                              (?empty-slot (~seg (~slotseg x))))
                  (Elist (??& (&* newseg) slot * * 'input)))))
         (in (sole-fed-input? (?supseg newseg) oldseg)))
    (cond ((and input-slotFs (null (cdr input-slotFs)) in)
           (let* ((slotseg (~slotseg (car input-slotFs)))
                  (new-in (>& slotseg input nil
                              (genobj (~seg slotseg) 'in) nil)))
             (>& new-in @io 'actual-arg '_ 1)
             (--& slotseg @seg)
             (>& slotseg @seg 'funcall '5noop nil)
             (implement-dflow (~from (-& * dflow * in)) new-in)
             (make-fact-be-out in)))))
  (2remove*assertions (??& * dflow * (&* (&* oldseg) port)))
  (2remove*assertions (??& * dflow (&* (&* oldseg) port)))
  (2remove*assertions (??& * tinput * * (&* (&* oldseg) port)))
  (2remove*assertions (??& * toutput * * (&* (&* oldseg) port)))
  (kill-seg-and-its-subsegs oldseg)
  newseg))
```

```
(defun 2imp-input-dflow (in name)
  (let* ((newseg (~seg~segcase in))
         (source (2find-var-source name newseg 'before))
         (top (top-segcase (& newseg))))
    (when (and (null source)
               (or (??& top @seg 'lambda-exp) (??& top cliche-def)))
      (setq source
            (~deep (>& (>& top input
                           nil (genobj (~seg top) name) nil)
                       @io 'free-var name nil))))
    (when source
      (implement-dflow source in)
      T)))

(defun 2find-side-effect-out (oldseg name)
  (if name
      (dolist (port (??output oldseg))
        (when (2side-effect-out port name)
          (return port)))))

(defun 2side-effect-out (port name)
  (or (??& port @io 'side-effect name)
      (let ((in (~01 (??& (&* (~seg~segcase port)) input * (~id port)))))
        (and (eq (~keyword port) 'output) (eq (~kind port) 'id) in
             (2ifv name in)))
      (let ((source (~01 ~from (??& * dflow * port))))
        (and source (2side-effect-out source name)))
      (and (eq (?segtype (~seg~segcase port)) 'join)
           (dolist (in (??& (&* (~seg~segcase port)) input
                            'id * (~obj port)))
             (when (2side-effect-out in name) (return port))))))

(defun 2splice-into-parallel-dflows (out name)
  out name nil) ;not implemented yet

(defun dummy-port? (port)
  (and (memq (~keyword port) '(tinput toutput))
       (or (null (~lower-io port))
           (dummy-port? (~lower-io port)))))

(defun 2make-dummy-toutput (seg)
  (if (not (eq (?plantype seg) 'ssr))
      (let* ((last-subseg (car (2brothers-before seg seg)))
             (sub-dummy (2make-dummy-toutput last-subseg))
             (port (>& (& seg) toutput (& last-subseg)
                       (genobj seg 'dummy) sub-dummy)))
        (>& port @io 'dummy nil nil)
        port)
      (let* ((rec (?1role seg 'rec))
             (lower-env (~from (?& * cflow * (&* rec))))
             (port (>& (& seg) toutput lower-env (genobj seg 'dummy) nil)))
        (>& port @io 'dummy nil nil)
        port)))

(defun 2make-dummy-tinput (seg)
  (if (not (eq (?plantype seg) 'ssr))
      (let* ((last-subseg
              (letS ((sub (Elist (reverse (2brothers-before seg seg)))))
                (if (memq (?plantype sub) '(ssr tempcomp)) (done sub))))
             (sub-dummy (2make-dummy-tinput last-subseg))
             (port (>& (& seg) tinput (& last-subseg)
                       (genobj seg 'dummy) sub-dummy)))
        (>& port @io 'dummy nil nil)
        port)
      (let* ((body (?1role seg 'body))
             (lower-env (~to (?& (& body) cflow (&* body))))
             (port (>& (& seg) tinput lower-env (genobj seg 'dummy) nil)))
        (>& port @io 'dummy nil nil)
```

```
        port)))

(defun sole-used-output? (supseg seg)
  (let ((ports (Reqset (~from (Elist (??& (&* supseg) dflow
                                          (&* (&* seg) port)))))))
    (cond ((and ports (null (cdr ports))) (car ports)))))

(defun sole-fed-input? (supseg seg)
  (let ((ports (Reqset (~to (Elist (??& (&* supseg) dflow
                                        * (&* (&* seg) port)))))))
    (cond ((and ports (null (cdr ports))) (car ports)))))
```

```
(defun 2input-from-var (seg name)
  (letS ((in (Elist (??input seg))))
    (if (2ifv name in) (done in))))

(defun 2ifv (name port)
  (if port
      (let ((var (2var-for-port port)))
        (cond ((eq var name) port)
              ((not (null var)) nil)
              ((2ifv name
                     (or (~01 ~from (??& * dflow * port))
                         (~01 (??& * tinput * * port))
                         (if (eq (~keyword port) 'toutput)
                             (~lower-io port)))))))))

(defun 2var-for-port (port)
  (or (~01 ~name (??& port @io))
      (~01 ~varname (??& port @createvar))
      (and (??& port @io 'returnval)
           (??& (~segcase port) @seg 'funcall '5noop)
           (2fed-input (car (??& (~segcase port) input)))
           (~01 ~name (??& (&* (~segcase port) input)
                           @io 'actual-arg * 1)))
      (~01 ~name (??& (&* * tinput * * port) @io 'free-var))
      (~01 ~name (??& (&* * toutput * * port) @io 'free-var))))

(defun 2fed-input (port)
  (and port
       (or (??& * dflow * port)
           (2fed-input (car (??& * tinput * * port))))))
```

```
;this looks over a plan and tries to find a near port before the
;specified seg which carries dflow in the indicated variable,
;The key problem is one of determining the appropriate ordering on nearby
;segments.  The order as last coded is used as a heuristic.  The argument
;START controls where the fn starts looking from.
;note it makes a top level port if it cannot find one.
;(Note this no longer allows you to use a slot name as if it were a
;variable.  However, it preferentially tries to find an output slot
;as the source for the specified variable.

(declare (special 2nearest-hit 2slot-hit))

(defun 2find-var-source (var seg start)
  (let (2nearest-hit 2slot-hit)
    (*catch '2fvs (2find-var-source1 var seg start))
    (or 2slot-hit 2nearest-hit)))

(defun 2fvs-test (var port)
  (when (eq (2var-for-port port) var)
    (if (null 2nearest-hit) (setq 2nearest-hit port))
    (when (2comes-from-output-slot port)
      (setq 2slot-hit port)
      (*throw '2fvs nil))))

(defun 2comes-from-output-slot (port)
  (when port
    (or (??& * slot (~segcase port) * 'output)
        (2comes-from-output-slot
          (or (~01 ~from (??& * dflow * port))
              (~01 (??& * tinput * * port))
              (if (eq (~keyword port) 'toutput)
                  (~lower-io port)))))))

(defun 2find-var-source1 (var seg start)
  (caseq start
    (at-front
      (dolist (in [!(??& (&* seg) input) !(??& (&* seg) tinput)])
        (2fvs-test var in))
      (2find-var-source1 var seg 'before))
    (at-end
     (dolist (out [!(??& (&* seg) output) !(??& (&* seg) toutput)])
       (2fvs-test var out))
     (2find-var-source1 var seg 'inside))
    (before
     (let ((supseg (?supseg seg)))
       (when supseg
         (over (2brothers-before supseg seg)
           (2find-var-source1 var _ 'at-end))
         (2find-var-source1 var supseg 'at-front))))
    (inside ;this is potentially buggy if several guys produce the var
     (over (2brothers-before seg seg)
       (2find-var-source1 var _ 'at-end)))))
```

```
;this gives every brother of the given seg which is not specificially
;constrained to be after the given seg.
;it returns a list in reverse order with the close ones first,
;and with all of the unordered ones at the tail.

(defun 2brothers-before (supseg seg)
  (let ((subsegs (??subseg supseg)) forced-after forced-before)
    (repeat ((to-do [seg]))
      (if (null to-do) (return nil))
      (over [!(~nset ~seg ~segcase ~to
                     (??& (&* supseg) dflow (&* (&* (top to-do)) port)))
             !(~nset ~seg ~to
                     (??& (&* supseg) cflow (&* (top to-do))))
             !(~nset ~seg~segcase ~lower-io
                     (??& (&* (pop to-do)) tinput))] ;if seg=supseg
            (cond ((and (memq _ subsegs) (not (memq _ forced-after)))
                   (push _ forced-after)
                   (push _ to-do)))))
    (repeat ((to-do [seg]))
      (if (null to-do) (return nil))
      (over [!(~nset ~seg ~segcase ~from
                     (??& (&* supseg) dflow * (&* (&* (top to-do)) port)))
             !(~nset ~seg ~from
                     (??& (&* supseg) cflow * (&* (top to-do))))
             !(~nset ~seg~segcase ~lower-io
                     (??& (&* (pop to-do)) toutput))] ;if seg=supseg
            (cond ((and (memq _ subsegs) (not (memq _ forced-before)))
                   (push _ forced-before)
                   (push _ to-do)))))
    (if (eq supseg seg)
        (if (> (length forced-before) (length forced-after))
            (nreconc forced-before
                     (reverse-order-by-dflow
                       (qset- subsegs forced-before)))
            (nconc forced-after
                   (reverse-order-by-dflow (qset- subsegs forced-after))))
        (nreconc forced-before
                 (reverse-order-by-dflow
                   (Rlist (filterS #'(lambda (x)
                                       (and (not (eq x seg))
                                            (not (memq x forced-before))
                                            (not (memq x forced-after))))
                                   (Elist subsegs)))))))))

;assumes segs are all in one supseg, and no loops in flow
;returns a list of them where no seg follows a seg it gets dflow from
;it works by repeatedly finding segs that don't get flow from anybody

(defun reverse-order-by-dflow (segs)
  (when segs
    (let (result (supseg (?supseg (car segs))))
      (repeat ((to-do (copy segs)))
        (if (null to-do) (return))
        (repeat ((seg (car to-do)))
          (when (not
                  (over [!(~nset ~seg ~segcase ~from
                                 (??& (&* supseg) dflow * (&* (&* seg) port)))
                         !(~nset ~seg ~from
                                 (??& (&* supseg) cflow * (&* seg)))]
                        (when (memq _ to-do) (setq seg _) (return T))))
            (push seg result)
            (setq to-do (delq seg to-do))
            (return))))
      result)))

;this fixes things so that everything about the old port
;now applies to the new one.
```

```
(defun 2replace-port (oldport newport)
  (over (--& * dflow * oldport) (assert-fact (&~to _ newport)))
  (over (--& * dflow oldport) (assert-fact (&~from _ newport)))
  ;this still isn't right if lower-env not simple.
  (over [!(--& * tinput * * oldport) !(--& * toutput * * oldport)]
        (2replace-port _ (>& (~segcase _) (~keyword _) (~segcase newport)
                             (~obj _) newport))))

;this just makes a lambda-exp into an open code thing.

(defun 2top->simple (seg)
  (--& (&* seg) @seg)
  (>& (& seg) @seg 'opencode nil nil)
  (over (--& (&* (&* seg) port) @io 'lambda-arg)
    (>& (~deep _) @io 'free-var (~name _) nil))
  seg)
```

```
;This is expected to get called a lot, so I try real hard here to
;be able to make a dflow between any two points. A key problem is
;deciding where we are going to have temporal dflow.
;2intermediate-steps does most of the hard work.

(defun implement-dflow (*from *to)
  (if (not (and *from *to))
      (barf "IMPLEMENT-DFLOW: must have from and to."))
  ;kill old dflow to destination
  (let* ((other-df (??& * dflow * *to)))
    (>& (~segcase *to) dflow *from *to) ;CLUDGE prevents deletion of to
    (2remove*assertions other-df)
    (--& * dflow *from *to))
  (let (interval)
    (letS (((fn arg) (Elist (2intermediate-steps *from *to))))
      (setq interval (funcall fn arg interval)))))

;This figures out what kinds of dflow has to be created in order
;to bridge the gap between these ports.  A big complexity comes
;in from needing to decide when temporal flow has to be used.
;Another problem is trying to find a good source for the dflow
;goven that we cannot in general convert a toutput into an output.
;The output is a list of steps to be performed in order.  each
;step is a function call to be evaluated.

(defun 2intermediate-steps (*from *to)
  (let ((steps (2intermediate-steps1 *from *to)))
    (cond ((listp steps) steps)
          ((eq steps 'failed-bad-non-ssr-from)
           (setq *from (2get-higher-from *from))
           (if (null *from) (barf "cannot make dflow ~:= to ~:=" *from *to))
           (2intermediate-steps *from *to))
          ((eq steps 'failed-bad-temporal-from)
           (2intermediate-steps (2get-better-from *from) *to))
          ((eq steps 'failed-bad-temporal-to)
           (2intermediate-steps *from (2get-better-to *to))))))

(defun 2get-higher-from (from)
  (let ((source (dflow-source from)))
    (if (not (equal source from)) source)))

;if *from is a port that we cannot make into a final value
;(and we need to) then this program tries to find some equivalent
;port that we can make into a final value.

(defun 2get-better-from (old-from)
  (let ((*from old-from))
    (if (repeat (did-something)
          (caseq (~keyword *from)
            (toutput (setq *from (~lower-io *from)))
            (tinput
              (let ((prior (~01 ~from (??& * dflow * *from))))
                (if (null prior) (return nil))
                (setq *from prior)))
            (input
              (if (eq (?plantype (~seg~segcase *from)) 'ssr-body)
                  (return did-something))
              (let ((upper (or (~01 (??& * tinput * * *from))
                               (~01 ~from (??& * dflow * *from)))))
                (if (null upper) (return nil))
                (setq *from upper)))
            (output
              (if (memq (?@seg-name (~seg~segcase *from))
                        '(5variable 5noop))
                  (setq *from (?& (&* (~seg~segcase *from)) input))
                  (return nil))))
          (setq did-something T))
        *from
```

```
        (force-to-final-value old-from))))

;this tries real hard to just bash something to a final value
;willy nilly.  it produces a plan that isn't really right,
;but the coder makes ok code anyway.

(defun force-to-final-value (*from)
  (let ((body (?supseg (~seg~segcase *from))))
    (if (not (and (eq (~keyword *from) 'output)
                  (eq (?plantype body) 'ssr-body)))
        (barf "2cannot coerce to final value"))
    (let* ((seg (?supseg body))
           (dfjoin (?1role seg 'dfjoin))
           (rec (?1role seg 'rec))
           (body-out (>& (& body) output nil (genobj body 'last) nil))
           (rec-out (>& (& rec) output nil
                        (genobj rec (~obj body-out)) nil))
           (dfjoin-out (>& (& dfjoin) output nil
                           (genobj dfjoin 'last) nil))
           (dfjoin-in-rec (>& (& dfjoin 'rec) input 'id
                              (genobj dfjoin 'last) (~obj dfjoin-out)))
           (dfjoin-in-body (>& (& dfjoin 'body) input 'id
                               (genobj dfjoin 'last) (~obj dfjoin-out))))
      (>& body-out @io 'free-var nil nil)
      (>& rec-out @io 'free-var nil nil)
      (>& dfjoin-out @io 'free-var nil nil)
      (>& dfjoin-in-rec @io 'free-var nil nil)
      (>& dfjoin-in-body @io 'free-var nil nil)
      (>& (& body) dflow *from dfjoin-in-body)
      (>& (& body) dflow rec-out dfjoin-in-rec)
      (>& (& body) dflow dfjoin-out body-out)
      body-out)))

;if *to is a port that we cannot make into an initial value
;(and we need to) then this program tries finds an equivalent
;port that we can make into an initial value.

(defun 2get-better-to (old-to)
  (let ((*to old-to))
    (if (repeat (did-something)
          (if (eq (~keyword *to) 'tinput)
              (setq *to (~lower-io *to))
              (return did-something))
          (setq did-something T))
        *to
        (barf "Cannot coerce input to initial-value ~:=" old-to))))
```

```
;note the cludgy way we just fail when we cannot make dflow
;from a temporal port.

(defun 2intermediate-steps1 (*from *to)
  (prog outer ()
    (let* ((from-seg (~seg~segcase *from))
           (to-seg (~seg~segcase *to))
           ((lca from-ancesters to-ancesters)
            (2intermediate-segs from-seg to-seg))
           (temporal-middle
            (and (eq (?plantype lca) 'tempcomp)
                 from-ancesters
                 (or (eq (?plantype (car from-ancesters)) 'ssr)
                     (and (eq (?plantype (car from-ancesters)) 'tempcomp)
                          (not (??& * role (&* (car from-ancesters))
                                    'init))))
                 to-ancesters
                 (or (eq (?plantype (car to-ancesters)) 'ssr)
                     (and (eq (?plantype (car to-ancesters)) 'tempcomp)
                          (not (??& * role (&* (car to-ancesters))
                                    'init))))))
           (steps '((2id-horizontal ,lca))))
      (if (or (eq (~keyword *from) 'tinput)
              (if from-ancesters
                  (and (eq (~keyword *from) 'input)
                       (not (eq (?plantype from-seg) 'ssr-body)))
                  (not (eq (~keyword *from) 'input)))
              (eq (~keyword *to) 'toutput)
              (if to-ancesters
                  (eq (~keyword *to) 'output)
                  (not (eq (~keyword *to) 'output))))
          (barf "impossible dflow ~:= -> ~:=" *from *to))
      (cond (temporal-middle ;plan outgoing temporal steps
             (repeat (seg)
               (cond ((null (cdr from-ancesters)) (return nil)))
               (setq seg (pop from-ancesters))
               (caseq (?plantype seg)
                 (tempcomp (push '(2id-temporal-outgoing ,seg) steps))
                 (ssr (push '(2id-temporal-outgoing ,seg) steps)
                      (if (cdr from-ancesters)  ;if port not on ssr-body
                          (pop from-ancesters)) ;skip ssr-body
                      (return nil))  ;done with temporal stuff
                 (T (return-from outer 'failed-bad-non-ssr-from))))
             (if (and (null (cdr from-ancesters))
                      (eq (?plantype from-seg) 'ssr)
                      (not (eq (~keyword *from) 'toutput)))
                 (barf "cannot make OUTPUT of ssr be TOUTPUT"))))
      (cond ((cdr from-ancesters) ;plan outgoing non temporal steps
             (if (eq (~keyword *from) 'toutput)
                 (return-from outer 'failed-bad-temporal-from))
             (repeat (seg)
               (cond ((null (cdr from-ancesters)) (return nil)))
               (setq seg (pop from-ancesters))
               (caseq (?plantype seg)
                 (ssr-body (return-from outer 'failed-bad-temporal-from))
                 (T (push '(2id-outgoing ,seg) steps))))
             (if (eq (?plantype from-seg) 'ssr-body)
                 (if (eq (~keyword *from) 'input)
                     (push '(2id-make-final-value ,from-seg) steps)))))
      (cond (temporal-middle ;plan ingoing temporal steps
             (repeat (seg)
               (cond ((null (cdr to-ancesters)) (return nil)))
               (setq seg (pop to-ancesters))
               (caseq (?plantype seg)
                 (tempcomp (push '(2id-temporal-ingoing ,seg) steps))
                 (ssr (push '(2id-temporal-ingoing ,seg) steps)
                      (if (cdr to-ancesters)  ;if port not on ssr-body
                          (pop to-ancesters)) ;skip ssr-body
```

```
                (return nil))  ;done with temporal stuff
            (T (barf "cannot make input of non-ssr be toutput"))))
        (if (and (null (cdr to-ancesters))
                 (eq (?plantype to-seg) 'ssr)
                 (not (eq (~keyword *to) 'tinput)))
            (barf "cannot make INPUT of ssr be TINPUT"))))
(cond ((cdr to-ancesters) ;plan ingoing non temporal steps
       (if (eq (~keyword *to) 'tinput)
           (return-from outer 'failed-bad-temporal-to))
       (repeat (seg)
         (cond ((null (cdr to-ancesters)) (return nil)))
         (setq seg (pop to-ancesters))
         (caseq (?plantype seg)
           (ssr-body (push '(2id-make-initial-value ,seg) steps))
           (T (push '(2id-ingoing ,seg) steps))))))
(push '(2id-initial-interval (,*from ,*to)) steps)
(return-from outer steps))))
```

```
;Finds all of the intervening segment boundaries encountered when
;going from the first seg to the second.  It works by first finding
;the lowest common ancestor (it assumes that there is one).
;Note the seg itself is listed as the closest ancester of itself.

(defun 2intermediate-segs (fromseg toseg)
  (let* ((from-ancestors (2ancestors fromseg))
         (to-ancestors (2ancestors toseg))
         (lca))
    (repeat ()
      (cond ((not (eq (car from-ancestors) (car to-ancestors)))
             (return nil)))
      (setq lca (pop from-ancestors))
      (pop to-ancestors))
    (if (null lca)
        (barf "2intermediate-segs: ~S and ~S not in same plan"
              fromseg toseg))
    [lca from-ancestors to-ancestors]))

(defun 2ancestors (seg)
  (repeat (result)
    (push seg result)
    (setq seg (?supseg seg))
    (if (null seg) (return result))))
```

```
;this is a litte dummy step to get things started right.
(defun 2id-initial-interval (interval ignore)
  (list (car interval) (cadr interval)))

(defun 2id-outgoing (to-seg interval)
  (let (((*from *to) interval))
    (if (not (and (eq (~keyword *from) 'output)
                  (eq (?supseg (~seg~segcase *from)) to-seg)
                  (not (eq (?plantype to-seg) 'ssr-body))))
        (barf "2id-outgoing"))
    (let ((port (car (~set ~to (??& (&* to-seg) dflow *from
                                    (&* (&* to-seg) output))))))
      (when (null port)
        (setq port (>& (& to-seg) output nil
                       (genobj to-seg 'obj) nil))
        (>& port @io 'free-var '_ nil)
        (>& (& to-seg) dflow *from port))
      (list port *to))))

(defun 2id-temporal-outgoing (to-seg interval)
  (let (((*from *to) interval))
    (if (not (and (memq (?plantype to-seg) '(tempcomp ssr))
                  (or (and (memq (~keyword *from) '(input toutput))
                           (eq (?supseg (~seg~segcase *from)) to-seg))
                      (and (eq (~keyword *from) 'output)
                           (eq (?supseg (?supseg (~seg~segcase *from)))
                               to-seg)))))
        (barf "2id-temporal-outgoing"))
    (let ((port (~01 (??& (&* to-seg) toutput (~segcase *from) * *from))))
      (when (null port)
        (setq port (>& (& to-seg) toutput (~segcase *from)
                       (genobj to-seg 'obj) *from))
        (>& port @io 'free-var '_ nil))
      (list port *to))))

;[Note that the OUTPUTs on segs inside an SSR-BODY cannot be routed outside
;  as final values because we cannot guarrantee that they will exist.]

(defun 2id-make-final-value (body interval)
  (let (((*from *to) interval))
    (if (not (and (eq (~keyword *from) 'input)
                  (eq (~seg~segcase *from) body)
                  (eq (?plantype body) 'ssr-body)))
        (barf "2id-make-final-value"))
    (let* ((seg (?supseg body))
           (dfjoin (?1role seg 'dfjoin))
           (rec (?1role seg 'rec))
           (body-out (>& (& body) output nil (genobj body 'last) nil))
           (rec-out (>& (& rec) output nil
                        (genobj rec (~obj body-out)) nil))
           (dfjoin-out (>& (& dfjoin) output nil
                           (genobj dfjoin 'last) nil))
           (dfjoin-in-rec (>& (& dfjoin 'rec) input 'id
                              (genobj dfjoin 'last) (~obj dfjoin-out)))
           (dfjoin-in-body (>& (& dfjoin 'body) input 'id
                               (genobj dfjoin 'last) (~obj dfjoin-out))))
      (>& body-out @io 'free-var nil nil)
      (>& rec-out @io 'free-var nil nil)
      (>& dfjoin-out @io 'free-var nil nil)
      (>& dfjoin-in-rec @io 'free-var nil nil)
      (>& dfjoin-in-body @io 'free-var nil nil)
      (>& (& body) dflow *from dfjoin-in-body)
      (>& (& body) dflow rec-out dfjoin-in-rec)
      (>& (& body) dflow dfjoin-out body-out)
      (list body-out *to))))
```

```
(defun 2id-ingoing (from-seg interval)
  (let (((*from *to) interval))
    (if (not (and (eq (~keyword *to) 'input)
                  (eq (?supseg (~seg~segcase *to)) from-seg)
                  (not (eq (?plantype from-seg) 'ssr-body))))
        (barf "2id-ingoing"))
    (let ((port (car (~set ~from (??& (&* from-seg) dflow
                                      (&* (&* from-seg) input) *to)))))
      (when (null port)
        (setq port (>& (& from-seg) input nil
                       (genobj from-seg 'obj) nil))
        (>& port @io 'free-var '_ nil)
        (>& (& from-seg) dflow port *to))
      (list *from port))))

(defun 2id-temporal-ingoing (from-seg interval)
  (let (((*from *to) interval))
    (if (not (and (memq (?plantype from-seg) '(tempcomp ssr))
                  (or (and (eq (~keyword *to) 'tinput)
                           (eq (?supseg (~seg~segcase *to)) from-seg))
                      (and (eq (~keyword *to) 'input)
                           (eq (?supseg (?supseg (~seg~segcase *to)))
                               from-seg)))))
        (barf "2id-temporal-ingoing"))
    (let ((port (~01 (??& (&* from-seg) tinput (~segcase *to) * *to))))
      (when (null port)
        (setq port (>& (& from-seg) tinput (~segcase *to)
                       (genobj from-seg 'obj) *to))
        (>& port @io 'free-var '_ nil))
      (list *from port))))

;[Note we do not have the kind of problem we had with final-value.]
(defun 2id-make-initial-value (body interval)
  (let (((*from *to) interval))
    (if (not (and (eq (~keyword *to) 'input)
                  (eq (?supseg (~seg~segcase *to)) body)
                  (eq (?plantype body) 'ssr-body)))
        (barf "2id-make-initial-value"))
    (let* ((seg (?supseg body))
           (rec (?1role seg 'rec))
           (body-in (>& (& body) input nil (genobj body 'init) nil))
           (rec-in (>& (& rec) input nil
                       (genobj rec (~obj body-in)) nil)))
      (>& body-in @io 'free-var nil nil)
      (>& rec-in @io 'free-var nil nil)
      (>& (& body) dflow body-in *to)
      (>& (& body) dflow body-in rec-in)
      (list *from body-in))))

(defun 2id-horizontal (lca interval)
  (let (((*from *to) interval))
    (>& (& lca) dflow *from *to)
    nil))
```

```
;something should be added here so that if you remove a seg that is
;a lot (or contains a slot or has a brother that is a slot) then the
;cliche instance with these slots gets erased out of the plan since it
;isn't accurate any more.

(defun 2remove-seg (thing)
  (when (not (and (??& thing segtype 'simple)
                  (memq (~type (?& thing plantype))
                        '(EXPR AND COMP SSR TEMPCOMP CONSTANT NEW-VIEW))
                  (memq (~kind (?& thing @seg))
                        '(OPENCODE FUNCALL CONSTANT))))
    (barf "2REMOVE-SEG: Cannot remove seg.~% Thing: ~:=" thing))
  (letS ((se (Elist (??& (&* thing output) @io 'side-effect)))
         (out (~deep se))
         (in (?& thing input * (~id out)))
         (in-dflow (-& * dflow * in))
         (out-dflow (-& * dflow out)))
    (>& (~segcase in-dflow) dflow (~from in-dflow) (~to out-dflow)))
  (2remove*assertions (??& * dflow * (&* (&* (~seg thing)) port)))
  (2remove*assertions (??& * dflow (&* (&* (~seg thing)) port)))
  (let ((cflow (~01 (--& * cflow * thing))))
    (if cflow (assert-fact (&~to cflow (~to (-& * cflow thing))))))
  (kill-seg-and-its-subsegs (~seg thing)))

;;; This actually removes assertions from the database.

(defun 2remove*assertions (things)
  (dolist (thing things)
    (cond ((not (fact? thing))
           (2remove-seg     thing))
          (T (caseq (~keyword thing)
               (dflow   (2remove-dflow   thing))
               (input   (2remove-input   thing))
               (output  (2remove-output  thing))
               (tinput  (2remove-tinput  thing))
               (toutput (2remove-toutput thing))
               (T (barf "2REMOVE*ASSERTIONS: cannot remove~% Thing: ~:="
                  thing)))))))

(defun 2remove-dflow (thing)
  (erase-facts thing)
  (if (and (find-facts (~to thing))
           (null (??& * dflow * (~to thing)))
           (null (??& * tinput * * (~to thing))))
      (cond ((and (input-fact? (~to thing))
                  (memq (?plantype (~seg~segcase (~to thing)))
                        '(EXPR AND COMP TEMPCOMP PRED XOR)))
             (2remove*assertions [(~to thing)])))
  (if (and (find-facts (~from thing))
           (null (??& * dflow (~from thing)))
           (null (??& * toutput * * (~from thing))))
      (cond ((and (input-fact? (~from thing))
                  (memq (?plantype (~seg~segcase (~from thing)))
                        '(EXPR AND COMP TEMPCOMP PRED XOR))
                  (null (??& (&* (~seg~segcase (~from thing)))
                             @seg 'lambda-exp)))
             (2remove*assertions [(~from thing)])))))

(defun 2remove-toutput (thing)
  (--& thing @io)
  (erase-facts thing)
  (2remove*assertions (??& * toutput * * thing))
  (2remove*assertions (??& * dflow thing)))

(defun 2remove-tinput (thing)
  (--& thing @io)
  (erase-facts thing)
  (2remove*assertions (??& * dflow * thing))
```

```
(2remove*assertions (??& * toutput * * thing))
(2remove*assertions (??& * tinput * * thing))
(if (eq (?@seg-kind (~seg~segcase (~lower-io thing))) 'opencode)
    (2remove*assertions [(~lower-io thing)])))
```

```
(defun 2remove-input (thing)
  (let* ((segcase (~segcase thing))
         (other-ports (cond ((~id thing)
                             (??& segcase output * (~id thing)))
                            (T
                             (??& segcase output 'id * (~obj thing))))))
    (let ((@io (~01 (??& thing @io))))
      (if (and @io (memq (~kind @io) '(lambda-arg actual-arg)))
          (dolist (other (??& (&* segcase input) @io 'lambda-arg))
            (when (> (~argno other) (~argno @io))
              (assert-fact (&~argno (erase-fact other)
                                    (1- (~argno other))))))))
    (2remove*assertions (??& * dflow * thing))
    (2remove*assertions (??& * dflow thing))
    (2remove*assertions (??& * toutput * * thing))
    (2remove*assertions (??& * tinput * * thing))
    (--& thing @io)
    (--& thing @createvar)
    (erase-facts thing) ;note this must be first to avoid looping!
    (let ((rec (~01 ~subseg (??& segcase recursive)))
          (sup (~01 ~segcase (??& * recursive segcase))))
      (if rec (setf other-ports
                    (append (??& rec input * (~obj thing)) other-ports)))
      (if sup (setf other-ports
                    (append (??& sup input * (~obj thing)) other-ports))))
    (2remove*assertions other-ports)))

(defun 2remove-output (thing)
  (let* ((segcase (~segcase thing))
         (other-ports (cond ((~id thing) (??& segcase input * (~id thing)))
                            (T (??& segcase input 'id * (~obj thing))))))
    (2remove*assertions (??& * dflow * thing))
    (2remove*assertions (??& * dflow thing))
    (2remove*assertions (??& * toutput * * thing))
    (2remove*assertions (??& * tinput * * thing))
    (--& thing @io)
    (--& thing @createvar)
    (erase-facts thing) ;note this must be first to avoid looping!
    (let ((rec (~01 ~subseg (??& segcase recursive)))
          (sup (~01 ~segcase (??& * recursive segcase))))
      (if rec (setq other-ports
                    (append (??& rec output * (~obj thing)) other-ports)))
      (if sup (setq other-ports
                    (append (??& sup output * (~obj thing)) other-ports))))
    (2remove*assertions other-ports)))
```

```
;;; Local Modes:;
;;; Mode:LISP;
;;; Lisp WHEN Indent:1;
;;; Lisp REPEAT Indent:1;
;;; Lisp REPEAT* Indent:1;
;;; End:;
```

```
;; -*- Mode:LISP; Package:KBE; Base:10; Fonts:MEDFNB; -*-

;;; (c) Copyright 1983, Massachusetts Institute of Technology
;;;     All Rights Reserved.
;;;     Permission to copy all or part of this material is granted,
;;;     provided that the copies are not made or distributed for resale,
;;;     the MIT copyright notice and reference to the source file and
;;;     the software distribution version appear, and that notice is
;;;     given that copying is by permission of Massachusetts Institute
;;;     of Technology.

;;; Support for letting the apprentice hack a small subset of ADA.
;;; this implements the various pseudo-lisp forms.

; in this implementation every ada value is a cons of its type and
; its value.

(defmacro ada-value-type (x) '(car ,x))
(defmacro ada-value-data (x) '(cadr ,x))
(defmacro ada-value-build (type value) '(list ,type ,value))

;The declarations create initial values for many things.
;setq (and hence output args) are handled by RPLACDing the values.

;the following is used in the support of USE.
;we don't do anything with WITH at the moment.  everything is WITHed.

(defvar ada-use-packages nil "full contour names USEd")
(defvar ada-prefix-stack nil "full name of current contour")

(defun ada-prefix-it (name) (cons name (car ada-prefix-stack)))

(defun ada-add-prefix (name)
  (push (ada-prefix-it name) ada-prefix-stack))

(defun ada-use (&quote &rest names)
  (mapc #'ada-add-use names)
  nil)

(defun ada-add-use (name)
  (push (cdar (ada-find 'package name T))
        ada-use-packages))
```

```
;the following DB shows what we know about each root name.  each entry
;has the form ((type root-name . qualifiers) . data).  The first part
;of the entry is called a TOKEN as is the retrieval key.  Several fns
;support looking up with a partial key.  All entries are on one huge
;ASSOC list ADA-INFO.  This alist is never bound below, but rather just
;keeps accumulating information.  This is done so that you can always
;get access to everything.  (You can bind it if you want to add some
;temporary information.)  Special care is taken so that if you
;redefine something this does not cause excess junk on ada-info.

;The following table shows the various types and what data goes with
;them.  Note that even if there is no data, the type itself and the
;qualifiers are often of interest.

;     object                      type            data
;function/operator/procedure     FN          alist of arg types and fn bodies
;generic package or function     GENERIC     parameter list and list of templates
;data type                       TYPE        the type defining form
;package name                    PACKAGE     the use list for the package body.
;exception name                  EXCEPTION   none
;variable name                   VARIABLE    none
;record field name               FIELD       none
;enumeration element             ENUM        none

(defvar ada-info nil "info on all ada identifiers")
```

```
;these functions are used for retrieval from ada-info in this file.
;for the basic get and put, we assume that a fully qualified token is
;supplied

(defun ada-find-fast (type name signal-error)
  (or (assoc (cons type name) ada-info)
      (if signal-error (barf "cannot find ~a" (cons type name)))))

(defun ada-store-fast (type name data)
  (let* ((token (cons type name))
         (spot (assoc token ada-info)))
    (cond (spot (rplacd spot data))
          (T (push (cons token data) ada-info)))))

;this finds the info associated with the supplied token following USE
;information.  Note it assumes that the order of uses is irrelevant.
;any hiding must be supported by order of definition.

(defun ada-find (type name signal-error)
  (or (car (ada-find-1 type name nil))
      (if signal-error (barf "cannot find ~A" (cons type name)))))

;this finds the first hit and can continue on.

(defun ada-find-1 (type name spot)
  (do ((entries (or spot ada-info) (cdr entries)))
      ((null entries) nil)
    (if (and (eq (car name) (cadaar entries))  ;test most likely to fail
             (eq type (caaar entries))
             (do ((ns (cdr name) (cdr ns))
                  (es (cddr (caar entries)) (cdr es)))
                 ((null es) (null ns))
               (cond ((null ns)
                      (return
                        (not (null (or (member es ada-prefix-stack)
                                       (member es ada-use-packages))))))
                     ((not (eq (car ns) (car es))) (return nil)))))
        (return entries))))
```

```
; this supports overloading on the assumption that you can tell what a
;thing refers to just from looking at it (and its arguments) (but
;never at how it is used).  This works out pretty well for
;most things (eg fns types).
; It fails badly for literal things like numbers, strings, and
;enumeration literals.  For numbers and strings we just assume that
;they will match up ok with any numeric (or string) parameter without
;trouble.  this is much less restrictive than ada but works fine.
; For enumeration literals we simply assume that the curresponding
;type is USEd and thaat there is no prefix on the literal and that the
;basic name is unique enough.  we don't check that it is in the
;correct type or anything.  this is pretty gross but works in the
;programs being demonstrated.
; we handle variable names and exception names the same simple way as
;enumeration literals even though you could figure them out pretty
;will by themselves.  It just that it is much simpler this way and
;works fine in our examples.
; Note all this implies that explicit prefixes can only appear on fns
;and types.  Everthing else must be globally unique namewise.

;this is a pretty gross cludge in that it assumes that
;the args to a fn can be evaled twice without harm!
;this is true in our demo, and saves a great deal of time.

(defmacro ada-fn (fn . args)
  (cons (cdr (ada-find-right-fn fn
               (mapcar #'(lambda (a) (ada-value-type (eval a))) args)
               nil))
        args))

(defmacro ada-attribute (var-or-type attribute)
  (let ((type (cond ((not (symbolp var-or-type)) var-or-type)
                    (T (ada-value-type (eval var-or-type))))))
    (cons (cdr (ada-find-right-fn (ncons attribute) (ncons type) nil))
          '',type)))

(defmacro ada-convert (type arg)
  '(ada-value-build ',(ada-find-true-type type) (ada-value-data ,arg)))

;You must always use a constructor name when making a record, because
;of the way we do overload resolution.  this has to do copying on the
;values because of the way ada-setq happens by side-effect.

(defmacro ada-construct (type . args)
  '(ada-construct-int ',(ada-find-true-type type) ., args))

(defun ada-construct-int (record-type &rest values)
  (ada-value-build record-type
                   (mapcar #'(lambda (v f)
                               (ada-value-build (cadr f)
                                                (ada-value-data v)))
                           values
                           (cddr (ada-find-fast 'type record-type T)))))
```

```
;this supports accessing of records.  Note the way it
;uses the stored type information.

(defmacro ada-select (var field-name)
  (let* ((record-dcl
          (cdr (ada-find-fast 'type (ada-value-type (eval var)) T))))
    (do ((i 0 (1+ i))
         (field-info (cdr record-dcl) (cdr field-info)))
        ((null field-info) (barf "bad record access"))
      (if (eq field-name (caar field-info))
          (return '(nth ,i (ada-value-data ,var)))))))

;returns cons of full fn name and internal name.

(defun ada-find-right-fn (fn type-list spot)
  (let* ((hit (ada-find-1 'fn fn spot))
         (internal-name
          (do ((alist (cdar hit) (cdr alist)))
              ((null alist) nil)
            (if (ada-types-match type-list (caar alist))
                (return (cdar alist))))))
    (cond (internal-name (cons (cdaar hit) internal-name))
          ((null (cdr hit)) (barf "cannot find fn ~a" fn))
          (T (ada-find-right-fn fn type-list (cdr hit))))))
(defun ada-find-true-type (type)
  (cond ((memq (car type) '(ada-string ada-exception ada-enumeration))
         type)
        ((eq (car type) 'ada-range)
         (cond ((cadr type)
                '(ada-range ,(ada-find-true-type (cadr type))
                            .,(cddr type)))
               (T type)))
        ((eq (car type) 'ada-rec)
         (cons 'ada-rec
               (mapcar #'(lambda (p)
                           (list (car p) (ada-find-true-type (cadr p))))
                       (cdr type))))
        ((memq (car type) '(any private limited-private))
         type)
        (t (or ;kludge for now so cliches can parse easier.
               (ignore-errors (cdar (ada-find 'type type T)))
               (and (string-search "_type" (string (car type)))
                    type)))))
```

```
;We treat NEW types just like subtypes.

;we have the following basic type guys
; ANY, PRIVATE, and LIMITED-PRIVATE.
; any symbol which is a TYPE on ada-info

;built in type constructors
; (ada-range {<type>} first last)
; (ada-enumeration . literals)
; (ada-string first last)
; (ada-rec . <field-name-<type>-pairs>)
; (ada-exception {superior exception})

(defun ada-type-match (arg-type def-type)
  (cond ((eq (car def-type) 'any) T)
        ((memq (car arg-type) '(ada-enumeration ada-exception ada-rec))
         nil)
        ((eq (car arg-type) 'ada-range)
         (ada-type-match (or (cadr arg-type) '(integer)) def-type))
        ((eq (car arg-type) 'ada-string) (equal def-type '(ada-string)))
        ((memq (car arg-type) '(nil private limited-private)) nil)
        ((equal arg-type def-type) T)
        ((memq (car arg-type)
               '(integer-literal string-literal enumeration-literal))
         (eq (car arg-type) (ada-literal-class def-type)))
        (T (ada-type-match (cdr (ada-find-fast 'type arg-type T))
                           def-type))))

(defun ada-literal-class (type)
  (cond ((eq (car type) 'ada-range)
         (ada-literal-class (or (cadr type) '(integer))))
        ((eq (car type) 'ada-string) 'string-literal)
        ((eq (car type) 'ada-enumeration) 'enumeration-literal)
        ((memq (car type) '(exception record)) nil)
        ((eq (car type) 'integer) 'integer-literal)
        ((memq (car type) '(nil any private limited-private))
         nil)
        (T (ada-literal-class (cdr (ada-find-fast 'type type T))))))

(defun ada-types-match (arg-type-list def-type-list)
  (and (= (length arg-type-list) (length def-type-list))
       (do ((arg-types arg-type-list (cdr arg-types))
            (def-types def-type-list (cdr def-types)))
           ((null arg-types) T)
         (if (not (ada-type-match (car arg-types) (car def-types)))
             (return nil)))))
```

```
(defun ada-type-dcl (&quote &rest body)
  (ada-do-subtype-dcl (car body) (cadr body)))

(defun ada-subtype-dcl (&quote &rest body)
  (ada-do-subtype-dcl (car body) (cadr body)))

(defun ada-do-subtype-dcl (subtype type)
  (let* ((true-subtype (ada-prefix-it subtype))
         (true-type (ada-find-true-type type)))
    (caseq (car true-type)
      (ada-rec
       (ada-store-fast 'type true-subtype true-type)
       (mapcar #'(lambda (x)
                   (ada-store-fast 'field (ada-prefix-it (car x)) nil))
               (cdr true-type)))
      (ada-enumeration
       (ada-store-fast 'type true-subtype true-type)
       (mapcar #'(lambda (x)
                   (ada-store-fast 'enum (ada-prefix-it x) nil))
               (cdr true-type)))
      (ada-exception (ada-store-fast 'exception true-subtype nil))
      (T (ada-store-fast 'type true-subtype true-type)))
    nil))

(defun str-lit (&quote &rest body)
  (ada-value-build '(string-literal) (car body)))
(defun enum-lit (&quote &rest body)
  (ada-value-build '(enumeration-literal) (car body)))
(defun int-lit (&quote &rest body)
  (ada-value-build '(integer-literal) (car body)))

;this makes variables of the given types.

(defun ada-dcl (&quote &rest body)
  (ada-do-dcl (car body) (cadr body) nil))
(defun ada-init-dcl (&quote &rest body)
  (ada-do-dcl (car body) (cadr body) (caddr body)))
(defun ada-constant-dcl (&quote &rest body)
  (ada-do-dcl (car body) (cadr body) (caddr body)))

(defun ada-do-dcl (var type value)
  (ada-store-fast 'variable (ncons var) nil)
  (if (not (eq (car type) 'ada-exception))
      '((,var (ada-value-build ',(ada-find-true-type type)
                               (ada-value-data ,value))))))
```

```
;def-ada-top needs to be a macro so that ada cliches will
;compile before they load.

(defmacro def-ada-top (name ignore use first-def &rest other-defs)
  (if (eq (car first-def) 'def-ada-cliche)
      first-def
      '(def-ada-top-internal ,name nil ,use ,first-def ., other-defs)))

(defun def-ada-top-internal
       (&quote name ignore use first-def &optional second-def)
  (let ((ada-use-packages nil))
    (apply 'ada-use use)
    (mapc #'(lambda (init)              ;global vars
              (if init (set (car init) (eval (cadr init)))))
          (eval first-def))
    (if second-def
        (mapc #'(lambda (init)
                  (if init (set (car init) (eval (cadr init)))))
              (eval second-def))))
  name)

(defun def-ada-package (&quote &rest body)
  (let* ((name (car body))
         (dcls (cadr body))
         (ada-prefix-stack ada-prefix-stack)
         (ada-use-packages ada-use-packages)
         (pkg-name (ada-prefix-it name))
         (generic (ada-find-fast 'generic pkg-name nil))
         inits)
    (cond (generic
           (rplacd (cdr generic) (ncons (cons 'def-ada-package body)))
           nil)
          (T (ada-add-prefix name)
             (setq inits (mapcan #'eval dcls))
             (ada-store-fast 'package pkg-name ada-use-packages)
             inits))))

(defun def-ada-package-body (&quote &rest body)
  (let* ((name (car body))
         (dcls (cadr body))
         (ada-prefix-stack ada-prefix-stack)
         (pkg-name (ada-prefix-it name))
         (ada-use-packages (or (cdr (ada-find-fast 'package pkg-name nil))
                               ada-use-packages))
         (generic (ada-find-fast 'generic pkg-name nil)))
    (cond (generic
           (rplacd (cddr generic)
                   (ncons (cons 'def-ada-package-body body)))
           nil)
          (T (ada-add-prefix name)
             (mapcan #'eval dcls)))))
```

```
(defun def-ada-fn (&quote &rest body)
  (let* ((spec (car body))
         (name (car spec)) (params (cadr spec)) (return (caddr spec))
         (dcls (cadr body)) (stmnts (caddr body)) (except (cadddr body)))
    (if stmnts
        (let* ((ada-use-packages ada-use-packages)
               (ada-prefix-stack ada-prefix-stack)
               (fn-name (ada-prefix-it name))
               (args (mapcar #'car params))
               (generic (ada-find-fast 'generic fn-name nil))
               inits)
          (cond (generic
                 (rplacd (cdr generic) (ncons (cons 'def-ada-fn body))))
                (T (ada-add-prefix name)
                   (setq inits (mapcan #'eval dcls))
                   (let* ((type-list
                           (mapcar #'(lambda (p)
                                       (ada-find-true-type (cadr p)))
                                   params))
                          (bod '(ada-begin ((prog () ,@stmnts))
                                           ,except)))
                     (cond (return (setq bod
                                         '(ada-value-build
                                           ',(ada-find-true-type return)
                                           (ada-value-data ,bod))))
                           (T (setq bod '(progn ,bod (values)))))
                     (def-ada-fn-internal fn-name type-list
                       '(lambda ,args
                          (let* ((ada-use-packages ',ada-use-packages)
                                 (ada-prefix-stack ',ada-prefix-stack)
                                 .,inits)
                            ,bod)))))))))
  nil)

(defun def-ada-fn-internal (name type-list lambda)
  (let* ((internal-name (ada-store-fn name type-list)))
    (eval '(defun ,internal-name .,(cdr lambda)))))

;we are careful to reuse names here in order to help maclisp GCing.

(defvar ada-gensym-counter 1 "protects from name collisions")

(defun ada-store-fn (fn type-list)
  (let* ((fn-info (cdr (ada-find-fast 'fn fn nil)))
         (spot (assoc type-list fn-info)))
    (cond ((or (null fn-info) (null spot))
           (setq ada-gensym-counter (1+ ada-gensym-counter))
           (let ((internal-name
                  (intern (format nil "~A-~D"
                                  (car fn) ada-gensym-counter))))
             (ada-store-fast 'fn fn (cons (cons type-list internal-name)
                                          fn-info))
             internal-name))
          (T (cdr spot)))))
```

```
; DEF-ADA-CLICHE is defined just like DEFINE-CLICHE and next to it.

;this handles generics by just doing wholesale substitutions

(defun def-ada-generic (&quote &rest body)
  (let* ((basic-spec (cadr body))
         (name (caseq (car basic-spec)
                 (def-ada-package (cadr basic-spec))
                 (def-ada-fn (caadr basic-spec))))
         (gen-name (ada-prefix-it name)))
    (ada-store-fast 'generic gen-name (ncons (cons 'def-ada-generic body)))
    (eval basic-spec) ;so if is package will store its body
    nil))

(defun ada-new-package (&quote &rest body)
  (ada-instance (cadr body) (car body) (cddr body)))

(defun ada-new-fn (&quote &rest body)
  (ada-instance (cadr body) (caar body) (cddr body)))

(defun specs-for-new-ada-fn (generic-name name)
  (let* ((generic-info (cdr (ada-find 'generic generic-name T)))
         (fn-template (cadr generic-info))
         (arg-info (cdadr fn-template)))
    (cons name arg-info)))

(defun ada-instance (generic-name name args)
  (let* ((generic-info (cdr (ada-find 'generic generic-name T)))
         (params (cadr (car generic-info)))
         (template (cdr generic-info))
         (substitutions
           (ada-determine-subs generic-name params name args)))
    (car (mapcar #'eval (ada-sublis substitutions template)))))

;works on equal not eq -- assumes no atomic cdrs

(defvar ada-sublis-alist)

(defun ada-sublis (ada-sublis-alist tree)
  (ada-sublis-1 tree))

(defun ada-sublis-1 (tree)
  (let ((hit (assoc tree ada-sublis-alist)))
    (cond (hit (cdr hit))
          ((and tree (listp tree)) (mapcar #'ada-sublis-1 tree))
          (T tree))))
```

```
(defun ada-determine-subs (generic-name params name args)
  (let ((subs '((,(car generic-name) ., name))))
    (do ((ps params (cdr ps))
         (as args (cdr as)))
        ((null ps))
      (let ((pair (ada-determine-sub (car ps) (car as))))
        (setq ps (ada-sublis (ncons pair) ps))
        (push pair subs)))
    subs))

(defun ada-determine-sub (param arg)
  (caseq (car param)
    ((ada-type-dcl ada-subtype-dcl)
     (cons (ncons (cadr param)) (ada-find-true-type arg)))
    (ada-with
     (if (and (null arg) (not (and (eq (car param) 'ada-with)
                                   (eq (car (last param)) '<>))))
         (barf "required generic arg missing"))
     (let* ((fn-name (ncons (caadr param)))
            (arg-name
              (car (ada-find-right-fn (or arg fn-name)
                                      (mapcar #'cadr
                                              (cadadr param)) nil))))
       (cons fn-name arg-name)))))
```

```
;unlike the things above, many of these forms are macros, because
;being macros explains them both to the evaluator and the analyzer.
;the things above have to have special analyzer understanding in any case.

(defvar ada-last-exception nil "supports ada-errors")

(defmacro ada-begin (body &optional exceptions)
  (setq body (cond ((cdr body) '(progn .,body)) (T (car body))))
  (cond ((null exceptions) body)
        (T '(if (eq 'ada-error (*catch 'ada-exception ,body))
                (cond .,(mapcar #'(lambda (except)
                                    '((ada-test-when ',(car except))
                                      .,(cdr except)))
                                exceptions))))))

(defun ada-test-when (exception-names)
  (or (member '(others) exception-names)
      (member ada-last-exception exception-names)))

(defmacro ada-loop (body &optional exceptions)
  '(ada-begin ((prog () L ,@body (go L))) ,exceptions))

(defmacro ada-while (test body &optional exceptions)
  '(ada-begin ((prog () L (ada-exit (ada-fn (not) ,test)) ,@body (go L)))
              ,exceptions))

(defmacro ada-cond (&rest clauses)
  '(cond .. (mapcar #'(lambda (c) '((ada-test-true ,(car c)) .. (cdr c)))
                    clauses)))

(defmacro ada-return (&optional value) ;not supported inside a loop
  '(return ,value))

(defmacro ada-exit (&optional condition) ;only local loop exit
  (cond ((null condition) '(return nil))
        (T '(if (ada-test-true ,condition) (return nil)))))

(defun ada-test-true (ada-bool)
  (ada-unbool (ada-value-data ada-bool)))

(defun ada-raise (&quote &rest names)
  (if names (setq ada-last-exception (car names)))
  (ada-reraise))

(defun ada-reraise ()
  (*throw 'ada-exception 'ada-error))

(defun ada-setq (left right)
  (setf (ada-value-data left) (ada-value-data right)))
```

```
;After this point we have the definitions of the standard ada fns and types
;Note that we have to define them all in occordance with the way things are
;handled above in the general case so that everything fits together.

(ada-type-dcl integer (private))
(ada-type-dcl boolean (ada-enumeration true false))
(ada-type-dcl character (private))

;this is used only in this file to define the standard ada fns below
;note you have to use the full correct names when you call this.

(defmacro def-ada (name var-type-pairs return-type &body body)
  (let* ((type-list (mapcar #'cadr var-type-pairs))
         (vars (mapcar #'car var-type-pairs))
         (full-vars (mapcar #'(lambda (v) (symbolconc v '|-full|)) vars)))
    (cond (return-type
           (cond ((cdr body) (setq body (cons 'progn body)))
                 (T (setq body (car body))))
           (setq body '((ada-value-build ',return-type ,body))))
          (T (setq body (append body '((values))))))
    '(def-ada-fn-internal ',name ',type-list
       '(lambda ,full-vars
          (let ,(mapcar #'(lambda (v fv) '(,v (ada-value-data ,fv)))
                        vars full-vars)
            ., body)))))

;this is used to change OUT parameters

(defmacro setq-param (param value)
  '(progn (setq ,param ,value)
          (setf (ada-value-data ,(symbolconc param '|-full|)) ,param)))
```

```
;basic operators and functions

(defun ada-bool (x)
  (cond (x 'true) (T 'false)))
(defun ada-unbool (x)
  (eq x 'true))

(def-ada (OR) ((x (boolean)) (y (boolean))) (boolean)
   (ada-bool (or (ada-unbool x) (ada-unbool y))))
(def-ada (XOR) ((x (boolean)) (y (boolean))) (boolean)
  (ada-bool (and (or (ada-unbool x) (ada-unbool y))
                 (not (and (ada-unbool x) (ada-unbool y))))))
(def-ada (AND) ((x (boolean)) (y (boolean))) (boolean)
  (ada-bool (and (ada-unbool x) (ada-unbool y))))
(def-ada (NOT) ((x (boolean))) (boolean)
  (ada-bool (not (ada-unbool x))))

(def-ada (=) ((x (ada-string)) (y (ada-string))) (boolean)
  (ada-bool (equal x y)))
(def-ada (//=) ((x (ada-string)) (y (ada-string))) (boolean)
  (ada-bool (not (equal x y))))

(def-ada (=) ((x (ada-enumeration)) (y (ada-enumeration))) (boolean)
  (ada-bool (eq x y)))
(def-ada (//=) ((x (ada-enumeration)) (y (ada-enumeration))) (boolean)
  (ada-bool (not (eq x y))))

(def-ada (=) ((x (integer)) (y (integer))) (boolean)
  (ada-bool (= x y)))
(def-ada (//=) ((x (integer)) (y (integer))) (boolean)
  (ada-bool (not (= x y))))

(def-ada (=) ((x (character)) (y (character))) (boolean)
  (ada-bool (= x y)))
(def-ada (//=) ((x (character)) (y (character))) (boolean)
  (ada-bool (not (= x y))))

(def-ada (&) ((x (ada-string)) (y (ada-string))) (ada-string)
   (format nil "~A~A" x y))

(def-ada (+) ((x (integer)) (y (integer))) (integer)
   (+ x y))
(def-ada (-) ((x (integer)) (y (integer))) (integer)
   (- x y))
(def-ada (u-) ((x (integer))) (integer)
   (- x))
(def-ada (u+) ((x (integer))) (integer)
   x)
(def-ada (ABS) ((x (integer))) (integer)
   (abs x))
(def-ada (*) ((x (integer)) (y (integer))) (integer)
   (* x y))
(def-ada (//) ((x (integer)) (y (integer))) (integer)
   (// x y))
(def-ada (MOD) ((x (integer)) (y (integer))) (integer)
   (rem x y))
(def-ada (rem) ((x (integer)) (y (integer))) (integer)
   (rem x y))
(def-ada (**) ((x (integer)) (y (integer))) (integer)
   (^ x y))
```

```
(def-ada (<) ((x (integer)) (y (integer))) (boolean)
   (ada-bool (< x y)))
(def-ada (>) ((x (integer)) (y (integer))) (boolean)
   (ada-bool (> x y)))
(def-ada (<=) ((x (integer)) (y (integer))) (boolean)
   (ada-bool (or (= x y) (< x y))))
(def-ada (>=) ((x (integer)) (y (integer))) (boolean)
   (ada-bool (or (= x y) (> x y))))

(def-ada (<) ((x (ada-string)) (y (ada-string))) (boolean)
   (ada-bool (alphalessp x y)))
(def-ada (>) ((x (ada-string)) (y (ada-string))) (boolean)
   (ada-bool (alphalessp y x)))
(def-ada (<=) ((x (ada-string)) (y (ada-string))) (boolean)
   (ada-bool (not (alphalessp y x))))
(def-ada (>=) ((x (ada-string)) (y (ada-string))) (boolean)
  (ada-bool (not (alphalessp x y))))

(def-ada (<) ((x (character)) (y (character))) (boolean)
   (ada-bool (< x y)))
(def-ada (>) ((x (character)) (y (character))) (boolean)
   (ada-bool (> x y)))
(def-ada (<=) ((x (character)) (y (character))) (boolean)
   (ada-bool (or (= x y) (< x y))))
(def-ada (>=) ((x (character)) (y (character))) (boolean)
   (ada-bool (or (= x y) (> x y))))
```

```
(def-ada-top calendar nil nil
 (def-ada-package CALENDAR
   ((ada-type-dcl time (private))
    (ada-subtype-dcl year_number (ada-range (integer) 1901 2099))
    (ada-subtype-dcl month_number (ada-range (integer) 1 12))
    (ada-subtype-dcl day_number (ada-range (integer) 1 31)))))

(def-ada (time_of calendar) ((year (year_number calendar))
                             (month (month_number calendar))
                             (day (day_number calendar)))
    (time calendar)
  (if (not (check-date year month day))
      (ada-raise (time_error)))
  (make-time year month day))

(defun make-time (year month day)
  (+ (* year 10000.) (* month 100.) day))

(def-ada (>= calendar) ((x (time calendar)) (y (time calendar)))
    (boolean)
  (ada-bool (> x y)))

(def-ada (year calendar) ((time (time calendar)))
  (year_number calendar)
  (// time 10000.))

(def-ada (month calendar) ((time (time calendar)))
    (month_number calendar)
  (remainder (// time 100.) 100.))

(def-ada (day calendar) ((time (time calendar)))
    (day_number calendar)
  (remainder time 100.))

(def-ada (clock calendar) () (time calendar)
  (multiple-value-bind (second minutes hours date month year)
      (time:get-time)
    (make-time (+ 1900 year) month date)))

(defun check-date (year month day)
  (and (fixnump year)
       (fixnump month)
       (fixnump day)
       (and (<= 1950 year) (<= year 2050))
       (and (<= 1 month) (<= month 12))
       (and (<= 1 day) (<= day (days-in-month month year)))))

(defun days-in-month (month year)
  (cond ((member month '(1 3 5 6 7 8 10 12)) 31)
        ((member month '(4 6 9 11)) 30)
        ((and (zerop (\ year 4))
              (or (not (zerop (\ year 100)))
                  (zerop (\ year 1000))))
         29)
        (T 28)))
```

```
;here are some file operation utilities.
;In lisp the files are just stored as lists, and the functions
;operate on these lists.  In other languages, real file IO would occure.
;Note that these functions are not themselves part of the demo
;they are just used by it.  As a result, they can use
;LISP features not understood by the PA.

;this stores the info about a file including the data.

(defstruct (file :conc-name (:type :named-list))
  name                ;name of the file
  mode                ;CLOSED, IN_FILE, OUT_FILE, INOUT_FILE
;for non-TEXT_IO files
  index               ;position pointer for sequential reading
  data                ;list of the records in the file
;for TEXT_IO files
  page                ;the current page number. init 1
  line                ;the current line number. init 1
  charpos             ;the current character number. init 1
  obj)                ;the external file object if open.

(defvar ada-old-files nil "for protection when errors happen")
(defvar ada-current-input)
(defvar ada-current-output)
(defvar ada-standard-input)
(defvar ada-standard-output)

;you must call this any time before calling a fn that does IO.

(defun ada-init ()
  (mapcar #'close ada-old-files)
  (setq ada-old-files nil)
  (setq ada-standard-input
        (make-file mode 'in_file obj standard-input))
  (setq ada-standard-output
        (make-file mode 'out_file
                   page 1
                   line 1
                   charpos 1
                   obj standard-output))
  (setq ada-current-input ada-standard-input)
  (setq ada-current-output ada-standard-output))
```

```
(def-ada-top text_io nil nil
 (def-ada-package text_io
  ((ada-type-dcl file_type (limited-private))
   (ada-type-dcl file_mode (ada-enumeration in_file out_file))
   (ada-type-dcl count (ada-range nil 0 10000))
   (ada-subtype-dcl positive_count (ada-range (count) 0 10000))

   (def-ada-generic
     ((ada-type-dcl num (ada-range nil <>)))
     (def-ada-package integer_io))))

;real package body needed for INTEGER_IO so that generic will work right.

 (def-ada-package-body text_io
  ((def-ada-package-body integer_io
     ((def-ada-fn (put ((obj (num)) (width (integer))))
        () ((ada-fn (ada_int_put) obj width)))
      (def-ada-fn (put ((obj (num))))
        () ((ada-fn (ada_int_put) obj (int-lit 9))))
      (def-ada-fn (put ((var (ada-string)) (obj (num))) )
        () ((ada-fn (ada_int_put_str) var obj)))
      (def-ada-fn (get ((var (num))) )
        () ((ada-fn (ada_int_get) var)))))))))

(def-ada (create text_io)
    ((file (file_type text_io))
     (mode (file_mode text_io))
     (name (ada-string))) nil
  (let ((disk-file (open name 'out)))
    (push disk-file ada-old-files)
    (setq-param file
                (make-file name name mode mode
                           index nil data nil
                           page 1 line 1
                           charpos 1 obj disk-file))))

(def-ada (close text_io) ((file (file_type text_io))) nil
  (close (file-obj file))
  (setf (file-mode file) 'closed))

(def-ada (set_output text_io) ((file (file_type text_io))) nil
  (setq ada-current-output file))

(def-ada (standard_output text_io) () (file_type text_io)
  ada-standard-output)

(def-ada (new_line text_io) () nil
  (setf (file-line ada-current-output)
        (1+ (file-line ada-current-output)))
  (setf (file-charpos ada-current-output) 1)
  (format (file-obj ada-current-output) "~%"))

(def-ada (new_line text_io) ((num (positive_count text_io))) nil
  (setf (file-line ada-current-output)
        (+ num (file-line ada-current-output)))
  (setf (file-charpos ada-current-output) 1)
  (format (file-obj ada-current-output) "~V%" num))

(def-ada (new_page text_io) () nil
  (setf (file-page ada-current-output)
        (1+ (file-page ada-current-output)))
  (setf (file-line ada-current-output) 1)
  (setf (file-charpos ada-current-output) 1)
  (format (file-obj ada-current-output) "~/|"))
```

```
(def-ada (set_col text_io) ((num (positive_count text_io))) nil
  (format (file-obj ada-current-output) "~VX"
          (- num (file-charpos ada-current-output)))
  (setf (file-charpos ada-current-output) num))

(def-ada (page text_io) () (positive_count text_io)
  (file-page ada-current-output))

(def-ada (line text_io) () (positive_count text_io)
  (file-line ada-current-output))

;TEXT_IO for characters

(def-ada (get text_io) ((var (character))) nil
  (setq-param var (tyi (file-obj ada-current-input))))

(def-ada (put text_io) ((obj (character))) nil
  (setf (file-charpos ada-current-output)
        (1+ (file-charpos ada-current-output)))
  (tyo obj (file-obj ada-current-output)))

;TEXT_IO for strings

(def-ada (get text_io) ((var (ada-string))) nil
  (setq-param var (readline (file-obj ada-current-input))))

(def-ada (put text_io) ((obj (ada-string))) nil
  (setf (file-charpos ada-current-output)
        (+ (string-length obj) (file-charpos ada-current-output)))
  (princ obj (file-obj ada-current-output)))


;TEXT_IO for integers

(def-ada (ada_int_put) ((obj (integer)) (width (integer))) nil
  (setf (file-charpos ada-current-output)
        (+ width (file-charpos ada-current-output)))
  (format (file-obj ada-current-output) "~VD" width obj))

(def-ada (ada_int_put_str) ((var (ada-string)) (obj (integer))) nil
  (setq-param var (format nil "~VD" (ada-string-length
                                     (ada-value-type |VAR-full|)) obj)))

(def-ada (ada_int_get) ((var (integer))) nil
  (setq-param var
              (readlist
                (exploden (readline (file-obj ada-current-input))))))

(defun ada-string-length (type)
  (cond ((eq (car type) 'ada-string) (1+ (- (caddr type) (cadr type))))
        (T (ada-string-length (cdr (ada-find 'type type T))))))
```

```
(def-ada-top sequential_io nil nil
 (def-ada-generic
  ((ada-type-dcl element_type (private)))
  (def-ada-package sequential_io nil
    ((ada-type-dcl file_type (limited-private))
     (ada-type-dcl file_mode (ada-enumeration in_file out_file)))))
 (def-ada-package-body sequential_io nil
  ((def-ada-fn (create
                ((file (file_type)) (mode (file_mode)) (name (ada-string))) )
    () ((ada-fn (ada-file-open) file mode name (enum-lit TRUE))))
   (def-ada-fn (create ((file (file_type)) (mode (file_mode))))
    () ((ada-fn (ada-file-open) file mode (str-lit "") (enum-lit TRUE))))
   (def-ada-fn (open
      ((file (file_type)) (mode (file_mode)) (name (ada-string))) )
    () ((ada-fn (ada-file-open) file mode name (enum-lit FALSE))))
   (def-ada-fn (reset ((file (file_type))) )
    () ((ada-fn (ada-file-reset) file)))
   (def-ada-fn (close ((file (file_type))) )
    () ((ada-fn (ada-file-close) file)))
   (def-ada-fn (read  ((file (file_type)) (var (element_type))) )
    () ((ada-fn (ada-file-read) file var (int-lit 0))))
   (def-ada-fn (write ((file (file_type)) (item (element_type))) )
    () ((ada-fn (ada-file-write) file item (int-lit 0))))
   (def-ada-fn (end_of_file ((file (file_type))) (boolean))
    () ((ada-return (ada-fn (ada-end_of_file) file)))))))
```

```
(def-ada-top direct_io nil nil
 (def-ada-generic
  ((ada-type-dcl element_type (private)))
  (def-ada-package direct_io
    ((ada-type-dcl file_type (limited-private))
     (ada-type-dcl file_mode (ada-enumeration in_file out_file))
     (ada-type-dcl count (ada-range nil 0 1000Q))
     (ada-subtype-dcl positive_count (ada-range (count) 1 10000)))))
 (def-ada-package-body direct_io
  ((def-ada-fn (create
       ((file (file_type)) (mode (file_mode)) (name (ada-string))) )
     () ((ada-fn (ada-file-open) file mode name (enum-lit TRUE))))
   (def-ada-fn (create ((file (file_type)) (mode (file_mode))))
     () ((ada-fn (ada-file-open) file mode (str-lit "") (enum-lit TRUE))))
   (def-ada-fn (open
       ((file (file_type)) (mode (file_mode)) (name (ada-string))) )
     () ((ada-fn (ada-file-open) file mode name (enum-lit FALSE))))
   (def-ada-fn (reset ((file (file_type))) )
     () ((ada-fn (ada-file-reset) file)))
   (def-ada-fn (close ((file (file_type))) )
     () ((ada-fn (ada-file-close) file)))
   (def-ada-fn (read  ((file (file_type)) (var (element_type))) )
     () ((ada-fn (ada-file-read) file var (int-lit 0))))
   (def-ada-fn (read
       ((file (file_type))
        (var (element_type))
        (index (positive_count))) )
     () ((ada-fn (ada-file-read) file var index)))
   (def-ada-fn (write ((file (file_type)) (item (element_type))) )
     () ((ada-fn (ada-file-write) file item (int-lit 0))))
   (def-ada-fn (write
       ((file (file_type))
        (item (element_type))
        (index (positive_count))) )
     () ((ada-fn (ada-file-write) file item index)))
   (def-ada-fn (end_of_file ((file (file_type))) (boolean))
     () ((ada-return (ada-fn (ada-end_of_file) file))))
   (def-ada-fn (set_index
       ((file (file_type)) (index (positive_count))) )
     () ((ada-fn (ada-file-set-index) file index)))
   (def-ada-fn (index ((file (file_type))) (positive_count))
     () ((ada-return (ada-fn (ada-file-index) file))))
   (def-ada-fn (size  ((file (file_type))) (count))
     () ((ada-return (ada-fn (ada-file-size) file)))))))
```

```
;these are the underlying implementation functions for direct and
;sequential IO.  Note use of (any) so these can get called by
;every instance.

(def-ada (ada-file-open)
    ((file (any))
     (mode (any))
     (name (ada-string))
     (createp (boolean))) nil
  (setq-param file
              (make-file name name
                         mode mode
                         index 1
                         data (if (not (ada-unbool createp))
                                  (ada-stored-data name)))))

(def-ada (ada-file-reset) ((file (any))) nil
  (setf (file-index file) 1))

(def-ada (ada-file-close) ((file (any))) nil
  (ada-store-data (file-name file) (file-data file))
  (setf (file-mode file) 'closed))

(def-ada (ada-file-read)
    ((file (any)) (var (any)) (index (integer))) nil
  (if (not (zerop index)) (setf (file-index file) index))
  (if (ada-end_of_file file) (ada-raise (end_error)))
  (setf (file-index file) (1+ (file-index file)))
  (setq-param var (nth (- (file-index file) 2) (file-data file))))

(def-ada (ada-file-write)
    ((file (any)) (item (any)) (index (integer))) nil
  (if (not (zerop index)) (setf (file-index file) index))
  (prog ()
    L (if (not (ada-end_of_file file)) (return nil))
      (setf (file-data file) (nconc (file-data file) (ncons nil)))
      (go L))
  (setf (file-index file) (1+ (file-index file)))
  (setf (nth (- (file-index file) 2) (file-data file)) item))

(def-ada (ada-file-set-index) ((file (any)) (index (integer))) nil
  (setf (file-index file) index))

(def-ada (ada-file-index) ((file (any))) (integer)
  (file-index file))

(def-ada (ada-file-size) ((file (any))) (integer)
  (length (file-data file)))

(def-ada (ada-end_of_file) ((file (any))) (boolean)
  (ada-bool (ada-end_of_file file)))

(defun ada-end_of_file (file)
  (> (file-index file) (length (file-data file))))
```

```
;This stores data items between uses of a file.

(defvar ada-data-storage nil "simulates disk storage")

(defun ada-stored-data (name)
  (cdr (assoc name ada-data-storage)))

(defun ada-store-data (name data)
  (let* ((spot (assoc name ada-data-storage)))
    (cond ((equal name ""))
          ((null spot)
           (push (cons name data) ada-data-storage))
          (T (rplacd spot data)))))

;after here we have basic stuff you need to run the demo.
;it isn't part of standard ada, but is part of what we take to be standard.

(def-ada-top functions
   ((CALENDAR) (IO_EXCEPTIONS) (TEXT_IO)) ((CALENDAR) (TEXT_IO))
 (def-ada-package functions
  ((ada-new-package INT_IO (INTEGER_IO) (INTEGER))
   (ada-subtype-dcl INDEX_TYPE
                    (ada-range (INTEGER) 1 (ada-attribute (INTEGER) last)))
   (ada-type-dcl DATE_STRING_TYPE (ada-string 1 10))
   (def-ada-fn (FORMAT_DATE ((DATE (TIME))) (date_string_type)))
   (def-ada-fn (QUERY_USER_FOR_DATE () (TIME)))

   (def-ada-generic
    ((ada-type-dcl ITEM_TYPE (private))
     (ada-type-dcl FILE_TYPE (limited-private))
     (ada-type-dcl INDEX_TYPE (ada-range nil <>))
     (ada-with (> ((X (ITEM_TYPE)) (Y (ITEM_TYPE))) (BOOLEAN)) <>)
     (ada-with (READ ((FILE (FILE_TYPE)) (ITEM (ITEM_TYPE) out))) <>)
     (ada-with (WRITE ((FILE (FILE_TYPE)) (ITEM (ITEM_TYPE))
                                          (TO (INDEX_TYPE)))) <>)
     (ada-with (SET_INDEX ((FILE (FILE_TYPE)) (TO (INDEX_TYPE)))) <>)
     (ada-with (INDEX ((FILE (FILE_TYPE))) (INDEX_TYPE)) <>)
     (ada-with (END_OF_FILE ((FILE (FILE_TYPE))) (BOOLEAN)) <>))
    (def-ada-fn (SORT_file ((FILE (FILE_TYPE))))))

   (def-ada-generic
    ((ada-type-dcl FROM_ITEM_TYPE (private))
     (ada-type-dcl FROM_FILE_TYPE (limited-private))
     (ada-type-dcl TO_ITEM_TYPE (limited-private))
     (ada-type-dcl TO_FILE_TYPE (limited-private))
     (ada-with (BUILD ((COUNT (INTEGER)) (ITEM (FROM_ITEM_TYPE)))
                      (TO_ITEM_TYPE)))
     (ada-with (READ ((FILE (FROM_FILE_TYPE))
                      (ITEM (FROM_ITEM_TYPE) out))) <>)
     (ada-with (WRITE ((FILE (TO_FILE_TYPE)) (ITEM (TO_ITEM_TYPE)))) <>)
     (ada-with (END_OF_FILE ((FILE (FROM_FILE_TYPE))) (BOOLEAN)) <>)
     (ada-with (RESET ((FILE (TO_FILE_TYPE)))) <>))
    (def-ada-fn (SUMMARIZE_file
      ((FROM_FILE (FROM_FILE_TYPE)) (TO_FILE (TO_FILE_TYPE))))))
```

```
(def-ada-generic
  ((ada-type-dcl ELEMENT_TYPE (private))
   (ada-type-dcl INDEX_TYPE (ada-range nil <>)))
  (def-ada-package CHAINED_IO
    ((ada-new-package UNDERLYING_IO (DIRECT_IO) (ELEMENT_TYPE))
     (ada-subtype-dcl FILE_TYPE (FILE_TYPE UNDERLYING_IO))
     (ada-type-dcl FILE_MODE (ada-enumeration IN_FILE INOUT_FILE OUT_FILE))
     (def-ada-fn (CREATE ((FILE (FILE_TYPE) in-out)
                          (MODE (FILE_MODE)) (NAME (ADA-STRING)))))
     (def-ada-fn (OPEN ((FILE (FILE_TYPE) in-out)
                        (MODE (FILE_MODE)) (NAME (ADA-STRING)))))
     (def-ada-fn (CLOSE ((FILE (FILE_TYPE) in-out))))
     (def-ada-fn (read ((FILE (FILE_TYPE))
                        (ITEM (ELEMENT_TYPE) out)
                        (FROM (INDEX_TYPE)))))
     (def-ada-fn (write ((FILE (FILE_TYPE))
                         (ITEM (ELEMENT_TYPE))
                         (to (INDEX_TYPE)))))
     (def-ada-fn (END_OF_FILE ((FILE (FILE_TYPE))) (BOOLEAN)))
     (def-ada-fn (free_index  ((FILE (FILE_TYPE))) (index_type)))
     (def-ada-fn (NULL_INDEX  ((INDEX (INDEX_TYPE))) (BOOLEAN)))
     (def-ada-fn (NULL_INDEX  () (INDEX_TYPE)))
     (ada-dcl STATUS_ERROR
              (ada-exception (STATUS_ERROR IO_EXCEPTIONS)))
     (ada-dcl MODE_ERROR (ada-exception (MODE_ERROR IO_EXCEPTIONS)))
     (ada-dcl NAME_ERROR (ada-exception (NAME_ERROR IO_EXCEPTIONS)))
     (ada-dcl USE_ERROR (ada-exception (USE_ERROR IO_EXCEPTIONS)))
     (ada-dcl DEVICE_ERROR
              (ada-exception (DEVICE_ERROR IO_EXCEPTIONS)))
     (ada-dcl END_ERROR (ada-exception (END_ERROR IO_EXCEPTIONS)))
     (ada-dcl DATA_ERROR
              (ada-exception (DATA_ERROR IO_EXCEPTIONS))))))

(def-ada-generic
  ((ada-type-dcl ELEMENT_TYPE (private))
   (ada-type-dcl key_TYPE (private)))
  (def-ada-package KEYED_IO
    ((ada-type-dcl combined_type
                   (ada-rec (key (key_type))
                            (element (element_type))))
     (ada-new-package UNDERLYING_IO (DIRECT_IO) (combined_TYPE))
     (ada-subtype-dcl FILE_TYPE (FILE_TYPE UNDERLYING_IO))
     (ada-type-dcl FILE_MODE (ada-enumeration IN_FILE INOUT_FILE OUT_FILE))
     (def-ada-fn (CREATE ((FILE (FILE_TYPE) in-out)
                          (MODE (FILE_MODE)) (NAME (ADA-STRING)))))
     (def-ada-fn (OPEN  ((FILE (FILE_TYPE) in-out)
                         (MODE (FILE_MODE)) (NAME (ADA-STRING)))))
     (def-ada-fn (CLOSE ((FILE (FILE_TYPE) in-out))))
     (def-ada-fn (read  ((FILE (FILE_TYPE))
                         (ITEM (ELEMENT_TYPE) out)
                         (FROM (KEY_TYPE)))))
     (def-ada-fn (write ((FILE (FILE_TYPE))
                         (ITEM (ELEMENT_TYPE))
                         (to (key_TYPE)))))
     (def-ada-fn (SET_KEY ((FILE (FILE_TYPE))
                           (KEY (KEY_TYPE)))))
     (def-ada-fn (END_OF_FILE ((FILE (FILE_TYPE))) (BOOLEAN)))
     (ada-dcl STATUS_ERROR
              (ada-exception (STATUS_ERROR IO_EXCEPTIONS)))
     (ada-dcl MODE_ERROR (ada-exception (MODE_ERROR IO_EXCEPTIONS)))
     (ada-dcl NAME_ERROR (ada-exception (NAME_ERROR IO_EXCEPTIONS)))
     (ada-dcl USE_ERROR (ada-exception (USE_ERROR IO_EXCEPTIONS)))
     (ada-dcl DEVICE_ERROR
              (ada-exception (DEVICE_ERROR IO_EXCEPTIONS)))
     (ada-dcl END_ERROR (ada-exception (END_ERROR IO_EXCEPTIONS)))
     (ada-dcl DATA_ERROR
              (ada-exception (DATA_ERROR IO_EXCEPTIONS))))))
```

```
(def-ada-package-body FUNCTIONS
 ((def-ada-fn (FORMAT_DATE ((DATE (TIME))) (DATE_STRING_TYPE))
    ((ada-use (INT_IO))
     (ada-dcl MONTH_STRING (ada-string 1 2))
     (ada-dcl DAY_STRING (ada-string 1 2))
     (ada-dcl YEAR_STRING (ada-string 1 4)))
    ((ada-fn (PUT) MONTH_STRING (ada-fn (MONTH) DATE))
     (ada-fn (PUT) day_STRING (ada-fn (day) DATE))
     (ada-fn (PUT) year_STRING (ada-fn (year) DATE))
     (ada-return (ada-fn (&) (ada-fn (&) (ada-fn (&) (ada-fn (&) MONTH_STRING
                                                              (str-lit "//"))
                                                     DAY_STRING)
                                            (str-lit "//"))
                                   YEAR_STRING))))
  (def-ada-fn (QUERY_USER_FOR_DATE ((PROMPT (ADA-STRING))) (TIME))
    ((ada-use (INT_IO))
     (ada-dcl month (month_number))
     (ada-dcl day (day_number))
     (ada-dcl year (year_number))
     (ada-dcl date (time)))
    ((ada-loop
       ((ada-begin
          ((ada-fn (new_line))
           (ada-fn (put) (ada-fn (&) (str-lit "Enter Date of ") PROMPT))
           (ada-fn (new_line)) (ada-fn (put) (str-lit "Month: "))
           (ada-fn (get) month)
           (ada-fn (put) (str-lit "Day: ")) (ada-fn (get) day)
           (ada-fn (put) (str-lit "Year: ")) (ada-fn (get) year)
           (ada-setq date (ada-fn (TIME_OF) year month day))
           (ada-exit))
          ((((DATA_ERROR)
             (CONSTRAINT_ERROR)
             (TIME_ERROR))
            (ada-fn (PUT) (str-lit "Invalid DATE")) (ada-fn (NEW_LINE)))))))
      (ada-return date)))
```

```
(def-ada-fn (SORT_file ((FILE (FILE_TYPE))))
  ((ada-init-dcl EDGE (INDEX_TYPE) (int-lit 1))
   (ada-dcl MAX (INDEX_TYPE))
   (ada-dcl EDGE_ITEM (item_type))
   (ada-dcl MAX_ITEM (item_type))
   (ada-dcl CURRENT_ITEM (item_type)))
  ((ada-loop
    ((ada-fn (SET_INDEX) FILE EDGE)
     (ada-exit (ada-fn (END_OF_FILE) FILE))
     (ada-fn (READ) FILE EDGE_ITEM)
     (ada-setq MAX EDGE)
     (ada-setq MAX_ITEM EDGE_ITEM)
     (ada-while (ada-fn (not) (ada-fn (END_OF_FILE) FILE))
      ((ada-fn (READ) FILE CURRENT_ITEM)
       (ada-cond ((ada-fn (>) CURRENT_ITEM MAX_ITEM)
                  (ada-setq max (ADA-FN (-) (ada-fn (INDEX) FILE) (INT-lit 1)))
                  (ada-setq MAX_ITEM CURRENT_ITEM)))))
     (ada-cond ((ada-fn (//=) max edge)
                (ada-fn (WRITE) FILE MAX_ITEM EDGE)
                (ada-fn (WRITE) FILE EDGE_ITEM MAX)))
     (ada-setq edge (ada-fn (+) edge (int-lit 1)))))
   (ada-fn (SET_INDEX) FILE (int-lit 1))))

(def-ada-fn (SUMMARIZE_file ((FROM_FILE (FROM_FILE_TYPE))
                             (TO_FILE (TO_FILE_TYPE))))
  ((ada-init-dcl COUNT (INTEGER) (int-lit 1))
   (ada-dcl CURRENT (from_item_type))
   (ada-dcl PRIOR (FROM_ITEM_TYPE)))
  ((ada-cond ((ada-fn (not) (ada-fn (END_OF_FILE) FROM_FILE))
              (ada-fn (READ) FROM_FILE PRIOR)
              (ada-while (ada-fn (not) (ada-fn (END_OF_FILE) FROM_FILE))
               ((ada-fn (READ) FROM_FILE CURRENT)
                (ada-cond ((ada-fn (//=) current prior)
                           (ada-fn (WRITE) TO_FILE
                                (ada-fn (BUILD) COUNT PRIOR))
                           (ada-setq count (int-lit 1))
                           (ada-setq prior current))
                          ((enum-lit TRUE)
                           (ada-setq count (ada-fn (+) count (int-lit 1)))))))
              (ada-fn (WRITE) TO_FILE (ada-fn (BUILD) COUNT CURRENT))
              (ada-fn (RESET) TO_FILE)))))
```

```
(def-ada-package-body CHAINED_IO
  ((def-ada-fn (FILE_MODE_CONVERT ((MODE (FILE_MODE)))
                (FILE_MODE UNDERLYING_IO))
     () ((ada-cond ((ada-fn (=) MODE (enum-lit IN_FILE))
                    (ada-return (enum-lit IN_FILE UNDERLYING_IO)))
                   ((ada-fn (=) MODE (enum-lit out_FILE))
                    (ada-return (enum-lit out_FILE UNDERLYING_IO)))
                   ((enum-lit True)
                    (ada-return (enum-lit inout_FILE UNDERLYING_IO))))))
   (def-ada-fn (CREATE ((FILE (FILE_TYPE) in-out)
                        (MODE (FILE_MODE)) (NAME (ADA-STRING))))
     () ((ada-fn (CREATE UNDERLYING_IO) FILE
                 (ada-fn (FILE_MODE_CONVERT) MODE) NAME)))
   (def-ada-fn (OPEN ((FILE (FILE_TYPE) in-out)
                      (MODE (FILE_MODE)) (NAME (ADA-STRING))))
     () ((ada-fn (OPEN UNDERLYING_IO) FILE
                 (ada-fn (FILE_MODE_CONVERT) MODE) NAME)))
   (def-ada-fn (CLOSE ((FILE (FILE_TYPE) in-out)))
     () ((ada-fn (CLOSE UNDERLYING_IO) FILE)))
   (def-ada-fn (read ((FILE (FILE_TYPE))
                      (ITEM (ELEMENT_TYPE) out)
                      (FROM (INDEX_TYPE))))
     () ((ada-fn (READ UNDERLYING_IO) FILE ITEM
                 (ada-convert (POSITIVE_COUNT UNDERLYING_IO) FROM))))
   (def-ada-fn (write ((FILE (FILE_TYPE))
                       (ITEM (ELEMENT_TYPE))
                       (to (INDEX_TYPE))))
     () ((ada-fn (write UNDERLYING_IO) FILE ITEM
                 (ada-convert (POSITIVE_COUNT UNDERLYING_IO) to))))
   (def-ada-fn (END_OF_FILE ((FILE (FILE_TYPE))) (BOOLEAN))
     () ((ada-return (ada-fn (END_OF_FILE UNDERLYING_IO) FILE))))
   (def-ada-fn (free_index ((FILE (FILE_TYPE))) (index_type))
     () ((ada-return (ada-fn (+) (int-lit 1)
                             (ada-fn (SIZE UNDERLYING_IO) FILE)))))
   (def-ada-fn (NULL_INDEX ((INDEX (INDEX_TYPE))) (BOOLEAN))
     () ((ada-return (ada-fn (=) index (int-lit 0)))))
   (def-ada-fn (NULL_INDEX () (INDEX_TYPE))
     () ((ada-return (int-lit 0))))))
```

```
(def-ada-package-body KEYED_IO
  ((def-ada-fn (FILE_MODE_CONVERT ((MODE (FILE_MODE)))
                (FILE_MODE UNDERLYING_IO))
     () ((ada-cond ((ada-fn (=) MODE (enum-lit IN_FILE))
                    (ada-return (enum-lit IN_FILE UNDERLYING_IO)))
                   ((ada-fn (=) MODE (enum-lit out_FILE))
                    (ada-return (enum-lit out_FILE UNDERLYING_IO)))
                   ((enum-lit True)
                    (ada-return
                     (enum-lit inout_FILE UNDERLYING_IO))))))
   (def-ada-fn (CREATE ((FILE (FILE_TYPE) in-out)
                        (MODE (FILE_MODE)) (NAME (ADA-STRING))))
     () ((ada-fn (CREATE UNDERLYING_IO) FILE
                 (ada-fn (FILE_MODE_CONVERT) MODE) NAME)))
   (def-ada-fn (OPEN ((FILE (FILE_TYPE) in-out)
                      (MODE (FILE_MODE)) (NAME (ADA-STRING))))
     () ((ada-fn (OPEN UNDERLYING_IO) FILE
                 (ada-fn (FILE_MODE_CONVERT) MODE) NAME)))
   (def-ada-fn (CLOSE ((FILE (FILE_TYPE) in-out)))
     () ((ada-fn (CLOSE UNDERLYING_IO) FILE)))
   (def-ada-fn (SET_KEY ((FILE (FILE_TYPE)) (KEY (KEY_TYPE))))
     ((ada-dcl COMBINED (COMBINED_TYPE))
      (ada-dcl I (POSITIVE_COUNT UNDERLYING_IO)))
     ((ada-fn (RESET UNDERLYING_IO) FILE)
      (ada-while (ada-fn (not) (ada-fn (END_OF_FILE UNDERLYING_IO) FILE))
       ((ada-fn (READ UNDERLYING_IO) FILE COMBINED)
        (ada-cond ((ada-fn (=) (ada-select COMBINED KEY) KEY)
                   (ada-setq I (ada-fn (-)
                                       (ada-fn (index underlying_io) file)
                                       (int-lit 1)))
                   (ada-fn (set_index underlying_io) FILE I)
                   (ada-exit)))))))
   (def-ada-fn (read ((FILE (FILE_TYPE))
                      (ITEM (ELEMENT_TYPE) out)
                      (FROM (KEY_TYPE))))
     ((ada-dcl combined (combined_type)))
     ((ada-fn (set_key) file from)
      (ada-fn (READ UNDERLYING_IO) FILE combined)
      (ada-setq item (ada-select combined element))))
   (def-ada-fn (read ((FILE (FILE_TYPE))
                      (ITEM (ELEMENT_TYPE) out)))
     ((ada-dcl combined (combined_type)))
     ((ada-fn (READ UNDERLYING_IO) FILE combined)
      (ada-setq item (ada-select combined element))))
   (def-ada-fn (write ((FILE (FILE_TYPE))
                       (ITEM (ELEMENT_TYPE)) (to (key_TYPE))))
     () ((ada-fn (set_key) file to)
         (ada-fn (write UNDERLYING_IO) FILE
                 (ada-construct (COMBINED_TYPE) TO ITEM))))
   (def-ada-fn (END_OF_FILE ((FILE (FILE_TYPE))) (BOOLEAN))
     () ((ada-return (ada-fn (END_OF_FILE UNDERLYING_IO) FILE)))))))
```

```
(def-ada-top maintenance_files
  ((CALENDAR) (FUNCTIONS) (TEXT_IO)) ((CALENDAR) (FUNCTIONS))
 (def-ada-package maintenance_files
  ((ada-dcl report (FILE_TYPE TEXT_IO))
   (ada-constant-dcl models_name (ada-string) (str-lit "models.data"))
   (ada-type-dcl model_key_type (ada-string 1 3))
   (ada-type-dcl model_type
       (ada-rec (NAME (ada-string 1 16))
                (MAKER (ada-string 1 16))))
   (ada-new-package model_io (keyed_io) (model_type) (model_key_type))
   (ada-dcl models (file_type model_io))

   (ada-constant-dcl defects_name (ada-string) (str-lit "defects.data"))
   (ada-type-dcl defect_key_type (ada-string 1 5))
   (ada-type-dcl defect_type
     (ada-rec (NAME (ada-string 1 32))
              (model (model_key_type))))
   (ada-new-package defect_io (keyed_io) (defect_type) (defect_key_type))
   (ada-dcl defects (file_type defect_io))

   (ada-constant-dcl repairs_name (ada-string) (str-lit "repairs.data"))
   (ada-subtype-dcl repair_index_type (index_type))
   (ada-type-dcl repair_type
     (ada-rec (DATE (TIME))
              (DEFECT (DEFECT_KEY_TYPE))
              (NEXT (REPAIR_INDEX_TYPE))
              (COMMENT (ada-string 1 32))))
   (ada-new-package repair_io (chained_io)
                    (repair_type) (repair_index_type))
   (ada-dcl repairs (file_type repair_io))

   (ada-constant-dcl units_name (ada-string) (str-lit "units.data"))
   (ada-type-dcl unit_key_type (ada-string 1 7))
   (ada-type-dcl unit_type
       (ada-rec (MODEL (MODEL_KEY_TYPE))
                (REPAIR (REPAIR_INDEX_TYPE))))
   (ada-new-package unit_io (keyed_io) (unit_type) (unit_key_type))
   (ada-dcl units (file_type unit_io)))))
```

```
(def-ada-top create_test_records
  ((CALENDAR) (Maintenance_FILES)) ((CALENDAR) (Maintenance_FILES))
 (def-ada-fn (CREATE_TEST_RECORDS ())
  ((ada-use (DEFECT_IO) (MODEL_IO) (REPAIR_IO) (UNIT_IO))
   (def-ada-fn (CLEAN_UP ()) ()
     ((ada-fn (CLOSE) DEFECTS)
      (ada-fn (CLOSE) MODELS)
      (ada-fn (CLOSE) REPAIRS)
      (ada-fn (CLOSE) UNITS))
     ((((status_error)) (ada-return)))))
  ((ada-fn (CREATE) DEFECTS (ENUM-lit OUT_FILE) defects_name)
   (ada-fn (CREATE) MODELS (ENUM-lit OUT_FILE) models_name)
   (ada-fn (CREATE) REPAIRS (ENUM-lit OUT_FILE) repairs_name)
   (ada-fn (CREATE) UNITS (ENUM-lit OUT_FILE) units_name)
   (ada-fn (WRITE) MODELS (ada-construct (model_type)
                    (str-lit "Opal Sorter     ")
                    (str-lit "Perth Mining    "))
            (str-lit "OS1"))
   (ada-fn (WRITE) MODELS (ada-construct (model_type)
                    (str-lit "Gas Analyzer    ")
                    (str-lit "Benson Labs     "))
            (str-lit "GA2"))
   (ada-fn (WRITE) DEFECTS (ada-construct (defect_type)
                     (str-lit "Control board cold solder joint ")
                     (str-lit "GA2"))
            (str-lit "GA-11"))
   (ada-fn (WRITE) DEFECTS (ada-construct (defect_type)
                     (str-lit "Clogged gas injection port      ")
                     (str-lit "GA2"))
            (str-lit "GA-32"))
   (ada-fn (WRITE) DEFECTS (ada-construct (defect_type)
                     (str-lit "Power supply thermistor blown   ")
                     (str-lit "OS1"))
            (str-lit "OS-03"))
   (ada-fn (WRITE) REPAIRS
         (ada-construct (repair_type)
          (ada-fn (TIME_OF) (int-lit 1982) (int-lit 9) (int-lit 14))
          (str-lit "GA-32")
          (int-lit 0)
          (str-lit "Probably caused by humidity.    "))
         (int-lit 1))
   (ada-fn (WRITE) REPAIRS
         (ada-construct (repair_type)
          (ada-fn (TIME_OF) (int-lit 1984) (int-lit 2) (int-lit 25))
          (str-lit "OS-03")
          (int-lit 0)
          (str-lit "Sorter arm got stuck.           "))
         (int-lit 2))
   (ada-fn (WRITE) REPAIRS
         (ada-construct (repair_type)
          (ada-fn (TIME_OF) (int-lit 1984) (int-lit 1) (int-lit 23))
          (str-lit "GA-11")
          (int-lit 1)
          (str-lit "Took two days to find.          "))
         (int-lit 3))
   (ada-fn (WRITE) REPAIRS
         (ada-construct (repair_type)
          (ada-fn (TIME_OF) (int-lit 1984) (int-lit 3) (int-lit 19))
          (str-lit "GA-32")
          (int-lit 3)
          (str-lit "Port Diameter seems below specs."))
         (int-lit 4))
   (ada-fn (WRITE) UNITS (ada-construct (unit_type) (str-lit "OS1") (int-lit 2))
            (str-lit "OS1-271"))
   (ada-fn (WRITE) UNITS (ada-construct (unit_type) (str-lit "GA2") (int-lit 4))
            (str-lit "GA2-342"))
   (ada-fn (CLEAN_UP)))
  ((((DEVICE_ERROR) (END_ERROR)
```

```
  (NAME_ERROR) (STATUS_ERROR))
 (ada-fn (CLEAN_UP)) (ada-fn (PUT) (str-lit "Data Base Inconsistent")))
(((others)) (ada-fn (CLEAN_UP)) (ada-raise)))))
```

```
; -*- Mode:LISP; Package: KBE; Base:10; Fonts:MEDFNB; -*-

;;; (c) Copyright 1983, Massachusetts Institute of Technology
;;;     All Rights Reserved.
;;;     Permission to copy all or part of this material is granted,
;;;     provided that the copies are not made or distributed for resale,
;;;     the MIT copyright notice and reference to the source file and
;;;     the software distribution version appear, and that notice is
;;;     given that copying is by permission of Massachusetts Institute
;;;     of Technology.

;NOTE WE MUST BE VERY CAREFULL IN THIS FILE NOT TO LET CODE
;CONSTANTS GET INTO THE DATA BASE!

;In the demo we have the notion of a logical file.  it is defined
;here because the coder takes advantage of it.

(defstruct (ada-file :conc-name (:type :named-list))
  name
  package-name
  record-type
  key-type
  name-variable
  record-def
  def)

(defvar *predefined-files* nil "alist of ada files defined by name")

;we call this here for convenience of testing.  it
;sets up the info about the standard files in MAINTENANCE_FILES.

(defun init-ada-files ()
  (setq *predefined-files*
        (copytree
        '((models .,
            (make-ada-file
              name 'models package-name 'model_io
              record-type 'model_type key-type 'model_key_type
              name-variable 'models_name
              record-def '(ada-rec (NAME (ada-string 1 16))
                                   (MAKER (ada-string 1 16)))
              def '(ada-file-def
                     (models model_io model_type
                             model_key_type models_name)
                     (ada-constant-dcl models_name (ada-string)
                                       (str-lit "models.data"))
                     (ada-type-dcl model_key_type (ada-string 1 3))
                     (ada-type-dcl model_type
                                   (ada-rec (name (ada-string 1 16))
                                            (maker (ada-string 1 16))))
                     (ada-new-package model_io
                       (keyed_io) (model_type) (model_key_type))
                     (ada-dcl models (file_type model_io)))))
          (defects .,
            (make-ada-file
              name 'defects package-name 'defect_io
              record-type 'defect_type key-type 'defect_key_type
              name-variable 'defects_name
              record-def '(ada-rec (NAME (ada-string 1 32))
                                   (model (model_key_type)))
              def '(ada-file-def
                     (defects defect_io defect_type
                       defect_key_type defects_name)
                     (ada-constant-dcl defects_name (ada-string)
                                       (str-lit "defects.data"))
                     (ada-type-dcl defect_key_type (ada-string 1 5))
                     (ada-type-dcl defect_type
                       (ada-rec (name (ada-string 1 32))
                                (model (model_key_type))))
```

```
                (ada-new-package defect_io
                  (keyed_io) (defect_type) (defect_key_type))
                (ada-dcl defects (file_type defect_io)))))
        (repairs .,
          (make-ada-file
            name 'repairs package-name 'repair_io
            record-type 'repair_type key-type 'repair_index_type
            name-variable 'repairs_name
            record-def '(ada-rec (date (time))
                                 (defect (defect_key_type))
                                 (next (repair_index_type))
                                 (comment (ada-string 1 32)))
            def '(ada-file-def
                   (repairs repair_io repair_type
                     repair_index_type repairs_name)
             (ada-constant-dcl repairs_name (ada-string)
                               (str-lit "repairs.data"))
             (ada-subtype-dcl repair_index_type (index_type))
             (ada-type-dcl repair_type
                           (ada-rec (date (time))
                                    (defect (defect_key_type))
                                    (next (repair_index_type))
                                    (comment (ada-string 1 32))))
             (ada-new-package repair_io
               (chained_io) (repair_type) (repair_index_type))
             (ada-dcl repairs (file_type repair_io)))))
        (units .,
          (make-ada-file
            name 'units package-name 'unit_io
            record-type 'unit_type key-type 'unit_key_type
            name-variable 'units_name
            record-def '(ada-rec (model (model_key_type))
                                 (repair (repair_index_type)))
            def '(ada-file-def
                   (units unit_io unit_type unit_key_type units_name)
             (ada-constant-dcl units_name (ada-string)
                               (str-lit "units.data"))
             (ada-type-dcl unit_key_type (ada-string 1 7))
             (ada-type-dcl unit_type
                           (ada-rec (model (model_key_type))
                                    (repair (repair_index_type))))
             (ada-new-package unit_io
               (keyed_io) (unit_type) (unit_key_type))
             (ada-dcl units (file_type unit_io)))))))))

(init-ada-files)
```

```
(defun get-ada-file-info (name)
  (when (or (symbolp name) (stringp name))
    (if (stringp name)
        (setq name (intern name)))
    (if (not (singular-form name))
        (setq name (intern (string-pluralize name))))
    (cdr (assq name *predefined-files*))))

(add-at-end-noticer 'check-for-new-files)

(declare (special *file-package-name*))

(defun check-for-new-files (form)
  (when (and (listp form) (eq (car form) 'def-ada-top)
             (eq (car (nth 4 form)) 'def-ada-package))
    (let ((*file-package-name* (cadr form)))
      (determine-ada-file-info (nth 2 (nth 4 form))))))

;given a declaration list, this looks to see if you
;are defining any new file objects, and stores the appropriate
;information this is also the place where we will create special
;cliches for the file.

(defun determine-ada-file-info (dcls)
  (setq dcls (clean-aggregate-file-defs (copytree dcls)))
  (dolist (dcl dcls)
    (when (and (eq (car dcl) 'ada-file-def)
               (not (get-ada-file-info (caadr dcl)))
               (assq 'ada-new-package (cddr dcl)))
      (handle-new-file dcl))))

(defun handle-new-file (dcl)
  (let* ((name (caadr dcl))
         (package-dcl (assq 'ada-new-package (cddr dcl)))
         (package-name (cadr package-dcl))
         (record-type (car (nth 3 package-dcl)))
         (record-def (dolist (d (cddr dcl))
                       (if (and (eq (car d) 'ada-type-dcl)
                                (eq (cadr d) record-type))
                           (return (caddr d)))))
         (key-type (car (nth 4 package-dcl)))
         (name-variable-dcl (assq 'ada-constant-dcl (cddr dcl)))
         (name-variable (cadr name-variable-dcl)))
    (push (cons name (make-ada-file name name
                                    package-name package-name
                                    record-type record-type
                                    key-type key-type
                                    name-variable name-variable
                                    record-def record-def
                                    def dcl))
          *predefined-files*)
    (make-new-cliches name)))
```

```
;this create the right PRINT_ and _HEADINGS cliches for a new file.

(defun make-new-cliches (name)
  (let* ((root (singular-form name))
         (print-name (symbolconc "PRINT_" root))
         (headings-name (symbolconc print-name "_HEADINGS"))
         (file-info (get-ada-file-info name))
         (record-def (ada-file-record-def file-info))
         ((print-stuff heading-string)
          (make-print-stuff root record-def)))
    (suggest-code-home print-name
                       '(:buffer "CLICHE-LIBRARY.ADA PS:<KBE.DEMO> OZ:"))
    (suggest-code-home headings-name
                       '(:buffer "CLICHE-LIBRARY.ADA PS:<KBE.DEMO> OZ:"))
    (code (create-plan-for-code
           (copytree
           '(def-ada-top ,print-name nil nil
              (def-ada-cliche ,print-name
                (primary-roles (,root)
                 described-roles (,root)
                 comment
                  (str-lit ,(string-append
                              "prints out the fields of {the "
                              (string-downcase root) "}")))
                ((ada-dcl ,root (,(ada-file-record-type file-info))))
                ((ada-setq ,root (kbe-/#-comment-macro ()
                                   (the input ,root)))
                 .,print-stuff))))))
    (code (create-plan-for-code
           (copytree
           '(def-ada-top ,headings-name ((text_IO)) ((text_IO))
              (def-ada-cliche ,headings-name
                (comment
                  (str-lit ,(string-append "prints headings for printing "
                                           (string-downcase root)
                                           " records")))
                ()
                ((ada-fn (PUT) (str-lit ,heading-string))
                 (ada-fn (NEW_LINE) (int-lit 2))))))))
    (typeout (text "The cliches ~A and ~A defined."
                   headings-name print-name))))
```

```
;this only works for things that can be printed on one line at the moment.

(defun make-print-stuff (var record-def)
  (let (print-stuff heading-stuff (current-column 1))
    (dolist (def (cdr record-def))
      (let* ((field-name (car def))
             (field-type (cadr def))
             ((field-printer field-length)
              (make-field-printer var field-name field-type)))
        (when field-printer
          (when (> current-column 1)
            (incf current-column)
            (push " " heading-stuff))
          (cond ((not (> (string-length field-name) field-length))
                 (setq field-name
                       (format nil "~V:@<~A~>" field-length
                               (string-capitalize
                                 (string-downcase (string field-name)))))
                 (when (> current-column 1)
                   (push '(ada-fn (set_col) (int-lit ,current-column))
                         print-stuff))
                 (push field-printer print-stuff)
                 (incf current-column field-length)
                 (push field-name
                       heading-stuff)
                 (push " " heading-stuff)
                 (incf current-column))
                (T (let* ((excess (- (string-length field-name)
                                     field-length))
                          (before (// excess 2))
                          (after (- excess before)))
                     (incf current-column before)
                     (when (> current-column 1)
                       (push '(ada-fn (set_col) (int-lit ,current-column))
                             print-stuff))
                     (push field-printer print-stuff)
                     (incf current-column field-length)
                     (push (string-capitalize
                             (string-downcase (string field-name)))
                           heading-stuff)
                     (incf current-column after)))))))
    (push (copytree '(ada-fn (NEW_LINE))) print-stuff)
    (setq heading-stuff (apply #'string-append (nreverse heading-stuff)))
    (list (nreverse print-stuff)
          (substring heading-stuff 0
                     (1+ (string-reverse-search-not-char
                           #\space heading-stuff))))))
```

```
(defun make-field-printer (var field-name field-type)
  (let (len)
    (cond ((eq (car field-type) 'ada-string)
           '((ada-fn (PUT) (ada-select ,var ,field-name))
             ,(caddr field-type)))
          ((eq (car field-type) 'time)
           '((ada-fn (PUT) (ada-fn (FORMAT_DATE)
                                   (ada-select ,var ,field-name)))
             10))
          ((eq field-name 'next) nil)
          ((equal field-type '(repair_index_type)) nil)
          ((setq len (get-predefined-type-len (car field-type)))
           '((ada-fn (PUT) (ada-select ,var ,field-name))
             ,len))
          (T (barf "unexpected type in file def ~A" field-type)))))

(defun get-predefined-type-len (type)
  (let* ((spot (string-search-char #/_ (string type)))
         (root (maybe-intern
                 (if spot (substring (string type) 0 spot) (string type))
                 (symbol-package type)))
         (file-def (ada-file-def (get-ada-file-info root))))
    (when file-def
      (dolist (dcl (cddr file-def))
        (if (and (eq type (cadr dcl))
                 (eq (caaddr dcl) 'ada-string))
            (return (caddr (caddr dcl)))))))))
```

```
;the following makes it possible for the analyzer to
;understand ada programs.  Everything is either expanded
;into lisp stuff, or just turned into a pseudo-function
;that is handled like an ordinary expr/lexpr.

;we have to have a definition here for everything that is
;a fexpr or macro above.

(comment a testing function
 (defun tt ()
  (setq segcase (analyze (surface-analyze (copytree test))))
  (code segcase)
  (setq code (?@seg-code (~seg segcase)))
  (if (not (equal (car code) test))
      (print "not equal"))
  (let (ada-supress) (pl code))) )

;note encoding of real fn name in symbol here.
;also we must add the file arg to the text_io fns in order for
;the analyzer to understand it easily.

(defvar *special-text_io-fns*
  '(new_line new_page set_col page line get put)
  "text_io fns that have an implicit file arg")

(defun (ada-fn k*expander) (form)
  (let* ((args (cddr form)) new-form)
    (if (memq (caadr form) *special-text_io-fns*)
        (push 'current_output args))
    (setq new-form '(,(symbolconc "ADA-FN~" (caadr form)) ., args))
    (when (eq (caadr form) 'read)
      (setq new-form
            '(setq ,(caddr new-form)
                   (ada-fn~read ,(cadr new-form) ., (cdddr new-form)))))
    (if (eq (caadr form) 'set_output)
        (setq new-form '(setq current_output ,new-form)))
    new-form))

;this should really be different if a var is being passed.
(defun (ada-attribute k*expander) (form)
  '(,(symbolconc "ADA-ATTRIBUTE~" (cadr form))))

(defun (ada-convert k*expander) (form)
  '(,(symbolconc "ADA-CONVERT~" (caadr form)) ,(caddr form)))

(defun (ada-construct k*expander) (form)
  '(,(symbolconc "ADA-CONSTRUCT~" (caadr form)) ., (cddr form)))

(defun (ada-select k*expander) (form)
  '(,(symbolconc "ADA-SELECT~" (caddr form))  ,(cadr form)))

(defun (str-lit k*expander) (form)
  '',form)
(defun (enum-lit k*expander) (form)
  '',form)
(defun (int-lit k*expander) (form)
  '',form)
```

```
;many declarations turn into things that just stay around.

(defun (ada-use k*expander) (form)
  '(ada-use~pseudo ',(cdr form)))

(defun (ada-file-def k*expander) (form)
  '(ada-file-def~pseudo ',(cdr form)))

(defun (ada-type-dcl k*expander) (form)
  '(ada-type-dcl~pseudo ',(cadr form) ',(caddr form)))

(defun (ada-subtype-dcl k*expander) (form)
  '(ada-subtype-dcl~pseudo ',(cadr form) ',(caddr form)))

;these also have to be processed by the forms that they bind in.

(defun (ada-init-dcl k*expander) (form)
  '(setq ,(cadr form)
         (ada-init-dcl~pseudo
           ',(cadr form) ',(caddr form) ,(cadddr form))))

(defun (ada-constant-dcl k*expander) (form)
  '(setq ,(cadr form)
         (ada-constant-dcl~pseudo
           ',(cadr form) ',(caddr form) ,(cadddr form))))

(defun (ada-dcl k*expander) (form)
 '(setq ,(cadr form)
         (ada-dcl~pseudo ',(cadr form) ',(caddr form))))

(defun (ada-new-fn k*expander) (form)
  (let* (((name params)
          (specs-for-new-ada-fn
            (append (caddr form) '(functions) nil) ;kludge
            (caadr form)))
         (internal-name (symbolconc '"ADA-FN~" name))
         (param-names (mapcar #'car params))
         (side-outs (mapcar #'(lambda (n)
                                '(,(symbolconc n "-OUT") side-effect ,n))
                            param-names)))
    (eval '(defun (,internal-name 6proto-seg) ()
             (6fast-defspecs
               '(,internal-name (z .,side-outs) .,param-names)))))
  '(ada-new-fn~pseudo ',(cdr form)))

(defun (ada-fn~create 6proto-seg) ()
  (6fast-defspecs
    '(ada-fn~create (z (file side-effect file-in)) file-in mode . name)))

(defun (ada-new-package k*expander) (form)
  '(ada-new-package~pseudo ',(cdr form)))

;these are for nested definitions only.  they don't
;understand hardly a thing.  they just remember what they
;are given so that they can produce it again.

(defun (def-ada-fn k*expander) (form)
  '(def-ada-fn~pseudo ',(cdr form)))

(defun (def-ada-generic k*expander) (form)
  '(def-ada-generic~pseudo ',(cdr form)))

(defun (def-ada-package k*expander) (form)
  '(def-ada-package~pseudo ',(cdr form)))

(defun (def-ada-package-body k*expander) (form)
  '(def-ada-package-body~pseudo ',(cdr form)))
```

```
;def-ada-top is handled specially in surface-analyze
;to generate these guys.

;these handle top level forms
;note that we must get rid of file closes because the loop analyzer
;would choke on them.  (This is just like the way with-open-file is
;handled in lisp.)  Note also that any exceptions and clean_up
;fn are similarly discarded.

(defun (ada-top-fn k*expander) (form)
  (let* (((nil (name params return) dcls stmnts except) form)
         (param-vars (mapcar #'car params))
         ((bound-vars . new-dcls) (discard-some-dcls dcls))
         (new-stmnts (discard-close-info stmnts)))
    (if return
        '(lambda ,param-vars
           (prog ,bound-vars ,@ new-dcls ,@ new-stmnts))
        '(lambda ,param-vars
           (let ,bound-vars ,@ new-dcls ,@ new-stmnts)))))

;note hack to let us no we have a package when we are done.

(defun (ada-top-package k*expander) (form)
  (let* (((nil name dcls) form))
    '(lambda nil ., (append dcls (ncons ''*package*)))))

;note that it would be a real problem to deal with return values
;from cliches without it forcing the program you put it in to
;become a function.  As it is since we only create procedures,
;we have no problem.

(defun (def-ada-cliche k*expander) (form)
  (let* (((nil name nil . body) form))
    (if (cdr body)
        (let* (((dcls stmnts except) body)
               ((bound-vars . new-dcls) (discard-some-dcls dcls))
               (new-stmnts (discard-close-info stmnts))
               (return? (assq 'ada-return new-stmnts)))
          (if return?
              '(prog ,bound-vars ,@ new-dcls ,@ new-stmnts)
              '(let ,bound-vars ,@ new-dcls ,@ new-stmnts)))
        (barf "not supported yet" form))))

(defun discard-some-dcls (dcls)
  (let (new-dcls bound-vars)
    (do ((d dcls (cdr d))) ((null d) (cons bound-vars (nreverse new-dcls)))
      (caseq (caar d)
        (def-ada-fn
          (if (not (eq (caadr (car d)) 'clean_up))
              (push (car d) new-dcls)))
        ((ada-dcl ada-init-dcl ada-constant-dcl)
         (push (cadar d) bound-vars)
         (push (car d) new-dcls))
        ((ada-use ada-type-dcl ada-subtype-dcl ada-new-package
          ada-new-fn def-ada-package def-ada-package-body
          def-ada-generic ada-file-def) (push (car d) new-dcls))
        (T (cbarf "unexpeted ada dcl ~a" (car d)))))))

(defun discard-close-info (stmnts)
  (let (new-stmnts)
    (do ((s stmnts (cdr s))) ((null s) (nreverse new-stmnts))
      (if (not (and (list (car s)) (eq (caar s) 'ada-fn)
                    (member (cadar s) '((close) (clean_up)))))
          (push (car s) new-stmnts)))))
```

```
(defun (ada-top-package-body k*expander) (form)
  (barf "not supported" form))

(defun (ada-top-generic k*expander) (form)
  (barf "not supported" form))

(defun (ada-begin k*expander) (form)
  (if (caddr form) (barf "ada exceptions not supported for begins"))
  '(progn ., (cadr form)))

;note that the only kind of exceptions we support are the following
;one kind in a begin in a loop.

(defun (ada-loop k*expander) (form)
  (let (((nil bod exceptions) form))
    (if exceptions (barf "ada exceptions not supported for loops"))
    (if (and (null (cdr bod)) (listp (car bod)) (eq (caar bod) 'ada-begin)
             (caddr (car bod))        ;has exceptions
             (equal (car (last (cadr (car bod))))
                    '(ada-exit))) ;exits if possible
        (let* ((begin (car bod))
               (nested-exceptions (caddr begin))
               (stmnts (nreverse (cdr (reverse (cadr begin))))))
          '(prog () L (if (no-ada-exceptions ',nested-exceptions)
                          (return nil))
                  ,@stmnts ,(append '(go L) nil)))
        '(prog () L ,@(cadr form) ,(append '(go L) nil)))))

(defun (ada-while k*expander) (form)
  (if (cadddr form) (barf "ada exceptions not supported for while"))
    '(prog () L (if (not ,(cadr form)) ,(append '(return) nil))
           ,@(caddr form) ,(append '(go L) nil)))

(defun (ada-cond k*expander) (form)
  '(cond ., (mapcar #'(lambda (clause)
                        (if (equal (car clause) '(enum-lit true))
                            (cons 't (cdr clause))
                            clause))
                    (cdr form))))

(defun (ada-return k*expander) (form)
  '(return ,(cadr form)))

(defun (ada-exit k*expander) (form)
 '(if ,(or (cadr form) (ncons 'ada-exit-zero-arg))
      ,(append '(return) nil)))

(defun (ada-raise k*expander) (form)
  (barf "ada exceptions not supported yet") form)

(defun (ada-setq k*expander) (form)
  '(setq ., (cdr form)))
```

```
;the following define which standard ada fns have side-effects.

(comment
   ;for the time being, we are pretending that these io fns
   ;do not side-effect their file input.

  (defun (ada-fn~create 6proto-seg) ()
    (6fast-defspecs
      '(ada-fn~create (z (file side-effect file-in)) file-in mode . name)))

  (defun (ada-fn~open 6proto-seg) ()
    (6fast-defspecs
      '(ada-fn~open (z (file side-effect file-in)) file-in mode . name)))

  (defun (ada-fn~reset 6proto-seg) ()
    (6fast-defspecs
      '(ada-fn~reset (z (file side-effect file-in)) file-in)))

  (defun (ada-fn~read 6proto-seg) ()
    (6fast-defspecs
      '(ada-fn~read (z (file side-effect file-in)) file-in . index)))

  (defun (ada-fn~set_index 6proto-seg) ()
    (6fast-defspecs
      '(ada-fn~set_index (z (file side-effect file-in)) file-in index)))  )

(defun (ada-fn~write 6proto-seg) ()
  (6fast-defspecs
    '(ada-fn~write (z (file side-effect file-in)) file-in item . index)))

;the following are for text_io fns with a file input.

(defun (ada-fn~new_line 6proto-seg) ()
  (6fast-defspecs
    '(ada-fn~new_line (z (file side-effect file-in)) file-in . num)))

(defun (ada-fn~set_col 6proto-seg) ()
  (6fast-defspecs
    '(ada-fn~set_col (z (file side-effect file-in)) file-in . num)))

(defun (ada-fn~new_page 6proto-seg) ()
  (6fast-defspecs
    '(ada-fn~new_page (z (file side-effect file-in)) file-in)))

(defun (ada-fn~get 6proto-seg) ()
  (6fast-defspecs
    '(ada-fn~get (z (file side-effect file-in)
                    (var-out side-effect var)) file-in var)))

(defun (ada-fn~put 6proto-seg) ()
  (6fast-defspecs
    '(ada-fn~put (z (file side-effect file-in)) file-in var . width)))
```

```
;the following undoes most of the above in a post pass after the coder
;is finished its normal work

(declare (special open-info type-info var-type-info var-list
                  use-info with-info fn-type did-set_output *saved-code*))

(comment
 (defun t2 ()
   (setq code (copytree *saved-code*))
   (cleanup-ada-code code)
  (let (all-code 5noop-tran)
    (5clean-noops1 code nil nil))
   (let (ada-supress) (pl code)))
) ;end comment

(defun cleanup-ada-code (code)
  (declare (special 5noop-tran all-code))
  (setq *saved-code* (copytree code)) ;for debugging ease
  (let (all-code 5noop-tran) ;we don't care if we loose pointing ability
    (5clean-noops1 code nil nil))
  (if (not (memq (car code) '(defun define-cliche)))
      (barf "can only code ada fns and cliches now, not expresions"))
  (cleanup-ada-fn-call code))

(defun cleanup-ada-fn-call (call)
  (when (and (listp call) (symbolp (car call)))
    (caseq (car call)
      (quote (clean-ada-quote call))
      (setq (clean-ada-setq call))
      (cond (clean-ada-cond call))
      (loop (clean-ada-loop call))
      (return (clean-ada-return call))
      (defun (clean-ada-defun call))
      (define-cliche (clean-ada-cliche call))
      (T (clean-ada-~fn call))))
  call)

;note that since we don't support ada-return in a loop, and
;the coder will never produce a return except in a loop, any
;return we run into must be an ada-exit.  But every such return
;will be nested in a cond from the way we do things, so
;if we run into a return, it is an error.

(defun clean-ada-return (call)
  (cbarf "continue and ignore" "erronious RETURN in ada code ~A" call)
  (clean-kill-call call))

(defun clean-ada-quote (call)
  (when (equal call '(quote *package*))
    (setq fn-type (cadr call))
    (clean-kill-call call))
  (when (and (listp (cadr call))
             (memq (caadr call) '(str-lit enum-lit int-lit)))
    (rplaca call (caadr call))
    (rplacd call (cdadr call))))

(defun clean-ada-setq (call)
  (caseq (if (listp (caddr call)) (caaddr call) nil)
    (ada-fn~read
      (let ((var (cadr call))) ;note work to preserve @seg pointers
        (rplaca call '5noop)
        (rplacd call (cddr call))
        (setq call (cadr call))
        (rplaca call 'ada-fn)
        (rplacd call (list* (ncons 'read) (cadr call) var (cddr call)))
        (mapc #'cleanup-ada-fn-call (cddr call))
        (clean-dcl-info call)
        (clean-open-info call)))
```

```
    (ada-fn~set_output
      (setq did-set_output t)
      (rplaca call 'ada-fn)
      (rplacd call (cons (ncons 'set_output) (cdaddr call)))
      (cleanup-ada-fn-call (caddr call)))
    (ada-init-dcl~pseudo
      (rplaca call 'ada-init-dcl)
      (rplacd call (list (cadr call)
                         (cadr (caddr (caddr call)))
                         (cadddr (caddr call))))
      (cleanup-ada-fn-call (cadddr call))
      (clean-dcl-info call)
      (clean-kill-call call))
    (ada-constant-dcl~pseudo
      (rplaca call 'ada-constant-dcl)
      (rplacd call (list (cadr call)
                         (cadr (caddr (caddr call)))
                         (cadddr (caddr call))))
      (cleanup-ada-fn-call (cadddr call))
      (clean-dcl-info call)
      (clean-kill-call call))
    (ada-dcl~pseudo
      (rplaca call 'ada-dcl)
      (rplacd call (list (cadr call)
                         (cadr (caddr (caddr call)))))
      (clean-dcl-info call)
      (clean-kill-call call))
    (T (cond ((eq (caddr call) 'nil)
              (clean-kill-call call))
             (T (rplaca call 'ada-setq)
                (cleanup-ada-fn-call (caddr call))
                (clean-dcl-info call))))))

(defun clean-kill-call (call)
  (rplaca call '5noop)
  (rplacd call nil))
```

```
;note that at the moment, there is nothing that can determine that
;a defun is supposed to be an ada function so we only create procedures.
;we would have to work something out to extend this.

(defun clean-ada-defun (call)
  (let* ((name (cadr call))
         (params (caddr call))
         (body (cdddr call))
         (aux-list (when (and (listp (car body))
                              (eq (caar body) 'let)
                              (null (cdr body)))
                     (prog1 (cadar body) (setq body (cddar body)))))
         (var-list (cons '*return* (append params aux-list)))
         open-info type-info var-type-info use-info with-info
         (final-thing (car (last call)))
         (fn-type (if (member final-thing '('*procedure* '*package*))
                      (cadr final-thing)
                      '*procedure*))
         did-set_output)
    (mapc #'cleanup-ada-fn-call body)
    (let* ((top-with (mapcar #'ncons
                             (sort (makeqset with-info) #'alphalessp)))
           (top-use (delete '(direct_io)
                            (delete '(sequential_io)
                                    (append top-with nil))))
           (opens (clean-get-opens open-info))
           (closes (clean-get-closes opens))
           (stmnts (append opens body closes))
           ;can only hack input args now
           (param-list (clean-get-params params))
           (dcls (clean-get-dcls aux-list)) ;must be after all other dcls
           (exceptions (clean-get-exceptions closes)))
      (rplaca call 'def-ada-top)
      (rplacd call
        (caseq fn-type
          (*package*
            '(,name ,top-with ,top-use
              (def-ada-package ,name ,dcls)))
          (T (let ((innards (if (or dcls stmnts exceptions)
                                (list dcls stmnts exceptions))))
               '(,name ,top-with ,top-use
                 (def-ada-fn (,name ,param-list) ., innards)))))))))

(defun clean-ada-cliche (call)
  (let* ((name (cadr call))
         (comments (caddr call))
         (stmnts (cdddr call))
         open-info type-info var-type-info use-info with-info
         (fn-type '*procedure*) did-set_output aux-list var-list)
    (when (and (null (cdr stmnts))
               (listp (car stmnts))
               (eq (caar stmnts) 'let))
      (setq aux-list (cadar stmnts))
      (setq stmnts (cddar stmnts)))
    (setq var-list aux-list)
    (mapc #'cleanup-ada-fn-call stmnts)
    (let* ((top-with (mapcar #'ncons
                             (sort (makeqset with-info) #'alphalessp)))
           (top-use (delete '(direct_io)
                            (delete '(sequential_io)
                                    (append top-with nil))))
           (opens (clean-get-opens open-info))
           (closes (clean-get-closes opens))
           (new-stmnts (append opens stmnts closes))
           (dcls (clean-get-dcls aux-list)) ;must be after get-closes
           (exceptions (clean-get-exceptions closes)))
      (rplaca call 'def-ada-top)
      (rplacd call
```

```
(caseq fn-type
  (*procedure*
    (let ((innards (if (or dcls new-stmnts exceptions)
                       (list dcls new-stmnts exceptions))))
      '(,name ,top-with ,top-use
        (def-ada-cliche ,name ,comments .. innards)))))))))
```

```
(defun dummy-ada-role ()
  (copytree '(kbe-/#-comment-macro () ())))

(defun clean-get-params (vars)
  (mapc #'(lambda (v)
            (if (not (or (assq v var-type-info) (assq v type-info)))
                (push '(,v ada-dcl ,v ,(dummy-ada-role))
                      var-type-info)))
        vars)
  (mapcar #'(lambda (p)
              (or (let* ((entry (assq p var-type-info)))
                    (setq var-type-info (delq entry var-type-info))
                    (cddr entry))
                  (let* ((entry (assq p type-info)))
                    (setq var-type-info (delq entry type-info))
                    (cddr entry))))
          vars))

(defun clean-get-dcls (aux)
  (mapc #'(lambda (v)
            (if (not (or (assq v var-type-info) (assq v type-info)))
                (push '(,v ada-dcl ,v ,(dummy-ada-role))
                      var-type-info)))
        aux)
  (let* ((dcls (append (mapcar #'cdr var-type-info)
                       (mapcar #'cdr type-info))))
    (setq dcls (clean-sort-dcls dcls))
    (when (eq (caar dcls) 'ada-use)
      (setq use-info (append (mapcar #'car (cdr (pop dcls))) use-info)))
    (setq use-info (sort (makeset use-info) #'alphalessp))
    (when (memq 'int_io use-info)
      (setq use-info (delq 'int_io use-info))
      (setq use-info (append use-info (ncons 'int_io))))
    (if use-info
        (push '(ada-use .,(mapcar #'ncons use-info)) dcls))
    dcls))

(defun clean-type-info (info)
  (if (clean-refers-to info 'time) (push 'calendar with-info))
  (if (clean-refers-to info 'text_io) (push 'text_io with-info))
  (if (memq (car info) '(ada-new-package ada-new-fn))
      (caseq (caaddr info)
        ((chained_io keyed_io sort summarize) (push 'functions with-info))
        ((sequential_io direct_io) (push (caaddr info) with-info))))
  (push (cons (cadr info) (copytree info)) type-info))

(defun clean-refers-to (list item)
  (or (eq list item)
      (and (listp list)
           (dolist (l list)
             (if (clean-refers-to l item) (return T))))))
```

```
(defun clean-sort-dcls (dcls)
  (setq dcls (clean-aggregate-file-defs dcls))
  (setq dcls (sort dcls #'clean-dcl-sort))
  (prog (packages)
      L (when (not (eq (caar dcls) 'ada-use))
        (if packages (push '(ada-use ., (makeset packages)) dcls))
        (return nil))
      (setq packages (append packages (cdr (pop dcls))))
      (go L))
  (map #'disaggregate-file-def dcls)
  dcls)

(defun clean-aggregate-file-defs (dcls)
  (let* (file-defs removed-dcls file-dcls)
    (dolist (dcl dcls)
      (when (and (eq (car dcl) 'ada-dcl) (eq (caaddr dcl) 'file_type))
        (push dcl file-defs)))
    (when file-defs
      (dolist (def file-defs)
        (let* ((root (singular-form (first-symbol (cadr def))))
               (package-dcl (dcl-for (symbolconc root "_IO") dcls))
               (name-dcl (dcl-for (symbolconc (cadr def) "_NAME") dcls))
               (type-dcls
                 (delq nil
                       (Rlist (dcl-for (car (Elist (reverse
                                                     (cdddr package-dcl))))
                                       dcls))))
               (use (car (member '(ada-use (,(cadr package-dcl))) dcls)))
               file-dcl-list)
          (push def file-dcl-list)
          (if use (push use file-dcl-list))
          (when package-dcl (push package-dcl file-dcl-list))
          (setq file-dcl-list (append type-dcls file-dcl-list))
          (when name-dcl (push name-dcl file-dcl-list))
          (setq removed-dcls (append file-dcl-list removed-dcls))
          (push '(ada-file-def ,(nreverse ;important is file var first
                                  (mapcar #'cadr file-dcl-list))
                               ., file-dcl-list)
                file-dcls)))
      (setq dcls (append file-dcls (qset- dcls removed-dcls))))
    dcls))

(defun dcl-for (symbol dcls)
  (car (mem #'(lambda (x y) (eq x (cadr y))) symbol dcls)))

(defun disaggregate-file-def (dcls)
  (when (eq (caar dcls) 'ada-file-def)
    (rplacd dcls (append (cdddar dcls) (cdr dcls)))
    (rplaca dcls (caddar dcls))))
```

```
(defun clean-dcl-sort (dcl1 dcl2)
  (let* ((class1 (clean-dcl-class dcl1))
         (class2 (clean-dcl-class dcl2)))
    (or (ordered-by-references dcl1 dcl2)
        (when (not (ordered-by-references dcl2 dcl1))
          (or (< class1 class2)
              (when (= class1 class2)
                (caseq class1
                  ((0 6) nil)
                  (1 (if (= (length (cadr dcl1)) (length (cadr dcl2)))
                         (alphalessp-dcl dcl1 dcl2)
                         (< (length (cadr dcl1)) (length (cadr dcl2)))))
                  ((2 3) (alphalessp-dcl dcl1 dcl2))
                  ((4 5) (alphalessp-dcl dcl1 dcl2)))))))))

(defun clean-dcl-class (dcl)
  (caseq (car dcl)
    ((ada-use) 0)
    ((ada-file-def) 1)
    ((ada-type-dcl ada-subtype-dcl ada-new-package) 2)
    ((ada-dcl ada-constant-dcl ada-init-dcl) 3)
    (def-ada-fn (if (equal (cadr dcl) '(clean_up nil)) 6 4))
    (ada-new-fn 5)))

(defun alphalessp-dcl (dcl1 dcl2)
  (alphalessp (first-symbol (cadr dcl1)) (first-symbol (cadr dcl2))))

(defun first-symbol (thing)
  (if (listp thing) (first-symbol (car thing)) thing))

(defun ordered-by-references (dcl1 dcl2)
  (caseq (car dcl1)
    (ada-use nil)
    (ada-file-def (Ror-fast (5po1 dcl2 (Elist (cadr dcl1)))))
    ((def-ada-fn ada-new-fn) (5po1 dcl2 (caadr dcl1)))
    (T (5po1 dcl2 (cadr dcl1)))))
```

```
(defun clean-dcl-info (info)
  (caseq (car info)
    ((ada-constant-dcl ada-init-dcl)
     (if (or (eq fn-type '*package*) (memq (cadr info) var-list))
         (let ((old-spot (assq (cadr info) var-type-info)))
           (if old-spot (rplacd old-spot (copytree info))
               (push (cons (cadr info) (copytree info)) var-type-info)))))
    (ada-dcl
      (if (clean-refers-to info 'text_io) (push 'text_io with-info))
      ;used to have (when (not (not-fully-specified info))
      (if (or (eq fn-type '*package*) (memq (cadr info) var-list))
          (let ((old-spot (assq (cadr info) var-type-info)))
            (if old-spot
                (unless (not-fully-specified info)
                  (rplacd old-spot (copytree info)))
                (push (cons (cadr info) (copytree info))
                      var-type-info)))))
    (ada-setq (clean-dcl-info-setq info))
    (ada-fn (clean-dcl-info-fn info))))

;Note fn names that are ambiguous type-wise or use/with-wise are ignored.
;This fn should be interacting with a real parser and overload resolution
;and everything.  It is just a gross hack for now.

(defun clean-dcl-info-fn (info)
  (caseq (caadr info) ;figure with
    ((time_of clock month day year) (push 'calendar with-info))
    ((set_output standard_output new_line new_page set_col page line)
     (push 'text_io with-info))
    ((get put)
     (cond ((cdddr info) ;is an integer PUT (GROSS KLUDGE here).
            (push 'functions with-info) (push 'int_io use-info))
           (T (push 'text_io with-info))))
    ((format_date query_user_for_date) (push 'functions with-info))
    ((read write close open create reset size index set_index end_of_file)
     (let ((package
             (ada-file-package-name (get-ada-file-info (caddr info)))))
       (when package
         (push 'maintenance_files with-info)
         (push package use-info)))))
;this part is very incomplete it works in the demo only understanding
;READ since almost every variable in the demo touches a read.
  (caseq (caadr info) ;figure types
    (read
     (let* ((file-info (get-ada-file-info (caddr info)))
            (record-type (or (ada-file-record-type file-info)
                             (intern (string-append
                                       (file-name-root (caddr info))
                                       "_TYPE"))))
            (key-type (or (ada-file-key-type file-info)
                          (intern (string-append
                                    (file-name-root (caddr info))
                                    "_KEY_TYPE")))))
       (clean-dcl-var-type (cadddr info) record-type)
       (if (car (cddddr info))
           (clean-dcl-var-type (car (cddddr info)) key-type))))))

;this is so gross it makes the fn above look great.  It should (and could)
;be really looking for the types of things in the type list and all,
;rather for ease in debugging, it just has appropriate canned knowledge
;about strings which is all it needs to know for the demo.

(defun clean-dcl-info-setq (info)
  (when (listp (caddr info))
    (cond ((or (eq (car (caddr info)) 'str-lit)
               (and (eq (car (caddr info)) 'ada-fn)
                    (eq (caadr (caddr info)) '&)))
           (let ((size (clean-string-size (caddr info))))
```

```
            (clean-dcl-var-type
              (cadr info)
              '(ada-string 1 ,(if (plusp size) size (dummy-ada-role))))))
          ((and (eq (car (caddr info))  'ada-fn)
                (eq (caadr (caddr info)) 'query_user_for_date))
           (clean-dcl-var-type (cadr info) 'time)))))

;this is awfully cludgy, it could be looking things up much better.

(defun clean-string-size (exp)
  (cond ((symbolp exp)
         (caseq exp
           ((model_key ) 3)
           ((defect_key) 5)
           ((unit_key) 7)
           (T -1000)))
        ((listp exp)
         (caseq (car exp)
           (str-lit (string-length (cadr exp)))
           (ada-select
             (cond ((equal (cdr exp) '(model name)) 16)
                   (T -1000)))
           (ada-fn
             (caseq (caadr exp)
               (format_date 10)
               (& (+ (clean-string-size (caddr exp))
                     (clean-string-size (cadddr exp))))))
           (T -1000)))
        (T -1000)))

(defun clean-dcl-var-type (var type)
  (when (symbolp var)
    (let ((spot (assq var var-type-info))
          (new-info
            '(ada-dcl ,var ,(if (symbolp type) (ncons type) type))))
      (cond ((not spot)
             (push (cons var new-info) var-type-info))
            ((not-fully-specified spot)
             (rplacd spot new-info))))))

(defun not-fully-specified (list)
  (cond ((not (listp list)) nil)
        ((eq (car list) 'kbe-/#-comment-macro) T)
        (T (dolist (item list)
             (if (not-fully-specified item) (return T))))))
```

```
(defun clean-open-info (info)
  (let* ((file (caddr info))
         (entry
           (copytree
             (caseq (caadr info)
               ((open create) info)
               (read '(ada-fn (open) ,file
                              (enum-lit in_file)
                              ,(symbolconc file  "_NAME")))
               (write '(ada-fn (open) ,file
                               (enum-lit out_file)
                               ,(symbolconc file "_NAME"))))))
         (old-entry (cdr (assq (caddr info) open-info))))
    (cond ((null old-entry) (push (cons file entry) open-info))
          (t (if (not (equal (cadr entry) (cadr old-entry)))
                 (rplaca (cdr old-entry) (ncons 'create)))
             (if (not (equal (cadddr entry) (cadddr old-entry)))
                 (rplaca (cdddr old-entry)
                         (copytree '(enum-lit inout_file))))))))

(defun clean-get-opens (open-info)
  (sort (mapcar #'cdr open-info)
        #'(lambda (a b)
            (if (equal (cadr a) (cadr b))
                (alphalessp (caddr a) (caddr b))
                (equal (cadr a) '(open))))))

(defun clean-get-closes (opens)
  (let* ((closes (mapcar #'(lambda (open) '(ada-fn ,(ncons 'close)
                                                   ,(caddr open)))
                         opens)))
    (if did-set_output
        (push (copytree '(ada-fn (set_output) (ada-fn (standard_output))))
              closes))
    (when (cdr closes)
      (clean-type-info '(def-ada-fn ,(copytree '(CLEAN_UP nil)) nil
                         ,closes
                         ,(copytree '((((STATUS_ERROR)) (ada-return))))))
      (setq closes (copytree '((ada-fn (clean_up))))))
    closes))

(defun clean-get-exceptions (closes)
  (if closes
      (copytree
        '((((DEVICE_ERROR) (END_ERROR) (NAME_ERROR) (STATUS_ERROR))
          ,@ closes
          (ada-fn (PUT) (str-lit "Data Base Inconsistent")))
         (((others)) ,@ closes (ada-raise))))))
```

```
(defun clean-ada-~fn (call)
  (let* ((~spot (string-search-char #/~ (string (car call)))))
    (if (null ~spot) (mapc #'cleanup-ada-fn-call (cdr call))
        (let* ((name-string (string (car call)))
               (prefix (intern (substring name-string 0 ~spot) 'kbe))
               (root (intern (substring name-string (1+ ~spot))
                             (symbol-package (car call)))))

          (rplaca call prefix)
          (caseq prefix
            ((ada-use ada-file-def)
              (clean-type-info (rplacd call (cadr (cadr call))))
              (clean-kill-call call))
            (ada-dcl
              (rplacd call (list (cadr (cadr call)) (cadr (caddr call))))
              (clean-dcl-info call)
              (clean-kill-call call))
            ((ada-constant-dcl ada-init-dcl)
             (rplacd call (list (cadr (cadr call))
                                (cadr (caddr call))
                                (cadddr call)))
             (cleanup-ada-fn-call (cadddr call))
             (clean-type-info call)
             (clean-kill-call call))
            ((ada-type-dcl ada-subtype-dcl)
             (clean-type-info
               (list prefix (cadr (cadr call)) (cadr (caddr call))))
             (clean-kill-call call))
            ((ada-new-fn ada-new-package)
             (rplacd call (cadr (cadr call)))
             (clean-type-info call)
             (clean-kill-call call))
            ((def-ada-fn def-ada-generic
              def-ada-package def-ada-package-body)
             (rplacd call (cadr (cadr call)))
             (clean-type-info call)
             (clean-kill-call call))
            (ada-select
              (rplacd call (list (cadr call) root))
              (if (not (symbolp (cadr call)))
;                 (barf "cannot select field of non variable ~A" call))
;
              )
            ((ada-construct ada-convert)
             (push (ncons root) (cdr call))
             (mapc #'cleanup-ada-fn-call (cddr call)))
            (ada-attribute
              (push root (cdr call)))
            (ada-fn
              (if (eq root 'read) ;here only if value of read not needed
                  ;we must think up a var name (GROSS!)
                  (push (singular-form (cadr call)) (cddr call)))
              (if (memq root *special-text_io-fns*)
                  (rplaca (cdr call) (ncons root))
                  (push (ncons root) (cdr call)))
              (mapc #'cleanup-ada-fn-call (cddr call))
              (clean-dcl-info call)
              (if (memq root '(open create read write))
                  (clean-open-info call))
              (if (memq root '(open create))
                  (clean-kill-call call))))))))
```

```
(defun clean-ada-loop (call)
  (let* ((stmnts (cddr call)) while-test exceptions moved-stmnt new-loop)
    (dolist (stmnt stmnts)
      (when (and (listp stmnt)
                 (eq (car stmnt) 'cond)
                 (null (cddr stmnt))
                 (listp (car (cadr stmnt)))
                 (eq (caar (cadr stmnt)) 'no-ada-exceptions))
        (setq exceptions (cadr (cadr (car (cadr stmnt)))))
        (setq moved-stmnt stmnt)
        (return nil))
      (when (and (listp stmnt)
                 (eq (car stmnt) 'cond)
                 (null (cddr stmnt))
                 (listp (car (cadr stmnt)))
                 (eq (caar (cadr stmnt)) 'not))
        (setq while-test (cadr (car (cadr stmnt))))
        (setq moved-stmnt stmnt)
        (return nil)))
    (if moved-stmnt (setq stmnts (delq moved-stmnt stmnts)))
    (setq new-loop
          (cond (while-test
                 (cleanup-ada-fn-call while-test)
                 (list 'ada-while while-test stmnts nil))
                (exceptions
                 (list 'ada-loop
                       '((ada-begin (,@ stmnts (ada-exit))
                                    ,exceptions))
                       nil))
                (T (list 'ada-loop stmnts nil))))
    (rplaca call (car new-loop))
    (rplacd call (cdr new-loop))
    (mapc #'cleanup-ada-fn-call stmnts)))

(defun clean-ada-cond (call)
  (cond ((and (null (cddr call))
              (null (cddr (cadr call)))
              (member (cadr (cadr call)) '((return nil) (return))))
         (rplaca call 'ada-exit)
         (if (equal (car (cadr call)) '(ada-exit-zero-arg))
             (rplacd call nil)
             (rplacd call (ncons (car (cadr call)))))
         (cleanup-ada-fn-call (cadr call)))
        (T (rplaca call 'ada-cond)
           (dolist (clause (cdr call))
             (if (eq (car clause) 't)
                 (rplaca clause (copytree ''(enum-lit true))))
             (mapc #'cleanup-ada-fn-call clause)))))
```

```
;; -*- Mode:LISP; Package:KBE; Base:10; Fonts:MEDFNB; -*-

;note idea of having points in rules that you
;cannot back up through in order to aid debugability.

;;; (c) Copyright 1983, Massachusetts Institute of Technology
;;;     All Rights Reserved.
;;;     Permission to copy all or part of this material is granted,
;;;     provided that the copies are not made or distributed for resale,
;;;     the MIT copyright notice and reference to the source file and
;;;     the software distribution version appear, and that notice is
;;;     given that copying is by permission of Massachusetts Institute
;;;     of Technology.

;;; Support for letting the apprentice hack a small subset of ADA.

;This file is a grammar for the subset of ADA supported in the 84 demo.
;The grammar below is executable and conforms to the following rules.

;Each production has the form:
;(def-rule ada-pattern lisp-pattern)
;the lisp-pattern is occasionally omitted this means that it
;is exactly like the ada-pattern
;If the correspondence between the two lines is not obvious, non terminal
;names of the form <non-terminal>1 <non-terminal>2 will sometimes be used.
;The numbers do not change the meaning of the non terminal, but
;just serve to identify individuals.

;<...>  is a non terminal grammar symbol.
;other lisp atoms in rules are terminal symbols

;(... ...) indicates a sequence of forms
;(optional ... ...) indicates an optional sequence of forms
;(one-of <...> <...>) indicates alteration of non-terminals
;  note that it is faster if the common cases are first, but
;  if one case is an initial substring of another than it must be first.
;(zero-or-more <...> after after-last?)
;  This indicates repetitions of a non-terminal.  it expects to find the
;  AFTER terminal (if any) between the items.  if the AFTER-LAST? flag
;  is present then the AFTER must be after every instance.
;(one-or-more <...> between after)
;  this is the same but requires at least one.

;The grammar here is quite weak.  (Ie it can generate a lot of
;junk that is not ada.)  However it can parse the ada subset correctly,
;and making it weaker makes it simpler in a lot of places.

(comment

(defun parse-test ()
  (declare (special pt test result))
  (setq result (ada-parse pt))
  (parse-comp)
  (let (ada-supress) (pl result)))

(defun parse-comp ()
  (declare (special test result ada-supress))
  (let ((ada-supress t))
    (parse-comp1 result test)))

(defun parse-comp1 (a b)
  (when (not (equal a b))
    (format T "~%~:=~% is not the same as the required~%~:=" a b)
    (if (and (listp a) (listp b) (not (= (length a) (length b))))
        (format t "~2% **** note their length is not the same"))
    (if (y-or-n-p "go deeper?")
        (mapcar #'parse-comp1 a b))))
)
```

```
;the parser here works in a very straightforward (and probably slow) way.
;it takes in a string and returns a parse tree list.
;As a first pass over the string it is converted to a list of tokens.
;the grammer rules parse this list of tokens producing a new list of
;tokens.  This is then grouped up into a parse tree.

(defvar *token-in-stream* nil "parse point for ada-parser")
(defvar *last-form-parsed* nil "last stmnt or dcl successfully parsed")

(defun ada-parse-to-end (&optional (stream standard-input)
                                   (end-chars '(#\End)))
  (ada-parse (readstring stream end-chars)))

(defun ada-parse (ada-string &optional (node '<sentence>))
  (declare (special ada-use-packages ada-info ada-prefix-stack))
  (let* ((ada-use-packages nil)
         (ada-info ada-info)
         (ada-prefix-stack (ncons '*parsing*))
         (ada-tokens (ada-tokenize ada-string))
         (lisp-tokens (ada-tokens-parse ada-tokens node)))
    (ada-gather-lisp-tokens lisp-tokens)))

(defun ada-gather-lisp-tokens (*token-in-stream* &optional error-supress)
  (setq *token-in-stream* (delq '*dummy* *token-in-stream*))
  (let* ((token (pop *token-in-stream*))
         (result (if (eq token '/() (sub-gather) token)))
    (if (and *token-in-stream* (not error-supress))
        (grumble "parse- output not just one form"))
    result))

(defun sub-gather ()
  (let (result token)
    (loop do
      (when (null *token-in-stream*)
        (grumble "parse- too few close parens in output")
        (return result))
      (setq token (pop *token-in-stream*))
      (caseq token
        (/( (push (sub-gather) result))
        (/) (return (nreverse result)))
        (T (push token result))))))

(defun ada-tokens-parse (*token-in-stream* node)
  (let* (*last-form-parsed* (result (funcall node)))
    (when (or (null result) *token-in-stream*)
      (barf "ada parse failed last dcl//stmnt parsed:~% ~:="
            (ada-gather-lisp-tokens *last-form-parsed* T)))
    result))
```

```
;this works from the back so that it can get the tokens in the
;right order.  also it just so happens that some things are easier
;to tokenize from the back. (and some aren't.)

(eval-when (eval load compile)

(defvar *ada-break-chars*
        (string-append #\newline #\tab #\lf "/" //.:;=<>,+-&*#{}()'"))

(defun ada-remove-comments (string)
  (let (chunks (start 0) hit)
    (loop do
      (setq hit (string-search "--" string start))
      (when (null hit)
        (when (null chunks) (return string))
        (push (substring string (min start (string-length string))) chunks)
        (return (apply #'string-append (nreverse chunks))))
      (push (substring string start hit) chunks)
      (setq start (or (string-search-char #\newline string hit)
                      (string-length string))))))

(defun ada-tokenize (string)
  (setq string (ada-remove-comments string))
  (let (tokens (scan (string-length string)) char)
    (loop do
      (setq scan
            (or (string-reverse-search-not-set
                  #.(string-append #\space #\newline #\tab #\lf)
                  string scan)
                -1))
      (if (minusp scan) (return tokens))
      (setq char (aref string scan))
      (cond
        ((= char #/")
         (let ((start (string-reverse-search-char #/" string scan)))
           (push (substring string (1+ start) scan) tokens)
           (setq scan start)))
        ((= char #/})
         (push '/} tokens)
         (let ((start (string-reverse-search-set "{." string scan)))
           (when start
             (push (substring string (1+ start) scan) tokens)
             (push (ascii (aref string start)) tokens)
             (setq scan start))))
        ((string-search-char char *ada-break-chars*)
         (let ((pair (and (> scan 0)
                          (substring string (1- scan) (1+ scan)))))
           (cond ((member pair '(".." "//=" "<=" ">=" "=>" "<>" ":="))
                  (push (intern pair) tokens)
                  (decf scan))
                 (T (push (intern (substring string scan (1+ scan)))
                          tokens)))))
        (T (let* ((break (or (string-reverse-search-set
                               *ada-break-chars* string scan) -1))
                  (non-number (or (string-reverse-search-not-set
                                    "0123456789" string (1+ scan)) -1))
                  (substring (substring string (1+ break) (1+ scan))))
             (if (= break non-number)
                 (push (read-from-string substring) tokens)
                 (push (intern (string-upcase substring)) tokens))
             (setq scan (1+ break))))))))
);end of eval-when
```

```
(defmacro def-rule (name ada-pattern &optional (lisp-pattern ada-pattern))
  (let* (((new-ada-pattern new-lisp-pattern vars)
          (process-rule-body ada-pattern lisp-pattern)))
    '(defun ,name ()
       (let ,vars
         (if ,new-ada-pattern ,new-lisp-pattern)))))

(eval-when (eval load compile)

(defun process-rule-body (ada-pattern lisp-pattern)
  (let* (((new-ada-pattern in-vars) (process-pattern-in ada-pattern))
         ((new-lisp-pattern out-vars) (process-pattern-out lisp-pattern)))
    (if (< (length in-vars) (length (makeqset in-vars)))
        (barf "def-rule: input var used twice."))
    (if (< (length out-vars) (length (makeqset out-vars)))
        (barf "def-rule: output var used twice."))
    (if (not (qset< out-vars in-vars))
        (barf "def-rule: output of rule refs vars not in input"))
    (list new-ada-pattern new-lisp-pattern in-vars)))

(defvar *pattern-vars* nil)

(defun process-pattern-in (pattern)
  (let (*pattern-vars*)
    (list (pp-in pattern) *pattern-vars*)))

(defun process-pattern-out (pattern)
  (let (*pattern-vars*)
    (list (pp-out pattern) *pattern-vars*)))

(defun non-terminal-root (symbol)
  (cond ((not (and (symbolp symbol)
                   (> (string-length symbol) 2)
                   (= (aref (string symbol) 0) #/<))) nil)
        ((= (aref (string symbol) (1- (string-length symbol))) #/>) symbol)
        (T (intern (substring symbol 0
                     (1+ (string-reverse-search-char #/> symbol)))))))
```

```
(defun pp-in (form)
  (cond ((symbolp form)
         (let ((root (non-terminal-root form)))
           (cond ((null root)
                  (let ((tokens (ada-tokenize (string form))))
                    (if (not (= (length tokens) 1))
                        (barf "def-rule: unrecognizable token ~a" form)))
                  '(terminal-in ,form))
                 (T (push form *pattern-vars*)
                    '(non-terminal-in ,form ,root)))))
        ((not (listp form)) (barf "def-rule: bad pattern form ~a" form))
        ((eq (car form) 'optional)
         '(optional-in ., (mapcar #'pp-in (cdr form))))
        ((eq (car form) 'one-of)
         (dolist (f (cdr form))
           (if (not (symbolp f))
               (barf "def-rule: one-of too complex ~a" form)))
         '(one-of-in ., (mapcar #'pp-in (cdr form))))
        ((memq (car form) '(one-or-more zero-or-more))
         (let ((root (non-terminal-root (cadr form))))
           (if (not (and root (symbolp (caddr form))
                         (not (non-terminal-root (caddr form)))
                         (memq (cadddr form) '(T nil))))
               (barf "def-rule: bad repetition ~a" form))
           (push (cadr form) *pattern-vars*)
           '(repetition-in ,(cadr form) ,root ,(eq (car form) 'one-or-more)
                           ., (cddr form))))
        ((eq (car form) 'computation)
         (if (not (and (symbolp (cadr form))
                       (non-terminal-root (caddr form))
                       (memq (caddr form) *pattern-vars*)
                       (null (cdddr form))))
             (barf "def-rule: computation too complex ~a" form))
         '(computation-in ., (cdr form)))
        (T '(sequence-in ., (mapcar #'pp-in form)))))
```

```
(defun pp-out (form)
  (cond ((symbolp form)
         (let ((root (non-terminal-root form)))
           (cond ((null root) '(terminal-out ,form))
                 (T (push form *pattern-vars*)
                    '(non-terminal-out ,form)))))
        ((not (listp form)) (barf "def-rule: bad pattern form ~a" form))
        ((eq (car form) 'optional)
         (dolist (f (cdr form))
           (if (not (or (symbolp f)
                        (and (listp f)
                             (memq (car f) '(one-or-more zero-or-more)))))
               (barf "def-rule: optional form too complex ~a" form)))
         '(optional-out ., (mapcar #'pp-out (cdr form))))
        ((eq (car form) 'one-of)
         (dolist (f (cdr form))
           (if (not (non-terminal-root f))
               (barf "def-rule: output one-of too complex ~a" form))
           (push f *pattern-vars*))
         '(one-of-out ., (cdr form)))
        ((memq (car form) '(one-or-more zero-or-more))
         (let ((root (non-terminal-root (cadr form))))
           (if (not (and root (null (cddr form))))
               (barf "def-rule: bad output repetition ~a" form))
           (push (cadr form) *pattern-vars*)
           '(repetition-out ,(cadr form))))
        ((eq (car form) 'computation)
         (if (not (and (symbolp (cadr form))
                       (non-terminal-root (caddr form))
                       (null (cdddr form))))
             (barf "def-rule: computation too complex ~a" form))
         ;note allowing any kind of var ref here
         ;you must be carfull of nconcs in code
         '(computation-out ., (cdr form)))
        (T '(sequence-out ., (mapcar #'pp-out form)))))

) ;end of eval-when
```

```
;a form returns NIL iff it fails in parsing.
;also every var assigned by a successful parse is non-NIL.

(defmacro sequence-in (&body forms)
  '(and ., forms))

(defmacro optional-in (&body forms)
  (let ((form (if (cdr forms) '(and ., forms) (car forms))))
    '(let ((old-token-in-stream *token-in-stream*))
       (cond ((null ,form) (setq *token-in-stream* old-token-in-stream)))
       T)))

(defmacro one-of-in (&body forms)
  '(let ((old-token-in-stream *token-in-stream*))
     (one-of-in1 ., forms)))

(defmacro one-of-in1 (&body forms)
  (if (null (cdr forms)) (car forms)
      '(cond (,(car forms))
             (T (setq *token-in-stream* old-token-in-stream)
                (one-of-in1 ., (cdr forms))))))

;note use of *dummy* so that NIL is never found.
(defmacro repetition-in (var name require-one?
                         &optional after after-every?)
  (let* ((new-var (symbolconc "NEW-" var))
         (list-var (symbolconc "LIST-" var))
         (form '(non-terminal-in ,new-var ,name)))
    (when after
      (if after-every?
          (setq form '(and ,form (terminal-in ,after)))
          (setq form '(and (or (null ,list-var)
                               (terminal-in ,after))
                           ,form))))
    '(prog (,new-var ,list-var saved-token-in-stream)
        L (setq saved-token-in-stream *token-in-stream*)
          (cond (,form
                 (setq ,list-var (append ,list-var ,new-var))
                 (go L)))
          (setq *token-in-stream* saved-token-in-stream)
          ,@(if require-one? '((cond ((null ,list-var) (return nil)))))
          (setq ,var (cons '*dummy* ,list-var))
          (return T))))

(defmacro non-terminal-in (var name)
  '(setq ,var (,name)))

(defmacro terminal-in (terminal)
  '(eq ',terminal (pop *token-in-stream*)))

(defmacro computation-in (fn var)
  '(,fn ,var))
```

```
(defmacro sequence-out (&body forms)
  '(append ., forms))

;cannot have a sequence-out optional-out or one-of-out form
(defmacro optional-out (&body forms)
  (if (null (cdr forms))
      (car forms)
      (let (test-vars)
        (dolist (form forms)
          (if (memq (car form) '(non-terminal-out 'repetition-out))
              (push (cadr form) test-vars)))
        (if (cdr test-vars)
            (setq test-vars '(and ., test-vars))
            (setq test-vars (or (car test-vars) T)))
        '(cond (,test-vars (append ., forms))))))

(defmacro one-of-out (&body vars)
  '(or ., vars))

;must make sure we always have something.
(defmacro repetition-out (var)
  '(or ,var (ncons '*dummy*)))

(defmacro non-terminal-out (var)
  '(progn ,var)) ;to help macroexpandall

(defmacro terminal-out (terminal)
  '(ncons ',terminal))

(defmacro computation-out (fn var)
  '(,fn ,var))
```

```
;A problem arises parsing ada from the fact that
;in ada all of the following things are realized as symbols and
;cannot be distinguished in isolation.  As a result I have a function
;ADA-SYMBOL-TYPE which when passed a symbol will return the indicated
;code showing its type in the local context.

;many of the problems with symbols are actually disambiguated by the
;syntactic context.  However, several things can still be confused which
;appear in the same context.  For example the following indistinguishable
;pairs of things can all be <exp>s.
; A zero arg <fn call> looks like a <variable>.
; A <converter> looks like a one argument <fn call>.
; A <field selector> looks like a zero arg <fn call>.
; A <variable> looks like an <enumeration literal>.

;we are experimenting with making these as weak as possible

;(defmacro <variable> () '(parse-token-type 'variable))
(defmacro <variable> () '(<symbol>))

;(defmacro <field-name> () '(parse-token-type 'field))
;should do better than this for the ada parsability problems.
;must think about it.
(defun <field-name> ()
  (let ((token (pop *token-in-stream*)))
    (if (or (memq token '(key count name))
            (memq (ada-symbol-type token) '(field variable)))
        (ncons token))))

(defmacro <attribute-name> () '(parse-token-type 'attribute))

(defmacro <fn-name-root> () '(parse-token-type 'fn))

(defmacro <generic-name-root> () '(parse-token-type 'generic))

(defmacro <enumeration-element-root> () '(parse-token-type 'enum))

(defmacro <package-name-root> () '(parse-token-type 'package "_IO"))

;(defmacro <exception-name-root> () '(parse-token-type 'exception))
(defun <exception-name-root> ()
  (let ((token (pop *token-in-stream*)))
    (if (or (eq token 'others) (eq (ada-symbol-class token) 'symbol))
        (ncons token))))

(defmacro <basic-type-name-root> () '(parse-token-type 'type "_TYPE"))

(def-rule <type-name-root>
  (one-of <limited-private> <basic-type-name-root>))

(def-rule <limited-private> (LIMITED PRIVATE) LIMITED-PRIVATE)

(defmacro <basic-mode> () '(parse-token-type 'mode))

(def-rule <mode> (one-of <in-out> <basic-mode>))

(def-rule <in-out> (IN OUT) IN-OUT)
```

```
(defmacro <symbol> () '(parse-token-class 'symbol))

(defmacro <operator> () '(parse-token-class 'operator))

(defmacro <integer> () '(parse-token-class 'integer))

(defmacro <string> () '(parse-token-class 'string))

(defun <word> ()
  (let ((token (pop *token-in-stream*)))
    (if (symbolp token) (ncons token))))

(defun parse-token-type (required-type &optional name_string)
  (when *token-in-stream*
    (let* ((token (pop *token-in-stream*))
           (type (ada-symbol-type token)))
      (if (or (eq required-type type)
              (and name_string (string-search name_string token)))
          (ncons token)))))

(defun parse-token-class (required-class)
  (when *token-in-stream*
    (let ((token (pop *token-in-stream*)))
      (if (eq required-class (ada-symbol-class token)) (ncons token)))))

(defun ada-symbol-class (token)
  (cond ((numberp token) 'integer)
        ((stringp token) 'string)
        ((eq token 'entry) 'keyword) ;KBE pseudo-op for dummy definitions.
        ((memq token
               '(/. /, /: /; => /:= <> /' /( /) with use package end body
                 procedure function generic cliche in out when exception
                 is new return constant type subtype renames record
                 string range begin loop while if then elsif else exit
                 raise private limited others))
         'keyword)
        ((memq token '(or xor and = //= <= >= < > + - &
                       not abs * // mod rem **))
         'operator)
        (T 'symbol)))

;In order for ADA-SYMBOL-TYPE to work, you have to bind the variable
;ADA-INFO each time you cross a <def form> contour and evaluate each
;<dcl> immediately after it is parsed.

;this takes in a root symbol and tells you what kind of thing it is.
;it can be buggy if a non local root symbol has been defined to be
;things of more than one catagory.  It will just tell you the first
;answer it finds.  It is not used in this file.  It is here to help
;the parser.

(defun ada-symbol-type (token)
  (declare (special ada-info))
  (cond ((memq token '(length last)) 'ATTRIBUTE)
        ((memq token '(in out)) 'mode)
        ((eq token 'others) 'exception)
        ((eq token 'private) 'type)
        ((do ((info ada-info (cdr info))) ((null info))
           (when (eq token (cadaar info))
             (return (caaar info)))))
        (T 'unknown)))
```

```
(def-rule <sentence> (one-of <top-level-form> <dcl> <stmnt> <exp>))

(def-rule <top-level-form>
  ((optional WITH <contour-names+> /;)
   (optional <top-use>)
   <def-form>1 /;
   (optional <def-form>2 /;))
  (/( DEF-ADA-TOP (computation get-def-name <def-form>1)
      /( <contour-names+> /)
      /( <top-use> /)
      <def-form>1 (optional <def-form>2) /) ))

(defun <top-use> ()
  (let* ((use (<basic-top-use>)))
    (when use
      (eval (ada-gather-lisp-tokens '( /( ada-use ,@ use /) )))
      use)))

(def-rule <basic-top-use>
  (USE <contour-names+> /;)
  <contour-names+>)

(def-rule <def-form>
  (one-of <fn> <cliche> <package> <package-body> <generic> <dummy>))

(defun get-def-name (defs)
  (ncons (caseq (cadr defs)
           ((def-ada-package def-ada-package-body def-ada-cliche)
            (nth 2 defs))
           ((def-ada-fn def-ada-entry) (nth 3 defs))
;          (def-ada-generic )
           (T (barf "parse- unknown def form")))))

(def-rule <package>
  (PACKAGE <symbol> (optional IS <dcls> END <symbol>2))
  (/( DEF-ADA-PACKAGE <symbol> /( <dcls> /) /)))

(def-rule <package-body>
  (PACKAGE BODY <symbol> <dcls> END (optional <symbol>2))
  (/( DEF-ADA-PACKAGE-BODY <symbol> /( <dcls> /) /)))

(def-rule <dummy> (ENTRY <fn-spec>)
  (/( DEF-ADA-ENTRY <fn-spec> /) ))

(def-rule <fn>
  ((one-of PROCEDURE FUNCTION) <fn-spec>
   (optional IS <dcls> BEGIN <stmnts+> <exception-clauses>
             END (optional <fn-designator>)))
  (/( DEF-ADA-FN <fn-spec>
      (optional /( <dcls> /) /( <stmnts+> /)
                /( <exception-clauses> /)) /) ))

(def-rule <generic>
  (GENERIC <dcls> (one-of <fn> <package>))
  (/( DEF-ADA-GENERIC /( <dcls> /) (one-of <fn> <package>) /)))

(def-rule <fn-spec>
  (<fn-designator> <params> (optional RETURN <type>))
  (/( <fn-designator> <params> (optional <type>) /) ))

(def-rule <fn-designator>
  (one-of <symbol> <operator-string>))

(def-rule <params>
  (optional /( (one-or-more <param> /;) /))
  (/( (zero-or-more <param>) /)))

(def-rule <param>
```

```
(<symbol> /: (optional <mode>) <type>)
(/( <symbol> <type> (optional <mode>) /)))
```

```
(def-rule <cliche>
   (one-of <full-cliche> <dcl-cliche> <expr-cliche> <empty-cliche>))

(def-rule <empty-cliche>
  (CLICHE <symbol>)
  (/( DEF-ADA-CLICHE <symbol> NIL /)))

(def-rule <full-cliche>
  (CLICHE <symbol> IS <cliche-comments>
          <dcls> BEGIN <stmnts+> <exception-clauses>
     END (optional <symbol>2))
  (/( DEF-ADA-CLICHE <symbol>
      <cliche-comments> /( <dcls> /) /( <stmnts+> /)
      /( <exception-clauses> /) /)))

(def-rule <dcl-cliche>
  (CLICHE <symbol> IS <cliche-comments> <dcls>
     END (optional <symbol>2))
  (/( DEF-ADA-CLICHE <symbol>
      <cliche-comments> /( <dcls> /) /( /) /( /) /)))

(def-rule <expr-cliche>
  (CLICHE <symbol> IS <cliche-comments> <exp>
     END (optional <symbol>2))
  (/( DEF-ADA-CLICHE <symbol> <cliche-comments> <exp> /)))

(def-rule <cliche-comments>
  ((optional <cliche-primary-roles-comment>)
   (optional <cliche-described-roles-comment>)
   (optional <cliche-comment-comment>)
   (optional <cliche-constraints-comment>))
  (/( (optional <cliche-primary-roles-comment>)
      (optional <cliche-described-roles-comment>)
      (optional <cliche-comment-comment>)
      (optional <cliche-constraints-comment>) /)))

(def-rule <cliche-primary-roles-comment>
  (PRIMARY ROLES (one-or-more <word> /,) /;)
  (PRIMARY-ROLES /( (one-or-more <word>) /)))

(def-rule <cliche-described-roles-comment>
  (DESCRIBED ROLES (one-or-more <word> /,) /;)
  (DESCRIBED-ROLES /( (one-or-more <word>) /)))

(def-rule <cliche-comment-comment>
  (COMMENT <string-literal> /;)
  (COMMENT <string-literal>))

(def-rule <cliche-constraints-comment>
  (CONSTRAINTS <stmnts+> END CONSTRAINTS /;)
  (CONSTRAINTS /( <stmnts+> /)))
```

```
(def-rule <dcls>
  (zero-or-more <dcl> /; T)
  (zero-or-more <dcl>))

(defun <dcl> ()
  (let* ((dcl (<basic-dcl>)))
    (when dcl
      (setq *last-form-parsed* dcl)
      ;to do vars right must add more cases here.
      (cond ((eq (cadr dcl) 'ada-use)
             (ignore-errors (eval (ada-gather-lisp-tokens dcl))))
            ((memq (cadr dcl) '(ada-type-dcl ada-subtype-dcl))
             (ignore-errors ;cludge for now
               (eval (ada-gather-lisp-tokens dcl))))
            ((memq (cadr dcl) '(def-ada-fn ada-new-fn))
             (push (ncons (list 'fn (cadddr dcl)))
                   ada-info))
            ((memq (cadr dcl) '(ada-new-package def-ada-package
                                def-ada-cliche def-ada-generic))
             'think)))
    dcl))

(def-rule <basic-dcl>
  (one-of <init-dcl> <constant-dcl> <variable-dcl>
          <use> <with> <type-dcl> <subtype-dcl>
          <new-package> <new-fn> <def-form>
          <role> <dcl-annotation>))

(def-rule <dcl-annotation>
  (/{ <dcl> /, <role-name-string> /})
  (/( kbe-/#-comment-macro /( <dcl> /)
      (computation fix-role-name-string <role-name-string>) /)))

(def-rule <new-package>
  (PACKAGE <symbol> IS NEW <instance>)
  (/( ADA-NEW-PACKAGE <symbol> <instance> /)))

(def-rule <new-fn>
  ((one-of FUNCTION PROCEDURE) <fn-spec> IS NEW <instance>)
  (/( ADA-NEW-FN <fn-spec> <instance> /)))

(def-rule <instance> (<generic-name> <generic-args>))

(def-rule <generic-args>
  (optional /( (zero-or-more <generic-arg> /,) /))
  (zero-or-more <generic-arg>))

(def-rule <generic-arg> (one-of <fn-name> <type-name>))

(def-rule <variable-dcl>
  (<symbol> /: <type>)
  (/( ADA-DCL <symbol> <type> /)))

(def-rule <init-dcl>
  (<symbol> /: <type> /:= <exp>)
  (/( ADA-INIT-DCL <symbol> <type> <exp> /)))

(def-rule <constant-dcl>
  (<symbol> /: CONSTANT <type> /:= <exp>)
  (/( ADA-CONSTANT-DCL <symbol> <type> <exp> /)))

(def-rule <use>
  (USE <contour-names+>)
  (/( ADA-USE <contour-names+> /)))

(def-rule <with>
  (WITH (one-of FUNCTION PROCEDURE) <fn-spec> (optional IS <box>))
  (/( ADA-WITH <fn-spec> (optional <box>) /)))
```

```
(def-rule <box> <>)
```

```
(def-rule <type-dcl>
  (TYPE <symbol> IS <type>)
  (/( ADA-TYPE-DCL <symbol> <type> /)))

(def-rule <subtype-dcl>
  (SUBTYPE <symbol> IS <type>)
  (/( ADA-SUBTYPE-DCL <symbol> <type> /)))

(def-rule <type>
  (one-of <type-name> <enumeration> <exception> <record>
          <string-type> <range> <role>))

(def-rule <enumeration>
  (/( (one-or-more <symbol> /,) /))
  (/( ADA-ENUMERATION (one-or-more <symbol>) /)))

(def-rule <exception>
  (EXCEPTION (optional RENAMES <exception-name>))
  (/( ADA-EXCEPTION (optional <exception-name>) /)))

(def-rule <record>
  (RECORD <fields+> END RECORD)
  (/( ADA-REC <fields+> /)))

(def-rule <fields+>
  (one-or-more <field> /; T)
  (one-or-more <field>))

(def-rule <field>
  (one-of <true-field> <role>))

(def-rule <true-field>
  (<symbol> /: <type>)
  (/( <symbol> <type> /)))

(def-rule <string-type>
  (STRING (optional <range-exp>))
  (/( ADA-STRING (optional <range-exp>) /)))

(def-rule <range>
  ((optional <type-name>) range <range-exp>)
  (/( ADA-range (computation stuff-or-nil <type-name>) <range-exp> /)))

(defun stuff-or-nil (stuff)
  (or stuff '(nil)))

(def-rule <range-exp> (one-of <range-pair> <box>))

(def-rule <range-pair>
  (/( <integer>1 |..| (one-of <integer>2 <role>) /))
  (<integer>1 (one-of <integer>2 <role>)))
```

```
(def-rule <stmnts+>
  (one-or-more <stmnt> /; T)
  (one-or-more <stmnt>))

(def-rule <stmnt>
  <basic-stmnt>
  (computation mark-check-point <basic-stmnt>))

(defun mark-check-point (stuff)
  (setq *last-form-parsed* stuff))

(def-rule <basic-stmnt>
  (one-of <begin> <loop> <while> <cond> <return> <exit>
          <raise> <setq> <fn-call> <role> <stmnt-annotation>))

(def-rule <stmnt-annotation>
  (/{ <stmnt> /, <role-name-string> /})
  (/( kbe-/#-comment-macro /( <stmnt> /)
      (computation fix-role-name-string <role-name-string>) /)))

(def-rule <begin>
  (BEGIN <stmnts+> <exception-clauses> END)
  (/( ADA-BEGIN /( <stmnts+> /) /( <exception-clauses> /) /)))

(def-rule <loop>
  (LOOP <stmnts+> <exception-clauses> END LOOP)
  (/( ADA-LOOP /( <stmnts+> /) /( <exception-clauses> /) /)))

(def-rule <while>
  (WHILE <exp> LOOP <stmnts+> <exception-clauses> END LOOP)
  (/( ADA-WHILE <exp> /( <stmnts+> /) /( <exception-clauses> /) /)))

(def-rule <cond>
  (<if> (zero-or-more <elsif>) (optional <else>) END IF)
  (/( ADA-COND <if> (zero-or-more <elsif>) (optional <else>) /)))

(def-rule <if>
  (IF <exp> THEN <stmnts+>)
  (/( <exp> <stmnts+> /)))

(def-rule <elsif>
  (ELSIF <exp> THEN <stmnts+>)      ;note ELSIF not ELSEIF
  (/( <exp> <stmnts+> /)))

(def-rule <else>
  (ELSE <stmnts+>)
  (/( /( ENUM-LIT TRUE /) <stmnts+> /)))

(def-rule <return>
  (RETURN (optional <exp>))
  (/( ada-return (optional <exp>) /)))

(def-rule <exit>
  (EXIT (optional WHEN <exp>))
  (/( ADA-EXIT (optional <exp>) /)))

(def-rule <raise>
  (RAISE (optional <exception-name>))
  (/( ADA-RAISE (optional <exception-name>) /)))

(def-rule <setq>
  (<variable> /:= <exp>)
  (/( ADA-SETQ <variable> <exp> /)))

(def-rule <exception-clauses>
  (optional EXCEPTION (one-or-more <exception-clause>))
  (zero-or-more <exception-clause>))
```

```
(def-rule <exception-clause>
  (WHEN (one-or-more <exception-name> /|) => <stmnts+>)
  (/( /( (one-or-more <exception-name>) /) <stmnts+> /)))
```

```
(def-rule <exps>
  (optional /( (one-or-more <exp> /,) /))
  (zero-or-more <exp>))

(def-rule <exps+>
  (/( (one-or-more <exp> /,) /))
  (one-or-more <exp>))

(def-rule <basic-exp-unit>
  (one-of <literal> <parenthesised-exp>
          <attribute> <record-constructor> <field-selector>
          <fn-call> <role> <converter> <variable>))

(def-rule <parenthesised-exp>
  (/( <exp> /))
  <exp>)

(def-rule <role> (one-of <complex-role> <simple-role> <exp-annotation>))

(def-rule <simple-role>
  (/{ <role-name-string> /})
  (/( kbe-/#-comment-macro /( /)
      (computation fix-role-name-string <role-name-string>) /)))

(def-rule <maybe-modified-exps+>
  (/( (one-or-more <maybe-modified-exp> /,) /))
  (one-or-more <maybe-modified-exp>))

(def-rule <maybe-modified-exp>
  (one-of <modified-exp> <exp>))

(def-rule <modified-exp>
  (/{ <exp> /, <modified> /})
  (&in-out <exp>))

(defun <modified> ()
  (let ((token (pop *token-in-stream*)))
    (if (and (stringp token)
             (or (string-equal token "modified")
                 (string-equal token " modified")))
        (ncons token))))

(def-rule <complex-role>
  (/{ <role-name-string> /} <maybe-modified-exps+>)
  (/( kbe-/#-comment-macro /( /( |...| <maybe-modified-exps+> /) /)
     (computation fix-role-name-string <role-name-string>) /)))

(defun fix-role-name-string (stuff)
  (ncons (string->kbe-comment-name (car stuff))))

(def-rule <exp-annotation>
  (/{ <exp> /, <role-name-string> /})
  (/( kbe-/#-comment-macro /( <exp> /)
      (computation fix-role-name-string <role-name-string>) /)))

(def-rule <fn-call>
  (<fn-name> <exps>)
  (/( ADA-FN <fn-name> <exps> /)))

(def-rule <converter>
  (<type> /( <exp> /))
  (/( ADA-CONVERT <type> <exp> /)))

(def-rule <attribute>
  (<type-name> /' <attribute-name>)
  (/( ADA-ATTRIBUTE <type-name> <attribute-name> /)))

(def-rule <record-constructor>
```

```
  (<type> /' <exps+>)
  (/( ADA-CONSTRUCT <type> <exps+> /)))

(def-rule <field-selector>
  ((one-of <role> <variable>) /. <field-name>)
  (/( ADA-SELECT (one-of <role> <variable>) <field-name> /)))
```

```
(defun <exp> ()
  (if (could-possibly-be-op-exp) (<op-exp>) (<basic-exp-unit>)))

;having this loop ahead speeds up the parser by a factor of more than 2.
(defun could-possibly-be-op-exp ()
  (let ((paren-count 0))
    (dolist (token *token-in-stream*)
      (caseq token
        ((or xor and = //= <= >= < > + - & not abs * // mod rem **)
         (return T))
        (/; (return nil))
        (/( (incf paren-count))
        (/) (if (plusp paren-count) (decf paren-count) (return nil)))
        (/, (if (not (plusp paren-count)) (return nil)))))))

;this takes care of operator precedence and the like
(defun <op-exp> ()
  (let (op-stack arg-stack (last-kind 'op))
    (loop do
      (let (((thing kind op-priority) (ada-get-exp-thing))
            (op-type 'binary))
        (caseq kind
          (arg
            (if (eq last-kind 'arg) (return nil))
            (setq last-kind 'arg)
            (push thing arg-stack))
          (op
            (cond ((eq last-kind 'op)
                   (if (not (memq thing '(not abs + -)))
                       (return nil))
                   (setq op-priority 4)
                   (setq op-type 'unary))
                  (T (if (memq thing '(not abs)) (return nil))
                     (setq last-kind 'op)))
            (if (loop do
                  (if (null op-stack) (return))
                  (let (((top-op top-type top-priority) (car op-stack)))
                    (if (> op-priority top-priority) (return nil))
                    (cond ((eq top-type 'binary)
                           (if (not (> (length arg-stack) 1)) (return t))
                           (let ((second-arg (pop arg-stack))
                                 (first-arg (pop arg-stack)))
                             (push '(/( ada-fn /( ,top-op /)
                                        ,@first-arg ,@second-arg /))
                                   arg-stack)))
                          (T (if (null arg-stack) (return nil))
                             (push '(/( ada-fn /( ,top-op /)
                                        ,@(pop arg-stack) /))
                                   arg-stack)))
                    (pop op-stack)))
                (return nil))
            (push (list thing op-type op-priority) op-stack)))
        (when (eq thing '*done*)
          (if (not (and (= (length arg-stack) 1)
                        (= (length op-stack) 1)))
              (return nil)
              (return (car arg-stack)))))))))
```

```
(defun ada-get-exp-thing ()
  (let* ((saved-token-in-stream *token-in-stream*)
         (thing (<operator>)))
    (cond (thing
           '(,(car thing) op
             ,(caseq (car thing)
                ((OR XOR AND) 1)
                ((= //= <= >= < >) 2)
                ((+ - &) 3)
                ((not abs) 4)
                ((* // mod rem) 5)
                ((**) 6))))
          (T (setq *token-in-stream* saved-token-in-stream)
             (setq thing (<basic-exp-unit>))
             (cond (thing '(,thing arg))
                   (T (setq *token-in-stream* saved-token-in-stream)
                      '(*done* op 0)))))))

;the operator priorities are used to disambiguate the nesting
;order of expressions (higher numbers bind more tightly, and things
;of equal priority are done leftmost first).  Parentheses can also be used.
```

```
(def-rule <literal>
  (one-of <integer-literal> <string-literal> <enumeration-literal>))

(def-rule <fn-name>
  (<qualifications> <fn-name-root>)
  (/( <fn-name-root> <qualifications> /)))

(def-rule <type-name>
  (<qualifications> <type-name-root>)
  (/( <type-name-root> <qualifications> /)))

(def-rule <package-name>
  (<qualifications> <package-name-root>)
  (/( <package-name-root> <qualifications> /)))

(def-rule <generic-name>
  (<qualifications> <generic-name-root>)
  (/( <generic-name-root> <qualifications> /)))

(def-rule <exception-name>
  (<qualifications> <exception-name-root>)
  (/( <exception-name-root> <qualifications> /)))

(def-rule <enumeration-element-name>
  (<qualifications> <enumeration-element-root>)
  (<enumeration-element-root> <qualifications>))

(def-rule <contour-name>
  (<qualifications> <contour-name-root>)
  (/( <contour-name-root> <qualifications> /)))

(def-rule <contour-names+>
  (one-or-more <contour-name> /,)
  (one-or-more <contour-name>))

(def-rule <contour-name-root>
  (one-of <package-name-root> <fn-name-root> <generic-name-root>))
```

```
(def-rule <qualifications>
  (zero-or-more <contour-name-root> /. T)
  (computation reverse <contour-name-root>))
;IN REVERSE ORDER eg: FOO.BAR.BAZ.  (BAZ BAR FOO)

;note that these literals depend on things having been tokenized right
;in the first place.

(def-rule <integer-literal> <integer> (/( int-lit <integer> /)))

(def-rule <string-literal> <string> (/( str-lit <string> /)))

(def-rule <enumeration-literal>
  <enumeration-element-name>
  (/( ENUM-LIT <enumeration-element-name> /)))

;the tokenizer has to work hard for this one.
(def-rule <role-name-string> <string>)

(defun <operator-string> ()
  (car (member (pop *token-in-stream*)
               '("<=" "+")))) ;etc
```

```
;;; -*- Mode:LISP; Package:KBE; Base:10; Fonts:MEDFNB; -*-

;;; (c) Copyright 1983, Massachusetts Institute of Technology
;;;     All Rights Reserved.
;;;     Permission to copy all or part of this material is granted,
;;;     provided that the copies are not made or distributed for resale,
;;;     the MIT copyright notice and reference to the source file and
;;;     the software distribution version appear, and that notice is
;;;     given that copying is by permission of Massachusetts Institute
;;;     of Technology.

;;; Support for letting the apprentice hack a small subset of ADA.
;;; this has gprint formats to print out ADA-pseudo-Lisp as ADA;


;this is a general program for printing code without abbreviation.
;Anybody anywhere who wants to print code in any langauge should call
;this fn.  Strictly speaking you should not be printing code any other way
;(except maybe in debugging).
;(there should be a better place for this fn than in this file.)
;Note that we use GPRINT to print ADA and PLI, but the intention
;is that we don't use it anywhere else in the kbe any more.

(defun print-code (code-list stream &optional pp:*print-lines*)
  (let ((prinmargin pp:*print-right-margin*)
        (prinlevel nil)
        (prinlength nil)
        (prinendline pp:*print-lines*)
        (base 10.)
        (*nopoint t))
    (caseq *display-language*
      (lisp (let ((*print-pretty* T))
              (prin1 code-list stream)))
      (pli (print-as-pli code-list stream))
      (ada (let (ada-supress)
             (declare (special ada-supress))
             ;;this sends messages that mess up any pprinting
             (Gprint1 code-list stream))))))

 (comment
(progn (setq prinlength 4)
       (setq prinmargin 83)
       (defun x () (pl test))) )

; some things for the editor interface.

(defprop def-ada-top "ADA definition" si:definition-type-name)

(defvar ada-functions nil "list of all ada functions")

(defun make-ada-fn (name)
  (if (not (memq name ada-functions)) (push name ada-functions)))

(defun ADA? (form)
  (and (listp form) (memq (car form) ada-functions)))

(defun ada-definition-type-name (form)
  (or (when (and (listp form) (eq (car form) 'def-ada-top))
        (let ((inner (nth 4 form)))
          (caseq (car inner)
            (def-ada-cliche "cliche")
            (def-ada-package "package")
            (def-ada-fn
              (if (caddr (cadr inner)) "function" "procedure")))))
      "program"))
```

```
(defvar ada-supress t "turns off ada printing")

(defmacro def-ada-gf (name ada-template)
  '(def-ada-gformat ,name form
     (gf ,ada-template form)))

(defmacro def-ada-gformat (name args &body ada-format)
  '(progn 'compile
          (make-ada-fn ',name)
          (defun (,name :Gformat) (|form|)
            (cond (ada-supress (non-ada-format |form|))
                  (T (let ,(ada-process-argl '|form| args)
                       .,ada-format))))))

(defun non-ada-format (form)
  (cond ((memq (car form) '(def-ada-fn def-ada-package def-ada-package-body
                            def-ada-top def-ada-generic ada-begin ada-loop
                            ada-while def-ada-cliche def-ada-entry))
         (gf "(1*<-*>)" form))
        (T (:Gfn-format form))))

(eval-when (eval load compile)
(defun ada-process-argl (var args)
  (do ((argl args (cdr argl))
       (i 1 (1+ i))
       (result))
      ((null argl) (reverse result))
    (cond ((listp argl) (push '(,(car argl) (nth ,i ,var)) result))
          (T (push '(,argl (nthcdr ,i ,var)) result) (setq argl nil))))))
```

```
;helping functions for common subparts of formats.

(defvar *nested-ada-fn-spec* nil)

(defun ada-fn-spec (fn-spec)
  (let ((name (car fn-spec))
        (params (cadr fn-spec))
        (return (caddr fn-spec)))
    (if *nested-ada-fn-spec*
        (cond (return (gf "{3'function'_"))
              (T (gf "{3'procedure'_")))
      (cond (return (gf "{0'function'_"))
            (T (gf "{0'procedure'_"))))
    (gf "*{3[.(1<%ada-param-gf ';',>)]" name params)
    (cond (return (gf "-'return '%ada-type " return)))
    (gf "}")))

(defun ada-param-gf (param-spec)
  (gf "{0*':'" (car param-spec))
  (if (cddr param-spec)
      (gf "_S" (cdr (assq (caddr param-spec)
                          '((in . |in|) (out . |out|)
                            (in-out . |in out|)))))))
  (gf "_%ada-type }" (cadr param-spec)))

(defun ada-exceptions (list)
  (if list (gf "{3'exception'[<!%ada-when-clause >]}-" list)))

(defun ada-when-clause (pair)
  (gf "{3'when'[[<,%ada-qname ' /|'>]' =>'-<*';',>]';'}" pair))
```

```
(defun ada-dcls (body)
  (let ((*nested-ada-fn-spec* T))
    (gf "+3")
    (do ((forms body (cdr forms))
         (last-form nil (car forms)))
        ((null forms))
      (if (and last-form (ada-gap-between-dcls last-form forms))
          (gf "!' '"))
      (gf "!*';'" (car forms))
      (setq last-form (car forms)))
    (gf "+-3!")))

(defun ada-gap-between-dcls (dcl1 dcls)
  (or (and (not (ada-interesting-def dcl1))
           (ada-interesting-def (car dcls)))
      (ada-starts-file-def dcl1 dcls)
      (ada-ends-file-def dcl1)))

(defun ada-interesting-def (x)
  (or (memq (car x) '(ada-new-fn def-ada-generic
                      def-ada-package def-ada-package-body))
      (and (eq (car x) 'def-ada-fn)
           (or (nth 2 x) (nth 3 x) (nth 4 x)))))

(defun ada-ends-file-def (dcl)
  (and (eq (car dcl) 'ada-dcl) (eq (caaddr dcl) 'file_type)))

(defun ada-starts-file-def (dcl1 dcls)
   (let (root)
     (dolist (dcl dcls)
       (when (ada-ends-file-def dcl)
         (setq root (singular-form (cadr dcl)))
         (return)))
     (and root
          (not (eq 0 (string-search (string root)
                                    (first-symbol (cadr dcl1)))))
          (eq 0 (string-search (string root)
                               (first-symbol (cadar dcls)))))))
```

```
(defun ada-statements (body)
  (gf "+3")
  (do ((forms body (cdr forms))
       (in-group nil)
       (after-group nil))
      ((null forms) (if in-group (gf "}")))
    (caseq (ada-statement-classify (car forms))
      (out-of-group
        (cond (in-group (gf "}-*';'" (car forms)))
              (after-group (gf "-*';'" (car forms)))
              (T (gf "-*';'" (car forms))))
        (setq in-group nil after-group nil))
      (in-group
        (cond (in-group (gf ",*';'" (car forms)))
              (after-group (gf ",{0*';'" (car forms)))
              (T (gf "-{0*';'" (car forms))))
        (setq in-group T after-group nil))
      (start-group
        (cond (in-group (gf "},{0*';'" (car forms)))
              (after-group (gf ",{0*';'" (car forms)))
              (T (gf "-{0*';'" (car forms))))
        (setq in-group T after-group nil))
      (end-group
        (cond (in-group (gf ",*';'}" (car forms)))
              (after-group (gf ",*';'" (car forms)))
              (T (gf "-*';'" (car forms))))
        (setq in-group nil after-group T))))
  (gf "+-3-"))

;four kinds - starts-group ends-group in-group out-of-group.

(defun ada-statement-classify (call)
  (cond ((not (eq (car call) 'ada-fn)) 'out-of-group)
        (T (caseq (caadr call)
             ((put open close create) 'in-group)
             ((set_col) 'start-group)
             ((get new_line new_page) 'end-group)
             (T 'out-of-group)))))
```

```
;formats for ada pseudo-lisp

(def-ada-gformat def-ada-top (ignore with use . defs)
  (let ((*nested-ada-fn-spec* nil))
    (if with (gf "{5'with '[<%ada-qname ',',>]';'}!" with))
    (if use (gf "{4'use '[<%ada-qname ',',>]';'}!" use))
    (gf "[<*';'!' '!>]';'" defs)))

(def-ada-gformat def-ada-package (name dcls)
  (gf "{0'package '*' is'%ada-dcls 'end '*}"
      name dcls name))

(def-ada-gformat def-ada-package-body (name dcls)
  (gf "{0'package body '*' is'%ada-dcls 'end '*}" name dcls name))

(def-ada-gformat def-ada-entry (fn-spec)
  (gf "'entry '*" (car fn-spec)))

(def-ada-gformat def-ada-fn (fn-spec dcls body except)
  (ada-fn-spec fn-spec)
  (if (or dcls body except)
      (gf "' is'%ada-dcls {0'begin'%ada-statements %ada-exceptions 'end '*}"
          dcls body except (car fn-spec)))
  (gf "}"))

(def-ada-gformat def-ada-cliche (name plist . rest)
  (GF "{0'cliche '*" name)
  (when (or plist (car rest) (cadr rest) (caddr rest))
    (GF "' is'%ada-cliche-plist " plist)
    (cond ((cdr rest)
           (if (not (or (cadr rest) (caddr rest)))
               (setq rest (ncons (car rest))))
           (gf "[%ada-dcls 'begin'%ada-statements %ada-exceptions ]" rest))
          (T (gf "!*" (car rest))))
    (GF "!'end '*" name))
  (gf "}"))

(defun ada-cliche-plist (plist)
  (gf "+3!{0")
  (do ((list plist (cddr list)))
      ((null list))
    (caseq (car list)
      (primary-roles (gf "!{14'primary roles '[<*',',>]';'}" (cadr list)))
      (described-roles (gf "!{16'described roles '[<*',',>]';'}"
                           (cadr list)))
      (comment (gf "!{8'comment /"'{0[<S,>]}'/";'}"
                   (decompose-cliche-comment (cadadr list))))
      (constraints
        (gf "!{0'constraints'%ada-statements 'end constraints;'}"
            (cadr list)))))
  (gf "!' '}+-3"))
```

```
(def-ada-gf def-ada-generic "{0['generic'%ada-dcls *]}")

(def-ada-gf ada-with "'with '[%ada-fn-spec ' is '*]}")

(def-ada-gf ada-new-package
  "{3'package '[*' is'-'new '%ada-qname .(1<%ada-qname ',',>)]}")

(def-ada-gf ada-new-fn
  "[%ada-fn-spec ' is'-'new '%ada-qname .(1<%ada-qname ',',>)]}")

(def-ada-gf ada-dcl "{3[*': '%ada-type ]}")
(def-ada-gf ada-init-dcl "{3[*': '%ada-type ' := '*]}")
(def-ada-gf ada-constant-dcl "{3[*': constant '%ada-type ' := '*]}")

(def-ada-gf ada-type-dcl "{3['type '*' is'-%ada-type ]}")
(def-ada-gf ada-subtype-dcl "{3['subtype '*' is'-%ada-type ]}")

(def-ada-gf ada-use "{4'use '[<%ada-qname ',',>]}")

(defun ada-type (args)
  (cond ((memq (car args) '(ada-enumeration ada-string ada-rec
                            ada-range ada-exception kbe-/#-comment-macro))
         (gf "*" args))
        (T (ada-qname args))))

(def-ada-gf ada-enumeration "(<*',',>)")
(def-ada-gf ada-string "'STRING'[.['('*'..'*']')']")
(def-ada-gf ada-exception "{3'exception'[-'renames '%ada-qname ]}")
(def-ada-gf ada-range "{0[[.%ada-qname _]'range '*'..'*]}")

(def-ada-gf ada-rec
  "{0{3'record'[<!%ada-field ';'>]';'}!'end record'}")

(defun ada-field (args)
  (cond ((eq (car args) 'kbe-/#-comment-macro) (gf "*" args))
        (T (gf "[*': '%ada-type ]" args))))

(def-ada-gformat kbe-/#-comment-macro (code name)
  (let* ((comment (kbe-comment-name->string name)))
    (cond ((null code) (GF "{1'{'S'}'}" comment))
          ((and (listp (car code)) (eq (caar code) '...))
           (GF "{1'{'S'}'" comment)
           (if (cdar code)
               (gf "(1<*','->)}" (handle-&in-out (cdar code)))))
          (T (GF "{2'{'[*', 'B.]S'}'}" code comment)))))

;note that for internal ease in general the comment guy
;has to have the same name it has elsewhere.
;unfortunately that means that the place it is
;defined has to know to call this to print it in ada.
;when appropriate.

(defun ada-kbe-/#-comment-macro-printer (form)
  (declare (special *input-language*))
  (if ada-supress
      (let ((*input-language* 'lisp))
        (gf "*" form))
      (let* (((code name) (cdr form))
             (comment (kbe-comment-name->string name)))
        (cond ((null code) (GF "{1'{'S'}'}" comment))
              ((and (listp (car code)) (eq (caar code) '...))
               (GF "{1'{'S'}'" comment)
               (if (cdar code)
                   (gf "(1<*','->)}" (handle-&in-out (cdar code)))))
              (T (GF "{2'{'[*', 'B.]S'}'}" code comment))))))
```

```
(def-ada-gf ada-begin "{0['begin'%ada-statements %ada-exceptions ]'end'}")
(def-ada-gf ada-loop
  "{0['loop'%ada-statements %ada-exceptions ]'end loop'}")
(def-ada-gf ada-while
  "{0['while '*' loop'%ada-statements %ada-exceptions ]'end loop'}")

(def-ada-gformat ada-cond clauses
  (prog (first)
        (setq first T)
      L (cond ((null clauses) (GF "!'end if'") (return nil)))
        (cond (first (setq first nil)
                     (GF "!'if '*' then '%ada-statements "
                         (caar clauses) (cdar clauses)))
              ((equal (caar clauses) '(enum-lit true))
               (GF "!'else '%ada-statements " (cdar clauses)))
              (T (GF "!'elsif '*' then '%ada-statements "
                     (caar clauses) (cdar clauses))))
        (setq clauses (cdr clauses))
        (go L)))

(def-ada-gf ada-return "'return'[_*]")
(def-ada-gf ada-exit "'exit'[' when '*]")
(def-ada-gf ada-raise "'raise'[_*]")
(def-ada-gf ada-setq "[*' := '*]")

(def-ada-gf ada-convert "{0[%ada-qname .(*)]}")
(def-ada-gf ada-attribute "{0[%ada-type ''''* ]}")
(def-ada-gf ada-construct "{0[%ada-qname ''''.(1<*',',>)]}")
(def-ada-gf ada-select "{0[*'.'*]}")

(def-ada-gf str-lit "{'/"'[S]'/"'}")
(def-ada-gf enum-lit "%ada-qname ")
(def-ada-gf int-lit "[*]")

;note that the qualifiers in a name are in revese order than in ada
;this is a bit confusing but makes much of the computation much easier
;Also note how three keywords are handled here.

(defun ada-qname (args)
  (cond ((equal args '(private)) (gf "'private'"))
        ((equal args '(limited-private)) (gf "'limited private'"))
        ((equal args '(others)) (gf "'others'"))
        (T (gf "{[<*'.'>]}" (reverse args)))))

(defun ada-lower-case (list)
  (mapcar #'ada-lower-case-1 list))

(defun ada-lower-case-1 (symbol)
  (maybe-intern (string-downcase (string symbol))
                (symbol-package symbol)))

(defun ada-lower-case-0 (char)
  (cond ((or (< char #/A) (> char #/Z)) char)
        (t (+ char #.(- #/a #/A)))))
```

```
;Operators are converted to infix.  Priority and parentheses are
;kept track of by the variable ADA-BIND.  This has the binding
;strength of the parent operator.  If an op is a left operand
;and has less binding strength than its parent then parentheses are
;use.  If an op is a right operand and has less than or equal to
;binding strength than its parent then parentheses are used.

;We take advantage of the fact that statements cannot occur in
;expressions and only bind ada-bind in the stuff below.

(defvar ada-bind 0 "the binding strength of the parent operation")

(def-ada-gformat ada-fn (fn . args)
  (let* ((parent-ada-bind ada-bind)
         (ada-bind (ada-bind-level fn args))
         (need-parens (not (> ada-bind parent-ada-bind))))
    (cond ((not (ada-op-? (car fn)))
           (GF "{0%ada-qname " fn)
           (if args (GF "(1<*',',>)" args))
           (GF "}"))
          (T (GF "{0")
             (if need-parens (GF "'('"))
             (if (cdr args)
                 (let ((ada-bind (1- ada-bind)))
                   (GF "*B" (pop args))))
             (GF "S*" (ada-op-name fn) (car args))
             (if need-parens (GF "')'"))
             (GF "}")))))

(defvar ada-binary-op-info
 '((OR 1 | or |)  (XOR 1 | xor |)  (AND 1 | and |) (= 2 | = |)
   (//= 2 | //= |)  (<= 2 | <= |) (>= 2 | >= |)  (< 2 | < |)
   (> 2 | > |) (& 3 | & |)  (+ 3 +)  (- 3 -)
   (* 5 *)  (// 5 //)
   (MOD 5 | mod |)  (rem 5 | rem |) (** 6 **)))

(defvar ada-unary-op-info
  '((NOT 4 |not |) (ABS 4 |abs |) (- 4 -) (+ 4 +)))

(defun ada-op-? (name)
  (or (assq name ada-binary-op-info) (assq name ada-unary-op-info)))

(defun ada-bind-level (fn args)
  (cond ((not (ada-op-? (car fn))) 0)
        ((cdr args) (cadr (assq (car fn) ada-binary-op-info)))
        (T (cadr (assq (car fn) ada-unary-op-info)))))

(defun ada-op-name (fn)
  (or (caddr (assq (car fn) ada-binary-op-info))
      (caddr (assq (car fn) ada-unary-op-info))))
```

```
;; -*- Mode:LISP; Package:KBE; Base:10; Fonts:MEDFNB; -*-
;;;
;;; (c) Copyright 1984, Massachusetts Institute of Technology
;;;     All Rights Reserved.
;;;     Permission to copy all or part of this material is granted,
;;;     provided that the copies are not made or distributed for resale,
;;;     the MIT copyright notice and reference to the source file and
;;;     the software distribution version appear, and that notice is
;;;     given that copying is by permission of Massachusetts Institute
;;;     of Technology.

;;;; Flavor support

;;; CODE-WINDOW-MIXIN

(defflavor code-window-mixin ((offset 0)
                              (dhunk nil) ;not needed
                              (focus nil)
                              (focus-name nil)) ()
  (:documentation
   :mixin "A mixin to support display of pieces of code.")
  :initable-instance-variables
  :gettable-instance-variables
  :settable-instance-variables)

(defmethod (code-window-mixin :before :clear-screen) (&rest ignore)
  (send self ':set-label "")
  (setq offset      0
        focus       nil
        focus-name  nil))

(defmethod (code-window-mixin :change-focus) (reference paths)
  (when reference
    (when (neq (car paths) (car focus))
      (send self ':clear-screen)
      (send self ':display-program (car paths)))
    (send self ':set-label
          '(:bottom
            :font fonts:medfnb
            :string ,(text "Focus: ~A." (string-capitalize reference))))
    (setq focus-name reference focus paths)))

(defmethod (code-window-mixin :display-program) (program)
  (with-user-package ;...sigh... --kmp
    (let ((entry (send (window-delegate self) ':belief ':code program)))
      (when entry
        (multiple-value-bind (x y)
            (send self ':size-in-characters)
          (let ((pp:*print-right-margin* x)) ;for PPRINT
            (if (not (zerop offset))
                (barf "non-zero offset not supported now")) ; dick
            (print-code entry self (1- (+ offset y)))))))))

(defmethod (code-window-mixin :scroll) (&optional (arg 1))
  (when focus
    (let ((new-offset (+ offset arg)))
      (cond ((minusp new-offset) (beep))
            (t
             (setq offset new-offset)
             (send self ':refresh))))))

(defmethod (code-window-mixin :home) ()
  (when focus
    (cond ((= offset 0) (beep))
          (t
           (setq offset 0)
           (send self ':refresh)))))

(defmethod (code-window-mixin :after :refresh) (&optional mode)
```

```
  (when (and (neq mode ':use-old-bits) focus)
    (send self ':set-cursorpos 0 0)
    (send self ':clear-eof)
    (send self ':display-program (car focus))))

(defmethod (code-window-mixin :who-line-documentation-string) ()
  (cond ((not focus) "R2: System Menu")
        ((zerop offset)
         "M: Scroll Up, M2: Scroll Half-Screen Up, R2: System Menu")
        (t
         #.(format nil "L:Home, ~
                        M:Scroll Up, ~
                        M2:Scroll Half-Screen Up, ~
                        R:Scroll Down, ~
                        R2:System Menu"))))

(defmethod (code-window-mixin :mouse-click) (button x y)
  x y ;ignored
  (selectq button
    (#\mouse-1-1 (send self ':home))
    (#\mouse-m-1 (send self ':scroll))
    (#\mouse-m-2 (send self ':scroll (multiple-value-bind (x y)
                                         (send self ':size-in-characters)
                                       x ;ignored
                                       (// y 2))))
    (#\mouse-r-1 (send self ':scroll -1))
    (#\mouse-r-2 (tv:mouse-call-system-menu))
    (t (beep)))
  t)
```

```
;;; CODE-WINDOW
;;;
;;;  This is just a TV:WINDOW with CODE-WINDOW-MIXIN

(defflavor code-window () (box-drawing-mixin code-window-mixin tv:window)
  (:documentation
   "A window which handles display of a piece of code.")
  (:default-init-plist
    :blinker-p nil
    :save-bits t
    :more-p nil))

;;; CODE-PANE
;;;
;;;  This is just adds TV:PANE-MIXIN to a CODE-WINDOW

(defflavor code-pane () (tv:pane-mixin code-window)
  (:documentation
   "A window pane which handles display of a piece of code."))

(compile-flavor-methods code-window code-pane)
```

```
;;; -*- Mode:LISP; Package:KBE; Base:10; Fonts:MEDFNB; -*-

;;; (c) Copyright 1983, Massachusetts Institute of Technology
;;;     All Rights Reserved.
;;;     Permission to copy all or part of this material is granted,
;;;     provided that the copies are not made or distributed for resale,
;;;     the MIT copyright notice and reference to the source file and
;;;     the software distribution version appear, and that notice is
;;;     given that copying is by permission of Massachusetts Institute
;;;     of Technology.

;;; PLI: Support for letting the apprentice hack PL/1.

;this lets you print out lisp as PLI.  It works by temporarily
;changing a lot of gprint formats.

;;; This fixes everything up to print out lisp in PLI.
;;; This page shows some samples of how things should print.

(comment (setq test '
 (defun sqrt (num &aux result)
   (prog ()
         (setq result 1)
      LP   (cond ((< (abs (- (* result result) num)) 0.0001)
                  (return result)))
         (setq result (// (+ (// num result) result) 2))
         (go LP))) ))

;SQRT: PROCEDURE(NUM) RETURNS (FLOAT);
;        DCL NUM FLOAT,
;            RESULT FLOAT;
;        RESULT = 1;
;    LP: IF ABS(RESULT * RESULT - NUM) < 0.0001 THEN RETURN (RESULT);
;        RESULT = (NUM / RESULT + RESULT) / 2;
;        GOTO LP;
;      END SQRT;

(comment (setq test '
 (defun delete (symbol &aux current prev)
   (prog ()
         (setq prev (hash symbol))
         (setq current (cdr prev))
      LP   (cond ((not current) (return nil)))
         (cond ((eq (car current) symbol)
                (rplacd prev (cddr prev))
                (return nil)))
         (setq prev current)
         (setq current (cdr current))
         (go LP))) ))

;DELETE: PROCEDURE(SYMBOL);
;          DCL SYMBOL ...,
;              CURRENT POINTER,
;              PREV POINTER,
;              HASH EXTERNAL ENTRY (...) RETURNS (...),
;              NODE_PTR POINTER,
;              1 NODE BASED (NODE_PTR),
;                2 VALUE ...,
;                2 NEXT POINTER;
;          PREV = HASH(SYMBOL);
;          CURRENT = PREV->NEXT;
;      LP: IF CURRENT=NULL() THEN RETURN;
;          IF CURRENT->VALUE = SYMBOL
;             THEN DO; PREV->NEXT = PREV->NEXT->NEXT; RETURN; END;
;          PREV = CURRENT;
;          CURRENT = CURRENT->NEXT;
;          GOTO LP;
;        END DELETE;
```

```
(defvar *printing-as-pli* nil "controls printing format")
(defvar *saved-fn-gformat* ':Gfn-format)

(defun print-as-pli (&rest gprint-args)
  (setq *saved-fn-gformat* gfn-format)
  (let ((base 10.) (*nopoint t)
        (gfn-format 'pli-fn-format)
        (gsymbol-car-format 'pli-fn-format)
        (gspecial-formatters
          (list* 'pli-test-nil 'pli-test-T gspecial-formatters))
        (*printing-as-pli* T))
    (unwind-protect
      (progn (pli-setup) (lexpr-funcall #'gprint1 gprint-args))
      (pli-unsetup))))

(defmacro def-pli-gformat (name args &body rest)
  '(progn 'compile
     (pli-prepare-for-setup ',name)
     (defun (,name pli-gformat) ,args
       (cond (*printing-as-pli* ., rest)
             (T (funcall (or (get ',name 'saved-gformat)
                             *saved-fn-gformat*)
                         ,(car args)))))))

(defvar *pli-gformat-atoms* nil
  "list of all atoms with special pli formats")

(defun pli-prepare-for-setup (name)
  (when (not (memq name *pli-gformat-atoms*))
    (push name *pli-gformat-atoms*)))

(defun pli-setup ()
  (dolist (symbol *pli-gformat-atoms*)
    (let ((old-gformat (get symbol ':gformat))
          (pli-gformat (get symbol 'pli-gformat)))
      (when (not (eq old-gformat pli-gformat))
        (putprop symbol old-gformat 'saved-gformat)
        (putprop symbol pli-gformat ':gformat)))))

(defun pli-unsetup ()
  (setq gfn-format *saved-fn-gformat*)
  (setq gspecial-formatters (delq 'pli-test-nil gspecial-formatters))
  (setq gspecial-formatters (delq 'pli-test-T gspecial-formatters))
  (setq  *printing-as-pli* nil)
  (dolist (symbol *pli-gformat-atoms*)
    (let ((saved-gformat (getl symbol '(saved-gformat))))
      (when saved-gformat
        (if (cadr saved-gformat)
            (putprop symbol (cadr saved-gformat) ':gformat)
            (remprop symbol ':gformat))))))
```

```
;This file has formats which cause gprint to print lisp out as
;PLI.  It only maps from a subset of lisp into a subset of PLI.
;It is only intended to support the simple demo, but there is no reason
;it couldn't be extended to cover everything.

; There is a fundamental problem with data types.  I will just punt
;that for now.  This leads to problems with fns that have different
;names for different data types in PLI but not in Lisp (eg NOT).
; Another problem is that everything can be nested in everything in
;Lisp, but not in PLI.  The coder has to fix this problem because
;additional variables have to be introduced.  I will assume here that
;this doesn't happen (eg no CONDs or PROGs embedded in expressions).
; also there are some Lips fns that just don't have very good PLI
;counterparts eg PSETQ and MULTIPLE-VALUE.  I will just do kludges
;for these things.
; note that there is no garbage collector in PLI and there is no way
;I can think of to get around the fact that the plans we have just leave
;out the computation needed to decide what to free.  (note delete above.)
; note that PLI identifiers cannot contain "-" signs.  I will translate
;them to "_".
; You cannot have any literal list data in a PLI program.  In fact any
;necessary use of QUOTE is liable to be big trouble.
; Note that some of the problem with the fact that there are lots of lisp
;programs that this won't work on is ameliorated by the fact that the coder
;produces a very restricted class of programs, and that this class is
;particularly amenable to translation to PLI.

(defvar pli-bind 0  "the binding strength of the parent operation")
(defvar pli-statement T "T iff at statement level")
(defvar pli-subroutine nil "controls printing of returns")
```

```
;;; This looks at a defun and sets things up to PLI print it right.
;;;
;;; Note: This doesn't do a very good job of getting the cursor positions
;;;       set up.

(def-pli-gformat defun (list)
  (let* (((_ name arglist . body) list)
         args
         (vars (do ((l arglist (cdr l)))
                   ((or (eq (car l) '&aux) (null l))
                    (setq args (reverse args))
                    (cdr l))
                 (push (car l) args)))
         (bod body)
         (pli-subroutine (pli-is-subroutine bod))
         ((ret-type list-type . decls)
          (pli-guess-decls (append args vars) bod))
         (pli-bind 0)
         (pli-statement T))
    (GF "*': '{0{2'PROCEDURE'(<*','_>)" name args)
    (cond ((not pli-subroutine) (GF "_'RETURNS ('C')'" ret-type)))
    (GF "';'")
    (cond (decls
           (GF "A{4'DCL '[<[*<_C>]','A>]" decls)
           (cond (list-type
                  (GF "','A'NODE_PTR POINTER,'
                              A{2'1 NODE BASED (NODE_PTR),'
                                    A'2 VALUE 'C','
                                    A'2 NEXT POINTER'}" list-type)))
           (GF "';'}")))
    (cond ((and (null (cdr body))
                (eq (caar body) 'prog))
           (pli-top-prog (car body)))
          ((and (null (cdr body))
                (eq (caar body) 'loop))
           (pli-top-loop (car body)))
          (T (pli-defun-body body)))
    (GF "}A'END '*';'}" name)))

;;; This looks at a fn body and guesses that it is a subroutine if:
;;;  (1) it is a PROG
;;;  (2) every return is a (RETURN NIL).

(defun pli-is-subroutine (list)
  (and (eq (caar list) 'PROG)
       (null (cdr list))
       (pli-is-s1 list)))

(defun pli-is-s1 (list)
  (cond ((atom list) T)
        ((eq (car list) 'RETURN) (eq (cadr list) nil))
        (T (do ((l list (cdr l)) (val nil))
               ((atom l) val)
             (setq val (pli-is-s1 (car l)))
             (if (not val) (return nil))))))
```

```
;;; This looks over a program body and tries to estimate what the data
;;; types of things are based on what functions things are used in.
;;; It assumes that all guesses will be consistant.  It also determines
;;; the right info for any free variables.

;;; THIS IS VERY BUGGY AT THE MOMENT, IT WORKS FOR THE EXAMPLES BUT
;;; PROBABLY NOT MUCH ELSE. --Dick

(declare (special pli-decls pli-ret-type pli-list-type))

(defun pli-guess-decls (vars list)
  (let ((pli-decls (mapcar #'(lambda (x) (list x '|...|)) vars))
        (pli-ret-type '|...|)
        pli-list-type)
    (mapc #'pli-gd list)
    (list* pli-ret-type pli-list-type pli-decls)))

(defun pli-gd (list)
  (cond ((atom list))
        ((memq (car list) '(quote go kbe-#-comment-macro)))
        ((or (not (atom (car list)))
             (memq (car list) '(prog loop)))
         (mapc #'pli-gd list))
        ((eq (car list) 'cond)
         (mapc #'(lambda (l) (mapc #'pli-gd (cdr l))) (cdr list)))
        ((eq (car list) 'return) ;assumes return not only ref to var.
         (cond ((eq pli-ret-type '|...|)
                (setq pli-ret-type
                      (or (cadr (assq (cadr list) pli-decls)) '|...|)))))
        ((eq (car list) 'multiple-value) (pli-gd (caddr list)))
        ((memq (car list) '(psetq setq))
         (do ((l (cdr list) (cddr l)))
             ((null l))
           (pli-gd2 (car l) (cadr l))
           (pli-gd (cadr l))))
        ((memq (car list) '(arrayfetch arraystore))
         (pli-gd2 (cadr list) list)
         (pli-gd (caddr list)))
        (T (cond ((and (null (assq (car list) pli-decls)) ;all crockish!
                       (null (get (car list) 'pli-gformat))
                       (not (memq (car list) '(abs remainder)))) ;etc.
                  (setq pli-decls
                        (append
                          pli-decls
                          (ncons (list (car list)
                                       '|EXTERNAL ENTRY (...) RETURNS (...)|)))))))
           (do ((l (cdr list) (cdr l)))
               ((null l))
             (pli-gd2 (car l) list)
             (pli-gd  (car l))))))

(defun pli-gd2 (symbol expr)
  (cond ((and (symbolp symbol) (not (memq symbol '(T nil))))
         (let ((type
                 (cond ((cadr (assq expr pli-decls)))
                       ((numberp expr) 'FLOAT)
                       ((not (listp expr)) '|...|)
                       ((memq (car expr)
                              '(< lessp > greaterp = + - * // expt ^
                                abs remainder))
                        'FLOAT)
                       ((memq (car expr)
                              '(car cdr caar cadr cdar cddr rplaca rplacd))
                        (setq pli-list-type '|...|) 'POINTER)
                       ((memq (car expr) '(arrayfetch arraystore))
                        '|(...) ...|)
                       (T '|...|)))
               (entry (assq symbol pli-decls)))
```

```
(cond ((null entry)
       (setq entry (list symbol '|...| 'EXTERNAL))
       (setq pli-decls (append pli-decls (list entry)))))
(cond ((not (eq type '|...|)) (setf (cadr entry) type)))))))
```

```
(defun pli-defun-body (list)
  (cond ((null list))
        ((eq (car (car list)) 'loop) ;allow tail-recursive loops
         (if (cdr list) (barf "Ill-formed DEFUN body"))
         (pli-top-loop (car list)))
        ((null (cdr list))
         (let ((pli-statement nil))
           (GF "A'RETURN ('*')'" (car list))))
        (T (GF "A*" (car list)) (pli-defun-body (cdr list)))))

;This does PROG when it is wrapped around the entire body of a defun.
;in this case the returns are actually returns from the fn and everything
;is hunky dory.  Other PROGs aren't handled at all now.  Assumes no
;multiple labels.

(defun pli-top-prog (list) (pli-prog-sexpr (cddr list)))

(defun pli-top-loop (list)
  (if (eq (cadr list) 'do) (pop list))
  (if (not (Rand-fast (not (symbolp (Elist (cdr list))))))
      (barf "Ill-formed LOOP expression."))
; (GF "A{-4A'LP:'{0[<_*A>]A_'GO LP;'}}" (cdr list))
  (GF "A{0'DO WHILE (''1''B);'A___{0[<*A>]}A'END;'}" (cdr list)))

(defun pli-prog-sexpr (list)
  (cond ((null list))
        ((not (atom (car list)))
         (GF "A*" (car list)) (pli-prog-sexpr (cdr list)))
        (T (GF "A{#A*': '*}" (- (+ 2 (flatsize (car list))))
                             (car list) (cadr list))
           (pli-prog-sexpr (cddr list)))))

;this does SETQs.

(def-pli-gformat setq (list)
  (let ((pli-statement nil))
    (GF "{0[<{3*_N'='_*';'}A>]}}" (cdr list)))
  (pli-statement? T))

;you need a lot more than this in general, but this will work for the demo
;so I am punting the rest.  The coder is a better place to put assignment
;sorting anyway.

;This prints CONDs as IF/THEN/ELSEs.

(def-pli-gformat if (list)
  (GF "*" '(cond (,(cadr list) ,(caddr list))
                 ,@(if (cdddr list) '((t ,@(cdddr list)))))))

(def-pli-gformat when (list)
  (GF "*" '(cond (,(cadr list) ,@(cddr list)))))

(def-pli-gformat cond (list)
  (GF "{3[$pli-cond-clause . %pli-cond-rest ]}" (cdr list)))

(defun pli-cond-rest (list)
  (cond (list
         (GF "A['ELSE'_+3$pli-cond-clause . %pli-cond-rest ]" list))))

(defun pli-cond-clause (list)
  (cond ((not (eq (car list) 'T))
         (let ((pli-statement nil))
           (GF "'IF'_*_N'THEN'_" (car list)))))
  (GF "%pli-progn " (cdr list)))

(def-pli-gformat progn (list) (pli-progn (cdr list)))
```

```
(defun pli-progn (list)
  (cond ((null list) (pli-statement? T))
        ((null (cdr list)) (GF "*" (car list)))
        (T (GF "{4'DO;'[<_*N>]_'END'}" list) (pli-statement? T))))
```

```
;these are function like things that can only be used as statements
;Note we print the ";"s as part of the statements so that the boxes will
;work out right.

(defmacro pli-expression-format-bind (level statement? body)
  '(progn
     (let ((need-parens (and (not (zerop ,level))
                             (not (> ,level pli-bind))))
           (pli-bind ,level)
           (pli-statement nil))
       (cond (need-parens (GF "{0'('"))
       ,body
       (cond (need-parens (GF "')'}"))))
     (pli-statement? ,statement?)))

(defun pli-statement? (required?)
  (cond (pli-statement (GF "';'"))
        (required? (warn "Nested statement in PLI mode"))))

(def-pli-gformat return (list)
 (pli-expression-format-bind 0 T
   (cond ((and pli-subroutine (eq (cadr list) nil))
          (GF "'RETURN'"))
         (T (GF "{0'RETURN'_'('*')'}" (cadr list))))))

(def-pli-gformat go (list)
  (pli-expression-format-bind 0 T
    (GF "{0'GOTO'_*}" (cadr list))))

;this does all other functions

(defun pli-fn-format (list)
  (cond (*printing-as-pli*
         (pli-expression-format-bind 0 nil
           (GF "{0*(1<*','_N>)}" (car list) (cdr list))))
        (T (funcall *saved-fn-gformat* list))))

;  T   is assumed to always be true (ie, '1'B) -- not a bad assumption.

(defun pli-test-t (thing)
  (when (and *printing-as-pli* (eq thing T)) (pli-t-format thing) T))

(defun pli-t-format (ignore)
  (pli-expression-format-bind 0 nil
    (GF "''1''B'")))

;  NIL is assumed to always be the null pointer (ie, NULL()) -- quite a
;      bad assumption; it is very often false (ie, '0'B).

(defun pli-test-nil (thing)
  (when (and *printing-as-pli* (null thing)) (pli-nil-format thing) T))

(defun pli-nil-format (ignore)
  (pli-expression-format-bind 0 nil
    (GF "'NULL()'")))
```

```
(def-pli-gformat kbe-#-comment-macro (x)
  (pli-expression-format-bind 20 nil
    (cond ((equal *kbe-role-marker* "{")
           (new-style-kbe-/#-comment-macro-print x))
          (T (old-style-kbe-/#-comment-macro-print x)))))

(defun new-style-kbe-/#-comment-macro-print (x)
  (let* (((code name) (cdr x))
         (comment (kbe-comment-name->string name)))
    (cond ((null code) (GF "{1'{'S'}'}" comment))
          ((and (listp (car code)) (eq (caar code) '...))
           (GF "{1'('{1'{'S'}'}-{0%role-args }')'}"
               comment (cdar code)))
          (T (GF "{2'{'[*', 'B.]S'}'}" code comment)))))

(defun old-style-kbe-/#-comment-macro-print (x)
  (let* (((code name) (cdr x))
         (comment (kbe-comment-name->string name)))
    (GF "{2'#{'[*', 'B.]S'}'}" code comment)))

(defun (|...| :gformat) (x)
  (gf "{5'(... '%role-args ')'}" (cdr x)))

(defun role-args (args)
  (let ((same-line T) (first-line T))
    (dolist (arg args)
      (if first-line (setq first-line nil)
          (cond ((not same-line) (gf "-"))
                (T (gf "_"))))
      (cond ((and (symbolp arg) (equal (string arg) "&IN-OUT"))
             (gf "*" arg)
             (setq same-line T))
            (T (gf "*" arg) (setq same-line nil))))))
```

```
;this converts expressions to infix.  there are several key problems:
;  priority and parentheses are kept track of by the variable
;PLI-BIND.  this has the finding strength of the containing expression
;(0 at top [statement] level).  If the operator to be printed has
;stronger binding than the parent then it is just printed, otherwise
;parentheses are used.  [Note that it doesn't take advantage of left to
;right precedence at the moment.]  The binding strengths are:
; 0 top level (weakest)
; 1 between commas (not in a statement, but super weak binding)
; 2 OR
; 3 AND
; 4 < lessp > greaterp = eq
; 5 + -
; 6 * /
; 7 expt ^
; 8 unary- NOT NULL (unary operators - highest)
;note some fns like CAR and NULL turn into expressions like things.
;  The fn NOT is used two ways in lisp: as NULL and as NOT.  Here I
;assume that no variable holds a boolean value, and that certain functions
;(like < and OR) return boolean values while the rest don't.
;THIS IS QUITE BUGGY!!!

(defmacro def-pli-expression-format (lisp pli level)
  '(def-pli-gformat ,lisp (list)
     (pli-expression-format-bind ,level nil
       ,(list 'GF (string-append "{0[<*_'" pli "'_N>]}") '(cdr list)))))

(def-pli-expression-format or /| 2)
(def-pli-expression-format and & 3)
(def-pli-expression-format < < 4)
(def-pli-expression-format lessp < 4)
(def-pli-expression-format > > 4)
(def-pli-expression-format greaterp > 4)
(def-pli-expression-format = = 4)
(def-pli-expression-format eq = 4)
(def-pli-expression-format + + 5)
(def-pli-expression-format * * 6)
(def-pli-expression-format // // 6)
(def-pli-expression-format expt ** 7)
(def-pli-expression-format ^ ** 8)

(def-pli-gformat - (list)
  (cond ((cddr list)
         (pli-expression-format-bind 5 nil
           (GF "{0[<*_'-'_N>]}" (cdr list))))
        (T (pli-expression-format-bind 8 nil
             (GF "{0['-'N*]}" (cdr list))))))
```

```
(def-pli-gformat not (list)
  (cond ((and (not (atom (cadr list)))
              (memq (car (cadr list))
                    '(< lessp > greaterp = atom or and)))
         (pli-expression-format-bind 8 nil
           (GF "{0'^E'*}" (cadr list))))
        (T (pli-expression-format-bind 4 nil
             (GF "{0*'=NULL()'}" (cadr list))))))

(putprop 'null (get 'not 'pli-gformat) 'pli-gformat)

(def-pli-gformat arrayfetch (list)
  (pli-expression-format-bind 0 nil
    (GF "{0[*'('*]')'}" (cdr list))))

(def-pli-gformat arraystore (list)
  (pli-expression-format-bind 0 T
    (GF "{0[*'('*') = '*]}}" (cdr list))))

;The list functions are recoded using pointers.  This works fine
;for CAR CDR RPLACA and the like, but not for CONS.  To get CONS to work,
;the coder would have to be told that CONS returns its value in a free
;variable.  Putting this a bit stronger, you cannot have any function
;that returns a pointer, you cannot nest it in a pointer qualification
;expression (that is to say you cannot take the CAR of it) unless you
;put it in a variable first.  this could easily be fixed by the coder
;easy enough.

(def-pli-gformat car (list)
  (pli-expression-format-bind 0 nil
    (GF "{*'->VALUE'}" (cadr list))))
(def-pli-gformat cdr (list)
  (pli-expression-format-bind 0 nil
    (GF "{*'->NEXT'}" (cadr list))))
(def-pli-gformat caar (list)
  (pli-expression-format-bind 0 nil
    (GF "{*'->VALUE->VALUE'}" (cadr list))))
(def-pli-gformat cadr (list)
  (pli-expression-format-bind 0 nil
    (GF "{*'->NEXT->VALUE'}" (cadr list))))
(def-pli-gformat cdar (list)
  (pli-expression-format-bind 0 nil
    (GF "{*'->VALUE->NEXT'}" (cadr list))))
(def-pli-gformat cddr (list)
  (pli-expression-format-bind 0 nil
    (GF "{*'->NEXT->NEXT'}" (cadr list))))

(def-pli-gformat rplaca (list)
  (pli-expression-format-bind 0 T
    (GF "{3*'->VALUE'_N'='_*}" (cadr list) (caddr list))))
(def-pli-gformat rplacd (list)
  (pli-expression-format-bind 0 T
    (GF "{3*'->NEXT'_N'='_*}" (cadr list) (caddr list))))
```

```
;;; -*- Mode:LISP; Package:KBE; Base:10; Fonts:MEDFNB; -*-
;;;
;;; (c) Copyright 1983, Massachusetts Institute of Technology
;;;     All Rights Reserved.
;;;     Permission to copy all or part of this material is granted,
;;;     provided that the copies are not made or distributed for resale,
;;;     the MIT copyright notice and reference to the source file and
;;;     the software distribution version appear, and that notice is
;;;     given that copying is by permission of Massachusetts Institute
;;;     of Technology.

;;; DEFLIB: The definition of defining forms DEFPROTOTYPE and DEFINE-CLICHE,
;;;         upon which SYSLIB, KBELIB, and DEMLIB depend.

;some general comments on cliches
;1. it is good for every cliche to have a return value, but for this not to
;be an explicit slot since it is easy to refer to anyway.
;2. By convention essentially every cliche is a simple segment.
;  it cannot be an ssr, ssrbody, split, or join.

;;; this allows you to create a cliche by giving pseudo code for it.
;;; note that the stuff in flows only creates slot assertions.
;;; this is the only place where the cliche assertion is made.
;;; you just specify what that should be yourself.
;;; Note that in order to run this macro you have to have the whole
;;; system up and going.

;Note that a naked defprototype is only used when you
;cannot use define-cliche for example if you need to create
;a naked pred or ssr, or need to have either one as a part.
;The funny little guys used by the system need this,
;(they are all in this file)
;but I don't think any of the normal ones do.

;Note that this cleans out the DB after creating the cliche
;if the DB was empty to start with.  This
;helps keep things fast and doesn't leave any garbage about.

(defmacro define-cliche (name comments &body body)
  (let ((default-cons-area working-storage-area)) ;Avoid screw related to
                                                  ;compiler temporary area
    (condition-case-if (not *debug*) (cond)
        '(local-declare ()
           ;; This really shouldn't work since it will create
           ;; DEFINE-CLICHE style source file info, but we later
           ;; say that DEFINE-CLICHE really creates DEFUN stuff.
           ;; It "works" because ZWEI's sectionization stuff is
           ;; such a mess that it finds the "wrong" definition...
           (record-source-file-name ',name 'define-cliche)
           ;; Now do main expansion...
           ,(expand-define-cliche name comments
              '(define-cliche ,name ,comments ., body)))
      (sys:error (grumble "The cliche (~A) failed to be defined." name)))))

(define-editor-definition-type DEFINE-CLICHE DEFUN)
```

```
(defmacro def-ada-cliche (name comments &body body)
  (let ((default-cons-area working-storage-area)) ;Avoid screw related to
                                                  ;compiler temporary area
    (condition-case-if (not *debug*) (cond)
        '(local-declare ()
           ;; This really shouldn't work since it will create
           ;; DEF-ADA-CLICHE style source file info, but we later
           ;; say that DEFINE-CLICHE really creates DEFUN stuff.
           ;; It "works" because ZWEI's sectionization stuff is
           ;; such a mess that it finds the "wrong" definition...
           (record-source-file-name ',name 'def-ada-top)
           ;; Now do main expansion...
           ,(expand-define-cliche name comments
              '(def-ada-top ,name nil nil
                 (def-ada-cliche ,name ,comments
                   ., body))))
      (sys:error (grumble "The cliche (~A) failed to be defined." name)))))

(define-editor-definition-type DEF-ADA-TOP DEFUN)
```

```
;note the way this converts the dffn into an instance
(defun expand-define-cliche (name comments body)
  ;must do this here because normal reader read this.
  (fix-up-semi-kbe-/#-comment-macros body)
  (let* ((empty-db (null *db-segs*))
         (segcase (create-plan-for-code body))
         (slot-names (get (locf comments) 'roles)))
    (-& segcase cliche-def)
    (--& segcase cliche)
    (+& segcase cliche name)
    (if (null slot-names)
        (setq comments (get-cliche-roles segcase comments)))
    (setq comments (get-cliche-role-tree segcase comments))
    (prog1 '(defprototype ,name ,comments
              ,(construct-plan-def (~seg segcase)))
           (if empty-db (db-init)))))

(defun get-cliche-roles (segcase comments)
  (let* ((primary-roles (get (locf comments) 'primary-roles))
         (actual-roles (~set ~slotname (??& segcase slot)))
         (roles (append primary-roles (qset- actual-roles primary-roles))))
    '(roles ,roles  .,comments)))

(defun get-cliche-role-tree (segcase comments)
  (let* ((roles (get (locf comments) 'roles))
         (role-tree))
    (dolist (role roles)
      (let ((slotseg (~slotseg (?& segcase slot * role))))
        (if (??& slotseg cliche 'pseudo-subcliche)
            (push (cons role (~set ~slotname (??& slotseg slot)))
                  role-tree)
            (push (ncons role) role-tree))))
    '(role-tree ,(nreverse role-tree) ., comments)))
```

```
(pp:define-list-print-function define-cliche (list s)
  (format s "(~@!~4I~=~^ ~@_~=~^~%~
                   (~!~@{~\kbe:define-cliche-plist-entry\~^~%~}~.)~
                 ~2I~{~%~=~^~}~.)"
          (car list) (cadr list) (pairup (caddr list)) (cdddr list)))

(defun pairup (list)
  (do ((l list (cddr l))
       (result nil (cons (list (car l) (cadr l)) result)))
      ((null l) (nreverse result))))

(pp:define-printing-directive define-cliche-plist-entry (entry s)
  (let ((prop (car entry)) (value (cadr entry)))
    (caseq prop
      ((primary-roles described-roles roles role-tree)
       (format s "~S ~@_~1\fill\" prop value))
      (constraints (format s "~@!~2I~S~_~1\linear\~." prop value))
      (comment
        (format s "~S /"~@!~{~A~^ ~:_~}~./""
                prop (decompose-cliche-comment value)))
      (T (format s "~S ~@_~=" prop value)))))

(defun decompose-cliche-comment (string)
  (if (not (stringp string)) (ncons string)
      (let ((*kbe-role-marker* "{"))
        (mapcar #'(lambda (word)
                    (if (symbolp word)
                        (string-downcase (string word))
                        (format nil "~:=" word)))
                (fix-up-semi-kbe-/#-comment-macros
                  (read-from-string
                    (string-append "(" string ")")))))))
```

```
;this is used to define cliches in the library.
;this does a bunch of things to interface with the editor
;as well as define the cliche.  Note that the CLICHE and SLOT assertions
;must be in the plan.  The cliches are not actually put into the
;data base but are rather put on property lists.  The plan-def
;goes on the CLICHE-PLAN property and the comments go on the
;CLICHE-COMMENTS property.

;each call on defprototype has the following form:
;(defprototype CLICHE-NAME      ;same as below
;              COMMENTS         ;general comments (the cdr of a plist)
;  (plan-def SEG-NAME           ;the name of the segment for the cliche
;    code                  ;for everything to work right there must be code
;    (deffacts SEG-NAME
;      (cliche (CLICHE-NAME))   ;the internal name of the cliche (one word)
;      ...)
;    ...))
;a lower case version of the cliche-name and each slot name is used
;externally

;for example

;(defprototype TRAILING-POINTER-LIST-ENUMERATION
;              (roles (LIST CURRENT PREVIOUS)
;               MAIN "trailing pointer list enumeration loop.")
;  (plan-def TPLE-LOOP
;    code
;    (deffacts TPLE-LOOP
;      (cliche (TRAILING-POINTER-LIST-ENUMERATION))
;      ...)
;    ...))
;the external names are: trailing-pointer-list-enumeration,
;list, current, and previous.

;note that one of the primary pieces of work this does is
;set up the name of the cliche and the names of the roles as things
;you can refer to in a command.

(defmacro defprototype (&body body)
  (let* (((cliche-name comments plan) body)
         (role-tree (get (locf comments) 'role-tree)))
    '(progn 'compile
            (if (get ',cliche-name 'cliche-plan)
                (grumble "Redefining cliche ~A" ',cliche-name))
            (*define-cliche ',cliche-name)
            (mapc #'(lambda (l)
                      (mapc #'*define-role l))
                  ',role-tree)
            (putprop ',cliche-name ',plan 'cliche-plan)
            (putprop ',cliche-name (cons nil ',comments) 'cliche-comments)
            ',cliche-name)))
```

```
;;; Local Modes:;
;;; Mode:LISP;
;;; Lisp WHEN Indent:1;
;;; Lisp UNLESS Indent:1;
;;; End:;
```

```
;;; -*- Mode:LISP; Package:KBE; Base:10; Fonts:MEDFNB; -*-
;;;
;;; (c) Copyright 1983, Massachusetts Institute of Technology
;;;     All Rights Reserved.
;;;     Permission to copy all or part of this material is granted,
;;;     provided that the copies are not made or distributed for resale,
;;;     the MIT copyright notice and reference to the source file and
;;;     the software distribution version appear, and that notice is
;;;     given that copying is by permission of Massachusetts Institute
;;;     of Technology.

;;; SYSLIB: All of the prototypes used by the system kernel

;CLICHES ARE IN THIS FILE AS OPPOSED TO ANOTHER CLICHE
;LIBRARY IF THE CLICHE IS REQUIRED FOR THE BASIC RUNNING OF THE
;SYSTEM EVEN IF THE USER DOES NOT MENTION ANY CLICHE BY NAME.
;OR IF A DEFPROTOTYPE MUST BE USED TO DEFINE IT.

;These first dummy guys are used by CREATE-CLICHE-FOR-FUNCTION.
;note that the renaming capability is used in a very cludgy way
;to get the right function names in the cliche.  Note that we
;cannot rename the cliche itself, or its slots because we
;have to be able to get at the slot-order information.

;note these are for internal use only.  If they were intended for
;external use, the fn would be a slot.

(define-cliche zeroary-function
    (main "Dummy zero argument function cliche")
  (fn))

(define-cliche unary-function
    (roles (x)
     main "Dummy one argument function cliche")
  (fn {input x}))

(define-cliche binary-function
    (roles (x y)
     main "Dummy one argument function cliche")
  (fn {input x} {input y}))
```

```
;These cliches are used for impedance matching by
;implement-seg.

(define-cliche simple-subseg
    (roles (subseg)
     main "Used to nest one seg inside another.")
  {subseg})

(DEFINE-CLICHE MAP
    (PRIMARY-ROLES (ITEM FN)
     DESCRIBED-ROLES (ITEM FN)
     COMMENT "maps {the function} over {the item}")
  (LOOP DO
    {({the fn} {the input item}), the output result}))
```

```
;;; -*- Mode:LISP; Package:KBE; Base:10.; Fonts:MEDFNB; -*-

;;; (c) Copyright 1983, Massachusetts Institute of Technology
;;;     All Rights Reserved.
;;;     Permission to copy all or part of this material is granted,
;;;     provided that the copies are not made or distributed for resale,
;;;     the MIT copyright notice and reference to the source file and
;;;     the software distribution version appear, and that notice is
;;;     given that copying is by permission of Massachusetts Institute
;;;     of Technology

;;; KBELIB: The main library of user cliches.

;the following is a dumb special little cliche so that you
;can say "a use of" something.  The only time you might
;want to say this is when you embed it in code
;eg '(eq (car {a use of the item}) 'foo)

(define-cliche use
  (primary-roles (thing))
  {thing})

;NOTE EACH CLICHE IS CODED IN THE EXACT STYLE THAT THE
;CODER PRODUCES IN ORDER TO FACILITATE COMPARISON.

;first we have the prototypes needed for the SQRT example

(DEFINE-CLICHE SUCCESSIVE-APPROXIMATION
   (PRIMARY-ROLES (INITIAL-APPROXIMATION)
    DESCRIBED-ROLES (INITIAL-APPROXIMATION CRITERION IMPROVEMENT-STEP)
    COMMENT "iteratively improves an approximation until {the criterion} is
             satisfied"

    main "successive approximation"
    detail ("Successive approximation.  Beginning with an initial"
            "approximation of" initial-approximation ", tries to find"
            "a value for which" criterion "succeeds."
            "If that test fails, a new approximation is generated"
            "from the old by" improvement-step
            ". This process is repeated until the test succeeds.")
    criterion "predicate expression")
  (LET* ((APPROXIMATION {the input initial-approximation}))
    (LOOP DO
      (IF ({the criterion} APPROXIMATION) (RETURN APPROXIMATION))
      (SETQ APPROXIMATION ({the improvement-step} APPROXIMATION)))))

(DEFINE-CLICHE AVERAGE
    (PRIMARY-ROLES (FIRST-NUMBER SECOND-NUMBER)
     DESCRIBED-ROLES (FIRST-NUMBER SECOND-NUMBER)
     COMMENT "computes the average of {the first-number}
              and {the second-number}"
     main ("average of" first-number "and" second-number)
     detail ("Averages" first-number "and" second-number
             "by summing them and dividing by two.")
     first-number "thing to be averaged"
     second-number "thing to be averaged")
  (// (+ {the input first-number} {the input second-number}) 2.0))

(DEFINE-CLICHE SQUARE
    (PRIMARY-ROLES (NUMBER)
     DESCRIBED-ROLES (NUMBER)
     COMMENT "computes the square of {the number}")
  (^ {the input number} 2))

(DEFINE-CLICHE EQUALITY-WITHIN-EPSILON
    (PRIMARY-ROLES (FIRST-NUMBER SECOND-NUMBER)
     DESCRIBED-ROLES (FIRST-NUMBER SECOND-NUMBER EPSILON)
     COMMENT "determines whether {the first-number} and
              {the second-number} differ by less than {the epsilon}"
```

```
    main ("equality within" epsilon "test")
    detail ("Succeeds when" first-number "and" second-number
            "are within" epsilon "of each other.")
    first-number "value being tested"
    second-number "value being tested"
    epsilon "epsilon")
  (< (ABS (- {the input first-number} {the input second-number}))
    {the epsilon}))
```

```
;here are the prototypes needed for the DELETE example.

;note we should have an element-accessor, but we cannot
;because it gets grouped up with the search-test wrecking the
;plan.  note the gastly kludge to get the search-test
;pulled separated out by the analyzer.  We could do more
;work in the analyzer to figure out that this should come out.

(compiler-let ((*separate-gens-and-terms* t))

(DEFINE-CLICHE LINEAR-SEARCH
    (PRIMARY-ROLES (ENUMERATOR)
     DESCRIBED-ROLES (ENUMERATOR SEARCH-TEST ACTION)
     COMMENT "searches (in linear time)
              {the input structure of the enumerator}
              for an element satisfying {the search-test}"

     main "linear time search"
     detail ("Searches" enumerator "for an element which"
             "satisfies" search-test ". If found, returns"
             action ". Otherwise, returns NIL.")
     structure-of-enumerator "thing being searched")
  (LET* ((DATA {the input structure of the enumerator}))
    (LOOP DO
      (IF ({the empty-test of the enumerator} DATA) (RETURN NIL))
      (IF ({the search-test} DATA) (RETURN {the action}))
      (SETQ DATA ({the step of the enumerator} DATA))))) )

;this has a lot of problems, but it is the only thing that I can
;make work.  It cannot have a real unitary output prev, because
;that would make it not be an ssr.  It cannot have a temporal output
;prev becuase then there is no way to get the last value of it out
;when you hook data flow up to it.  This causes the variable PREV
;to exist when you add it into a plan so that you can use it later.
;Unfortunately, if you analyze and recode then this variable will
;get deleted since it isn't really used for anything.  That is
;to bad, we will just have to avoid doing that for now.

(DEFINE-CLICHE TRAILING-POINTER-LIST-ENUMERATION
    (PRIMARY-ROLES (LIST)
     DESCRIBED-ROLES (LIST)
     COMMENT "enumerates the elements of the tail of {the list}
              maintaining a trailing pointer"

     main ("trailing pointer list enumeration of" list)
     detail ("Enumerates the elements in the tail of the list returned by"
             list "maintaining a backpointer to successive values of"
             current ".")
     list    "initial value computation"
     element    "element under examination")
  (LET* ((PREVIOUS-SUBLIST {the input list})
         (LIST (CDR PREVIOUS-SUBLIST)))
    (LOOP DO
      (IF (NULL LIST) (RETURN PREVIOUS-SUBLIST))
      {(CAR LIST), the output element}
      (SETQ PREVIOUS-SUBLIST LIST)
      (SETQ LIST (CDR LIST)))))

(DEFINE-CLICHE SPLICE-OUT
    (PRIMARY-ROLES (POSITION)
     DESCRIBED-ROLES (POSITION)
     COMMENT "splices an element out of a list after {the position}"

     main ("splice out of the successor to" position)
     detail ("Splices an element out of a list;"
             position "has its tail modified to be its tail's tail.")
     position "previous sublist")
  (LET* ((PREVIOUS-SUBLIST {the input position})))
```

```
    (RPLACD PREVIOUS-SUBLIST (CDDR PREVIOUS-SUBLIST))
    NIL))
```

```
;some related cliches

(DEFINE-CLICHE TRAILING-POINTER-SUBLIST-ENUMERATION
    (PRIMARY-ROLES (LIST)
     DESCRIBED-ROLES (LIST)
     COMMENT "enumerates the sublists of the tail of {the list}
              maintaining a trailing pointer"

     main ("trailing pointer sublist enumeration of" list)
     detail ("Enumerates the sublsits in the tail of the list returned by"
             list "maintaining a backpointer to successive values of"
             sublist ".")
     list    "initial value computation"
     sublist "sublist under examination")
  (LET* ((PREVIOUS-SUBLIST {the input list})
         (LIST (CDR PREVIOUS-SUBLIST)))
    (LOOP DO
      (IF (NULL LIST) (RETURN PREVIOUS-SUBLIST))
      {LIST, the output sublist}
      (SETQ PREVIOUS-SUBLIST LIST)
      (SETQ LIST (CDR LIST)))))

(DEFINE-CLICHE SPLICE-IN
    (PRIMARY-ROLES (POSITION NEW-ELEMENT)
     DESCRIBED-ROLES (POSITION NEW-ELEMENT)
     COMMENT "splices {the new-element} into a list after {the position}"

     main ("splice in of" new-element "after" position)
     detail ("Splices" new-element
             "into a list.  A new cell is created with head"
             new-element "and tail the same as the tail of" position ";"
             position "has its tail modified to be" new-element ".")
     position "previous sublist"
     new-element "new element")
  (LET* ((PREVIOUS-SUBLIST {the input position}))
    (RPLACD PREVIOUS-SUBLIST
            (CONS {the input new-element} (CDR PREVIOUS-SUBLIST)))
    NIL))
```

```
;Here are the basic kinds of loop fragments.
;Note that since adding new fragments into loops doesn't work, and
;just combining the fragmants together into a loop expression
;before filling them in doesn't work much better, these are of
;rather limited utility.

(DEFINE-CLICHE ENUMERATION
    (PRIMARY-ROLES (STRUCTURE)
     DESCRIBED-ROLES (STRUCTURE ELEMENT-ACCESSOR EMPTY-TEST STEP)
     COMMENT "enumerates the elements of {the structure}")
  (LET* ((DATA {the input structure}))
    (LOOP DO
      (IF ({the empty-test} DATA) (RETURN))
      {({the element-accessor} DATA), the output element}
      (SETQ DATA ({the step} DATA)))))

(DEFINE-CLICHE GENERATION
    (PRIMARY-ROLES (STRUCTURE)
     DESCRIBED-ROLES (STRUCTURE ELEMENT-ACCESSOR STEP)
     COMMENT "generates the elements of {the structure}")
  (LET* ((DATA {the input structure}))
    (LOOP DO
      {({the element-accessor} DATA), the output element}
      (SETQ DATA ({the step} DATA)))))

;note that a cliche map is defined in syslib.

(DEFINE-CLICHE ACCUMULATION
    (PRIMARY-ROLES (ITEM)
     DESCRIBED-ROLES (ITEM OPERATOR ZERO-FOR-OPERATOR)
     COMMENT "accumulates {the item}")
  (LET* ((RESULT {the input zero-for-operator}))
    (LOOP DO
      (SETQ RESULT ({the operator} RESULT {the input item})))
    RESULT))

(DEFINE-CLICHE SELECT
    (PRIMARY-ROLES (ITEM SELECTION-TEST)
     DESCRIBED-ROLES (ITEM SELECTION-TEST)
     COMMENT "selects a subseries of {the item}")
  (LET* (DATUM)
    (LOOP DO
      (SETQ DATUM {the input item})
      (IF ({the selection-test} DATUM)
          {DATUM, the output selected-item}))))
```

```
;Here are a few other example cliches just to test things.

(DEFINE-CLICHE LIST-ENUMERATION
    (PRIMARY-ROLES (LIST)
     DESCRIBED-ROLES (LIST)
     COMMENT "enumerates the elements of {the list}"

     main ("list enumeration of" list)
     detail ("Enumerates the successive elements of" list ".")
     element "element")
  (LET* ((LIST {the input list}))
    (LOOP DO
      (IF (NULL LIST) (RETURN))
      {(CAR LIST), the output element}
      (SETQ LIST (CDR LIST)))))

(DEFINE-CLICHE SUBLIST-ENUMERATION
    (PRIMARY-ROLES (LIST)
     DESCRIBED-ROLES (LIST)
     COMMENT "enumerates the sublists of {the list}")
  (LET* ((LIST {the input list}))
    (LOOP DO
      (IF (NULL LIST) (RETURN))
      {LIST, the output sublist}
      (SETQ LIST (CDR LIST)))))

(DEFINE-CLICHE VECTOR-ENUMERATION
    (PRIMARY-ROLES (VECTOR)
     DESCRIBED-ROLES (VECTOR)
     COMMENT "enumerates the elements of {the vector}")
  (LET* ((I 0)
         (VECTOR {the input vector})
         (SIZE (ARRAY-ACTIVE-LENGTH VECTOR)))
    (LOOP DO
      (IF (NOT (< I SIZE)) (RETURN))
      {(AREF VECTOR I), the output element}
      (SETQ I (1+ I)))))

(DEFINE-CLICHE LIST-GENERATION
    (PRIMARY-ROLES (LIST)
     DESCRIBED-ROLES (LIST)
     COMMENT "generates the elements of {the list}")
  (LET* ((LIST {the input list}))
    (LOOP DO
      {(CAR LIST), the output element}
      (SETQ LIST (CDR LIST)))))
```

```
(DEFINE-CLICHE SUM
    (PRIMARY-ROLES (NUMBER)
     DESCRIBED-ROLES (NUMBER)
     COMMENT "accumulates the sum of {the number}")
  (LET* ((SUM 0))
    (LOOP DO
      (SETQ SUM (+ SUM {the input number})))
    SUM))

(DEFINE-CLICHE PRODUCT
    (PRIMARY-ROLES (NUMBER)
     DESCRIBED-ROLES (NUMBER)
     COMMENT "accumulates the product of {the number}")
  (LET* ((PRODUCT 1))
    (LOOP DO
      (SETQ PRODUCT (* PRODUCT {the input number})))
    PRODUCT))
```

```
;;; -*- Mode: lisp; Package: KBE; Base:10.; Fonts:MEDFNB; -*-

;;; (c) Copyright 1984, Massachusetts Institute of Technology

;this program should be run just before a demo in order
;to set things up to work right.

(defvar original-with-open-file (fsymeval 'with-open-file))
(fset 'original-with-open-file  original-with-open-file)

(defun demo-setup ()
  (declare (special *predefined-files*))
;general things
  (pkg-goto 'kbe) ;won't work in any other package.
  (setq si:gc-report-stream nil)
  (setq si:address-space-warning-low-words 2000000.)
;for lisp stuff
  (fset 'with-open-file (fsymeval 'with-one-open-file))
;for ada stuff
  (ada-init)
  (eval '(ada-fn (create_test_records)))
  (if (y-or-n-p "do you want to test implmentation of MAINTENANCE_FILES ")
      (setq *predefined-files*
            (delq (assq 'repairs *predefined-files*) *predefined-files*)))
  'done)

(defun demo-unsetup ()
; (setq si:gc-report-stream T)
  (fset 'with-open-file original-with-open-file))

(pp:define-list-print-function format (list s)
  (format s "(~!~=~^ ~:I~@_~=~^ ~:_~=~^ ~_~
             ~@!~@{~=~^ ~:_~}~.~.)" list))
```

```
(DEFINE-CLICHE SIMPLE-REPORT
    (PRIMARY-ROLES (ENUMERATOR PRINT-ITEM EPILOG)
     DESCRIBED-ROLES (FILE-NAME TITLE ENUMERATOR
                      COLUMN-HEADINGS PRINT-ITEM EPILOG)
     COMMENT "prints a report of {the input structure of the enumerator}"
     CONSTRAINTS
      ((DEFAULT {the file-name} "report.txt")
       (DERIVED {the line-limit}
                (- 65
                   (SIZE-IN-LINES {the print-item})
                   (SIZE-IN-LINES {the epilog}))))))
  (WITH-OPEN-FILE (REPORT {the file-name} ':OUT)
    (LET* ((DATE (TIME:PRINT-CURRENT-TIME NIL))
           (LINE 66)
           (PAGE 0)
           (TITLE {the title})
           (DATA {the input structure of the enumerator}))
      (FORMAT REPORT "~5%~66:@<~A~>~2%~66:@<~A~>~%" TITLE DATE)
      (LOOP DO
        (IF ({the empty-test of the enumerator} DATA) (RETURN))
        (WHEN (> LINE {the line-limit})
          (SETQ PAGE (+ PAGE 1))
          (FORMAT REPORT "~/|~%Page:~3D~50:@<~A~>~17A~2%" PAGE TITLE DATE)
          (SETQ LINE 3)
          ({the column-headings} {REPORT, modified} {LINE, modified}))
        ({the print-item}
         {REPORT, modified}
         {LINE, modified}
         ({the element-accessor of the enumerator} DATA))
        (SETQ DATA ({the step of the enumerator} DATA)))
      ({the epilog} {REPORT, modified}))))
```

```
;note cannot use the name number-of-lines due to internal artifacts of KBE

(DEFINE-CLICHE PRINT-OUT
    (PRIMARY-ROLES (FORMAT-STRING ITEM)
     DESCRIBED-ROLES (FORMAT-STRING ITEM)
     COMMENT "prints out {the item}"
     CONSTRAINTS
      ((DEFAULT {the format-string} "~%~A")
       (DERIVED {the size-in-lines} (SIZE-IN-LINES {the format-string}))))
   (FORMAT REPORT {the format-string} {the input item})
   (SETQ LINE (+ LINE {the size-in-lines})))

;note if we count columns here this becomes a loop fragment and
;implement cannot hack it.

(DEFINE-CLICHE TABULARIZED-PRINT-OUT
    (PRIMARY-ROLES (FORMAT-STRING ITEM)
     DESCRIBED-ROLES (FORMAT-STRING ITEM NUMBER-OF-COLUMNS)
     COMMENT "prints out {the item} in columns"
     CONSTRAINTS
      ((DEFAULT {the format-string} "~16A")
       (DERIVED {the maximum-charpos}
                (- 75 (SIZE-IN-CHARACTERS {the format-string})))))
  (WHEN (> (CHARPOS REPORT) {the maximum-charpos})
    (FORMAT REPORT "~&")
    (SETQ LINE (+ LINE 1)))
  (FORMAT REPORT {the format-string} {the input item}))

;coder gets confused when depending-on is around +

(DEFINE-CLICHE COUNT
    (PRIMARY-ROLES (ITEM)
     DESCRIBED-ROLES (ITEM)
     COMMENT "accumulates a count of {the item}")
  (LET* ((COUNT 0))
    (LOOP DO
      (SETQ COUNT (+ COUNT {1, depending on the input item})))
    COUNT))

;note this reads in as the following old form.
;  (+ COUNT (DEPENDING-ON {the input object} 1))

(comment
;this only works in loops.  In order to get a final value out
;you would have to use a slightly different cliche.

(DEFINE-CLICHE PREVIOUS-VALUE
    (PRIMARY-ROLES (ITEM ITEM-BEFORE-FIRST-ITEM)
     DESCRIBED-ROLES (ITEM ITEM-BEFORE-FIRST-ITEM)
     COMMENT "keeps track of the previous value of {the item}")
  (LET* ((PREVIOUS {the item-before-first-item}))
    (LOOP DO
      {PREVIOUS, the output previous}
      (SETQ PREVIOUS {the input item}))))
  )

;a very interesting bug happens if instead of using the
;following you you use the cliche previous-value and set
;its init to (NCONS NIL) and then try to use the value of
;this NCONS in the body of a loop.  The trouble is that
;the NCONS gets analyzed into just one of the ssr's and
;you cannot use it in the others.  This is yet another
;problem with over analysis.

;also if you make there be a place for an action in this it
;ends up being a filter, and the code chokes for some
;reason I never figured out.
```

```
(DEFINE-CLICHE LEADING-BOUNDARY-DETECTOR
    (PRIMARY-ROLES (ITEM BOUNDARY-TEST)
     DESCRIBED-ROLES (ITEM BOUNDARY-TEST)
     COMMENT "locates the beginning of each subgroup in {the item}")
  (LET* ((DATUM)
         (UNIQUE-DATUM (NCONS NIL))
         (PRIOR-DATUM UNIQUE-DATUM))
    (LOOP DO
      (SETQ DATUM {the input item})
      {(OR (EQ PRIOR-DATUM UNIQUE-DATUM)
           ({the boundary-test} PRIOR-DATUM DATUM)),
       the output boundary-indicator}
      (SETQ PRIOR-DATUM DATUM))))
```

```
;helping fn for lisp demo.

(defun display-report ()
  (with-open-file (file "report.txt" ':in)
    (do ((char (tyi file nil) (tyi file nil))
         (newlines 1))
        (nil)
      (do () ((not (eq char #\newline)))
        (incf newlines)
        (setq char (tyi file nil)))
      (cond ((= newlines 1) (terpri))
            ((= newlines 2) (terpri) (terpri))
            ((> newlines 2)
             (format T "~%<~D blank lines>~%" (1- newlines))))
      (setq newlines 0)
      (if (null char) (return))
      (tyo char)))
  (values))

;this stuff supports charpos for the lisp demo.
;this is supported in later lispm systems but not in the current one.

;;; (WITH-OPEN-FILES (spec1 spec2 spec3 ...) . body)
;;;
;;; is like
;;;
;;; (WITH-OPEN-FILE spec1
;;;  (WITH-OPEN-FILE spec2
;;;   (WITH-OPEN-FILE spec3
;;;    ... . body )))
;;;
;;; except that it does additional work to assure that the
;;; given file is open in a mode which :READ-CURSORPOS
;;; and :SET-CURSORPOS messages work in a limited way on the
;;; stream (so that FORMAT's ~T can work).
;;;
;;; Also, this file defines (CHARPOS stream).

(defun charpos (stream)
  (values (send stream ':read-cursorpos ':character)))

(defmacro with-open-files (bindings &body forms)
  (cond ((null bindings) '(progn ,@forms))
        (t
         (let ((reversed-bindings (reverse bindings)))
           (do ((f '(with-one-open-file ,(car reversed-bindings) ,@forms)
                   '(with-one-open-file ,(car b) ,f))
                (b (cdr reversed-bindings) (cdr b)))
               ((null b) f))))))

(defmacro with-one-open-file (spec &body forms)
  (let ((var1 (gensym)))
    '(original-with-open-file (,var1 ,@(cdr spec))
       (let ((,(car spec) (make-instance 'stream-with-charpos
                                         ':indirect-stream ,var1)))
         ,@forms))))
```

```
(defflavor stream-with-charpos ((indirect-stream nil) (charpos 0)) ()
  (:initable-instance-variables indirect-stream))

(defmethod (stream-with-charpos :read-cursorpos) (&optional units)
  (check-arg units (eq units ':character) "the keyword :CHARACTER")
  (values charpos nil)) ;ugh, no second value

(defmethod (stream-with-charpos :set-cursorpos) (x y &optional units)
  x y ;ignored
  (check-arg units (eq units ':character) "the keyword :CHARACTER")
  (when (> x charpos)
    (dotimes (i (- x charpos))
      (send self ':tyo #\space))))

(defmethod (stream-with-charpos :fresh-line) ()
  (unless (zerop charpos)
    (send self ':tyo #\newline)))

(defmethod (stream-with-charpos :tyo) (char)
  (cond ((= char #\newline)
         (setq charpos 0))
        ((= char #\tab)
         (setq charpos (+ charpos (- 8 (\ charpos 8)))))
        (t
         (incf charpos)))
  (send indirect-stream ':tyo char))

(defmethod (stream-with-charpos :operation-handled-p) (op)
  (or (memq op (send self ':which-operations))
      (send indirect-stream ':operation-handled-p op)))

(defmethod (stream-with-charpos :unclaimed-message) (op &rest args)
  (selectq op
    ((:line-out :string-out)
     (stream-default-handler self op (car args) (cdr args)))
    (otherwise
     (lexpr-send indirect-stream op args))))
```

```
;;; -*- Mode: lisp; Package: KBE; Base:10.; Fonts:MEDFNB; -*-

;;; (c) Copyright 1984, Massachusetts Institute of Technology

;this makes the lisp constraint things available for ada
;we have several cludges here because {} is exactly the same in lisp
;and so cannot be nested in an ada-guy unless he always knows it will.

(eval-when (eval load compile) ;because need ada fn defs to parse

(def-ada (display_report) () nil (display_report))

(defun display_report ()
  (with-open-file (file "report.txt" ':in)
    (do ((char (tyi file nil) (tyi file nil))
         (newlines 1))
        (nil)
      (do () ((not (eq char #\newline)))
        (incf newlines)
        (setq char (tyi file nil)))
      (cond ((= newlines 1) (terpri))
            ((= newlines 2) (terpri) (terpri))
            ((> newlines 2)
             (format T "~%<~2D blank lines>~%" (1- newlines))))
      (setq newlines 0)
      (if (null char) (return))
      (tyo char)))
  (values))

(def-ada (rename) ((old (ada-string)) (new (ada-string))) nil
  (rename-all old new))

(def-ada (derived) ((x (any)) (y (any))) nil
  (derived |X-full| y))

(def-ada (default) ((x (any)) (y (any))) nil
  (default |X-full| y))

(def-ada (FILE_NAME_ROOT) ((thing (any))) (ada-string)
  (file-name-root |THING-full|))

;nausiating kludge for now.  Should really look up and figure
;out what kinds of records are in the file.
(def-ada (RECORD_KEY_ROOT) ((thing (any))) (ada-string)
  (record_key_root-stub |THING-full|))

(defun record_key_root-stub (ignore) "DEFECT_KEY")

(def-ada (size_in_lines) ((x (any))) (integer)
  (ada-line-size |X-full|))

(defun ada-line-size (segcase)
  (declare (special total-size))
  (if (not segcase) (*throw '2constraint-abort nil))
  (let ((total-size 0))
    (cond ((not-implemented? (~seg segcase)) 1)
          (T (mapcar #'ada-line-size-0 (all-subsegs (~seg segcase)))
             total-size))))

(defun ada-line-size-0 (seg)
  (declare (special total-size))
  (when (??& (&* seg) @seg 'funcall 'ada-fn~new_line)
    (let* ((code (car (?@seg-code seg)))
           (numeric-arg (caddr code)))
      (if (listp numeric-arg) (setq numeric-arg (cadr numeric-arg)))
      (if (not (numberp numeric-arg)) (setq numeric-arg 1))
      (setq total-size (+ total-size numeric-arg)))))

(def-ada (corresponding_file_name) ((thing (any))) (any)
```

```
  (corresponding-file-name |THING-full|))

(defun corresponding-file-name (segcase)
  (let (root file-info)
    (if (not (and segcase
                  (setq root (file-name-root segcase))
                  (setq file-info (get-ada-file-info root))))
        (*throw '2constraint-abort nil))
    (ada-file-name-variable file-info)))

(def-ada (corresponding_headings) ((thing (any))) (any)
  (corresponding-headings |THING-full|))

(defun corresponding-headings (segcase)
  (let (cliche-name corresponding-name)
    (if (not (and segcase
                  (setq cliche-name (~01 ~clichename (??& segcase cliche)))
                  (setq corresponding-name
                        (symbolconc cliche-name "_HEADINGS"))
                  (get corresponding-name 'cliche-plan)))
        (*throw '2constraint-abort nil))
    (& (2get-cliche corresponding-name))))
```

```
(def-ada (chain_field) ((ff (any)) (tf (any))) (any)
  (chain-field |FF-full| |TF-full|))

(defun chain-field (from-file to-file)
  (let (from-file-name to-file-name from-file-info to-file-info
        to-key-type field-name)
    (if (not (and from-file to-file
                  (setq from-file-name (file-name-root from-file))
                  (setq from-file-info (get-ada-file-info from-file-name))
                  (setq to-file-name (file-name-root to-file))
                  (setq to-file-info (get-ada-file-info to-file-name))
                  (setq to-key-type
                        (ncons (ada-file-key-type to-file-info)))
                  (setq field-name
                        (dolist (field (cdr (ada-file-record-def
                                             from-file-info)))
                          (if (equal (cadr field) to-key-type)
                              (return (car field)))))))
        (*throw '2constraint-abort nil))
    ;note how we turn it into a one arg thing here.
    '(ada-select #{the input rec} ,field-name)))


(def-ada (corresponding_reporting) ((thing (any))) (any)
  (corresponding-reporting |THING-full|))

(defun corresponding-reporting (segcase)
  (let (cliche-name record-read corresponding-name)
    (if (not (and segcase
                  (setq cliche-name (~01 ~clichename (??& segcase cliche)))
                  (not (eq cliche-name 'pseudo-subcliche))
                  (setq record-read (determine-record-enumerated segcase))
                  (setq corresponding-name
                        (symbolconc "PRINT_" record-read))
                  (get corresponding-name 'cliche-plan)))
        (*throw '2constraint-abort nil))
    (& (2get-cliche corresponding-name))))

(defun determine-record-enumerated (segcase)
  (let (outroles source record-name)
    (if (not (and segcase
                  (setq outroles (??& segcase slot * * 'output))
                  (null (cdr outroles))
                  (setq source (dflow-source
                                (?& (~slotseg (car outroles)) input)))
                  (setq record-name (~01 ~varname (??& source @createvar)))
                  (symbolp record-name)))
        (*throw '2constraint-abort nil))
    record-name))
```

```
(defmacro def-ada-text-cliche (string)
  (ada-parse string))

(def-ada (SUGGEST_VARIABLE_NAME) ((thing (any))) (ada-string)
  (suggest-variable-name |THING-full|))

(defun suggest-variable-name (segcase)
  (let (outputs source var-name)
    (when (not (and segcase
                    (setq outputs (??& segcase output))
                    (null (cdr outputs))
                    (setq source (dflow-source (car outputs)))
                    (setq var-name
                          (3make-up-name-for-returnval
                            (~seg~segcase source)))))
      (*throw '2constraint-abort nil))
    var-name))

;nausiating kludge for now.  Should really look up and figure
;out what kinds of records are in the file.  This would
;in fact be easier if we were making the whole print-summary
;cliche new each time.
(def-ada (CORRESPONDING_SUMMARIZED_FILE) ((thing (any))) (ada-string)
  (corresponding_summarized_file-stub |THING-full|))

(defun corresponding_summarized_file-stub (ignore)
  'DEFECTS)
 ) ;end of eval-when needed for parsing of ada cliches

;This reads everything up to the next $ (dollar sign) which is the
;last character on a line and returns it as one huge string.

(eval-when (eval compile load)

(defun read-literal-string (ignore stream)
  (let ((lines nil))
    (do ((line (readline stream nil) (readline stream nil)))
        ((null line))
      (when (and (not (string-equal line ""))
                 (= (aref line (1- (string-length line))) #/$))
        (push (substring line 0 (1- (string-length line))) lines)
        (return))
      (push line lines)
      (push #\return lines))
    (apply #'string-append (nreverse lines))))

(set-syntax-/#-macro-char #/$ 'read-literal-string) ) ;dollar sign

(def-ada-text-cliche #$
cliche PRE_LOOP_COMPUTATION is
   primary roles ITEM;
   described roles ITEM;
   comment "makes the value {the item} available inside of a loop";
   constraints
      RENAME("DATUM", SUGGEST_VARIABLE_NAME({the item}));
   end constraints;

   DATUM: {};
begin
   DATUM := {the item};
   loop
      {DATUM, the output datum};
   end loop;
end PRE_LOOP_COMPUTATION;$
)
```

```
;note it is important that no cliche name is the same as a role name.

;note we cannot make {the data} really be a role because it ends
;up inside of a quoted list due to the way we handle ada.
;we can have the following so that it looks like a role however.
;a gross cludge to be sure.

(def-ada-text-cliche #$
with FUNCTIONS;
use FUNCTIONS;
cliche CHAIN_FILE_DEFINITION is
   primary roles FILE_NAME;
   described roles FILE_NAME;
   comment "defines a file named {the file_name} of chain records";
   constraints
      RENAME("DATA_RECORD", FILE_NAME_ROOT({the file_name}));
   end constraints;

   DATA_RECORDS_NAME: constant STRING := {the file_name};
   subtype DATA_RECORD_INDEX_TYPE is INDEX_TYPE;
   type DATA_RECORD_TYPE is
      record
         {the data};
         NEXT: DATA_RECORD_INDEX_TYPE;
      end record;
   package DATA_RECORD_IO is
      new CHAINED_IO(DATA_RECORD_TYPE, DATA_RECORD_INDEX_TYPE);
   DATA_RECORDS: DATA_RECORD_IO.FILE_TYPE;
end CHAIN_FILE_DEFINITION;$
)

;The fake role key_length is the same kind of cludge as type above.

(def-ada-text-cliche #$
with FUNCTIONS;
use FUNCTIONS;
cliche KEYED_FILE_DEFINITION is
   primary roles FILE_NAME;
   described roles FILE_NAME;
   comment "defines a keyed file named {the file_name} of records";
   constraints
      RENAME("DATA_RECORD", FILE_NAME_ROOT({the file_name}));
   end constraints;

   DATA_RECORDS_NAME: constant STRING := {the file_name};
   subtype DATA_RECORD_KEY_TYPE is STRING(1..{the key_length});
   type DATA_RECORD_TYPE is
      record
         {the data};
      end record;
   package DATA_RECORD_IO is new KEYED_IO(DATA_RECORD_TYPE, DATA_RECORD_KEY_TYPE);
   DATA_RECORDS: DATA_RECORD_IO.FILE_TYPE;
end KEYED_FILE_DEFINITION;$
)
```

```
;you must give this cliche an argument in order for the renaming
;to work.  (Also setq of FILE is out of order in that case.)
;note that FILE must not get renamed, or the dflow will not be right.
;you cannot call the record RECORD because that is an ADA keyword
;you cannot call it REC because that is a KBE keyword.

;all of the above is true for most of the cliches below.

(def-ada-text-cliche #$
cliche READ is
   primary roles FILE, KEY;
   described roles FILE, KEY;
   comment "reads the record indexed by {the key} from {the file}";
   constraints
      RENAME("DATA_RECORD", FILE_NAME_ROOT({the file}));
      DEFAULT({the file_name}, CORRESPONDING_FILE_NAME({the file}));
   end constraints;

   DATA_RECORD: {};
   FILE: {};
begin
   FILE := {the file};
   OPEN(FILE, IN_FILE, {the file_name});
   READ(FILE, DATA_RECORD, {the key});
   {DATA_RECORD, the  output data_record};
   CLOSE(FILE);
exception
   when DEVICE_ERROR | END_ERROR | NAME_ERROR | STATUS_ERROR =>
      CLOSE(FILE); PUT("Data Base Inconsistent");
   when others => CLOSE(FILE); raise;
end READ;$
)
```

```
(def-ada-text-cliche #$
with TEXT_IO;
use TEXT_IO;
cliche QUERY_USER_FOR_KEY is
   primary roles FILE;
   described roles FILE;
   comment "queries the user for a key to a record in {the file}";
   constraints
      RENAME("DATA_RECORD", FILE_NAME_ROOT({the file}));
      DEFAULT({the file_name}, CORRESPONDING_FILE_NAME({the file}));
   end constraints;

   DATA_RECORD: {};
   DATA_RECORD_KEY: {};
   FILE: {};
begin
   FILE := {the file};
   OPEN(FILE, IN_FILE, {the file_name});
   loop
      begin
         NEW_LINE; PUT("Enter DATA_RECORD Key: "); GET(DATA_RECORD_KEY);
         READ(FILE, DATA_RECORD, DATA_RECORD_KEY);
         exit;
      exception
         when END_ERROR => PUT("Invalid DATA_RECORD Key"); NEW_LINE;
      end;
   end loop;
   CLOSE(FILE);
   return DATA_RECORD_KEY;
exception
   when DEVICE_ERROR | END_ERROR | NAME_ERROR | STATUS_ERROR =>
      CLOSE(FILE); PUT("Data Base Inconsistent");
   when others => CLOSE(FILE); raise;
end QUERY_USER_FOR_KEY; $
)
```

```
(def-ada-text-cliche #$
cliche FILE_ENUMERATION is
   primary roles FILE;
   described roles FILE;
   comment "enumerates the records in {the file}";
   constraints
      RENAME("DATA_RECORD", FILE_NAME_ROOT({the file}));
      DEFAULT({the file_name}, CORRESPONDING_FILE_NAME({the file}));
   end constraints;

   FILE: {};
   DATA_RECORD: {};
begin
   FILE := {the file};
   OPEN(FILE, IN_FILE, {the file_name});
   while not END_OF_FILE(FILE) loop
      READ(FILE, DATA_RECORD);
      {DATA_RECORD, the output data_record};
   end loop;
   CLOSE(FILE);
exception
   when DEVICE_ERROR | END_ERROR | NAME_ERROR | STATUS_ERROR =>
      CLOSE(FILE); PUT("Data Base Inconsistent");
   when others => CLEAN_UP; raise;
end FILE_ENUMERATION;$
)
```

```
(def-ada-text-cliche #$
cliche CHAIN_ENUMERATION is
   primary roles MAIN_FILE, CHAIN_FILE, MAIN_FILE_KEY;
   described roles MAIN_FILE, CHAIN_FILE, MAIN_FILE_KEY;
   comment "enumerates the chain records in {the chain_file} starting
            from the header record indexed by {the main_file_key}";
   constraints
      RENAME("MAIN_RECORD", FILE_NAME_ROOT({the main_file}));
      RENAME("CHAIN_RECORD", FILE_NAME_ROOT({the chain_file}));
      DEFAULT({the main_file_name},
              CORRESPONDING_FILE_NAME({the main_file}));
      DEFAULT({the chain_file_name},
              CORRESPONDING_FILE_NAME({the chain_file}));
      DEFAULT({the main_file_chain_field},
              CHAIN_FIELD({the main_file}, {the chain_file}));
      DEFAULT({the chain_file_chain_field},
              CHAIN_FIELD({the chain_file}, {the chain_file}));
   end constraints;

   CHAIN_FILE: {};
   CHAIN_RECORD: {};
   CHAIN_RECORD_INDEX: {};
   MAIN_FILE: {};
   MAIN_RECORD: {};

   procedure CLEAN_UP is
   begin
      CLOSE(CHAIN_FILE); CLOSE(MAIN_FILE);
   exception
      when STATUS_ERROR => return;
   end CLEAN_UP;
begin
   CHAIN_FILE := {the chain_file};
   MAIN_FILE := {the main_file};
   OPEN(CHAIN_FILE, IN_FILE, {the chain_file_name});
   OPEN(MAIN_FILE, IN_FILE, {the main_file_name});
   READ(MAIN_FILE, MAIN_RECORD, {the main_file_key});
   CHAIN_RECORD_INDEX := {the main_file_chain_field}(MAIN_RECORD);
   while not NULL_INDEX(CHAIN_RECORD_INDEX) loop
      READ(CHAIN_FILE, CHAIN_RECORD, CHAIN_RECORD_INDEX);
      {CHAIN_RECORD, the output chain_record};
      CHAIN_RECORD_INDEX := {the chain_file_chain_field}(CHAIN_RECORD);
   end loop;
   CLEAN_UP;
exception
   when DEVICE_ERROR | END_ERROR | NAME_ERROR | STATUS_ERROR =>
      CLEAN_UP; PUT("Data Base Inconsistent");
   when others => CLEAN_UP; raise;
end CHAIN_ENUMERATION;$
)
```

```
(def-ada-text-cliche #$
with FUNCTIONS;
use FUNCTIONS;
cliche ALL_CHAINS_ENUMERATION is
   primary roles MAIN_FILE, CHAIN_FILE;
   described roles MAIN_FILE, CHAIN_FILE;
   comment "enumerates the header records in {the main_file} and
            enumerates the chain records in {the chain_file} starting
            from each header record";
   constraints
      RENAME("MAIN_RECORD", FILE_NAME_ROOT({the main_file}));
      RENAME("CHAIN_RECORD", FILE_NAME_ROOT({the chain_file}));
      DEFAULT({the main_file_name},
              CORRESPONDING_FILE_NAME({the main_file}));
      DEFAULT({the chain_file_name},
              CORRESPONDING_FILE_NAME({the chain_file}));
      DEFAULT({the main_file_chain_field},
              CHAIN_FIELD({the main_file}, {the chain_file}));
      DEFAULT({the chain_file_chain_field},
              CHAIN_FIELD({the chain_file}, {the chain_file}));
   end constraints;

   CHAIN_FILE: {};
   CHAIN_RECORD: {};
   CHAIN_RECORD_INDEX: {};
   MAIN_FILE: {};
   MAIN_RECORD: {};

   procedure CLEAN_UP is
   begin
      CLOSE(CHAIN_FILE); CLOSE(MAIN_FILE);
   exception
      when STATUS_ERROR => return;
   end CLEAN_UP;
begin
   CHAIN_FILE := {the chain_file};
   MAIN_FILE := {the main_file};
   OPEN(CHAIN_FILE, IN_FILE, {the chain_file_name});
   OPEN(MAIN_FILE, IN_FILE, {the main_file_name});
   CHAIN_RECORD_INDEX := NULL_INDEX;
   loop
      while NULL_INDEX(CHAIN_RECORD_INDEX) and not END_OF_FILE(MAIN_FILE) loop
         READ(MAIN_FILE, MAIN_RECORD);
         CHAIN_RECORD_INDEX := {the main_file_chain_field}(MAIN_RECORD);
      end loop;
      exit when NULL_INDEX(CHAIN_RECORD_INDEX);
      READ(CHAIN_FILE, CHAIN_RECORD, CHAIN_RECORD_INDEX);
      {CHAIN_RECORD, the output chain_record};
      CHAIN_RECORD_INDEX := {the chain_file_chain_field}(CHAIN_RECORD);
   end loop;
   CLEAN_UP;
exception
   when DEVICE_ERROR | END_ERROR | NAME_ERROR | STATUS_ERROR =>
      CLEAN_UP; PUT("Data Base Inconsistent");
   when others => CLEAN_UP; raise;
end ALL_CHAINS_ENUMERATION;$
)
```

```
(def-ada-text-cliche #$
with FUNCTIONS, TEXT_IO;
use FUNCTIONS, TEXT_IO;
cliche SIMPLE_REPORT is
   primary roles ENUMERATOR, PRINT_ITEM, EPILOG;
   described roles FILE_NAME, TITLE, ENUMERATOR, COLUMN_HEADINGS,
                   PRINT_ITEM, EPILOG;
   comment "prints a report of {the input structure of the enumerator}";
   constraints
      DEFAULT({the file_name}, "report.txt");
      DERIVED({the line_limit},
              65-SIZE_IN_LINES({the print_item})-SIZE_IN_LINES({the epilog}));
      DEFAULT({the print_item}, CORRESPONDING_REPORTING({the enumerator}));
      DEFAULT({the column_headings},
              CORRESPONDING_HEADINGS({the print_item}));
   end constraints;

   use INT_IO;
   CURRENT_DATE: constant STRING := FORMAT_DATE(CLOCK);
   DATA: {};
   TITLE: STRING(1..{});

   procedure CLEAN_UP is
   begin
      SET_OUTPUT(STANDARD_OUTPUT);
      CLOSE(REPORT);
   exception
      when STATUS_ERROR => return;
   end CLEAN_UP;
begin
   CREATE(REPORT, OUT_FILE, {the file_name});
   DATA := {the input structure of the enumerator};
   SET_OUTPUT(REPORT);
   TITLE := {the title};
   NEW_LINE(4); SET_COL(20); PUT(CURRENT_DATE); NEW_LINE(2);
   SET_COL(13); PUT(TITLE); NEW_LINE(60);
   while not {the empty_test of the enumerator}(DATA) loop
      if LINE > {the line_limit} then
         NEW_PAGE; NEW_LINE; PUT("Page: "); PUT(INTEGER(PAGE-1), 3);
         SET_COL(13); PUT(TITLE);
         SET_COL(61); PUT(CURRENT_DATE); NEW_LINE(2);
         {the column_headings}({CURRENT_OUTPUT, modified});
      end if;
      {the print_item}({CURRENT_OUTPUT, modified},
                       {the element_accessor of the enumerator}(DATA));
      DATA := {the step of the enumerator}(DATA);
   end loop;
   {the epilog};
   CLEAN_UP;
exception
   when DEVICE_ERROR | END_ERROR | NAME_ERROR | STATUS_ERROR =>
      CLEAN_UP; PUT("Data Base Inconsistent");
   when others => CLEAN_UP; raise;
end SIMPLE_REPORT;$
)
```

```
(def-ada-text-cliche #$
with DIRECT_IO;
cliche FILE_SELECTION is
   primary roles SOURCE_FILE;
   described roles SOURCE_FILE, SELECTION_TEST, RECORD_KEY_ACCESSOR;
   comment "creates a file containing selected record keys for
            {the source_file}";
   constraints
      RENAME("RECORD_KEY", RECORD_KEY_ROOT({the source_file}));
   end constraints;

   package SELECTION_IO is new DIRECT_IO(RECORD_KEY_TYPE);
   use SELECTION_IO;
   SELECTIONS: SELECTION_IO.FILE_TYPE;

   DATA: {};
   DATUM: {};
begin
   DATA := {the source_file};
   CREATE(SELECTIONS, INOUT_FILE);
   while not {the empty_test of the enumerator}(DATA) loop
      DATUM := {the element_accessor of the enumerator}(DATA);
      if {the selection_test}(DATUM) then
         WRITE(SELECTIONS, {the record_key_accessor}(DATUM));
      end if;
      DATA := {the step of the enumerator}(DATA);
   end loop;
   {SELECTIONS, the output selection_file};
   CLOSE(SELECTIONS);
exception
   when DEVICE_ERROR | END_ERROR | NAME_ERROR | STATUS_ERROR =>
      CLOSE(SELECTIONS); PUT("Data Base Inconsistent");
   when others => CLOSE(SELECTIONS); raise;
end FILE_SELECTION;$
)
```

```
;note you cannot let the input role source_file get renamed.
;to have source_record as a prefix whether you like it or not.

(def-ada-text-cliche #$
with DIRECT_IO;
cliche FILE_SUMMARIZATION is
   primary roles SOURCE_FILE;
   described roles SOURCE_FILE;
   comment "creates a file summarizing the frequency of occurance of
            the record keys stored in {the source file}";
   constraints
      RENAME("SOURCE_RECORD", FILE_NAME_ROOT({the source_file}));
      RENAME("RECORD_KEY", RECORD_KEY_ROOT({the source_file}));
      DEFAULT({the source_file_name},
              CORRESPONDING_FILE_NAME({the source_file}));
   end constraints;

   type SUMMARY_TYPE is
      record
         COUNT: INTEGER;
         KEY: RECORD_KEY_TYPE;
      end record;
   package SUMMARY_IO is new DIRECT_IO(SUMMARY_TYPE);
   use SUMMARY_IO;
   SUMMARIES: SUMMARY_IO.FILE_TYPE;

   SOURCE_FILE: {};

   function BUILD_SUMMARY(COUNT: INTEGER; KEY: RECORD_KEY_TYPE)
                              return SUMMARY_TYPE is
      begin return SUMMARY_TYPE'(COUNT, KEY); end BUILD_SUMMARY;
   function SUMMARY_GREATER_THAN(X: SUMMARY_TYPE; Y: SUMMARY_TYPE)
                                    return BOOLEAN is
      begin return X.COUNT > Y.COUNT; end SUMMARY_GREATER_THAN;
   procedure SORT_SOURCE_RECORDS is
      new SORT_FILE(RECORD_KEY_TYPE, SOURCE_RECORD_IO.FILE_TYPE,
                    SOURCE_RECORD_IO.POSITIVE_COUNT);
   procedure SORT_SUMMARIES is
      new SORT_FILE(SUMMARY_TYPE, SUMMARY_IO.FILE_TYPE,
                    SUMMARY_IO.POSITIVE_COUNT, SUMMARY_GREATER_THAN);
   procedure SUMMARIZE_SOURCE_RECORDS is
      new SUMMARIZE_FILE(RECORD_KEY_TYPE, SOURCE_RECORD_IO.FILE_TYPE,
                         SUMMARY_TYPE, SUMMARY_IO.FILE_TYPE, BUILD_SUMMARY);
   procedure CLEAN_UP is
      begin
         CLOSE(SOURCE_FILE); CLOSE(SUMMARIES);
      exception
         when STATUS_ERROR => return;
      end CLEAN_UP;
begin
   SOURCE_FILE := {the source_file};
   OPEN(SOURCE_FILE, INOUT_FILE, {the source_file_name});
   CREATE(SUMMARIES, INOUT_FILE);
   SORT_SOURCE_RECORDS(SOURCE_FILE);
   SUMMARIZE_SOURCE_RECORDS(SOURCE_FILE, SUMMARIES);
   SORT_SUMMARIES(SUMMARIES);
   {SUMMARIES, the output summaries};
   CLEAN_UP;
exception
   when DEVICE_ERROR | END_ERROR | NAME_ERROR | STATUS_ERROR =>
      CLEAN_UP; PUT("Data Base Inconsistent");
   when others => CLEAN_UP; raise;
end FILE_SUMMARIZATION;$
)
```

```
;These could be automatically generated for each particlar kind
;of summary file, but we just use a predefined general one for simplicity.

(def-ada-text-cliche #$
with TEXT_IO;
use TEXT_IO;
cliche PRINT_SUMMARY_HEADINGS is
   comment "prints headings for printing repair records";

begin
  PUT("  # Key    Name"); NEW_LINE(2);
end PRINT_SUMMARY_HEADINGS;$
)
;note that you cannot have a variable name the same as the name of
;a function becuase my parser will parse it as a zero arg fn call.
;we have to use the big expression for rename because renames
;actually must happen first.

(def-ada-text-cliche #$
with TEXT_IO;
use TEXT_IO;
cliche PRINT_SUMMARY is
   primary roles SUMMARY;
   described roles SUMMARY, SUMMARIZED_FILE;
   comment "prints out the fields of {the summary}";
   constraints
      RENAME("SUMMARIZED_RECORD",
             FILE_NAME_ROOT(CORRESPONDING_SUMMARIZED_FILE({the summary})));
      DEFAULT({the summarized_file},
               CORRESPONDING_SUMMARIZED_FILE({the summary}));
      DEFAULT({the summarized_file_name},
               CORRESPONDING_FILE_NAME({the summarized_file}));
   end constraints;

   SUMMARIZED_FILE: {};
   SUMMARIZED_RECORD: {};
   SUMMARY: {};
begin
   SUMMARIZED_FILE := {the summarized_file};
   OPEN(SUMMARIZED_FILE, IN_FILE, {the summarized_file_name});
   SUMMARY := {the input summary};
   READ(SUMMARIZED_FILE, SUMMARIZED_RECORD, SUMMARY.KEY);
   PUT(SUMMARY.COUNT, 3); SET_COL(5); PUT(SUMMARY.KEY);
   SET_COL(12); PUT(SUMMARIZED_RECORD.NAME); NEW_LINE;
   CLOSE(SUMMARIZED_FILE);
exception
   when DEVICE_ERROR | END_ERROR | NAME_ERROR | STATUS_ERROR =>
      CLOSE(SUMMARIZED_FILE); PUT("Data Base Inconsistent");
   when others => CLOSE(SUMMARIZED_FILE); raise;
end PRINT_SUMMARY;$
)
```

```
;these cliches are created during the demo, but are defined
;here to make testing easier.

(def-ada-text-cliche #$
with TEXT_IO;
use TEXT_IO;
cliche PRINT_REPAIR_HEADINGS is
   comment "prints headings for printing repair records";

begin
  PUT("   Date     Defect   Description/Comment"); NEW_LINE(2);
end PRINT_REPAIR_HEADINGS;$
)
(def-ada-text-cliche #$
with FUNCTIONS, MAINTENANCE_FILES, TEXT_IO;
use FUNCTIONS, MAINTENANCE_FILES, TEXT_IO;
cliche PRINT_REPAIR is
   primary roles REPAIR;
   described roles REPAIR;
   comment "prints out the fields of {the repair}";

   use DEFECT_IO;
   DEFECT: DEFECT_TYPE;
   REPAIR: REPAIR_TYPE;
begin
   OPEN(DEFECTS, IN_FILE, DEFECTS_NAME);
   REPAIR := {the input repair};
   READ(DEFECTS, DEFECT, REPAIR.DEFECT);
   PUT(FORMAT_DATE(REPAIR.DATE));
   SET_COL(13); PUT(REPAIR.DEFECT);
   SET_COL(20); PUT(DEFECT.NAME); NEW_LINE;
   SET_COL(22); PUT(REPAIR.COMMENT); NEW_LINE;
   CLOSE(DEFECTS);
exception
   when DEVICE_ERROR | END_ERROR | NAME_ERROR | STATUS_ERROR =>
      CLOSE(DEFECTS); PUT("Data Base Inconsistent");
   when others => CLOSE(DEFECTS); raise;
end PRINT_REPAIR;$
)
```